

Fish-Net: Networking Evolved

Version v4

Generated 2026-09-22

Documentation for Fish-Net Pro, the high-performance Unity networking solution.

Contents

Fish-Network-Discovery	8
Asset Integrations	10
Configuration and Tools	12
Building a Dedicated Server	14
Complete Games	21
Common Tasks	22
Making a Custom Player Spawner	23
Spawning Players Manually	24
On Scene Load	27
Overview	31
Set Player Number	34
Spawning Selected Player	38
Making a Loading Screen	41
Simulating Bad Network Connections	44
Starting FishNet's Connections	46
Testing with Multiple Editors	51
Community Resources	56
ColliderRollback (Pro Feature)	57
DefaultScene	58
Components	59
ClientManager	60
DebugManager	61
HashGrid	62
Managers	63
IntermediateLayer	64
NetworkManager	67
ObserverManager	69
PredictionManager	70
RollbackManager (Pro Feature)	71
SceneManager	72
ServerManager	73
StatisticsManager	75
TimeManager	76
TransportManager	78
NetworkBehaviour	79
NetworkObject	80
NetworkObserver	82
NetworkTransform	83
NetworkAnimator	85

Contents

PlayerSpawner	87
Prediction	88
Network Collider	89
NetworkCollision	90
NetworkCollision2D	91
NetworkTrigger	92
NetworkTrigger2D	93
OfflineRigidbody	94
PredictedOwner	95
PredictedSpawn	96
ServerSpawner	97
DetachableNetworkTickSmoother \[Obsolete]	98
Tick Smoothers	99
MonoTickSmoother \[Obsolete]	100
NetworkTickSmoother	101
OfflineTickSmoother	103
Authenticator	105
BandwidthDisplay	106
Utilities	107
PingDisplay	108
Configuration and Tools	109
Addressables	111
Code Stripping (Pro Feature)	114
Custom Serializers Guides	117
Inheritance Serializers	119
Interface Serializers	122
Data Serialization	125
Features	127
InstanceFinder	128
Lag Compensation	129
Projectiles	130
Raycast	135
States	138
Network State Events	140
Broadcasts	142
Network Communication	145
Remote Procedure Calls	146
Custom Synctype	150
Customizing Behavior	151

Contents

SyncTypes	153
Syncdictionary	154
Synhashset	156
Synclist	159
Syncstopwatch	162
Synctimer	164
Syncvar	168
Synchronizing	170
Networked GameObjects and Scripts	171
Network Behaviour Guides	172
Networkobjects	176
Nested NetworkObjects	177
Spawning and Despawning	178
Object Pooling	179
Predicted Spawning	183
Spawn Payloads	184
Spawning	187
Custom Conditions	188
Area of Interest (Observer System)	191
Modifying Conditions	193
Ownership	194
Using Ownership To Read Values	196
Configuring NetworkObject	202
Configuring PredictionManager	205
Configuring TimeManager	207
Advanced Controls	208
Controlling an Object	212
Creating Code	218
Non-Controlled Object	219
Understanding ReplicateState	222
Predicting States in Code	223
Using States in Code	225
Custom Comparers	229
Prediction	231
Interpolations	232
Offline Rigidbodies	233
PredictionRigidbody	234
Using NetworkColliders	235
What Is Client-Side Prediction	238

Contents

Addressables	239
Custom Scene Processors	240
Scene Management	241
Automatic Online And Offline Scenes	242
Loading Scenes	243
Loading Scenes	248
Persisting Networkobjects	252
Scene Caching	254
Scene Data	255
SceneLoadData	256
Scenelookupdata	259
SceneUnloadData	260
Scene Events	262
Scene Stacking	264
Scene Visibility	266
Unloading Scenes	267
Executing On Server Or Client	268
Server and Client Identification	270
Networkconnections	271
Transports	273
Multipass	274
Beyond the Basics	276
Connecting to Remote Devices	278
Getting Connected	281
Getting Started	284
Installing Fish-Networking	285
Moving Your Player Around	287
Preparing Your Player	294
Basic Setup with Cinemachine	299
Setting Up a Camera	300
Instantiating a Local Camera	301
Overview	302
Using the Scene Camera	305
Spawning and Despawning Items	308
Using SyncVars to Sync Colors	314
Fundamentals	316
High-Level Overview	318
Networking Models	321
Communicating	323

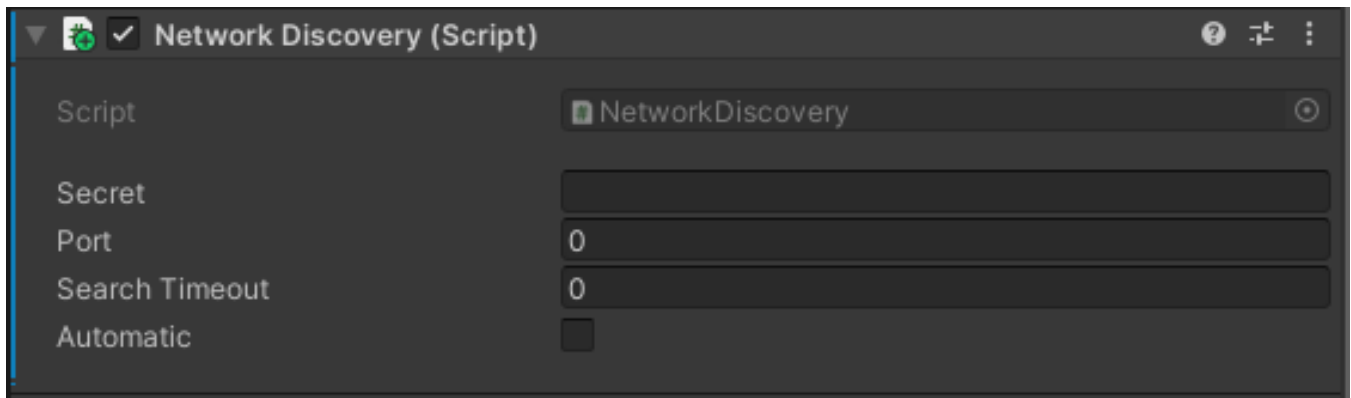
Contents

Terminology	325
Miscellaneous	326
Server, Client, Host	327
Transports	328
Branding	329
Business Support	330
Development	331
Features	332
Benchmark Setup	335
Fish-Networking vs Mirror	341
Performance	343
Unity Compatibility	344
Legal Restrictions	345
Pro, Projects, and Support	346
What Is FishNet?	350
Moving to Fish-Networking	352
Prefabs	355
NetworkHudCanvas	356
NetworkManager	357
ScriptableObjects	358
LevelLoggingConfiguration	359
DistanceCondition	360
GridCondition	361
HostOnlyCondition	362
ObserverConditions	363
MatchCondition	364
OwnerOnlyCondition	365
SceneCondition	366
DefaultPrefabObjects	367
DualPrefabObjects	368
SpawnablePrefabs	369
SinglePrefabObjects	370
Server Hosting	371
Getting Started with AWS	372
Services	374
Easy WebGL Hosting with PlayFlow	376
Getting Started with PlayFlow	383
PlayFlow Cloud (Recommended)	393
Services	395

Contents

Terminology	397
Showcase	399
Upcoming Releases	405
Bayou	407
FishyEOS (Epic Online Services)	409
FishyFacepunch (Steam)	411
FishyRealtime (Photon)	413
FishySteamworks (Steam)	414
FishyUnityTransport	415
FishyWebRTC	417
Transports	418
Multipass	419
Tugboat	420
Yak (Pro Feature)	422
Creating Bug Reports	423
Report Example	426
Creating Bug Reports	427
FAQ	429
Troubleshooting	433
Technical Limitations	434
Updating FishNet	435
Changes in API	436
Upgrading to FishNet Pro	441

Fish-Network-Discovery



General

Fish-Networking-Discovery allows you to find and connect to devices on the same network without needing to adjust firewall or router rules.

Compatibility

System	Supported?
Windows	Fully Supported
MacOS	Fully Supported
IOS	Fully Supported
Android	Fully Supported
Linux	Fully Supported
Xbox	Not Supported
Playstation	Not Supported
Nintendo	Not Supported

How to Install

For installation and usage instructions please visit the GitHub page.

- Main: <https://github.com/Abdelfattah-Radwan/Fish-Networking-Discovery>
- Fork: <https://github.com/FirstGearGames/Fish-Networking-Discovery>

Component Settings

Default Fish Network Discovery

- **Secret:** Secret message to use when advertising or searching for servers.
- **Port:** Port to use when advertising or searching for servers.
- **Search Timeout:** How long (in seconds) to wait for a response when advertising or searching for servers.
- **Automatic:** If true, will automatically start advertising or searching for servers when the NetworkManager starts or stops.

Asset Integrations

You'll find here a collection of third-party integrations for Fish-Networking in other assets, as well as assets that work well with FishNet.

Info: If you have a contribution you would like to add to this page, please feel free to open a pull-request on the [documentation repository](#) or leave us a [suggestion](#).

The following resources are put together by developers such as yourself. The Fish-Networking team has no official involvement in these resources, thus we may not be able to troubleshoot or resolve any issues you experience with them.

Asset Integrations

[MetaVoiceChat](#) is a completely free, simple, and self-contained proximity voice chat solution for Unity's popular client-server networking libraries with all of the features you need.

[Dissonance Voice Chat](#) is a real-time Voice over IP (VoIP) system designed to be built directly into Unity games. [Dissonance Voice Chat for Fish-Networking](#) is a community integration for using it with FishNet, provided by [LambdaTheDev](#) and the creator of Dissonance.

[FishNet Multi-Room Manager](#) is a lightweight, single-process multi-room solution for FishNet that allows you to host multiple isolated game rooms on one server using Unity's additive scenes and FishNet's built-in scene-based visibility. Players connect to a central lobby, where they can view active rooms, create new ones, and join existing rooms. Each room runs in its own additive scene with completely isolated network traffic.

[Turtle Pass](#) allows you to send large byte arrays over several frames to prevent data from overwhelming bandwidth limited transports like Steam and Unity Transport.

[Network Audio Sync](#) is an easy way to synchronize audio clips over the network.

[Master Server Toolkit](#) is a powerful solution to many back-end problems you may encounter when developing multiplayer games or applications. It includes user registration and authorization, game server list, user data management, database interactions, chats and chat channels. A Fish-Networking demo is included!

[Network Particle System](#) will synchronize particles, lifespan, and more.

[GameKit for Fish-Networking](#) is an efficient solution to managing inventory, chat, and several other common multiplayer game mechanics. GameKit is well commented and is intended to be used as a learning resource.

[ParrelSync](#) allows you to run multiple editors of the same project, making multiplayer testing easier.

[MultiPlay](#) lets you open multiple Unity editor windows to test multiplayer games without building every time.

[Facepunch.Steamworks](#) is a C# wrapper on top of Valve's Steamworks library.

[Steamworks.NET](#) is a C# Wrapper for Valve's Steamworks API, it can be used either with Unity or your C# based application.

[Toolkit for Steamworks 2025](#) is a powerful set of tools, systems and editor extensions to make your integration with the Steam API easier, faster, and more robust.

[Steamworks Foundation](#) is a free, limited version of Toolkit for Steamworks.

[Fish-Networking-Discovery](#) is a simple LAN network discovery component for Fish-Networking.

[Vault Inventory](#) supports your design requirements from Editor to Runtime with an intuitive designer

dashboard and runtime UI components for a smooth multiplayer inventory experience.

Edgegap's Unity plugin makes game server hosting and orchestration easy and simple.

PlayFlow Cloud simplifies multiplayer game server hosting. Use the PlayFlow Cloud Unity Plugin to spin up multiplayer game servers for your Unity game.

Crayon: Multiplayer Coloring & Voting Game Template is a Unity Asset Store template designed for multiplayer coloring and voting gameplay.

COZY: Stylized Weather 3 is a customizable weather system offering dynamic time-of-day cycles, seasonal transitions, and diverse atmospheric effects. It has a multiplayer module that supports FishNet: COZY: Link - Multiplayer Module.

Master Audio 2024: AAA Sound is a powerful audio framework for Unity, offering real-time parameter commands, full multiplayer functionality, and advanced occlusion features. It integrates seamlessly with FishNet.

Multiplayer for Malbers Character Controller is a networking integration for Animal Controller, designed to work with FishNet. It extends MalberS to support multiplayer functionality with minimal workflow changes.

Smooth Sync is a network synchronization tool for Unity that enhances object movement across multiplayer sessions. It improves NetworkTransform by offering better configurability, reduced bandwidth usage, and smoother, more accurate syncing for Rigidbodies and Transforms.

Transports

There are a variety of transports that can be implemented in your project for uses such as WebGL, Steam, Unity Relay, and more. For a complete collection see: Transports.

Configuration and Tools

FishNet comes with some useful settings and editor tools to be used with it; these can be accessed through the **(Tools ? Fish-Networking)** toolbar menu.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Beta

The settings here will vary depending on your FishNet version and will enable you to activate and deactivate the latest beta features. This can be helpful if you want to stay with the most tested code for your project or if alternatively if you want to try out features that are coming out soon.

Utility

Quality of Life Attributes

With this option you will be able to disable and enable certain attributes FishNet uses. It's not recommended to use this unless a developer or community helper advises you do so for debugging purposes.

Upgrading From Mirror

When converting a project from Mirror Networking, it can be helpful to use these tools.

Replace Components

This tool helps you replace common Mirror components with what FishNet may use instead.

Remove Defines

This tool removes the many scripting defines that Mirror includes in your Unity project.

Refresh Default Prefabs

This option will cause FishNet's Default Prefab Objects tool to be refreshed by searching your project and keeping track of the Network Object Prefabs within it. This is done automatically for you, but you may want to trigger it if you start seeing the wrong prefabs being spawned in a client build or editor.

Reserialize NetworkObjects

This tool allows you to reserialize prefabs and scenes. Most often you will use this if you get an error message by FishNet telling you to do so. It can also help you if you are running into errors reporting missing sceneids.

{% hint style="info" %} If you run it on the open scenes, be sure to save them after it runs. {% endhint %}

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Configuration

The FishNet configuration (available in the Project Settings) has options for FishNet Pro's **Code Stripping**

feature as well as the Prefab Objects Generator and NetworkBehaviour Template.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Code Stripping

FishNet Pro allows you to strip server-only code from non-server builds^[^1], this includes the code inside ServerRpcs, Server marked methods, and server specific callbacks such as OnStartServer and OnSpawnServer. You can use one of the two available methods for this, either **redirection** or **emptying**.

New NetworkBehaviour Template

This allows you to customize the NetworkBehaviour asset template as well as choose a new location for it. If you want to create a NetworkBehaviour using this template, simply go to **Assets ? FishNet ?**

NetworkBehaviour Script.

Prefab Objects Generator

The Prefab Objects Generator is used by the Default Prefab Objects search your project and keep track of the Network Object Prefabs within it. You can enable and disable it as well as enable and disable its logging.

The **Full Rebuild** option will tell FishNet to search all prefabs instead of just ones that were recently modified.

Spawnable Only instructs the generator to only find NetworkObject prefabs that have IsSpawnable enabled.

The **Default Prefab Objects Path** option allows you to change the default path of the asset in your Unity project.

And finally you can choose which folders to exclude or include when it searches for network prefabs.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Getting Started

The getting started option will show FishNet's **Getting Started** menu, giving you quick access to relevant FishNet links, including the Documentation, Discord, GitHub, and more.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

[^1]: Regular build profiles that aren't headless/dedicated server

Building a Dedicated Server

Instructions for how to build a dedicated FishNet server.

FishNet supports running your project as a dedicated server, meaning Unity runs without rendering graphics and focuses purely on network logic. This is the recommended way to host multiplayer games at scale. You can deploy this build to a server hosting service, your own physical servers, or even give it to your players to allow them to host their own servers.

Tip: Learn more about Unity's Dedicated Server Build Profiles [here](#).

Required Unity Modules

Before building your server, ensure you have the correct Unity modules installed. You will need the **Dedicated Server Build Support** module for your server's operating system (usually Linux, but can be Windows or macOS too).

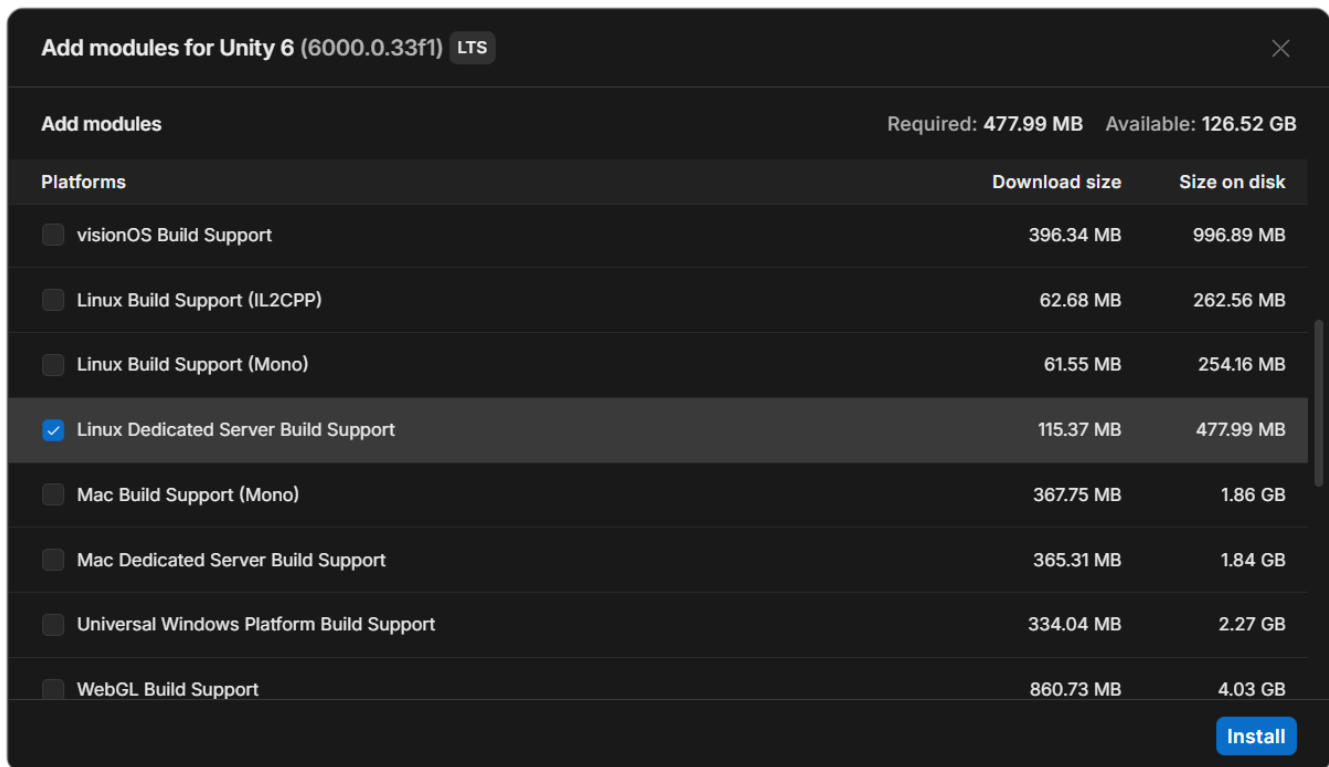
To install through the Unity Hub:

Open the Unity Hub.

Go to the **Installs** tab.

Find your desired Unity version, click **? Manage**, then **Add Modules**.

Tick the relevant build targets and click **Continue**.



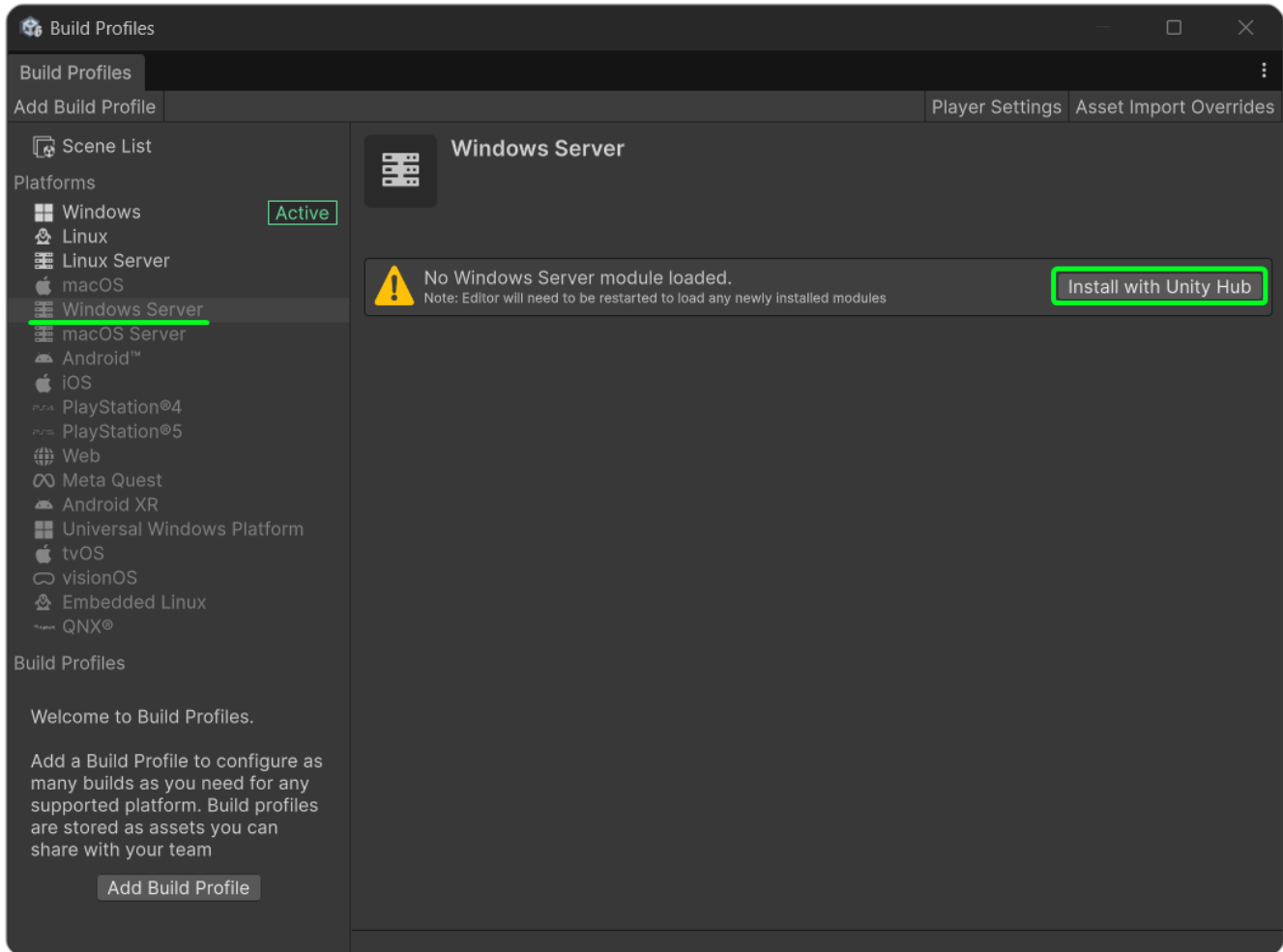
To install through the Unity Editor:

Go to **File ? Build Profiles**.

Select your server's target platform (e.g., **Linux Server** or **Windows Server**).

If the module is not installed, Unity will prompt you to install it.

Click **Install with Unity Hub** as necessary.



Ensuring FishNet's Server Starts

With the default settings, FishNet detects when the game is built as a dedicated server and will start the FishNet server as soon as the NetworkManager is loaded.

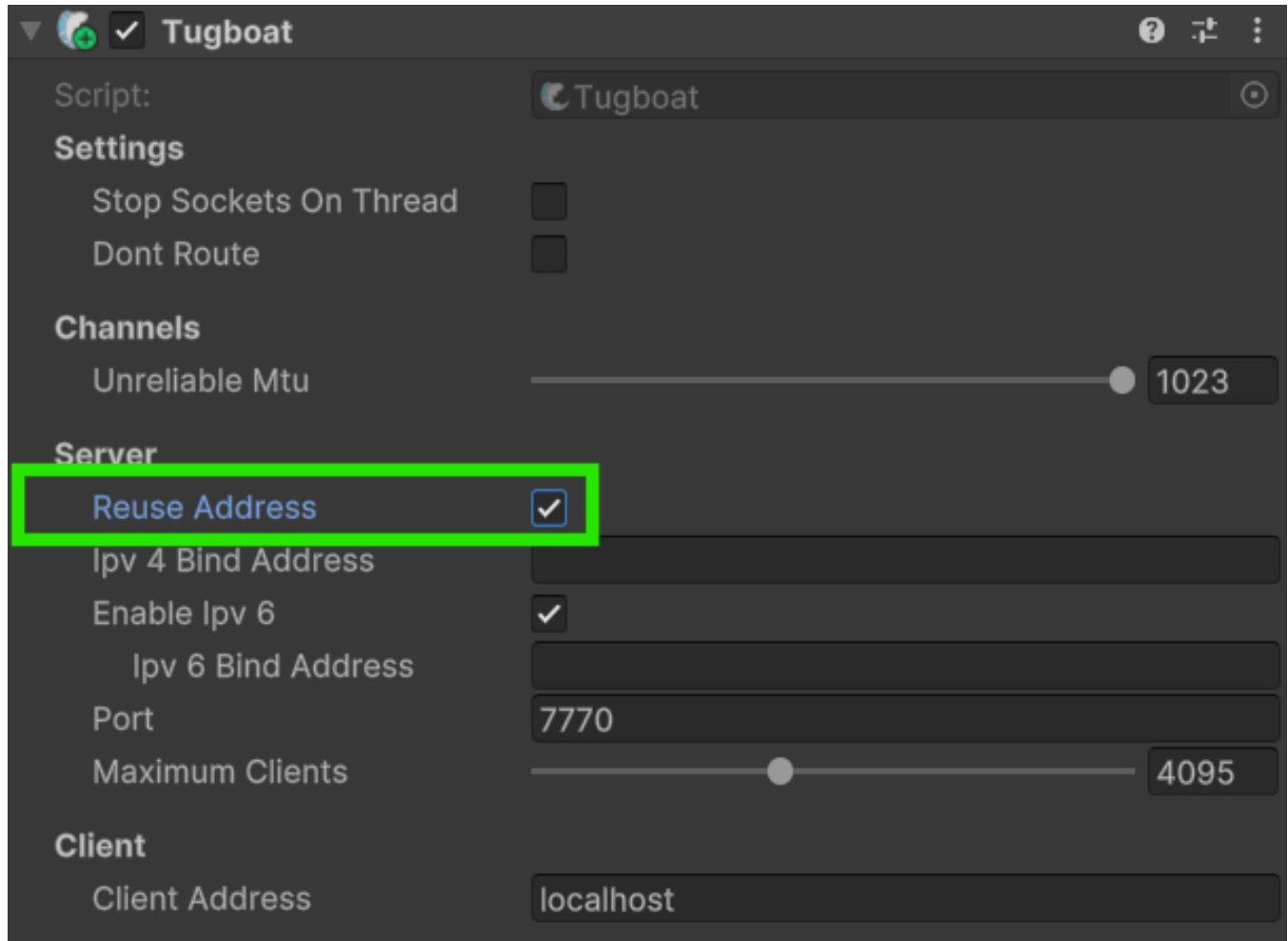
You can change this by adding the **ServerManager** component to your NetworkManager and adjusting the **Start on Headless** option. If you disable this functionality you may want to implement it yourself:

```
using FishNet.Managing;
using UnityEngine;

[RequireComponent(typeof(NetworkManager))]
public class NetworkServerStarter : MonoBehaviour
{
    private void Start()
    {
        #if UNITY_SERVER
            GetComponent<NetworkManager>().ServerManager.StartConnection();
        #endif
    }
}
```

Configuring the Transport

Set up your chosen transport's settings for your dedicated server. The default transport **Tugboat** should in most cases have the **Reuse Address** option enabled.



The screenshot shows the Tugboat configuration window with the following settings:

- Script:** Tugboat
- Settings**
 - Stop Sockets On Thread: ☐
 - Dont Route: ☐
- Channels**
 - Unreliable Mtu: 1023
- Server**
 - Reuse Address: ☒ (highlighted with a green box)
 - Ipv 4 Bind Address: [empty field]
 - Enable Ipv 6: ☒
 - Ipv 6 Bind Address: [empty field]
 - Port: 7770
 - Maximum Clients: 4095
- Client**
 - Client Address: localhost

You may also want to set your **Port** and **Server Bind Address**. Here's how to set these dynamically from command line arguments:

```

using UnityEngine;
using FishNet.Managing;
using System.Diagnostics;

[RequireComponent(typeof(NetworkManager))]
public class NetworkCommandLineArgs : MonoBehaviour
{
    private NetworkManager _networkManager;

    private void Start()
    {
        _networkManager = GetComponent<NetworkManager>();

        ushort? port = GetPortFromCommandLine();
        if (port.HasValue)
        {
            _networkManager.TransportManager.Transport.SetPort(port.Value);
            _networkManager.Log($"Port set to {port} via command line.");
        }
        else
        {
            _networkManager.Log("No valid port found in command line args. Using default."
        );
        }

        StartDedicatedServer();
    }

    [Conditional("UNITY_SERVER")]
    private void StartDedicatedServer()
    {
        _networkManager.ServerManager.StartConnection();
    }

    private ushort? GetPortFromCommandLine()
    {
        string[] args = System.Environment.GetCommandLineArgs();
        for (int i = 0; i < args.Length; i++)
        {
            if ((args[i] == "-port") && i + 1 < args.Length)
            {
                if (ushort.TryParse(args[i + 1], out ushort port))
                    return port;
            }
        }
        return null;
    }
}

```

This code looks for command arguments in this format: `-port <number>`. For example, running `game.exe -port 7777` will set the transport to use port 7777.

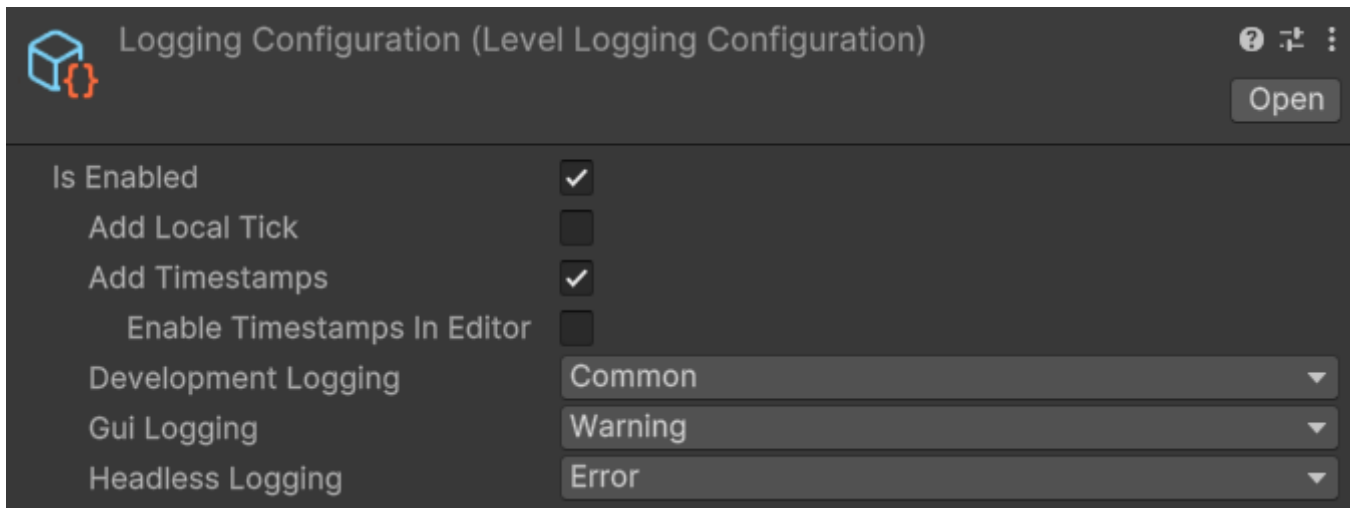
Configuring the Server's Logging

FishNet's Logging Level

The NetworkManager has a **Logging** field which can be used to select a **Level Logging Configuration** asset to customize FishNet logging settings. Create one via **Assets ? Create ? FishNet ? Logging ? Level**

Logging Configuration.

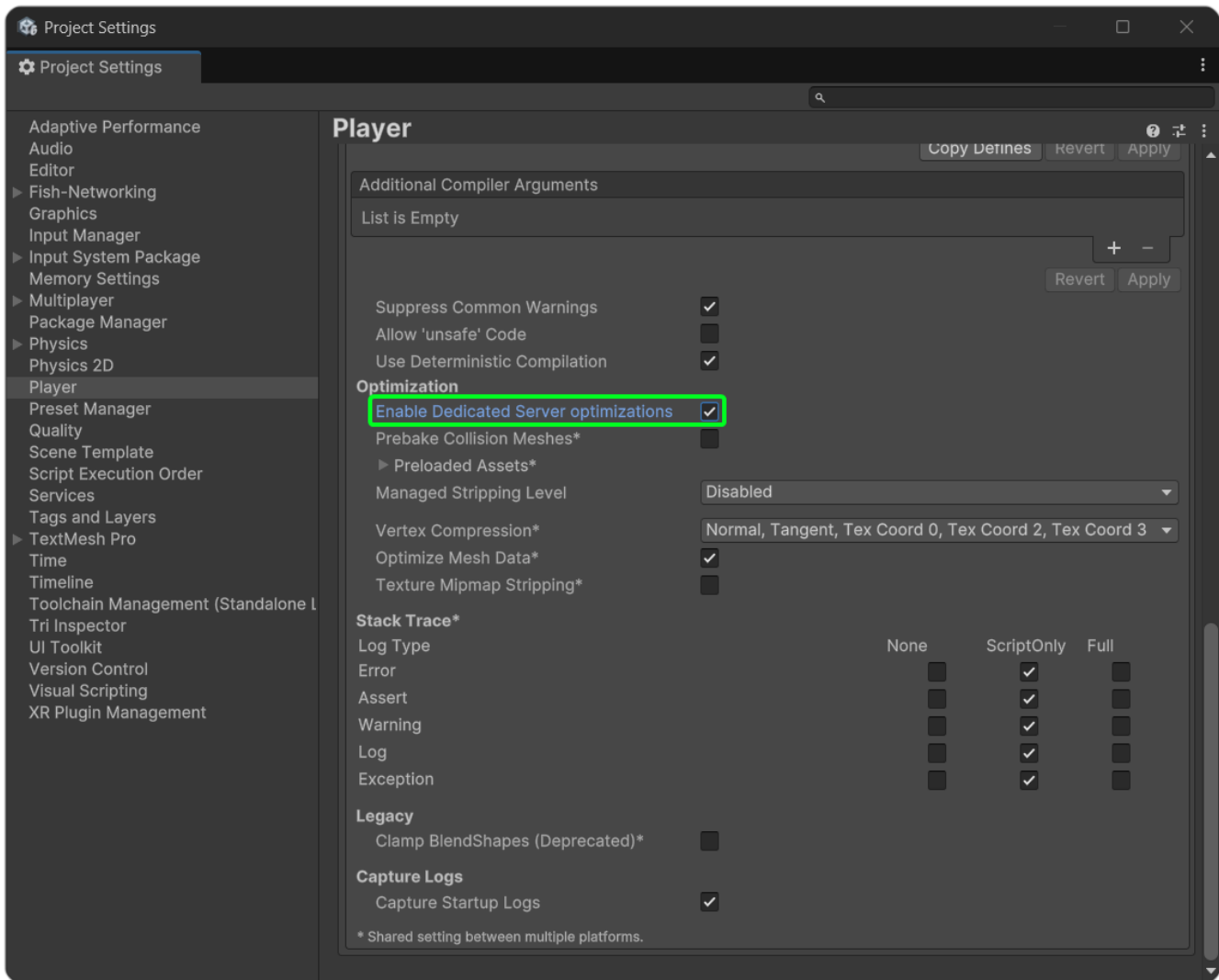
Set the **Headless Logging** option to **"Common"** to see regular debug logs, including connection events such as clients joining and leaving and the server starting.



Shader Errors

When you run your server you may notice errors related to shaders — this is normal as Unity is running without graphics. Enable full shader stripping:

Go to **Edit ? Project Settings ? Player ? Other Settings**, and under **Optimization**, enable **"Dedicated Server Optimizations"**.



Excluding Files from the Build

With server builds you will often want to exclude assets that aren't necessary on the server, such as audio files and textures. Recommended tools:

- [Exclude From Build](#)
- [File Excluder](#)
- [BuildExcluder](#)

Excluding Code from the Server

Use Assembly Definitions and choose to exclude the dedicated server platforms. You can also exclude specific code using the `!UNITY_SERVER` [preprocessor directive](#).

Excluding Code from the Client

Use Assembly Definitions or the `UNITY_SERVER` preprocessor directive — this removes the code from non-dedicated server builds.

If you have **FishNet Pro**, you can make use of its built-in code stripping to strip server-specific code from client-only builds. This includes code inside methods marked with `ServerRpc` or `[Server]` attributes, as

well as server-only callbacks such as `OnStartServer`. Access this option in the **FishNet Configuration** menu through the Unity Toolbar.

Info: Be sure to let the code recompile after changing the FishNet Code Stripping option.

Creating a Server Build

Go to **File ? Build Profiles** and select your target server platform.

Include your game scenes carefully. Ensure the `NetworkManager` is present in the starting scene, a scene loaded via code, or instantiated at runtime.

Click **Build** and choose an output folder.

Once it's done you can upload it to your server or server hosting service and then launch your game servers.

Complete Games

Step-by-step tutorials on how to create simple games in order to show you how the core concepts fit together.

Common Tasks

Simple step-by-step tutorials on how to perform common tasks in FishNet.

Making a Custom Player Spawner

Step-by-step instructions for how to write a custom player spawner for various scenarios.

In many games, you'll often need to spawn a `GameObject` to represent a player. This can happen at various points during gameplay — such as when a player first connects, once a specific number of players have joined, after a particular scene finishes loading, or when the host initiates it with a button press.

FishNet includes a default **PlayerSpawner** component designed primarily as a quick-start tool and reference example. It automatically spawns a player object for each client upon successful connection and authentication. It also has the option to define specific player spawn points, and if multiple spawn points are provided, the component will rotate through them sequentially.

If this functionality is sufficient for your game, then you can use the default one. Otherwise, see the [Overview](#) for custom spawning scenarios:

- **Spawning Players When Connected** — The default `PlayerSpawner` handles this for you.
- **Spawning Players Manually** — Create a script to manually spawn your players when you call a method.
- **Spawning a Selected Player** — Allow your players to choose a character object before spawning it.
- **Spawning Players When Set Number of Players Joined** — Spawn players as soon as a set number of clients have joined.
- **Spawning Players on Scene Load** — Spawn players as soon as they are loaded by FishNet into a specific scene.

Spawning Players Manually

Tutorial for creating a script to manually spawn your players when you call a method.

You may find yourself needing to spawn your players from some custom action. This script exposes a public method you can call to trigger spawning whenever you need to.

Create the Script

Create a new script in your project for your custom player spawner (e.g. `ManualPlayerSpawner`).

Add Namespaces and Inheritance

The script inherits from `NetworkBehaviour` to make use of its exposed properties.

```
using FishNet.Connection;  
using FishNet.Managing;  
using FishNet.Object;  
using UnityEngine;  
  
public class ManualPlayerSpawner : NetworkBehaviour  
{  
}
```

Info: Since this class is a `NetworkBehaviour`, it should not be placed on the `NetworkManager` `GameObject` or any of its children, but rather on a `Networked GameObject` in the desired scene.

Expose the Prefab in the Inspector

Add a `NetworkObject` variable for the player prefab reference.

```
[SerializeField] private NetworkObject _playerPrefab;
```

Create a `SpawnPlayers` Method

The method loops through all connected clients and spawns a prefab for each authenticated client. The `[Server]` attribute prevents accidentally calling this on the client side and enables code stripping in FishNet Pro.

```

[Server]
public void SpawnPlayers()
{
    if (_playerPrefab == null)
    {
        Debug.LogWarning("Player prefab is not assigned and thus cannot be spawned.");
        return;
    }

    foreach (NetworkConnection client in ServerManager.Clients.Values)
    {
        if (!client.IsAuthenticated) continue;

        // If the client isn't observing this scene, make them an observer of it.
        if (!client.Scenes.Contains(gameObject.scene))
            SceneManager.AddConnectionToScene(client, gameObject.scene);

        NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer:
true);
        Spawn(obj, client, gameObject.scene);
    }
}

```

Using `NetworkManager.GetPooledInstantiated` instead of `Instantiate` makes the script compatible with FishNet's Object Pooling system.

Final Script

```

using FishNet.Connection;
using FishNet.Managing;
using FishNet.Object;
using UnityEngine;

public class ManualPlayerSpawner : NetworkBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;

    [Server]
    public void SpawnPlayers()
    {
        if (_playerPrefab == null)
        {
            Debug.LogWarning("Player prefab is not assigned and thus cannot be spawned.");
            return;
        }

        foreach (NetworkConnection client in ServerManager.Clients.Values)
        {
            if (!client.IsAuthenticated) continue;

            if (!client.Scenes.Contains(gameObject.scene))
                SceneManager.AddConnectionToScene(client, gameObject.scene);

            NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
            Spawn(obj, client, gameObject.scene);
        }
    }
}

```

Add this script to a game object in your desired scene and assign the player prefab to it.

On Scene Load

Tutorial for spawning players as soon as they are loaded by FishNet into a specific scene.

To spawn a player when they load into a scene, add a `ScenePlayerSpawner` script to a `NetworkObject` in your desired scene.

Basic Version

Create the Script

Create a new script in your project named `ScenePlayerSpawner.cs`.

Make the Script a `NetworkBehaviour`

Inherit from `NetworkBehaviour` to use its callbacks and properties.

```
using FishNet.Connection;  
using FishNet.Managing;  
using FishNet.Object;  
using UnityEngine;  
  
public class ScenePlayerSpawner : NetworkBehaviour  
{  
}
```

Info: Since this class is a `NetworkBehaviour`, it should not be placed on the `NetworkManager` `GameObject` or any of its children, but rather on a `Networked GameObject` in the desired scene.

Expose the Prefab in the Inspector

```
[SerializeField] private NetworkObject _playerPrefab;
```

Override the `OnSpawnServer` Method

`OnSpawnServer` is called by FishNet on the server as soon as the object is being spawned for a client, which tells us the client is in the scene and ready to receive a player object.

```
public override void OnSpawnServer(NetworkConnection connection)  
{  
    NetworkObject obj = Instantiate(_playerPrefab);  
    Spawn(obj, connection, gameObject.scene);  
}
```

Use FishNet's Object Pooling

Replace `Instantiate` with `NetworkManager.GetPooledInstantiated` to make the script compatible with FishNet's Object Pooling system. This works even without Object Pooling, so it is a direct improvement.

```
NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
```

Basic Final Script

```

using FishNet.Connection;
using FishNet.Managing;
using FishNet.Object;
using UnityEngine;

public class ScenePlayerSpawner : NetworkBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;

    // This method runs on the server when the client is about to spawn this object.
    // Since the player is about to spawn this object, we know he is in this scene.
    public override void OnSpawnServer(NetworkConnection connection)
    {
        NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer:
true);
        Spawn(obj, connection, gameObject.scene);
    }
}

```

Warning: This works great as long as the scene is not one of the starting scenes. If it is, FishNet may warn that you are trying to spawn an object before all starting scenes have loaded. See the section below to handle that case.

Handling Starting Scenes

If this scene is loaded before or as soon as the client connects, the basic script will produce a warning. To fix this, we only spawn the player after the client has loaded all starting scenes.

Listening for the Event

Override `OnStartServer` and `OnStopServer` to subscribe to `SceneManager.OnClientLoadedStartScenes`.

```

public override void OnStartServer()
{
    SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
}

public override void OnStopServer()
{
    if (SceneManager != null)
        SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
}

```

Handling the Event

Spawn the player when the event fires, but only if the client is actually in this scene.

```

private void OnClientLoadedStartScenes(NetworkConnection connection, bool asServer)
{
    // Check if the client is observing this object (i.e. in this scene).
    // Alternatively: connection.Scenes.Contains(gameObject.scene)
    if (asServer && Observers.Contains(connection))
        SpawnPlayer(connection);
}

```

Adjusting the OnSpawnServer Method

Update `OnSpawnServer` to only spawn a player if the client has already loaded the starting scenes.


```
// This method runs on the server when the client is about to spawn this object.  
// BUT he may not have loaded all start scenes yet, so we check that.  
public override void OnSpawnServer(NetworkConnection connection)  
{  
    if (connection.LoadedStartScenes(true))  
        SpawnPlayer(connection);  
}
```

The SpawnPlayer Method

Move the actual spawning logic into its own helper method.

```
private void SpawnPlayer(NetworkConnection connection)  
{  
    NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true  
);  
    Spawn(obj, connection, gameObject.scene);  
}
```

The two-path approach ensures exactly one player is spawned per client: `OnSpawnServer` handles clients who loaded starting scenes before this object spawned, and `OnClientLoadedStartScenes` handles clients who load starting scenes after this object was already spawned.

Final Script

```

using FishNet.Connection;
using FishNet.Managing;
using FishNet.Object;
using UnityEngine;

public class ScenePlayerSpawner : NetworkBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;

    public override void OnStartServer()
    {
        SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
    }

    public override void OnStopServer()
    {
        if (SceneManager != null)
            SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
    }

    private void OnClientLoadedStartScenes(NetworkConnection connection, bool asServer)
    {
        // Check if the client is observing this object (i.e. in this scene).
        // Alternatively: connection.Scenes.Contains(gameObject.scene)
        if (asServer && Observers.Contains(connection))
            SpawnPlayer(connection);
    }

    // This method runs on the server when the client is about to spawn this object.
    // Since the player is about to spawn this object, we know he is in this scene.
    // BUT he may not have loaded all start scenes yet, so we check that.
    public override void OnSpawnServer(NetworkConnection connection)
    {
        if (connection.LoadedStartScenes(true))
            SpawnPlayer(connection);
    }

    private void SpawnPlayer(NetworkConnection connection)
    {
        NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer:
true);
        Spawn(obj, connection, gameObject.scene);
    }
}

```

Overview

Spawning Players Manually

Tutorial for creating a script to manually spawn your players when you call a method.

You may find yourself needing to spawn your players from some custom action. We can write a script that has a public method you can easily call to trigger it whenever you need to.

Step 1 — Create the Script

Create a new script in your project for your custom player spawner. We've called ours `ManualPlayerSpawner`.

Step 2 — Add the Namespaces and Inheritance

Our script will make use of a few different FishNet namespaces and will inherit from `NetworkBehaviour` to make use of its exposed properties.

```
using FishNet.Connection;  
using FishNet.Managing;  
using FishNet.Object;  
using UnityEngine;  
  
public class ManualPlayerSpawner : NetworkBehaviour  
{  
}
```

Info: Since this class is a `NetworkBehaviour`, it should not be placed on the `NetworkManager` `GameObject` or any of its children, but rather on a `Networked GameObject` in the desired scene.

Step 3 — Expose the Prefab in the Inspector

Add a `NetworkObject` variable for holding a reference to our player prefab.

```
[SerializeField] private NetworkObject _playerPrefab;
```

Step 4 — Create a SpawnPlayers Method

Now we can create the method that we will call when we want to spawn in the players.

The method will first do a simple null check to make sure we don't get any null reference errors if we forget to assign the player prefab. After that it will loop through all the connected clients and, as long as they are authenticated, it will instantiate and spawn the prefab for them.

```

public void SpawnPlayers()
{
    if (_playerPrefab == null)
    {
        Debug.LogWarning("Player prefab is not assigned and thus cannot be spawned.");
        return;
    }

    foreach (NetworkConnection client in ServerManager.Clients.Values)
    {
        // Since the ServerManager.Clients collection contains all clients (even non-authenticated ones),
        // we need to check if they are authenticated first before spawning a player object for them.
        if (!client.IsAuthenticated) continue;

        NetworkObject obj = Instantiate(_playerPrefab);
        Spawn(obj, client, gameObject.scene);
    }
}

```

Step 5 — Prevent Accidentally Running on Client

Let's prevent ourselves from accidentally running this method on the client side — spawning should only happen by the server.

Add the `[Server]` attribute to the method:

```

[Server]
public void SpawnPlayers()

```

This will also allow FishNet to strip out this code from client builds when using FishNet Pro's code stripping feature.

Step 6 — Enable Use of Object Pooling

We can also make the script work with FishNet's Object Pooling system by changing the `Instantiate` method call to one provided by FishNet. This code will work even if we don't use Object Pooling, so it's a direct improvement:

```

NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);

```

Step 7 — Handle Starting Scenes

Now the script works well for situations where clients are loaded into the scene by the FishNet `SceneManager`, but what if you start the game in this scene and don't use the FishNet `SceneManager` to load it? In that case we can tell FishNet that the client has loaded this scene and should observe it:

```

// If the client isn't observing this scene, make him an observer of it.
if (!client.Scenes.Contains(gameObject.scene))
    SceneManager.AddConnectionToScene(client, gameObject.scene);

```

Alternatively, if you only want clients who are already loaded in this scene to have a player spawned:

```

// If the client isn't in this scene, ignore him.
if (!client.Scenes.Contains(gameObject.scene)) continue;

```

Step 8 — The Final Script

Here is the complete `ManualPlayerSpawner` script. Add it to a game object in your desired scene and

assign the player prefab to it.

```
using FishNet.Connection;
using FishNet.Managing;
using FishNet.Object;
using UnityEngine;

public class ManualPlayerSpawner : NetworkBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;

    // The Server attribute here prevents this method from being called except on the server.
    [Server]
    public void SpawnPlayers()
    {
        if (_playerPrefab == null)
        {
            Debug.LogWarning("Player prefab is not assigned and thus cannot be spawned.");
            return;
        }

        foreach (NetworkConnection client in ServerManager.Clients.Values)
        {
            // Since the ServerManager.Clients collection contains all clients (even non-authenticated ones),
            // we need to check if they are authenticated first before spawning a player object for them.
            if (!client.IsAuthenticated) continue;

            // If the client isn't observing this scene, make him an observer of it.
            if (!client.Scenes.Contains(gameObject.scene))
                SceneManager.AddConnectionToScene(client, gameObject.scene);

            NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
            Spawn(obj, client, gameObject.scene);
        }
    }
}
```

Set Player Number

Tutorial for spawning players as soon as a set number of clients have joined your game.

Create the Script

Create a new script in your project for your custom player spawner (e.g. CountBasedPlayerSpawner).

Add the Needed Fields and Namespaces

Our class will take a `NetworkObject` prefab which will be spawned for our players and an integer for how many players need to connect before we spawn the players.

We're also storing a reference to the `NetworkManager` this script will use. This is necessary as we won't be inheriting from `NetworkBehaviour` for this player spawner script — as a result, we will be able to place this component directly on the `NetworkManager`.

```
using FishNet;
using FishNet.Connection;
using FishNet.Managing;
using FishNet.Managing.Server;
using FishNet.Object;
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

public class CountBasedPlayerSpawner : MonoBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;
    [SerializeField] private int _requiredPlayerCount;
    private NetworkManager _networkManager;
}
```

Get a Reference to the NetworkManager

Get a reference to the `NetworkManager`, first trying to get it from this object or one of its parents, and falling back to `InstanceFinder` if that fails.

```
private void Awake()
{
    _networkManager = GetComponentInParent<NetworkManager>();
    if (_networkManager == null)
        _networkManager = InstanceFinder.NetworkManager;

    if (_networkManager == null)
    {
        Debug.LogWarning($"CountBasedPlayerSpawner cannot work as a NetworkManager couldn't be found.");
        return;
    }
}
```

Listen for Clients Joining

With access to the `NetworkManager`, we can monitor when clients join the game using the `SceneManager.OnClientLoadedStartScenes` event, which is triggered once a client finishes loading

the initial scenes after connecting.

Subscribe to the event at the end of the Awake method:

```
_networkManager.SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
```

And unsubscribe when this object is destroyed:

```
private void OnDestroy()
{
    if (_networkManager != null)
        _networkManager.SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
}
```

Implement the OnClientLoadedStartScenes Method

This event runs for both the server and client, so we exit early if not running on the server. We then build a list of all authenticated connected clients. If the count hasn't reached `_requiredPlayerCount` yet, we return early. Otherwise, we spawn a player object for each authenticated client.

```
private void OnClientLoadedStartScenes(NetworkConnection _, bool asServer)
{
    if (!asServer) return;

    List<NetworkConnection> authenticatedClients = _networkManager.ServerManager.Clients.Values
        .Where(conn => conn.IsAuthenticated).ToList();

    if (authenticatedClients.Count < _requiredPlayerCount) return;

    foreach (NetworkConnection client in authenticatedClients)
    {
        NetworkObject obj = Instantiate(_playerPrefab);
        _networkManager.ServerManager.Spawn(obj, client);
    }
}
```

Enable Use of Object Pooling

Use `NetworkManager.GetPooledInstantiated` instead of `Instantiate` to make the script compatible with FishNet's Object Pooling system. This works even if you don't use Object Pooling, so it is a direct improvement.

```
NetworkObject obj = _networkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
```

Handle Starting Scenes

If the game begins in this scene without using the FishNet SceneManager to load it, we need to manually inform FishNet that the client has entered the scene. Insert the following code right after the Spawn call:

```
// If the client isn't observing this scene, make them an observer of it.
if (!client.Scenes.Contains(gameObject.scene))
    _networkManager.SceneManager.AddOwnerToDefaultScene(obj);
```

Final Script

You can add this script to your `NetworkManager` object or any object that exists before clients start connecting. Assign the `_playerPrefab` and `_requiredPlayerCount` fields in the Inspector.

```

using FishNet;
using FishNet.Connection;
using FishNet.Managing;
using FishNet.Managing.Server;
using FishNet.Object;
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

public class CountBasedPlayerSpawner : MonoBehaviour
{
    [SerializeField] private NetworkObject _playerPrefab;
    [SerializeField] private int _requiredPlayerCount;
    private NetworkManager _networkManager;

    private void Awake()
    {
        _networkManager = GetComponentInParent<NetworkManager>();
        if (_networkManager == null)
            _networkManager = InstanceFinder.NetworkManager;

        if (_networkManager == null)
        {
            Debug.LogWarning($"CountBasedPlayerSpawner cannot work as a NetworkManager couldn't be found.");
            return;
        }

        _networkManager.SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
    }

    private void OnDestroy()
    {
        if (_networkManager != null)
            _networkManager.SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
    }

    private void OnClientLoadedStartScenes(NetworkConnection _, bool asServer)
    {
        if (!asServer) return;

        List<NetworkConnection> authenticatedClients = _networkManager.ServerManager.Clients.Values
            .Where(conn => conn.IsAuthenticated).ToList();

        if (authenticatedClients.Count < _requiredPlayerCount) return;

        foreach (NetworkConnection client in authenticatedClients)
        {
            NetworkObject obj = _networkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
            _networkManager.ServerManager.Spawn(obj, client);

            // If the client isn't observing this scene, make them an observer of it.
            if (!client.Scenes.Contains(gameObject.scene))

```



```
        _networkManager.SceneManager.AddOwnerToDefaultScene(obj);  
    }  
}  
}
```

Spawning Selected Player

Tutorial for allowing your players to choose a character object before spawning it.

This guide lets clients call a `SpawnPlayer` `ServerRpc` to have the server spawn their chosen character. A list of permitted prefabs prevents clients from spawning arbitrary network objects.

Create the Script

Create a new script called `SelectablePlayerSpawner` inheriting from `NetworkBehaviour`.

```
using FishNet.Connection;  
using FishNet.Managing;  
using FishNet.Object;  
using System.Linq;  
using UnityEngine;  
  
public class SelectablePlayerSpawner : NetworkBehaviour  
{  
}
```

Info: Since this class is a `NetworkBehaviour`, it should not be placed on the `NetworkManager` `GameObject` or any of its children.

Create an Array of Permitted Prefabs

```
[SerializeField] private NetworkObject[] _playerPrefabs;
```

Create a SpawnPlayer Method

The method is a `ServerRpc` with `RequireOwnership = false` so any client can call it. The `NetworkConnection` sender parameter is filled in automatically by FishNet.

```
[ServerRpc(RequireOwnership = false)]  
public void SpawnPlayer(NetworkObject _playerPrefab, NetworkConnection sender = null)  
{  
    if (sender.FirstObject != null)  
    {  
        Debug.LogWarning($"Client {sender.ClientId} already has a player object; not spawning another.");  
        return;  
    }  
  
    if (!_playerPrefabs.Contains(_playerPrefab))  
    {  
        Debug.LogWarning("Invalid player prefab selected, cannot spawn.");  
        sender.Kick(FishNet.Managing.Server.KickReason.ExploitAttempt);  
        return;  
    }  
  
    NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);  
    Spawn(obj, sender, gameObject.scene);  
}
```

Handle Starting Scenes

Subscribe to `SceneManager.OnClientLoadedStartScenes` so clients are added to the scene's observer list automatically.

```
public override void OnStartServer()
{
    SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
}

public override void OnStopServer()
{
    if (SceneManager != null)
        SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
}

private void OnClientLoadedStartScenes(NetworkConnection conn, bool asServer)
{
    if (asServer && !conn.Scenes.Contains(gameObject.scene))
        SceneManager.AddConnectionToScene(conn, gameObject.scene);
}
```

Final Script

```

using FishNet.Connection;
using FishNet.Managing;
using FishNet.Object;
using System.Linq;
using UnityEngine;

public class SelectablePlayerSpawner : NetworkBehaviour
{
    [SerializeField] private NetworkObject[] _playerPrefabs;

    public override void OnStartServer()
    {
        SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
    }

    public override void OnStopServer()
    {
        if (SceneManager != null)
            SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
    }

    private void OnClientLoadedStartScenes(NetworkConnection conn, bool asServer)
    {
        if (asServer && !conn.Scenes.Contains(gameObject.scene))
            SceneManager.AddConnectionToScene(conn, gameObject.scene);
    }

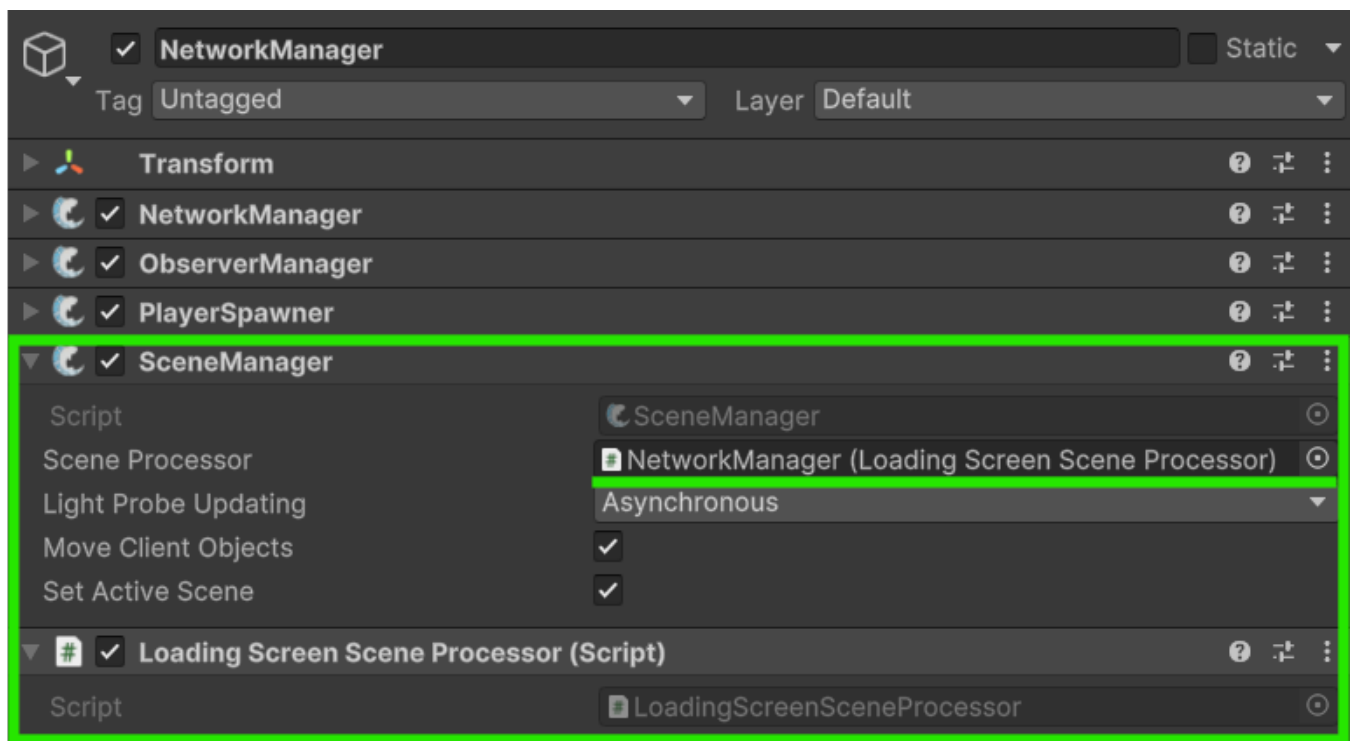
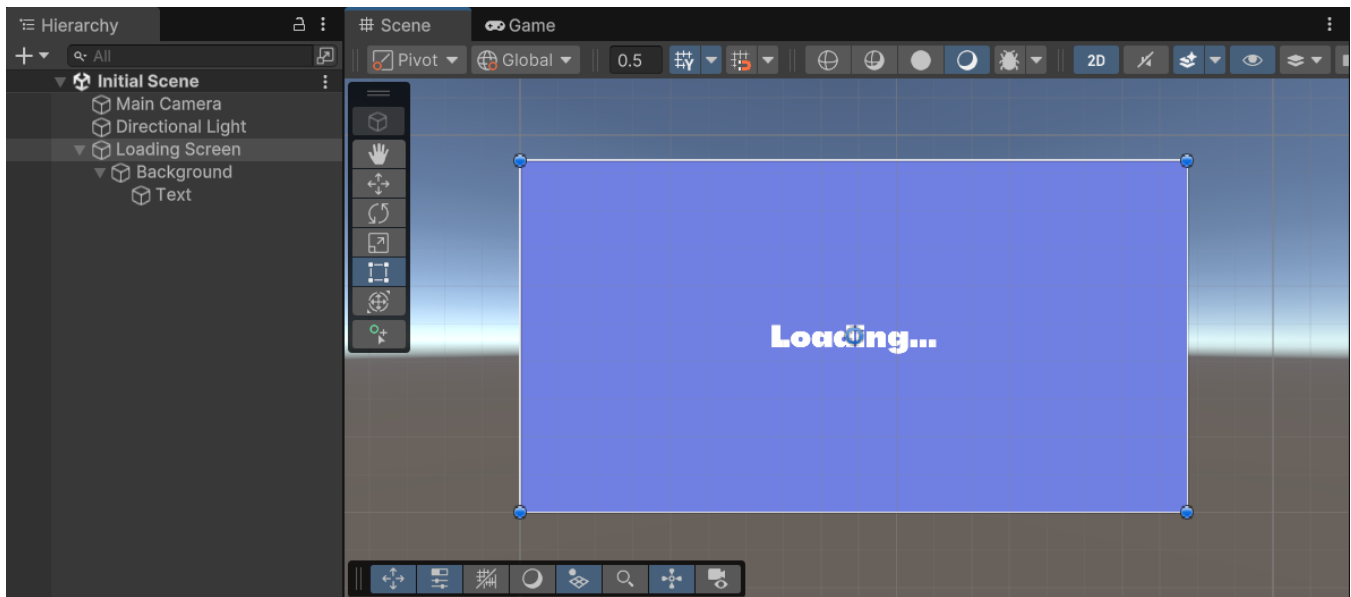
    [ServerRpc(RequireOwnership = false)]
    public void SpawnPlayer(NetworkObject _playerPrefab, NetworkConnection sender = null)
    {
        if (sender.FirstObject != null)
        {
            Debug.LogWarning($"Client {sender.ClientId} already has a player object; not spawning another.");
            return;
        }

        if (!_playerPrefabs.Contains(_playerPrefab))
        {
            Debug.LogWarning("Invalid player prefab selected, cannot spawn.");
            sender.Kick(FishNet.Managing.Server.KickReason.ExploitAttempt);
            return;
        }

        NetworkObject obj = NetworkManager.GetPooledInstantiated(_playerPrefab, asServer: true);
        Spawn(obj, sender, gameObject.scene);
    }
}

```

Making a Loading Screen



[Image: untitled]

A tutorial for creating a loading screen when using FishNet's Scene Management instead of Unity's default one.

You may be used to making loading screens when using Unity's regular scene loading, but how can you do it for networked scenes? A good and simple solution is to use FishNet's **Scene Processor** to add (or override) functionality during when FishNet loads scenes.

Step 1 — Creating a Loading Screen UI

Start by adding a canvas with an image and some text to cover the screen and let the user know that the game is currently loading.

We will disable and enable this object when needed, and to prevent it getting destroyed, we will mark it as `DontDestroyOnLoad`.

Step 2 — Adding a Simple Loading Screen Script

Write the following script and place it on our **Loading Screen** object.

This script acts as a singleton — we will only have one loading screen in our game. We add a static reference to it and register it with the NetworkManager in `Start`, or destroy it if an instance is already registered. We will hide the loading screen after this, so it doesn't block our game before we want it to.

```
using FishNet;
using UnityEngine;

public class LoadingScreen : MonoBehaviour
{
    private void Start()
    {
        if (InstanceFinder.NetworkManager.HasInstance<LoadingScreen>())
        {
            Destroy(gameObject);
            return;
        }

        InstanceFinder.NetworkManager.RegisterInstance(this);
        DontDestroyOnLoad(gameObject);
        HideLoadingScreen();
    }

    public static void ShowLoadingScreen()
    {
        if (InstanceFinder.NetworkManager.TryGetInstance(out LoadingScreen loadingScreen))
            loadingScreen.gameObject.SetActive(true);
    }

    public static void HideLoadingScreen()
    {
        if (InstanceFinder.NetworkManager.TryGetInstance(out LoadingScreen loadingScreen))
            loadingScreen.gameObject.SetActive(false);
    }
}
```

We also give it public static methods to show and hide the loading screen. We can call these whenever we want to, and we will call them from our scene processor.

Info: This example script uses FishNet's instance registering to handle the singleton reference, but you can use your own or any other instead if you prefer.

Step 3 — Creating a Custom Scene Processor

Now create a script that will inherit from FishNet's `DefaultSceneProcessor` and add our extra functionality when loading a scene. Simply override the `LoadStart` and `LoadEnd` methods, call the base method to retain the default functionality, and then tell our loading screen to be shown and hidden.

```
using FishNet.Managing.Scened;  
  
public class LoadingScreenSceneProcessor : DefaultSceneProcessor  
{  
    public override void LoadStart(LoadQueueData queueData)  
    {  
        base.LoadStart(queueData);  
        LoadingScreen.ShowLoadingScreen();  
    }  
  
    public override void LoadEnd(LoadQueueData queueData)  
    {  
        base.LoadEnd(queueData);  
        LoadingScreen.HideLoadingScreen();  
    }  
}
```

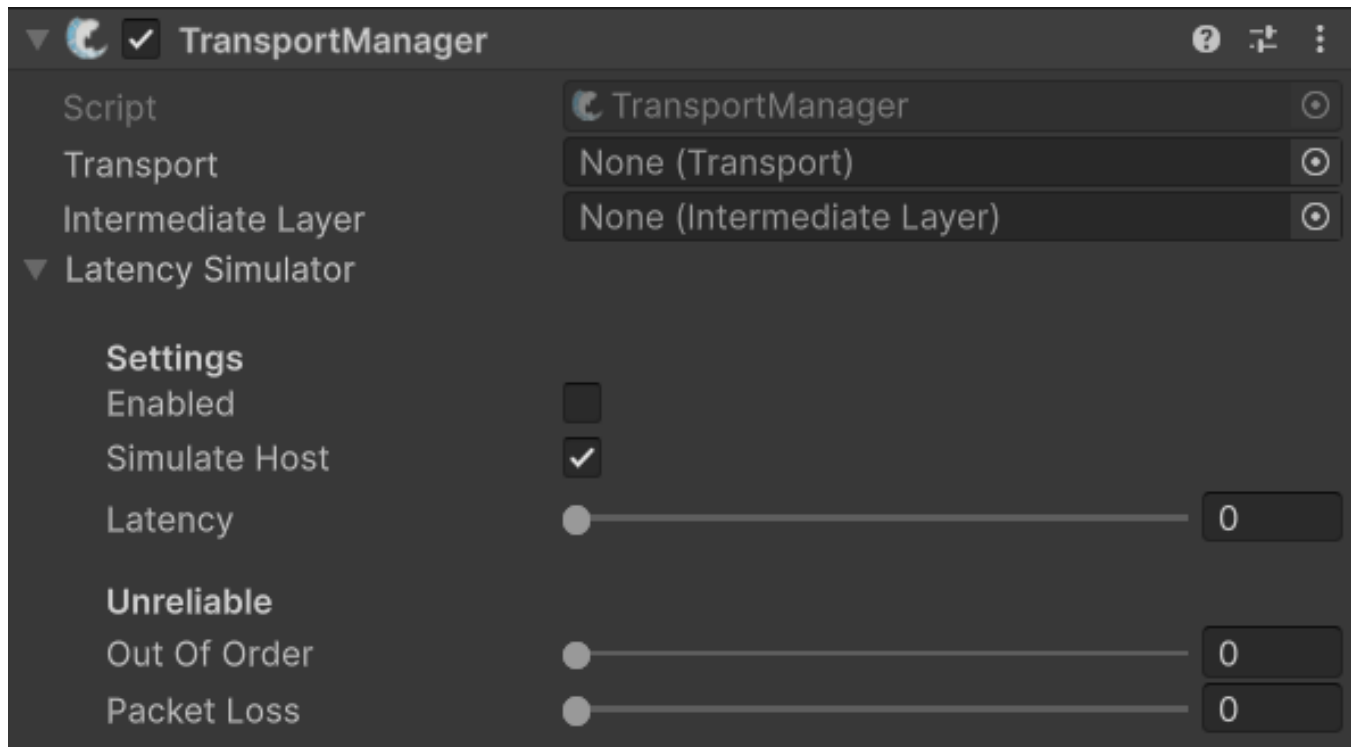
Step 4 — Add the Scene Processor to the NetworkManager

Add the `LoadingScreenSceneProcessor` script to your **NetworkManager** game object. Also add the **SceneManager** component if it isn't there already. The `SceneManager` has a **Scene Processor** field — drag your `LoadingScreenSceneProcessor` component into it.

Step 5 — Success!

With that all set up, your loading screen should work whenever `FishNet` loads new scenes.

Simulating Bad Network Connections



This page covers how to simulate poor network conditions when testing with FishNet.

When developing a multiplayer game, it's not enough to test only under perfect network conditions. Real-world players will often experience latency, jitter, out of order packets, and packet loss, and your game needs to tolerate these. If you only test on your local machine with a flawless connection, you risk shipping a game that breaks down as soon as it's played online.

What Conditions Affect Networked Games?

- **Latency (Ping)** — delays between sending and receiving packets.
- **Jitter** — variations in latency over time (unstable ping).
- **Out-of-order Packets** — packets may arrive out of the order from when they were sent.
- **Packet Loss** — some messages never arrive.

Testing with these factors early helps you tune interpolation, reconciliation, lag-compensation, and other systems. It also reveals edge cases that only show up at higher latencies (e.g., 500ms+).

What Values Should I Test With?

There's no one-size-fits-all answer, but some general guidelines:

- **Desktop broadband:** 100–150ms latency, 1–3% packet loss
- **Mobile networks:** 100–300ms latency, 2–5% packet loss, plus jitter
- **Stress testing:** also check 500ms, 1000ms+, and extreme packet loss to ensure your game fails gracefully

Info: Remember: don't test with just "added delay." Real networks are affected by multiple factors

simultaneously.

FishNet's Built-in Simulator

FishNet includes a basic latency simulator that can add artificial latency, packet loss, and tamper with packet order. These can also affect the client host player.

To use this, add the **TransportManager** component to your network manager. You will now be able to enable the **Latency Simulator** and tweak its settings.

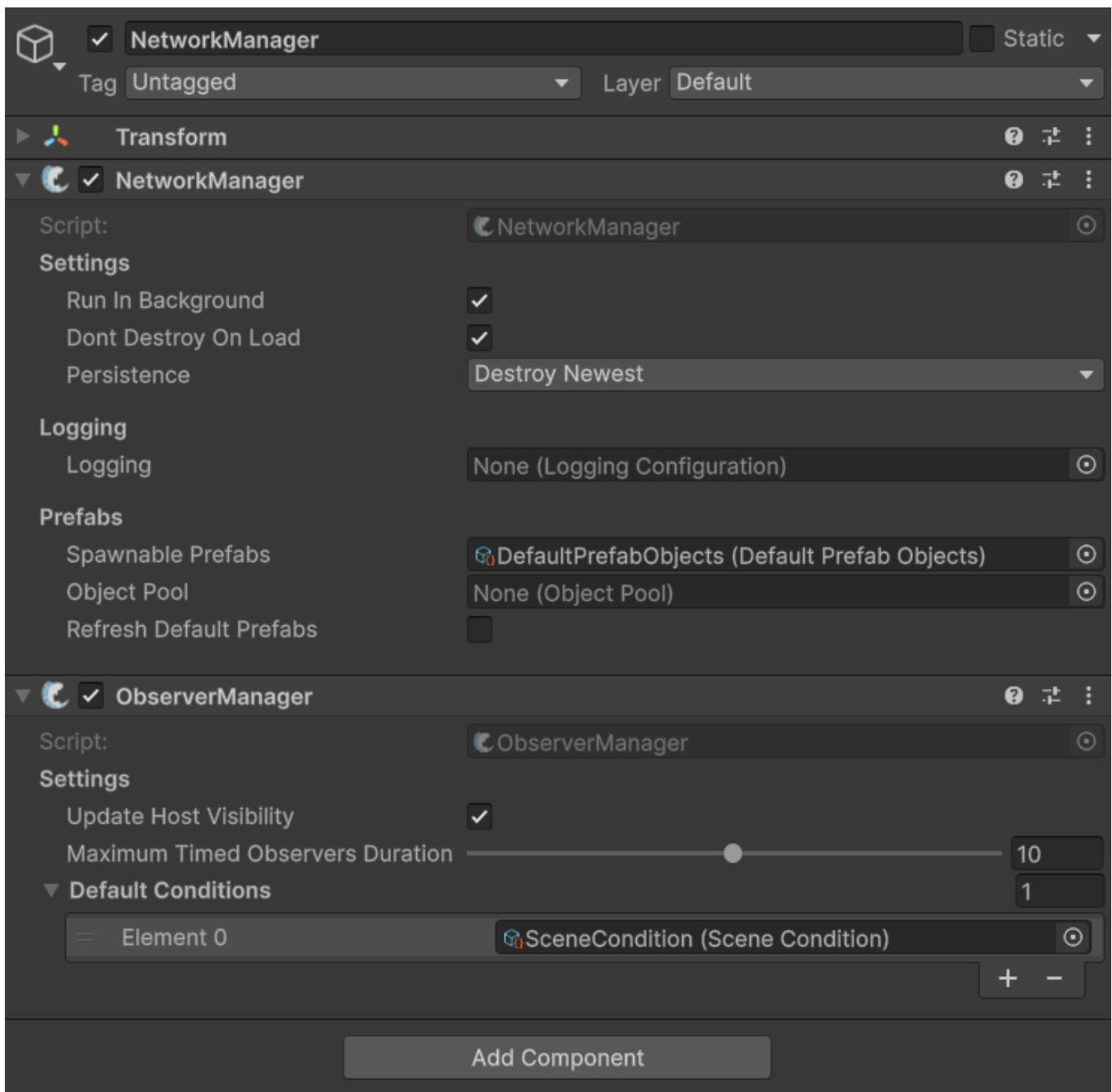
External Tools

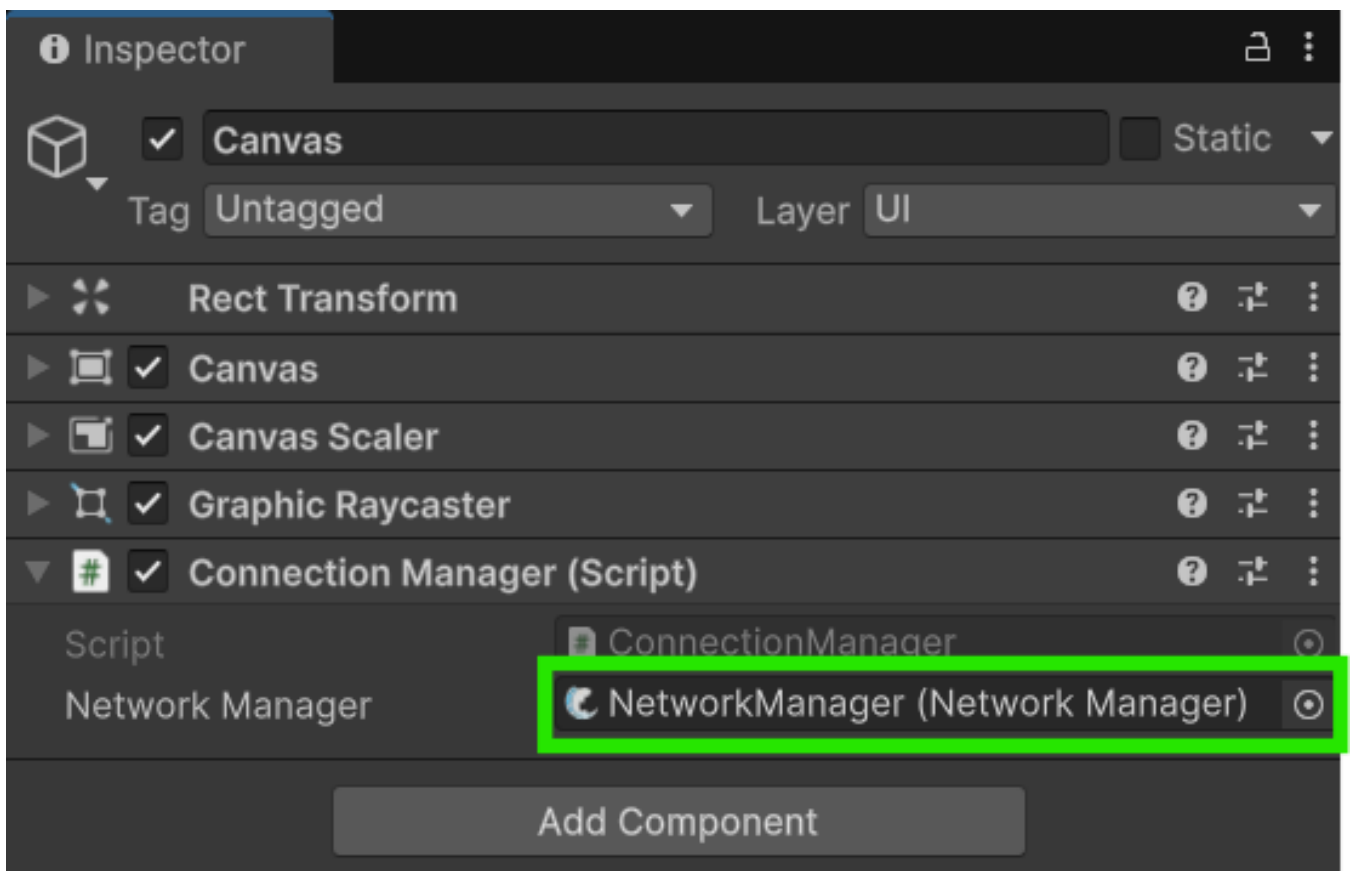
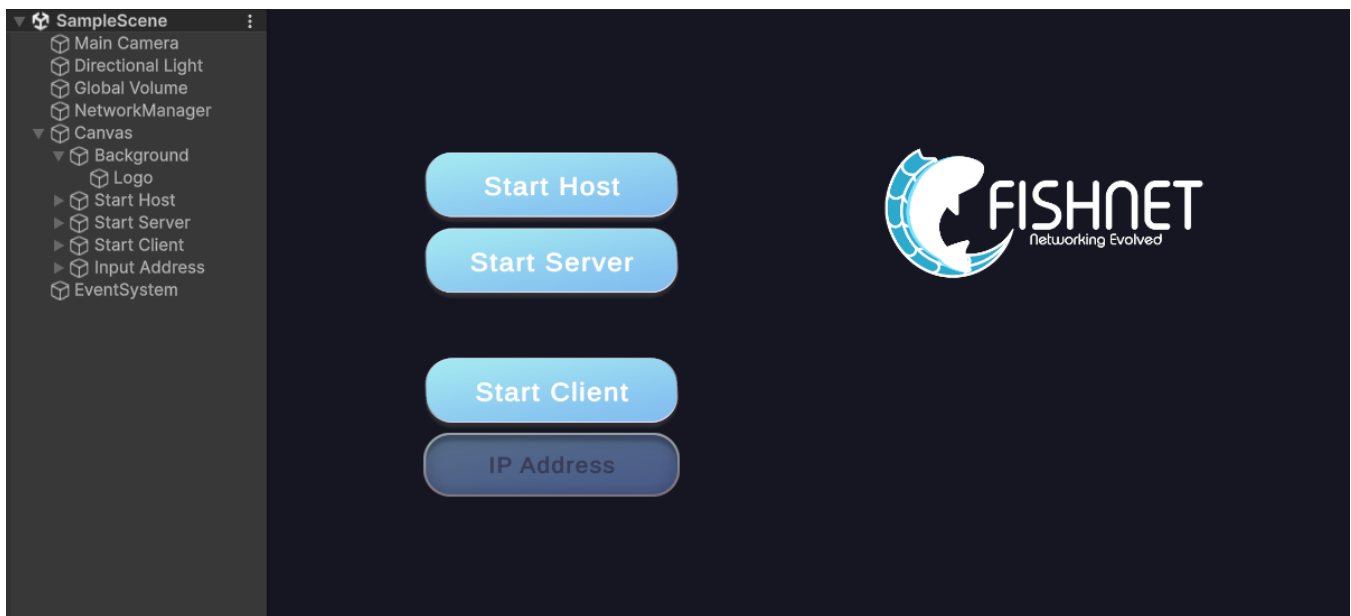
For more options it's recommended to use an external tool, such as one of the ones listed below:

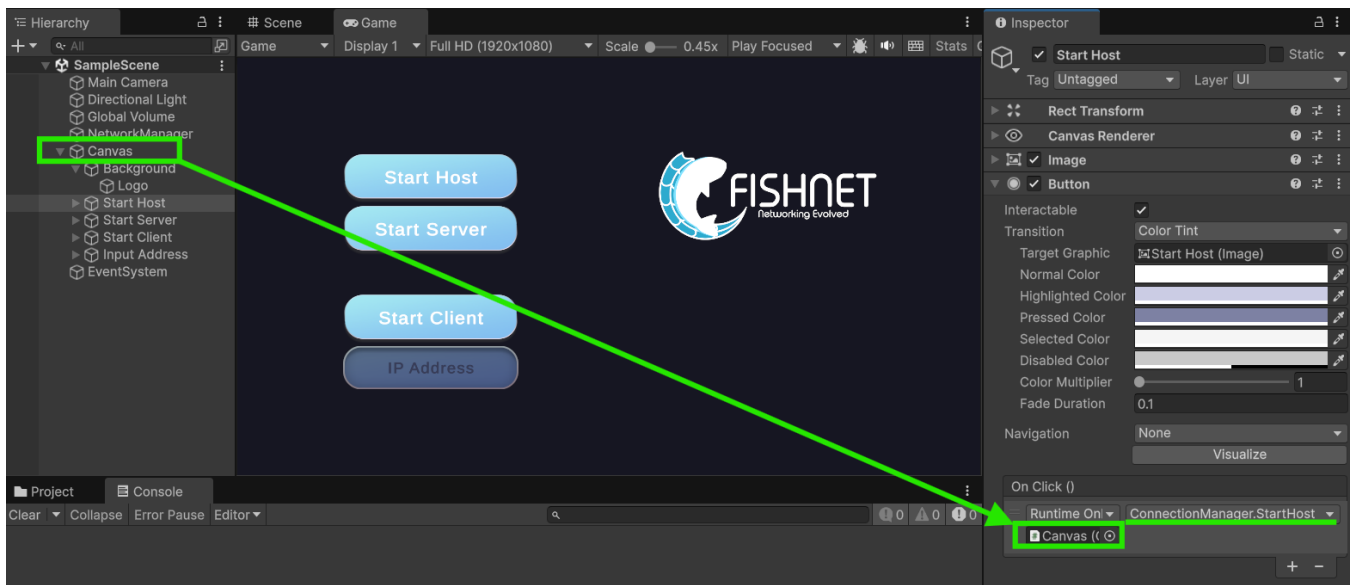
- **Windows:** [Clumsy](#)
- **macOS/iOS:** Network Link Conditioner (in Apple's Additional Tools for Xcode)
- **macOS (advanced):** [dummynet](#), `dnctl` and `pfctl` (built into macOS)
- **Linux:** [netem](#)

These tools let you add latency, packet loss, bandwidth limits, and more. They operate at the system level, so every application (including your game) experiences the degraded connection.

Starting FishNet's Connections







[Image: untitled]

A guide on starting the FishNet server and client connections yourself.

This guide will show you how you can start the FishNet server and client yourself through code instead of relying only on the example NetworkHudCanvas or default autostart option.

Step 1 — Install FishNet

Before we begin, don't forget to install FishNet if you haven't already. You can follow this guide if needed: [Installing Fish-Networking](#).

Step 2 — Create a NetworkManager

Create a new game object in your scene and give it a nice name such as NetworkManager.

Add the **NetworkManager** component to it.

Finally, add an **ObserverManager** component and in the **Default Conditions** field, add a new element and select the **SceneCondition**.

Step 3 — Setup Basic UI

To get started, let's add a basic user interface which we can use to start the FishNet server, client, and allow

the user to enter an IP Address to connect to.

Add three buttons to your game, label them **Start Host**, **Start Server**, and **Start Client**. Now add an input field for the address the client will attempt to connect to.

Step 4 — Create a ConnectionManager Script

Create the following script and add it to your **Canvas** game object.

```
using FishNet.Managing;
using UnityEngine;

public class ConnectionManager : MonoBehaviour
{
    [SerializeField] private NetworkManager _networkManager;

    public void StartHost()
    {
        StartServer();
        StartClient();
    }

    public void StartServer()
    {
        _networkManager.ServerManager.StartConnection();
    }

    public void StartClient()
    {
        _networkManager.ClientManager.StartConnection();
    }

    public void SetIPAddress(string text)
    {
        _networkManager.TransportManager.Transport.SetClientAddress(text);
    }
}
```

One thing to observe is that this script does not inherit from `NetworkBehaviour`. This is because this script will be active before the network is started and after it has stopped.

Step 5 — Assign the NetworkManager

Our `ConnectionManager` needs a reference to the `NetworkManager` component — assign that in the editor now.

Step 6 — Hook-Up the UI to the Code

Select your UI components and attach their **OnClick** events to the relevant methods in our `ConnectionManager`.

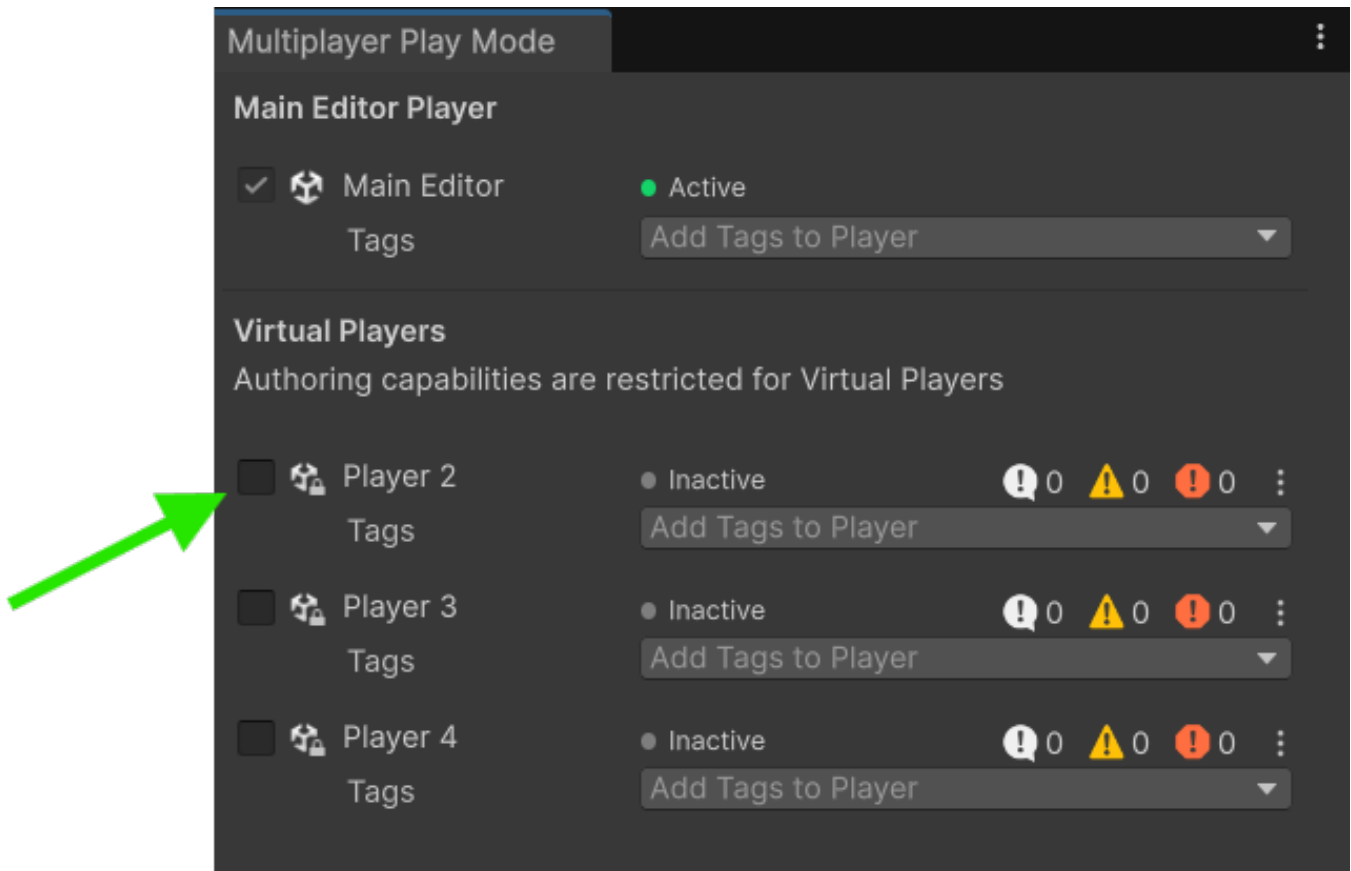
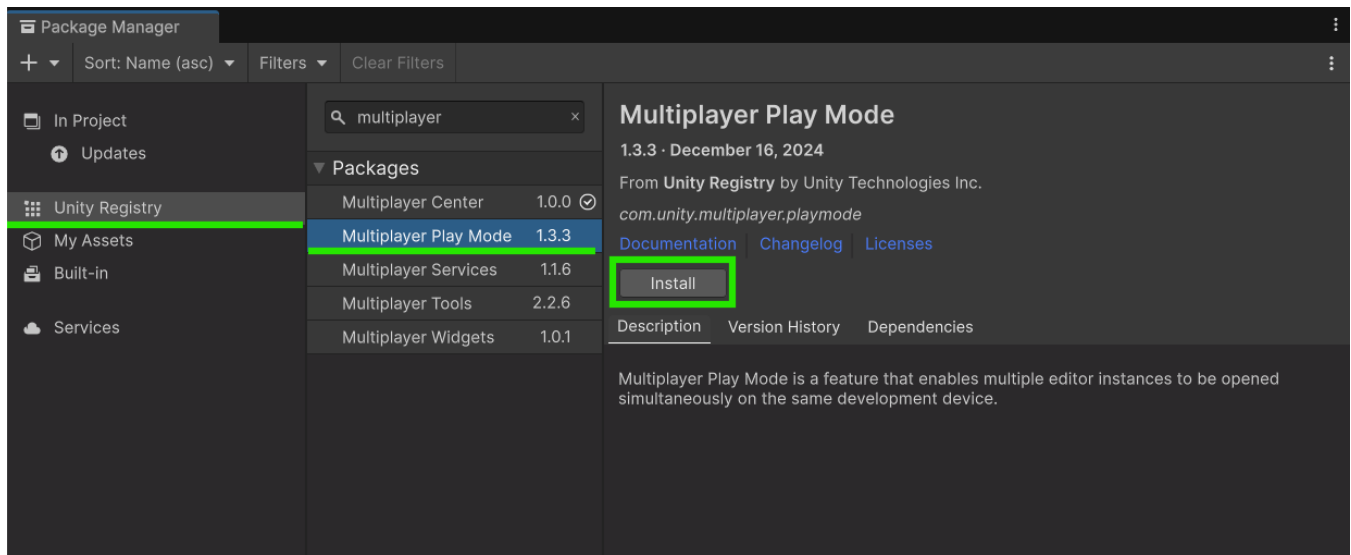
For the **IP Address Input Field**, hook up the **On Value Changed** event to the `ConnectionManager.SetIPAddress` method we created.

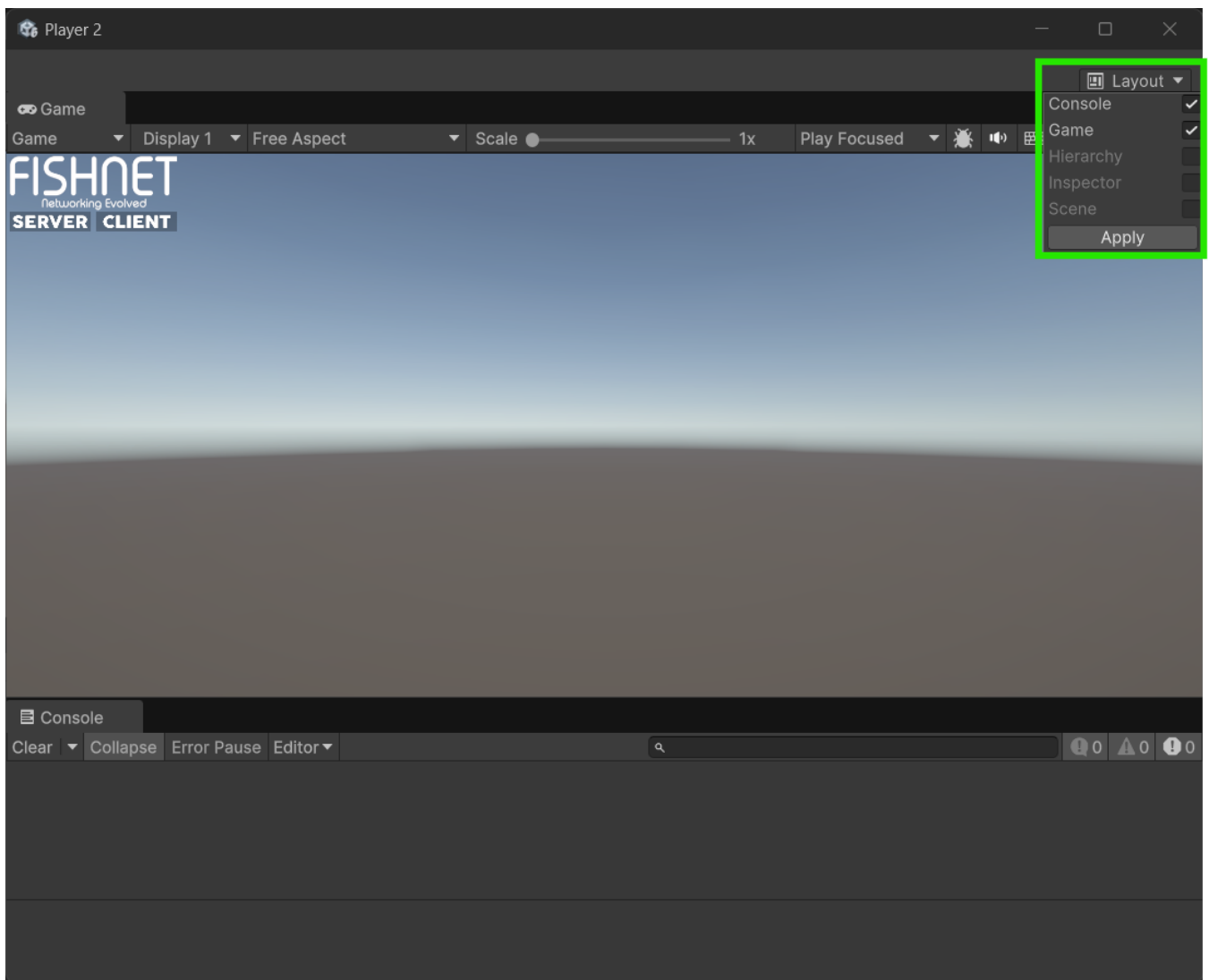
Step 7 — Test It Out!

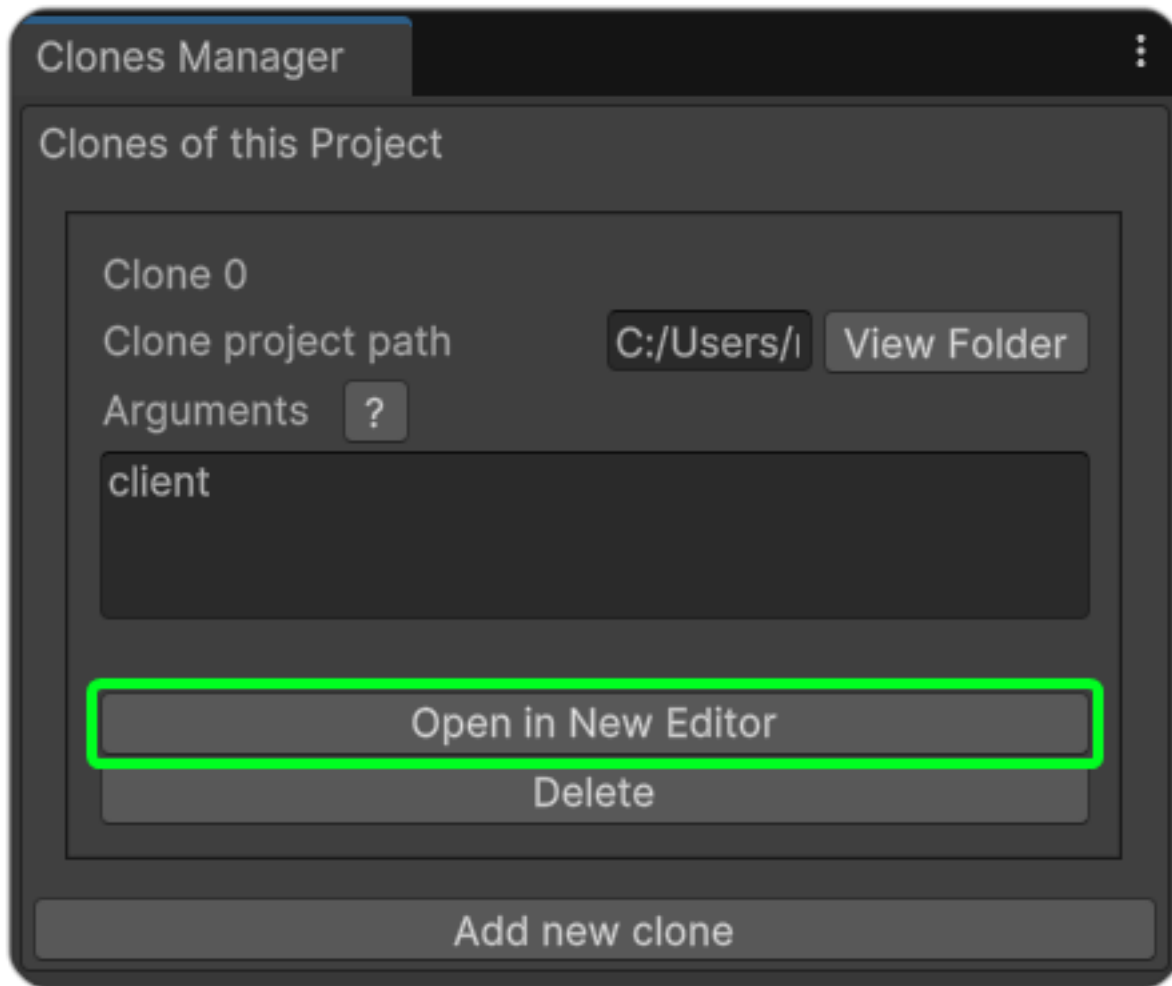
With that all done you should be able to run the game and use the UI we created to start the FishNet server and/or client.

Tip: Try extending these buttons to also include stopping the server and clients. You can do those actions directly from the `ServerManager` and `ClientManager` components as well.

Testing with Multiple Editors







[Image: untitled]

A tutorial for running multiple Unity Editor windows for testing multiplayer.

During your multiplayer game development, you may find it useful to run multiple Unity editors to test out your project locally. This can be especially helpful if you want to use the Unity Inspector or Scene Hierarchy to analyze your game at runtime.

This tutorial will go over two solutions: Unity's own **Multiplayer Play Mode** and the community extension **ParrelSync**.

Multiplayer Play Mode vs ParrelSync

	ParrelSync	Multiplayer Play Mode
Author	Community (VeriorPies)	Unity
Compatibility	Works with most Unity versions	Unity 6 and later
Editor Limit	No Limit	4 Editor Clones
Limitations	Changes should be made on main editor	Clone instances are stripped down versions of the Unity editor with limited functionality

If you are using a Unity version older than Unity 6, you will have to use ParrelSync. Otherwise you can choose either option, or even both.

Multiplayer Play Mode (MPPM)

The Multiplayer Play Mode package is a newer, official Unity solution for local and remote multiplayer testing. It allows you to simulate multiple players (up to four) directly within a single editor instance.

Step 1 — Install MPPM

Open the **Package Manager** in Unity (Window ? Package Manager). Select **"Unity Registry"** in the dropdown, find **Multiplayer Play Mode** in the packages listing, and click **Install**.

Or alternatively, click the + dropdown button, select **"Install package by name"**, and enter:

```
com.unity.multiplayer.playmode
```

Step 2 — Create Your Clones

Open the MPPM window (**Window ? Multiplayer ? Multiplayer Play Mode**) and click the checkbox next to one of the four Virtual Player options. After the clone creates the necessary files, it will open a new editor window.

Step 3 — Customize Your Clone Window

Now that your clone window has opened, you can choose which Unity tabs you want to show in it, such as the scene hierarchy, inspector, and console.

That's all you need to do! Now as soon as you start your main project, the clones will all enter play mode as well.

Info: If the options are disabled, try changing them at run-time instead. You can learn more from the [official documentation](#).

ParrelSync

ParrelSync is a popular, open-source Unity extension that creates multiple clones of your project, allowing you to run multiple editor windows that reference the same core assets.

Step 1 — Install ParrelSync

The recommended method is to use the Unity Package Manager. Open the **Package Manager** (Window ? Package Manager), click the + dropdown, select **"Install package from git URL"**, and enter:

```
https://github.com/VeriorPies/ParrelSync.git?path=/ParrelSync
```

Warning: Ensure you have the [Git client](#) (minimum version 2.14.0) installed on your system and added to the PATH system environment variable.

Or alternatively, download the latest .unitypackage file from the [releases](#) page and import it via **Assets ? Import Package ? Custom Package**.

Step 2 — Create and Test Your Clones

From the menu bar, open the **ParrelSync Clones Manager** (ParrelSync ? Clones Manager). From the Clones Manager window, click **Create new clone**. This will create a new folder next to your original project and copy the necessary files. After the clone is created, select **Open in New Editor** to launch the cloned project in a new Unity window.

You can now press **Play** in both the original and the cloned editor windows to test your multiplayer game with two separate "players."

Warning: It's important to only make changes to your project from the **original editor instance**, not the cloned one, to avoid potential asset conflicts. ParrelSync disables asset modification in clones by default.

Final Comments

With either of those options set up, you can now easily test your game with multiple editors without needing to rebuild each time.

Community Resources

You'll find here a collection of third-party tutorials and resources.

Info: If you have a contribution you would like to add to this page, please feel free to open a pull-request on the [documentation repository](#) or leave us a [suggestion](#).

The following resources are put together by developers such as yourself. The Fish-Networking team has no official involvement in these resources, thus we may not be able to troubleshoot or resolve any issues you experience with them.

Videos

Networking Concepts and Fundamentals

[I Shot You First: Networking the Gameplay of Halo: Reach](#) by GDC. GDC cover a range of developmental topics including game design, programming, audio, visual arts, business management, production, online games, and much more.

[Winterbolt Games YouTube](#) educates on a variety of Fish-Networking topics in great detail. If you're looking for step by step teachings, this is the spot.

[Fuzzy Man Studios YouTube](#) is a great landing page for visual scripting coders. The author uses visual scripting to cover a variety of tasks commonly needed to network your game.

[Bobsi's FPS tutorial series](#) shows how to make a simple client-auth First Person Shooter game using FishNet.

[Let's explore Unity!](#) has a tutorial series in German covering FishNet.

ColliderRollback (Pro Feature)

Description

This component is used for rolling back colliders on objects for the purpose of lag compensation. You can find out more about it in the dedicated guide for this feature: [raycast.md](#)

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Bounding Box

This configures how a bounding box is added to the object. When set to **Disabled** a bounding box will not be added. Additional settings are displayed when not set to **Disabled**.

:gear: Physics Type

This is which type of physics you plan on rolling back, and determines if a Physics collider is added, or Physics2D collider.

:gear: Bounding Box Size

This determines how large of a bounding box to add. Typically a value three times larger than the world space of your object is sufficient. For example, if your object is a Vector3.one cube, this value would be Vector3(3f, 3f, 3f). If an object is exceptionally fast moving consider making this larger.

:gear: Collider Parents

These are transforms you wish to send back in time when performing a rollback. For best results put your hitboxes on their own transform, which is a child of the object the hitbox is for. A setup example is provided as a demo within FishNet\Demos\ColliderRollback.

DefaultScene

Description

This component will automatically change between online and offline scenes depending on server and client connective state. When a connection is started the online scene is loaded, and when disconnected the offline scene is loaded. This component must be placed on or beneath your [NetworkManager](#).

{% hint style="success" %} Check out the API page for specific methods [here](#). {% endhint %}

{% hint style="warning" %} For this component to work, both the online scene and the offline scene must be selected in the inspector, and they must not be the same scene. {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Enable Global Scenes

This will load the scenes as global when enabled, or as connection when not. For more information on the differences see [Scene Loading](#).

:gear: Start In Offline

This will load the server and clients into the offline scene when the game starts.

:gear: Offline Scene

This is the scene to load when offline.

:gear: Online Scene

This is the scene to load when online.

:gear: Replace Scenes

This can be set to replace all loaded scenes, or just online scenes with the Online or Offline scene. For more information see loading [Scene Loading - replacing scenes](#).

Components

Many components aim to make development easier, and quicker. For example, if you wish to synchronize animations, you attach the NetworkAnimator component. There are a wide variety of components to choose from.

Pages within this section summarize what each component accomplishes as well provides more in-depth knowledge to the component's settings. A few select components which offer a significant amount of functionality are covered in the [Guides](#).

ClientManager

Description

The **ClientManager** is FishNet's primary orchestrator for client-side networking, managing connections, state, event handling, and all data flow between the local client and the server.

It maintains a reference to the network connection of the local client, `Connection`, keeps track of all known networked objects on the client through `ClientObjects`, as well as optionally tracks all currently connected clients (when server ID sharing is enabled), using a dictionary of client connections. The `ClientManager` also has many useful events, such as when a client is authenticated and when the local client or a remote client changes their network state. It also is used for sending broadcasts to the server.

When enabled, this component will also timeout the client from the server if the server doesn't respond after a set time.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left" data-full-width="false"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Remote Server Timeout

This decides if the client should disconnect when the server seems unresponsive. This feature can be set to **disabled**, work in **development** and releases, or only **releases**. *Timeout** is how long the server must be unresponsive before they are kicked.

:gear: Change Frame Rate

While enabled, this will change the frame rate limitation when acting as client only. *Frame Rate** is the frame rate to use while only the client is active. There is also a setting for client frame rate on the ServerManager. When both client and server are active the higher of the two frame rates will be used.

DebugManager

Description

The **Debug Manager** component can be used to gain extra information and temporarily tweak certain networking aspects to assist in debugging errors in your game. For example, if you are running into Scene ID errors you can enable the Detailed Writing of Scene Objects to know the object names instead of just IDs. These settings often come at the cost of a little more bandwidth used and are only enabled in the Editor and Development builds of your game.

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Scene Objects

This adds information about scene objects instead of just sending IDs. You can use this when debugging missing sceneld / spawn-related issues.

:gear: RPC Lengths

This will validate written versus read length of RPCs when enabled. Use in development to detect serialization mismatches in RPCs (written length vs read length).

:gear: ObserversRpcs

This disables ObserverRpc links.

:gear: TargetRpcs

This disables TargetRpc links.

:gear: ServerRpcs

This disables ServerRpcs links.

:gear: ReplicateRpcs

This disables ReplicateRpc links.

:gear: ReconcileRpcs

This disables ReconcileRpc links.

HashGrid

Description

The **HashGrid** is required when using the [Grid Condition](#). This component is responsible for managing the grid and providing information to grid objects. While grids are less accurate than using the [Distance Condition](#) they do provide better performance. HashGrid must be placed on or beneath the [NetworkManager](#) object.

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Grid Axes

These are which axes to base the grid on. Whichever axis is excluded will not have it's values considered in calculations. For example, if you are developing a 2D game most likely only XY matters, so you would choose the XY axes.

:gear: Accuracy

This determines how close a client's [FirstObject](#) must be (in Unity units) to be considered in range. Note that accuracy is not as precise as when using a [Distance Condition](#).

Managers

The NetworkManager will automatically add the needed manager components to itself at run-time, if they aren't already added. You can add them yourself to customize their settings in the editor.

IntermediateLayer

Using the IntermediateLayer is simple as overriding two methods, one for when data is sent and one for when data is received.

Below is an example included with FishNet which demonstrates using a Caesar cipher to encrypt data.

You can find this example in the Demos folder, file name **IntermediateLayerCipher.cs** .

```

using FishNet.Managing.Transporting;
using System;

namespace FishNet.Example.IntermediateLayers
{
    /* Below is an example of creating a basic Caesar Cipher.
     * Bytes are modified by a set value of CIPHER_KEY, and then
     * the original src ArraySegment is returned.
     *
     * It's very important to only iterate the bytes provided
     * as the segment. For example, if the ArraySegment contains
     * 1000 bytes but the Offset is 3 and Count is 5 then you should
     * only iterate bytes on index 3, 4, 5, 6, 7. The code below
     * shows one way of properly doing so.
     *
     * If you are to change the byte array reference, size, or segment
     * count be sure to return a new ArraySegment with the new values.
     * For example, if your Offset was 0 and count was 10 but after
     * encrypting data the Offset was still 0 and count 15 you would
     * return new ArraySegment<byte>(theArray, 0, 15); */
    public class IntermediateLayerCipher : IntermediateLayer
    {
        private const byte CIPHER_KEY = 5;
        // Decipher incoming data.
        public override ArraySegment<byte> HandleIncoming(ArraySegment<byte> src, bool fromServer)
        {
            byte[] arr = src.Array;
            int length = src.Count;
            int offset = src.Offset;

            for (int i = src.Offset; i < (offset + length); i++)
            {
                short next = (short)(arr[i] - CIPHER_KEY);
                if (next < 0)
                    next += 256;
                arr[i] = (byte)next;
            }

            return src;
        }
        // Cipher outgoing data.
        public override ArraySegment<byte> HandleOutgoing(ArraySegment<byte> src, bool toServer)
        {
            byte[] arr = src.Array;
            int length = src.Count;
            int offset = src.Offset;

            for (int i = offset; i < (offset + length); i++)
            {
                short next = (short)(arr[i] + CIPHER_KEY);
                if (next > byte.MaxValue)
                    next -= 256;
                arr[i] = (byte)next;
            }
        }
    }
}

```

```

        return src;
    }
}

```

In some cases you may need to inject headers into data sent, such as if you are validating each packet with an authorization key.

If such is the case it may be wise to reserve a number of bytes needed for your header. You can do this by calling `SetMTUReserve()` on your `TransportManager`.\\ Here is an example of doing such.

```

public class MyIntermediateLayer : IntermediateLayer
{
    public override void InitializeOnce(TransportManager manager)
    {
        base.InitializeOnce(manager);
        const int bytesForMyHeader = 10;
        manager.SetMTUReserve(bytesForMyHeader);
    }
    // ...rest omitted
}

```

NetworkManager

Description

The **NetworkManager** is the central component of FishNet that manages the networking lifecycle, including establishing connections, hosting sessions, and handling client-server communication. Although this component manages the network, it should not itself be a networked object, and must not contain a Network Object component on the same object, or as a parent or child of it. Additionally, the Network Manager can be used to register components and access them directly from it. This can be useful for other manager like scripts.

The Network Manager utilizes several sub-manager components, which it will add at runtime if they do not already exist on the object.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Run In Background

This allows the application to run in the background when true. Running in the background is often essential for clients, and especially for the server.

:gear: Don't Destroy On Load

This will ensure the Network Manager persist between scene changes. If you are using only one Network Manager it's best to leave this true.

:gear: Persistence

This option specifies how to behave when multiple NetworkManagers exist in your game.

:gear: Logging

This lets you specify what actions to log for builds, editor, and headless mode. When the field is not populated, the default logging settings are used. To make a custom logging settings open your create menu and choose: **Tools > Fish-Networking > Logging > Logging Configuration**.

:gear: Spawnable Prefabs

This option dictates which prefabs collection to use for networked objects. By default this field is automatically set to DefaultPrefabObjects for any saved scene; you can however make your own PrefabObjects class with customized rules and applications.

:gear: Object Pool

Which object pooling script to use. When not set, DefaultObjectPool is added automatically at run-time. You may inherit from ObjectPool to create your own.

:gear: Refresh Default Prefabs

While true, this will refresh the DefaultPrefabCollection every time play mode is entered. This is generally not needed to be enabled, but can be useful if your prefab collection is regularly becoming

corrupted through symlinks.

ObserverManager

Description

ObserverManager is used to globally customize the observer system. Observer conditions within the ObserverManager will be automatically added to NetworkObjects, unless the NetworkObserver component is set to ignore the manager.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Update Host Visibility

This will hide renderers on networked objects which are hidden to host client. When true, all networked objects will be visible to the host client even if these objects would normally be despawned for the client.

:gear: Maximum Timed Observers

The maximum duration that the server will take to update timed observer conditions as server load increases. Lower values will result in timed conditions being checked quicker at the cost of performance.

:gear: Default Conditions

These are Observer Conditions which will be added to all NetworkObjects by default. Almost all games should utilize at least the Scene Observer Condition.

PredictionManager

Description

The **PredictionManager** is a core component responsible for managing the prediction system, which allows clients to predict their own actions and then reconcile (correct) their state when authoritative updates are received from the server. It allows you to tweak various global prediction settings and get certain prediction callbacks.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Create Local States

When enabled this allows the client to create local reconcile states which means reconciles can be sent less frequently and provide data to use for reconciles when packets are lost.

:gear: State Interpolation

This is how many states to try and hold in a buffer before running them on clients. Larger values add resilience against network issues at the cost of running states later.

:gear: State Order

The order in which clients run states. **Future** favors performance and does not depend upon reconciles, while **Past** favors accuracy but clients must reconcile every tick.

:gear: Drop Excessive Replicates

Toggle the setting to true to drop excessive client replicates, helping prevent attacks but risking temporary desynchronization during connectivity issues. Set to false to store up to 3 seconds of replicates, processing multiple per tick for quicker cache clearing. This approach ensures all inputs are executed but could enable speed hacking. When false, replicated data is consumed more quickly if the cache limit is exceeded.

:gear: Maximum Server Replicates

This is shown when the **Drop Excessive Replicates** is enabled. This value indicates the maximum number of replicates which the server will cache from client before dropping old ones. Higher values will reduce the chance of dropped input when the client's connection is unstable, but will potentially add latency to the client's object both on the server and client. Generally, cached replicates will never significantly exceed Queued Inputs.

RollbackManager (Pro Feature)

Description

The RollbackManager must be added and configured properly for lag compensation to function properly. Objects you wish to rollback must contain the [ColliderRollback](#) script on them. You can learn more about using it on this page: [lag-compensation](#)

{% hint style="success" %} Check out its API page for more specific methods [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Bounding Box Layer

This is the layer to first test against before rolling back colliders. When a layer is specified a collider will be added to your [ColliderRollback](#) objects; because of this, be sure to use a layer which has no physics intersections and is not used for anything else.

:gear: Maximum Rollback Time

This is the maximum of seconds colliders may rollback. Using a value of 1f would allow colliders to rollback at most one second in the past, which is a very reasonable amount of time given typical player latencies are less than 100ms.

:gear: Interpolation

This is the amount of interpolation you are using on your [NetworkTransform](#) components. Or if you are using rigidbodies with prediction this would be the **interpolation** value on the smoothing component.

SceneManager

Description

FishNet's **SceneManager** provides robust, synchronized, and event-driven scene management for networked Unity games, ensuring all clients and the server remain in sync throughout complex scene operations. It is responsible for coordinating which scenes are loaded or unloaded across all clients and the server, ensuring synchronization and exposes many options for the loading of scenes. There are multiple guides that are worth reading about it: [scene-management](#)

The callbacks within the SceneManager are informative and quite useful, well worth looking into!

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Scene Processor

This determines how scene loads occur. When left empty the default scene processor is used. For more information on scene processing and addressables see [here](#).

:gear: Light Probe Updating

This controls how light probes are updated after scenes are loaded.

:gear: Move Client Host Objects

When enabled this will move objects visible to the host client to a temporary scene rather than let them be destroyed when a scene is unloaded. Objects are then destroyed next tick by the host client. This ensures that server and client side callbacks will work properly on the moved objects.

:gear: Set Active Scene

This will allow the **SceneManager** to pick which scene to set as the active scene when loading and unloading scenes. By default global scenes are used, and if no global scenes then the clients single scene.

ServerManager

Description

The **ServerManager** is a central component responsible for handling server-related data and actions. It maintains information and handles operations for server-side networked objects as well as tracking connected clients, both authenticated and unauthenticated. Additionally, it monitors client timeouts and disconnects unresponsive clients.

The **ServerManager** can be used directly to start and stop the FishNet server as well as monitor its state and send broadcasts to the clients.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: SyncType Rate

This is the default send rate for SyncTypes. Setting it to 0 will send updates every tick. This value can be overridden for individual SyncTypes.

:gear: Spawn Packing

This determines how well transform properties will be packed^[^1] when being sent in a spawn message. If transforms spawn marginally off reducing the packing may help. These can be edited for **Position**, **Rotation** and **Scale** independently.

:gear: Change Frame Rate

When enabled, this will change the frame rate limitation when acting as server only. *Frame Rate** is the frame rate to use while only the server is active. There is also a setting for client frame rate on the ClientManager. When both server and client are active the higher of the two frame rates will be used.

:gear: Start On Headless

When this is enabled, FishNet will automatically start the server when loaded on a server (headless) build.

:gear: Remote Client Timeout

This decides if the server should disconnect clients which seem unresponsive. This feature can be set to disabled, work in development and releases, or only releases. *Timeout** is how long the client must be unresponsive before they are kicked.

:gear: Share Ids

While true, this enables clients to be aware of other clients in-game and objects owned by other clients. Objects owned by other clients are only known if they are available to the local client, such as through the observer system. Client Ids are not sensitive information but leaving this option enabled will use slightly additional bandwidth.

:gear: Authenticator

This is where you can specify which authenticator to use. When left empty clients may join the server without specialized authentication.

:gear: Allow Predicted Spawning

This option lets prefabs and scene objects be setup to use Predicted Spawning. You can use this setting to enable or disable predicted spawning without having to change the settings for every object. This value may also be set at runtime. If changing at runtime be certain to also change on the client; otherwise they could be kicked for trying to use a disabled feature. *Reserved Object Ids** is the number of ObjectIds to reserve per client for predicted spawning. Clients will start out with the specified number of Ids and receive new ones as the server validates their predicted spawning requests. For example: if this value was set to 15 and a client with a 100ms ping sent 3 predicted spawns in one tick then they would have only 12 predicted spawns left to use until the server responded with 3 new Ids, which would be approximately 50ms later.

[^1]: compressed together to save bandwidth

StatisticsManager

Description

The **Statistics Manager** component is responsible for tracking and reporting network traffic statistics for your multiplayer game. Its **Network Traffic** can be used to gain basic information about how much network traffic your game is using. These values must be enabled for the [BandwidthDisplay](#) component to function.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Update Interval

This is how often network traffic related operations occur, such as invoking update events.

:gear: Update Client

This will invoke client traffic updates when enabled.

:gear: Update Server

This will invoke server traffic updates when enabled.

TimeManager

Description

The **TimeManager** component is responsible for managing network time and tick-based systems within the networking framework. It is central to keeping networked objects synchronized in time, handling ticks, timing conversions, physics step control, and providing hooks for network-related timing events in FishNet. It does this by managing how network timing is performed, either using fixed ticks or variable intervals and by providing precise timing for sending and receiving network data, ensuring synchronization between server and clients. It also controls the physics simulation mode (manual, automatic, or disabled) to maintain consistent simulation across the network.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Update Order

Controls when the TimeManager will invoke its version of Unity callbacks. BeforeTick would be a good option if you wanted to collect input in OnUpdate before the tick occurred.

:gear: Timing Type

Dictates when data is sent and read. When set to Tick data is only processed on frames which also tick. When Variable is selected data will be sent and read every frame, when available.

:gear: Allow Tick Dropping

Will let the client skip ticks when they occur several times over a single frame. This can be useful to prevent the client from running an increasing number of simulations per frame, resulting in more performance loss.

:gear: Maximum Frame Ticks

This is shown when **Allow Tick Dropping** is enabled. This value is the maximum number of ticks that can occur per frame before the client begins dropping ticks to recover performance.

:gear: Tick Rate

This is an average of how many times per second the TimeManager will invoke tick events, as well how often data may be sent or received.

:gear: Ping Interval

How often, in seconds, a user receives a ping update. A larger ping has a very small chance of server tick synchronization losing accuracy. These changes do not affect prediction.

:gear: Timing Interval

This is how many seconds between prediction timing updates. Lower values will result in marginally more accurate timings at the cost of bandwidth.

:gear: Physics Mode

The Physics Mode determines how physics are run. **Unity** will let the engine manage physics.

TimeManager simulates physics on ticks. When using Client-Side Prediction you must use the **TimeManager** setting.

TransportManager

Description

The **TransportManager** in FishNet is a central component responsible for communicating with the underlying network transport to send and receive data. It acts as the bridge between FishNet's networking system and the actual transport layer, enabling networked games to send/receive packets to and from clients and servers. There is also support for latency simulation for testing different network conditions.

With the TransportManager you can optionally utilize an intermediate layer to modify data before it's sent or after it's received (e.g., encryption, compression).

If you want to modify network data such as for encryption, or use any one of our number of transports, you will want to add this component to your [NetworkManager](#) game object.

{% hint style="success" %} Check out its API page for more specific methods and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Transport

This field indicates which [transport](#) to use. When left empty, the default transport ([Tugboat](#)) is used, or the first transport manually added to the object which the TransportManager resides.

:gear: Intermediate Layer

This is used to specify a custom [intermediate layer](#) to use.

:gear: Latency Simulator

These settings allow you to simulate a variety of latency scenarios on any transport. *Enabled: Toggles the enabled state of the simulator.* **Simulate Host:** When enabled, this will also simulate latency for host client. *Latency: Is the amount of latency to simulate in milliseconds.* **Unreliable:** Features only used for unreliable packets. *Out Of Order: The percentage chance to send an out of order packet.* **Packet Loss:** The chance to drop a packet.

NetworkBehaviour

Description

NetworkBehaviour(s) cannot be added to objects directly. This script must be inherited from before it may be included on an object. Inheriting from NetworkBehaviour allows access to network communications and other useful data about the network, server, and client.

Each NetworkBehaviour must be attached to a GameObject that also has a NetworkObject component on it or one of its parent objects.

{% hint style="danger" %} Adding a NetworkBehaviour derived component at run-time is not yet supported.
{% endhint %}

Please check out the guide page for more information about all the things you can do with NetworkBehaviours:

```
<table data-view="cards"><thead><tr><th align="center"></th><th data-hidden data-card-target
data-type="content-ref"></th></tr></thead><tbody><tr><td align="center"><span data-gb-custom-inline
data-tag="emoji" data-code="1f4d8">?</span> <mark style="color:blue;">NetworkBehaviour
Guide</mark></td><td><a
href="../../guides/features/networked-gameobjects-and-scripts/network-behaviour-guides.md">network-behaviour-guides
align="center">? <mark style="color:yellow;">NetworkBehaviour API</mark></td><td><a
href="https://fish-networking.com/FishNet/api/api/FishNet.Object.NetworkBehaviour.html">https://fish-networking.com/Fis
```

NetworkObject

Description

The **NetworkObject** component is essential for any game object that needs to be synchronized or controlled over the network using FishNet. It acts as the anchor for all networked behaviors and manages identification, ownership, and the object's lifecycle within the networking system.

You can nest network objects, place them in scenes, or in prefabs, but the component itself cannot be added to a game object at run-time.

{% hint style="success" %} Check out the API page for more specific methods, properties, and events [here](#).
{% endhint %}

Settings

{% tabs %} {% tab title="Settings" %} <div align="left" data-full-width="false"><figure><figcaption><p>Default settings in the Settings tab</p></figcaption></figure></div>

:gear: **Is Networked**

Indicates if an object should always be considered a networked object. When false the object will not initialize as networked. This may be useful if you have objects that you sometimes want to only run locally, and other times spawn over the network. Anytime an object is spawned using ServerManager.Spawn *Is Networked* automatically becomes true for that instantiated copy.

:gear: **Is Spawnable**

This should be enabled if the object can spawn at runtime; this is generally false for scene prefabs as you do not need to instantiate and manually spawn them. While true, this object's prefab will be added to DefaultPrefabObjects.

:gear: **Is Global**

When enabled, this will make the **NetworkObject** known to all clients at all times, the object will also be added to the Don't Destroy on Load scene. This setting will have **no** effect on scene objects, but for instantiated objects it may be set in the prefab or changed at run-time immediately after instantiating the object.

:gear: **Initialize Order**

Determines the order in which **NetworkObjects** spawned in the same tick will run their initialization callbacks. A lower value will have higher priority and execute first. The default value is 0 and negative values are allowed. On the server or clientHost, this value is not used and initialization order dependent upon the order Spawn is called.

:gear: **Prevent Despawn On Disconnect**

This will ensure the object will not be destroyed or despawned when the owning client disconnects.

:gear: **Default Despawn Type**

This is the default behavior when despawning the object. Objects are typically destroyed when despawned, but this can be set to other values, such as *Pool*, to save performance. Check out the object-pooling.md guide for more information.

{% endtab %}

```
{% tab title="Prediction" %} <div align="left"><figure><figcaption><p>Default settings in the <strong>Prediction</strong>
tab</p></figcaption></figure></div>
```

:gear: **Enable Prediction**

This should be used to set whether the object is making use of prediction or not. Enabling this will make available the following settings:

:gear: **Prediction Type**

You can use this to set if you are using rigidbodies or not for the predicted object. For example, if you were updating your transform with a CharacterController component this would be set to *Other*. If you were using rigidbodies you would choose *Rigidbody* or *Rigidbody2D*.

:gear: **Enable State Forwarding**

This is used to forward the replicate and reconcile states to all clients. This is ideal for games where you want all clients and server to run the same inputs. Disabling this setting will cause prediction to only be used on the owner; you will then have to synchronize to spectators using other means, such as a NetworkTransform or custom script.

:gear: **Graphical Object**

This is the object which holds the graphics for your predicted object. When using client-side prediction all predicted objects must have their graphics as a child of the predicted object. For example, if your rigidbody and collider are on the root object, the graphics must remain beneath the root and set as the graphical object.

:gear: **Detach Graphical Object**

When **true**, this will detach and re-attach the graphical object at runtime when the client initializes/de-initializes the item. This can resolve camera jitter or be helpful objects child of the graphical which do not handle reconiliation well, such as certain animation rigs. Transform is detached after OnStartClient, and reattached before OnStopClient.

:gear: **Interpolation**

Is how many ticks to interpolate the graphics on client owned objects. A setting as low as 1 can usually be sufficient to smooth over the frames between ticks.

:gear: **Enable Teleport**

This will allow the graphical object to teleport to it's actual position – also known as the root position – if the position changes are drastic. Ideally you will not need this setting, but it's an available option should you desire to use it.

:gear: **Teleport Threshold**

This is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position.

```
{% endtab %} {% endtabs %}
```

NetworkObserver

Description

The **NetworkObserver** uses conditions to determine if a client qualifies to be an observer of an object; any number of conditions may be used and a client will be considered an observer of the object if he passes all active conditions. The NetworkObserver component can be used to override the [ObserverManager](#), or add additional conditions onto the object to which the NetworkObserver is added.

{% hint style="warning" %} When adding this component manually, be careful not to unintentionally override the default scene observer condition of the object by using the **Ignore Manager** override type setting. {% endhint %}

This component will be added to a network object at run-time if it doesn't already exist.

{% hint style="success" %} Check out the API page for specific methods, and properties [here](#). {% endhint %}

Conditions

There are [several included conditions](#), which may be used together. ObserverCondition may also be inherited from [to make your own conditions](#). When all conditions are true, the object will become visible to the client.

Each condition must be created as a scriptable object, and dropped into the Network Observer component.

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Override Type

This is used to change how the **NetworkObserver** component uses the [ObserverManager](#) settings. *Add Missing* will add any conditions from the ObserverManager which are not already on the NetworkObserver. *UseManager* replaces conditions with those from the manager. *Ignore Manager* will keep the NetworkObserver conditions, ignoring the ObserverManager entirely. *Add Missing* is usually the recommended option.

:gear: Update Host Visibility

This will change the visibility of renderers for the host client when server objects are not visible to the client. If you wish to enable and disable other aspects during a visibility change consider using the [`NetworkObject.OnHostVisibilityUpdated`](#) event.

:gear: Observer Conditions

This is a list of which observer conditions to use. You can find out more about the specific ones here: [observerconditions](#)

NetworkTransform

Description

NetworkTransform is FishNet's robust solution for syncing transform data (position, rotation, scale) over the network, supporting both server and client authority, smooth movement, and scene hierarchy handling. It is essential for multiplayer games where objects need to appear at the same place for all players.

You may place as many NetworkTransforms as you like on children. A single NetworkTransform will synchronize the object it is directly on.

{% hint style="info" %} Check out the [moving-your-player-around.md](#) guide for an example of using the **Network Transform** to sync basic player movement. {% endhint %}

{% hint style="success" %} Check out the API page for more specific methods, properties, and events [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Component Configuration

The **Component Configuration** attempts to automatically configure components used to move your object. For example, if you were to use a **CharacterController** you could change this setting to **CharacterController** and the CharacterController will be automatically configured based on other **NetworkTransform** settings. This feature does not change owner smoothing; for example, if you are using a client authoritative rigidbody setting the Component Configuration to rigidbody will not interpolate the rigidbody for owner, but rather configures the rigidbody on non-owners and server so the NetworkTransform can smooth it properly.

:gear: Synchronize Parents

When enabled, this will automatically synchronize which objects the transform is attached to.

:gear: Packing

These settings determine the level of packing for each transform property. In some instances you may want more precision; **Unpacked** will give you the most precision; **Packed** will give you the least amount of precision, and **Packed Less** will give you an in-between amount. Less packing equals more precision at the cost of more network bandwidth used.

:gear: Interpolation

This is how long of a buffer to create when replicating the transform. Larger interpolation values will reduce the chance of jitter should there be network lag in favor of the transform being further in the past.

:gear: Extrapolation (pro feature)

This is how long the transform will try to predict movement when new data is expected, but does not arrive. Using a low interpolation value mixed with extrapolation is a great way to get responsive movement without showing network latency.

:gear: Enable Teleport

This option allows the **NetworkTransform** to teleport the object instead of smoothly moving it over several frames once the distance moved in an instance is greater than the set threshold.

:gear: Teleport Threshold

This is how far the transform must travel in a single update to cause a teleport rather than smoothing. Using a value of 0f will teleport every frame and will result in no smoothing at all.

:gear: Client Authoritative

as true allows the owning client to make changes to their transform locally, and those changes will be sent to the server and other clients. While false the server must change transforms to have them sent to clients.

:gear: Send Interval

determines at most how often in ticks the NetworkTransform may send. A value of 1 indicates the NetworkTransform can send every tick, if there is change. A value of 5 would mean that the NetworkTransform will send at most every 5 ticks, even if there is change between each tick. For example: if using an interval of 5 and the transform changes and sends on tick 100, then changes on 101, the next update will not send until 105.

:gear: Synchronize Position, Rotation, Scale

The **Synchronize** settings let you choose which properties to synchronize. Only changed values will send over the network, but if you do not want a value to update at all you can turn off synchronization for a transform property. The **Snapping** settings will allow the transform to snap axes rather than smooth them over time. This feature is commonly used for 2D games, such if you wanted to flip the Y axis on rotation immediately.

NetworkAnimator

Description

FishNet's **Network Animator** is a component designed to synchronize Unity's Animator state and parameters over the network. It supports both server-authoritative and client-authoritative modes. In client authority mode, the owner client can send animation parameter changes to the server, which then relays them to other clients.

{% hint style="info" %} This component does not align with our Client-Side prediction API. You may still use the NetworkAnimator with prediction but the animations likely be updated slightly before the prediction state runs. {% endhint %}

You may place as many NetworkAnimators as you like on children or root. When an animator is not specified on the Animator field the first one on the same object is used.

{% hint style="success" %} Check out the API page for specific methods, and properties [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Animator

This field indicates which Animator to synchronize over the network. The referenced animator can be anywhere on the object. You may also change the Animator at runtime using Network Animator API.

:gear: Synchronize When Disabled

When enabled, this will allow the **NetworkAnimator** to synchronize changes even when the **Animator** component is disabled.

:gear: Interpolation

This is how many ticks of interpolation to use. Like the [NetworkTransform](#), the animator will be this number of ticks behind before iterating data. To use tick alignment with a NetworkTransform, use the same value of interpolation as on the NetworkTransform.

:gear: Smooth Floats

This will ensure floats are moved over time on those receiving animator updates. This is commonly left true to allow blending between animation trees.

:gear: Client Authoritative

When this is enabled it allows the owning client to make changes to their animations locally, and those changes will be sent to the server and other clients. While disabled the server must change animations to have them sent to clients.

:gear: Send To Owner

This will only be displayed when **Client Authoritative** is false. While **Synchronize To Owner** is true the server will also send animation changes to the object owner; while false the owner will not get the animation changes by the server. This can be useful if you want to run animations locally in real time on owning clients.

:gear: Synchronized Parameters (pro feature)

These allow only specified parameters to be synchronized over the network. This can be useful if some parameters should only be used for the local client, or do not necessarily need to be networked.

Runtime

The NetworkAnimator will detect values, layer weights, and speed changes automatically, and synchronize these changes efficiently.

However, most projects also depend on using crossfade, play, and other common Animator APIs.

To synchronize these actions over the network you will simply call each desired method on your NetworkAnimator reference.

```
// This will perform Play on your animator, and synchronize the
// action over the network.
_myNetworkAnimator.Play(stringOrHash);
```

There are several more common methods in our NetworkAnimator component that you will want to use. To see all methods you're encouraged to view our [API](#) or simply open the NetworkAnimator source file.

Here are some examples of commonly used methods.

```
_myNetworkAnimator.SetTriger(...)
_myNetworkAnimator.ResetTriger(...)
_myNetworkAnimator.CrossFade(...)
```

Other less commonly used actions are **SetController** and **SetAnimator**. You are encouraged to review the API on these when using, or even the XML when calling the method.

PlayerSpawner

Description

The **PlayerSpawner** component in FishNet is responsible for spawning player objects for clients when they connect to the server. You set the player prefab using the inspector or via the `SetPlayerPrefab` method. You can also define an array of spawn points (Transforms) for players (the **Spawns** array). If no spawn points are set, the prefab's own position and rotation are used. You also have access to the `OnSpawned` event that fires on the server when a player is spawned.

{% hint style="success" %} Check out the API page for specific fields and events [here](#). {% endhint %}

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Player Prefab

This is used to select the prefab this component will instantiate for clients when they connect to the server.

:gear: Add to Default Scene

This is used to add a client to the active scene when no global scenes are specified through the [FishNet SceneManager](#). This is important because FishNet doesn't force a client to start in the same scene as the server, so FishNet needs to be told if a client should observe a specific scene and receive information about the Network Objects in that scene.

:gear: Spawns

This is a list of GameObject Transforms that the **PlayerSpawner** will use as locations to spawn the players at. It will use them in order, one after another, looping once it reaches the final one. If none are given, then it will use the prefab's base position.

Prediction

There are a number of components specifically used for prediction. These include components specifically tied to the Client-Side Prediction system^[1] as well as ones that are not^[2]. Other components such as the NetworkObject can also have settings related to prediction, but they are not included in this subsection.

{% hint style="info" %} Also consider viewing the PredictionManager to customize global prediction settings.
{% endhint %}

[^1]: For example the NetworkColliders

[^2]: For example the Predicted Spawn and Predicted Owner

Network Collider

Each component offers callbacks for OnEnter, OnStay, and OnExit, which work even during the prediction cycle. These components are needed because of a limitation in Unity's physics system that affects their OnCollisionEnter and OnCollisionExit methods, causing them to not always be executed.

{% hint style="success" %} Fun fact: Fish-Networking is the only framework which provides a solution for using Enter/Exit collider callbacks with prediction! {% endhint %}

{% hint style="warning" %} Due to the complexity of physics with prediction we currently only support these components on primitive shapes: box, cube, sphere, circle, etc. {% endhint %}

NetworkCollision

Description

This component is used for receiving collision events for prediction enabled objects. It is very much equivalent to a Collider's `OnCollisionEnter`, `OnCollisionExit`, and `OnCollisionStay` messages, except that to receive these you would add this component to your game object and subscribe to its public events.

{% hint style="success" %} Check out this API page [here](#) for the specific details. {% endhint %}

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Maximum Simultaneous Hits

This is the maximum number of simultaneous hits that the component will check for. You can use this field to customize how many overlapping colliders the component should be able to detect. It should be noted that having too large of a value will decrease its performance. In most cases, the default value of 16 suffices.

:gear: Additional Size

The **Additional Size** determines the distance in units by which collision traces are extended. This extension helps prevent missed overlaps when colliders do not intersect sufficiently. Depending on the scale used in your game you may want to raise or lower this value.

:gear: Layers

These are the Layers to detect collisions on. This is used when value is not set as **Nothing**.

NetworkCollision2D

Description

This component is used for receiving collision events for prediction enabled objects for 2D Colliders. It is very much equivalent to a Collider's `OnCollisionEnter`, `OnCollisionExit`, and `OnCollisionStay` messages, except that to receive these you would add this component to your game object and subscribe to its public events.

{% hint style="success" %} Check out this API page [here](#) for the specific details. {% endhint %}

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Maximum Simultaneous Hits

This is the maximum number of simultaneous hits that the component will check for. You can use this field to customize how many overlapping colliders the component should be able to detect. It should be noted that having too large of a value will decrease its performance. In most cases, the default value of 16 suffices.

:gear: History Duration

This determines how long collision history is retained. Lower values optimize memory usage slightly, but may lead to the collision records becoming out of sync on clients with excessively high latency.

:gear: Additional Size

The **Additional Size** determines the distance in units by which collision traces are extended. This extension helps prevent missed overlaps when colliders do not intersect sufficiently. Depending on the scale used in your game you may want to raise or lower this value.

:gear: Layers

These are the Layers to detect collisions on. This is used when value is not set as **Nothing**.

NetworkTrigger

Description

This component is used for receiving trigger events for prediction enabled objects for 2D Colliders. It is very much equivalent to a Collider's `OnTriggerEnter`, `OnTriggerExit`, and `OnTriggerStay` messages, except that to receive these you would add this component to your game object and subscribe to its public events.

{% hint style="success" %} Check out this API page [here](#) for the specific details. {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Maximum Simultaneous Hits

This is the maximum number of simultaneous hits that the component will check for. You can use this field to customize how many overlapping colliders the component should be able to detect. It should be noted that having too large of a value will decrease its performance. In most cases, the default value of 16 suffices.

:gear: Additional Size

The **Additional Size** determines the distance in units by which collision traces are extended. This extension helps prevent missed overlaps when colliders do not intersect sufficiently. Depending on the scale used in your game you may want to raise or lower this value.

:gear: Layers

These are the Layers to detect collisions on. This is used when value is not set as **Nothing**.

NetworkTrigger2D

Description

This component is used for receiving trigger events for prediction enabled objects for 2D Colliders. It is very much equivalent to a Collider's `OnTriggerEnter`, `OnTriggerExit`, and `OnTriggerStay` messages, except that to receive these you would add this component to your game object and subscribe to its public events.

{% hint style="success" %} Check out this API page [here](#) for the specific details. {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Maximum Simultaneous Hits

This is the maximum number of simultaneous hits that the component will check for. You can use this field to customize how many overlapping colliders the component should be able to detect. It should be noted that having too large of a value will decrease its performance. In most cases, the default value of 16 suffices.

:gear: History Duration

This determines how long collision history is retained. Lower values optimize memory usage slightly, but may lead to the collision records becoming out of sync on clients with excessively high latency.

:gear: Additional Size

The **Additional Size** determines the distance in units by which collision traces are extended. This extension helps prevent missed overlaps when colliders do not intersect sufficiently. Depending on the scale used in your game you may want to raise or lower this value.

:gear: Layers

These are the Layers to detect collisions on. This is used when value is not set as **Nothing**.

OfflineRigidbody

Description

When using predictions, especially physics prediction, the client will often re-simulate physics to make corrections. You generally want to add an OfflineRigidbody component to prevent non-networked objects from simulating multiple times during a correction.

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Rigidbody Type

This is to specify if the offline object uses a rigidbody3d, or 2d.

:gear: Get In Children

This setting should be enabled if you have child rigidbodies. Keeping this disabled when child rigidbodies are not present will save a very small amount of performance.

Combining Smoothers

There is a fair chance you will want to have the graphical object on your rigidbodies smoothed between ticks. There are a variety of components to accomplish this; see them on the [tick-smoothers](#) page.

PredictedOwner

Description

The **PredictedOwner** allows clients to immediately simulate ownership on an object without waiting for the server's response. This overcomes delays from client latency, allowing for a seamless transition of ownership. Placing this component on an object enables this feature for that network object.

A client can use the ``TakeOwnership`` method to anticipatorily claim ownership of an object. You can also access the ``PreviousOwner`` of the object and by inheriting from this component you can add checks to the server to decide whether to accept the ownership change or reject it. This can be done by overriding the ``OnTakeOwnership`` method and not handing over the ownership for whatever reason you may have.

{% hint style="success" %} Check out its full API page for more specific methods and properties [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Allow Take Ownership

This enables or disables the functionality of this component, and can be set at run-time from the server with this method: ``SetAllowTakeOwnership``.

PredictedSpawn

Description

Adding this component to a [NetworkObject](#) will allow you to adjust predicted spawning settings for the object. To enable this feature you must also enable predicted spawning within the [ServerManager](#).

The **PredictedSpawn** component allows you to inherit from it and then override any of several available virtual methods to customize and validate predicted spawns.

{% hint style="success" %} Check out its full API page for the specific methods [here](#). {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Allow Spawning

This allows clients to predicted spawn this object. You can change this at run-time with the ``SetAllowSpawning`` method. Just be sure to do so on the client and server side.

:gear: Allow Despawning

This allows clients to predicted despawn this object. You can change this at run-time with the ``SetAllowDespawning`` method. Just be sure to do so on the client and server side.

{% hint style="info" %} You can implement `WritePayload` and `ReadPayload` in your classes which inherit `NetworkBehaviour` to send data with spawn messages. This can even be used for predicted spawns to the server, and clients!

Learn more about that here: [spawn-payloads.md](#) {% endhint %}

ServerSpawner

Description

The **ServerSpawner** component is a helpful tool that you can use for optionally instantiating and spawning objects as soon as the FishNet server successfully starts running (or on your own command if preferred). This is very often used to instantiate Global Network Objects^[^1], which need to be instantiated at run-time instead of existing inside a scene directly.

Settings

<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>

:gear: Automatically Spawn

This dictates whether the component will instantiate and spawn the objects as soon as the server starts, or only after you manually call Spawn on the **ServerSpawner** component. You can disable this if you want to manually control when these objects are spawned.

:gear: Network Objects

This is a list of the Network Objects the component will instantiate once the server starts. The order of this list is respected when instantiating the objects, and the component will attempt to use FishNet's object pooling for the objects if they are added to the an object pool.

[^1]: These are NetworkObjects that are added to the DontDestroyOnLoad scene and visible to all clients.

DetachableNetworkTickSmoother \[Obsolete]

{% hint style="warning" %} This component is obsolete, you can use the [networkticksmoother.md](#) instead of this now. {% endhint %}

Description

The DetachableNetworkTickSmooth will detach from parents at runtime and follow the configured follow object. Generally this is placed on an empty object beneath the parent, and you will set your camera target to this.

```
<figure><figcaption><p>For use with prediction. You may follow NetworkObjectRoot if not using prediction.</p></figcaption></figure>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Attach On Stop

This will re-attach this object to it's previous parent when the [NetworkObject](#) for this component has completed network stop callbacks.

:gear: Follow Object

This is the object to follow. This is typically the graphical object for the same NetworkObject.

:gear: Interpolation

The **Interpolation** is how long in ticks to smooth over. Depending on your game type and camera settings, typically 1 or 2 works well.

:gear: Enable Teleport

This will allow this object to teleport to Follow Object's position after a distance between tick has passed. *Teleport Threshold** is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position.

:gear: Teleport Threshold

This is shown while [Enable Teleport](#) is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position.

:gear: Synchronize Position

When enabled this will move this transforms position towards the follow object's.

:gear: Synchronize Rotation

Synchronize Rotation will move this transforms rotation towards the follow object's if it is enabled.

:gear: Synchronize Scale

When true, will move this transforms scale towards the follow object's.

Tick Smoothers

You may put multiple tick smoother components beneath an object if you have several different graphical pieces in your hierarchy you want smoothed. A good example of this is a vehicle where the wheels move independently of the vehicle body.

The smoother will be placed on the graphical object you wish to smooth; this object must not be the same as the target, nor higher in the hierarchy of the target; being a child of the target is fine.

{% hint style="success" %} Tick Smoothers are only used by clients. These components will not function when only the server is started. {% endhint %}

MonoTickSmoother \[Obsolete]

{% hint style="warning" %} This component is obsolete, you can use the [offlineticksmoother.md](#) instead of this now. {% endhint %}

Description

The main difference between NetworkTickSmoother and MonoTickSmoother is that MonoTickSmoother does not need to be placed on a networked object, and initializes by default using InstanceFinder; NetworkTickSmoother initializes using NetworkBehaviour callbacks.

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Use Instance Finder

This option determines whether or not to use the [InstanceFinder](#) to find the [TimeManager](#). When this is false you will need to specify which TimeManager to use by calling `SetTimeManager`. This can be useful in cases where you have multiple [NetworkManagers](#) and therefore multiple TimeManagers and you want to specify which one to use.

:gear: Graphical Object

This is the object which holds the graphics you want to smooth. The graphics should be a child of the game object with this component.

:gear: Enable Teleport

This will allow the graphical object to teleport to it's actual position – also known as the root position – if the position changes are drastic. Ideally you will not need this setting, but it's an available option should you desire to use it.

:gear: Teleport Threshold

The **Teleport Threshold** is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position.

NetworkTickSmoother

Description

The **Network Tick Smoother** is responsible for smoothing an object's transformation properties between network ticks. Its main purpose is to make movement, rotation, and scale, transitions appear fluid, even though network updates may arrive in discrete steps. The **NetworkTickSmoother** initializes its settings using [NetworkBehaviour callbacks](#), and may only be placed below a [NetworkObject](#); if you need this form of smoothing for objects that aren't NetworkObjects you can use the [offlineticksmoother.md](#) instead.

There are several public APIs available for changing the smoother settings at runtime. An example of some are changing adaptive interpolation type, interpolation value, teleporting the smoothed object, and more.

{% hint style="success" %} Please review our [API](#) for all runtime settings. {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Favor Prediction Network Transform

This setting prevents jitter by automatically disabling the NetworkTickSmoother when a predicted NetworkObject is under the active control of a NetworkTransform. If a specific transform is assigned in the prediction field, the smoother will yield control to the NetworkTransform's internal smoothing logic; otherwise, the smoother remains active to handle state interpolation.

:gear: Initialization Settings

These are essential to the component working. *Target Transform is the transform you want to follow. This is commonly the root NetworkObject but may be other nested NetworkBehaviours when using multiple smoothers beneath a single NetworkObject.* **Detach On Start** when true will unparent the object which the smoother is attached, placing it as root in world space. Detach is commonly used when the smoothed object is a camera target, given cameras do not handle rollback or larger tick steps well. *Attach On Stop** when true will reparent the graphical object when the network stop callbacks occur on the object, as detach will unparent on network start callbacks. Typically you want to reattach graphical objects. If the Target Transform is destroyed while detached the object the smoother is attached to will also destroy itself.

:gear: Controller Smoothing

The **Controller Smoothing** settings determine how the object is smoothed for the controller of the object, which is either the owner or if the object has no owner then it is the server. *Enable Teleport will allow the graphical object to teleport to it's actual position – also known as the root position – if the position changes are drastic. Ideally you will not need this setting, but it's an available option should you desire to use it.* **Teleport Threshold** is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position. *Adaptive Interpolation, when not disabled, will increase the interpolation amount as the local client's latency becomes higher. Low settings of adaptive interpolation will increase the interpolation at lower amounts, while high will increase the interpolation more. When Adaptive Interpolation is disabled, a flat amount of interpolation will be used at all times.* \ \ Flat interpolation is often used in competitive or reaction based games to keep the interpolation consistent for all players. Flat

interpolation is also necessary for accurate collider rollback, given our [collider rollback system](../../guides/features/lag-compensation/) needs to know the amount of interpolation a client sees to provide accurate raycast hit results. \ \ Adaptive interpolation is best used with casual games where you want the absolute smoothed experiences regardless of local client latency. **Interpolation Value** is displayed when Adaptive Interpolation is off. This is a flat amount of interpolation that does not increase when the local client's latency does. *Smoothed Properties** determines which properties of the transform will be smoothed. If smoothing is not set for a specific property then that particular value will persist in the same world space at all times. For example, if you uncheck Position from Smoothed Properties and move the root, the object which this component sits will never move in space, unless you of course move it manually.

:gear: Spectator Smoothing

The **Spectator Smoothing** settings determine how the object is smoothed for client's that are **not** the owner. *Enable Teleport will allow the graphical object to teleport to it's actual position – also known as the root position – if the position changes are drastic. Ideally you will not need this setting, but it's an available option should you desire to use it.* **Teleport Threshold** is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position. *Adaptive Interpolation, when not disabled, will increase the interpolation amount as the local client's latency becomes higher. Low settings of adaptive interpolation will increase the interpolation at lower amounts, while high will increase the interpolation more. When Adaptive Interpolation is disabled, a flat amount of interpolation will be used at all times.* \ \ Flat interpolation is often used in competitive or reaction based games to keep the interpolation consistent for all players. *Flat interpolation is also necessary for accurate collider rollback, given our [collider rollback system](../../guides/features/lag-compensation/) needs to know the amount of interpolation a client sees to provide accurate raycast hit results.* \ \ Adaptive interpolation is best used with casual games where you want the absolute smoothed experiences regardless of local client latency. **Interpolation Value** is displayed when Adaptive Interpolation is off. This is a flat amount of interpolation that does not increase when the local client's latency does. *Smoothed Properties** determines which properties of the transform will be smoothed. If smoothing is not set for a specific property then that particular value will persist in the same world space at all times. For example, if you uncheck Position from Smoothed Properties and move the root, the object which this component sits will never move in space, unless you of course move it manually.

Scale Smoothing

Scale behaves a little differently when smoothed. Since nested objects use a localScale they will not change with the parent scale, and thus cannot be easily interpolated.

If you wish to smooth scale transitions then **Detach** (and optionally **Reattach**) must be enabled in the initialization settings so that the smoothed object does not have it's global (lossy) scale modified by Unity when the target transform or any higher transform has it's scale changed.

OfflineTickSmoother

Description

The OfflineTickSmoother behaves just like the [networkticksmoother.md](#), except it initializes it's settings using the [InstanceFinder](#) and can thus be placed beneath objects which are not networked. It is responsible for smoothing an offline object's transformation properties between network ticks.

There are several public APIs available for changing the smoother settings at runtime. An example of some are changing adaptive interpolation type, interpolation value, teleporting the smoothed object, and more.

{% hint style="success" %} Please review our [API](#) for all runtime settings. {% endhint %}

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Automatically Initialize

This option will configure this smoother in Awake using the [InstanceFinder](#) to listen to network callbacks. When false, you must manually call the `Initialize` method on this component.

:gear: Initialization Settings

These are essential to the component working. *Target Transform is the transform you want to follow. This is commonly the root*

[NetworkObject](../../guides/features/networked-gameobjects-and-scripts/networkobjects/) but may be other nested NetworkBehaviours when using multiple smoothers beneath a single NetworkObject.

Detach On Start when true will unparent the object which the smoother is attached, placing it as root in world space. Detach is commonly used when the smoothed object is a camera target, given cameras do not handle rollback or larger tick steps well. *Attach On Stop** when true will reparent the graphical object when the network stop callbacks occur on the object, as detach will unparent on network start callbacks. Typically you want to reattach graphical objects. If the Target Transform is destroyed while detached the object the smoother is attached to will also destroy itself.

:gear: Smoothing

The **Smoothing** settings determine how the object is smoothed. *Enable Teleport will allow the graphical object to teleport to it's actual position – also known as the root position – if the position changes are drastic. Ideally you will not need this setting, but it's an available option should you desire to use it.*

Teleport Threshold is shown while teleporting is enabled. If the graphical object's position is this many units away from the actual position, then the graphical object will teleport to the actual position. *Adaptive Interpolation, when not disabled, will increase the interpolation amount as the local client's latency becomes higher. Low settings of adaptive interpolation will increase the interpolation at lower amounts, while high will increases the interpolation more. When Adaptive Interpolation is disabled, a flat amount of interpolation will used at all times.* \ \ *Flat interpolation is often used in competitive or reaction based games to keep the interpolation consistent for all players. Flat interpolation is also necessary for accurate collider rollback, given our [collider rollback system](../../guides/features/lag-compensation/) needs to know the amount of interpolation a client sees to provide accurate raycast hit results.* \ \ *Adaptive interpolation is best used with casual games where you want the absolute smoothed experiences regardless of local client latency.* **Interpolation Value** is displayed when Adaptive

Interpolation is off. This is a flat amount of interpolation that does not increase when the local client's latency does. *Smoothed Properties** determines which properties of the transform will be smoothed. If smoothing is not set for a specific property then that particular value will persist in the same world space at all times. For example, if you uncheck Position from Smoothed Properties and move the root, the object which this component sits will never move in space, unless you of course move it manually.

{% hint style="info" %} Since a non-networked object cannot be owned there is only one set of smoothing settings, compared to NetworkTickSmoother which can have unique settings if Owner or Spectator. {% endhint %}

Authenticator

Authenticator logic can be fully customized by inheriting from the ***Authenticator*** script, and placing your created authenticator within the Server Manager component. There are a few example authenticators that come installed with the FishNet package within the FishNet/Demos/Authenticator/Scripts directory.

BandwidthDisplay

Description

Using this component in combination with the [Statistics Manager](#) will show bandwidth being used by Fish-Networking.

```
<div align="left"><figure><figcaption><p>Example of the BandwidthDisplay being used</p></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Color

This is which color to display results as.

:gear: Placement

The **Placement** is in what part of the screen to display results.

:gear: Seconds Averaged

This is for the number of seconds used to gather data. Lower values will show more up to date usage per second while higher values provide a better over-all estimate.

:gear: Show Outgoing

When enabled, this will display the bandwidth used from data being sent. This can be changed at runtime with the ``SetShowOutgoing`` method.

:gear: Show Incoming

When enabled, this will display the bandwidth used from data being received. This can be changed at runtime with the ``SetShowIncoming`` method.

Utilities

PingDisplay

Description

Often times you may want to display the local client's ping while developing your game, this can easily be done with the **PingDisplay** component. This component is commonly added to the NetworkManager object, but that is not a requirement.

```
<div align="left"><figure><figcaption><p>Example of the PingDisplay component in-use</p></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Color

This is which color to use for the displayed text.

:gear: Placement

The **Placement** indicates in which part of the screen to display the client's ping.

:gear: Hide Tick Rate

This option will remove tick rate latency from the ping results otherwise the displayed ping will include the delay caused by the server's tick rate.

Configuration and Tools

FishNet comes with some useful settings and editor tools to be used with it; these can be accessed through the **(Tools ? Fish-Networking)** toolbar menu.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Beta

The settings here will vary depending on your FishNet version and will enable you to activate and deactivate the latest beta features. This can be helpful if you want to stay with the most tested code for your project or if alternatively if you want to try out features that are coming out soon.

Utility

Quality of Life Attributes

With this option you will be able to disable and enable certain attributes FishNet uses. It's not recommended to use this unless a developer or community helper advises you do so for debugging purposes.

Upgrading From Mirror

When converting a project from Mirror Networking, it can be helpful to use these tools.

Replace Components

This tool helps you replace common Mirror components with what FishNet may use instead.

Remove Defines

This tool removes the many scripting defines that Mirror includes in your Unity project.

Refresh Default Prefabs

This option will cause FishNet's Default Prefab Objects tool to be refreshed by searching your project and keeping track of the Network Object Prefabs within it. This is done automatically for you, but you may want to trigger it if you start seeing the wrong prefabs being spawned in a client build or editor.

Reserialize NetworkObjects

This tool allows you to reserialize prefabs and scenes. Most often you will use this if you get an error message by FishNet telling you to do so. It can also help you if you are running into errors reporting missing sceneids.

{% hint style="info" %} If you run it on the open scenes, be sure to save them after it runs. {% endhint %}

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Configuration

The FishNet configuration (available in the Project Settings) has options for FishNet Pro's **Code Stripping**

feature as well as the Prefab Objects Generator and NetworkBehaviour Template.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Code Stripping

FishNet Pro allows you to strip server-only code from non-server builds^[^1], this includes the code inside ServerRpcs, Server marked methods, and server specific callbacks such as OnStartServer and OnSpawnServer. You can use one of the two available methods for this, either **redirection** or **emptying**.

New NetworkBehaviour Template

This allows you to customize the NetworkBehaviour asset template as well as choose a new location for it. If you want to create a NetworkBehaviour using this template, simply go to **Assets ? FishNet ?**

NetworkBehaviour Script.

Prefab Objects Generator

The Prefab Objects Generator is used by the Default Prefab Objects search your project and keep track of the Network Object Prefabs within it. You can enable and disable it as well as enable and disable its logging.

The **Full Rebuild** option will tell FishNet to search all prefabs instead of just ones that were recently modified.

Spawnable Only instructs the generator to only find NetworkObject prefabs that have IsSpawnable enabled.

The **Default Prefab Objects Path** option allows you to change the default path of the asset in your Unity project.

And finally you can choose which folders to exclude or include when it searches for network prefabs.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Getting Started

The getting started option will show FishNet's **Getting Started** menu, giving you quick access to relevant FishNet links, including the Documentation, Discord, GitHub, and more.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

[^1]: Regular build profiles that aren't headless/dedicated server

Addressables

Both addressable scenes and prefabs work over the network.

Scene Addressables

Scene addressables utilize Fish-Networking's Custom Scene Processors. With a few overrides you can implement addressable scenes.

Prefab Addressables

To work reliably, each addressables package must have a unique `ushort` ID between 1 and 65535. Never use 0 as the ID, as the preconfigured `SpawnablePrefabs` uses this ID. You may assign your addressable IDs however you like, for instance using a dictionary that tracks your addressable names with IDs.

Registering addressable prefabs with Fish-Networking is easy once each package has been given an ID.

The code below shows one way of loading and unloading addressable prefabs for the network.

```

/// <summary>
/// Reference to your NetworkManager.
/// </summary>
private NetworkManager _networkManager => InstanceFinder.NetworkManager;

/// <summary>
/// Used to load and unload addressables in async.
/// </summary>
private AsyncOperationHandle<IList<GameObject>> _asyncHandle;

/// <summary>
/// Loads an addressables package by string.
/// </summary>
public IEnumerator LoadAddressables(string addressablesPackage)
{
    /* FishNet uses an Id to identify addressable packages
    * over the network. A simple way is to use GetStableHash
    * helper methods to return a unique key for the package name.
    * This does require the package names to be unique. */
    ushort id = addressablesPackage.GetStableHash16();

    /* GetPrefabObjects will return the prefab
    * collection to use for Id. Passing in true
    * will create the collection if needed. */
    SinglePrefabObjects spawnablePrefabs = (SinglePrefabObjects)_networkManager.GetPrefabOb
jects<SinglePrefabObjects>(id, true);

    /* Get a cache to store NetworkObject references from our helper object pool.
    * FishNet has the helpful CollectionCaches and ObjectCaches to prevent allocations. */
    /
    List<NetworkObject> cache = CollectionCaches<NetworkObject>.RetrieveList();

    /* Load addressables normally. If the object is a NetworkObject prefab
    * then add it to our cache! */
    _asyncHandle = Addressables.LoadAssetsAsync<GameObject>(addressablesPackage, addressab
le =>
    {
        NetworkObject nob = addressable.GetComponent<NetworkObject>();
        if (nob != null)
            cache.Add(nob);
    });
    yield return _asyncHandle;

    /* Add the cached references to spawnablePrefabs. */
    spawnablePrefabs.AddObjects(cache);

    // Store the collection cache for use later to prevent garbage.
    CollectionCaches<NetworkObject>.Store(cache);
}

/// <summary>
/// Unloads an addressables package by string.
/// </summary>
public void UnloadAddressables(string addressablesPackage)
{
    // Get the Id the same way as we did for loading.
    ushort id = addressablesPackage.GetStableHash16();

```

```

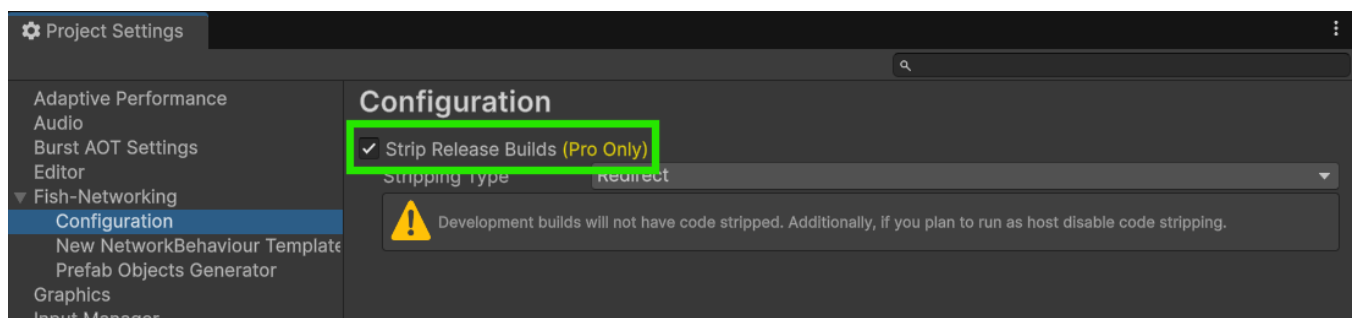
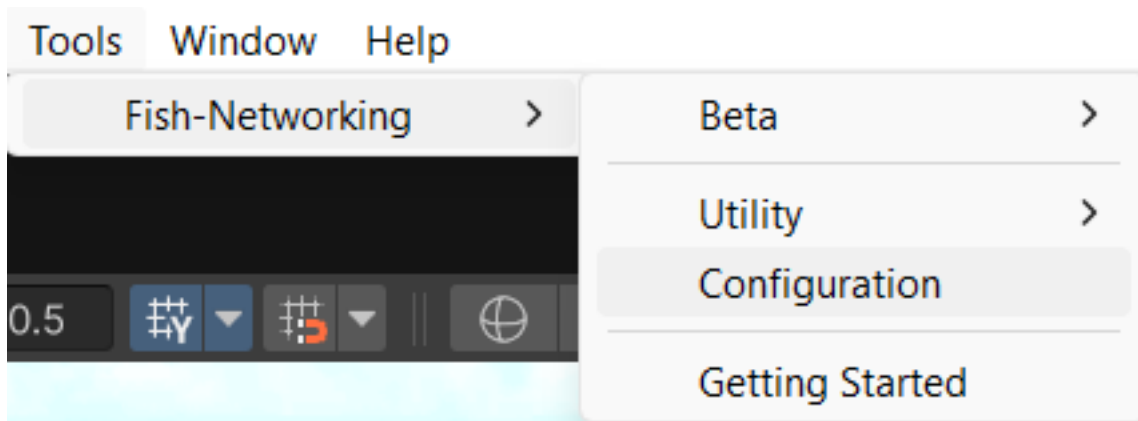
    /* Get the prefab collection for the Id and
    * clear it so that there are no references of the objects in memory. */
    SinglePrefabObjects spawnablePrefabs = (SinglePrefabObjects)_networkManager.GetPrefabOb
jects<SinglePrefabObjects>(id, true);
    spawnablePrefabs.Clear();

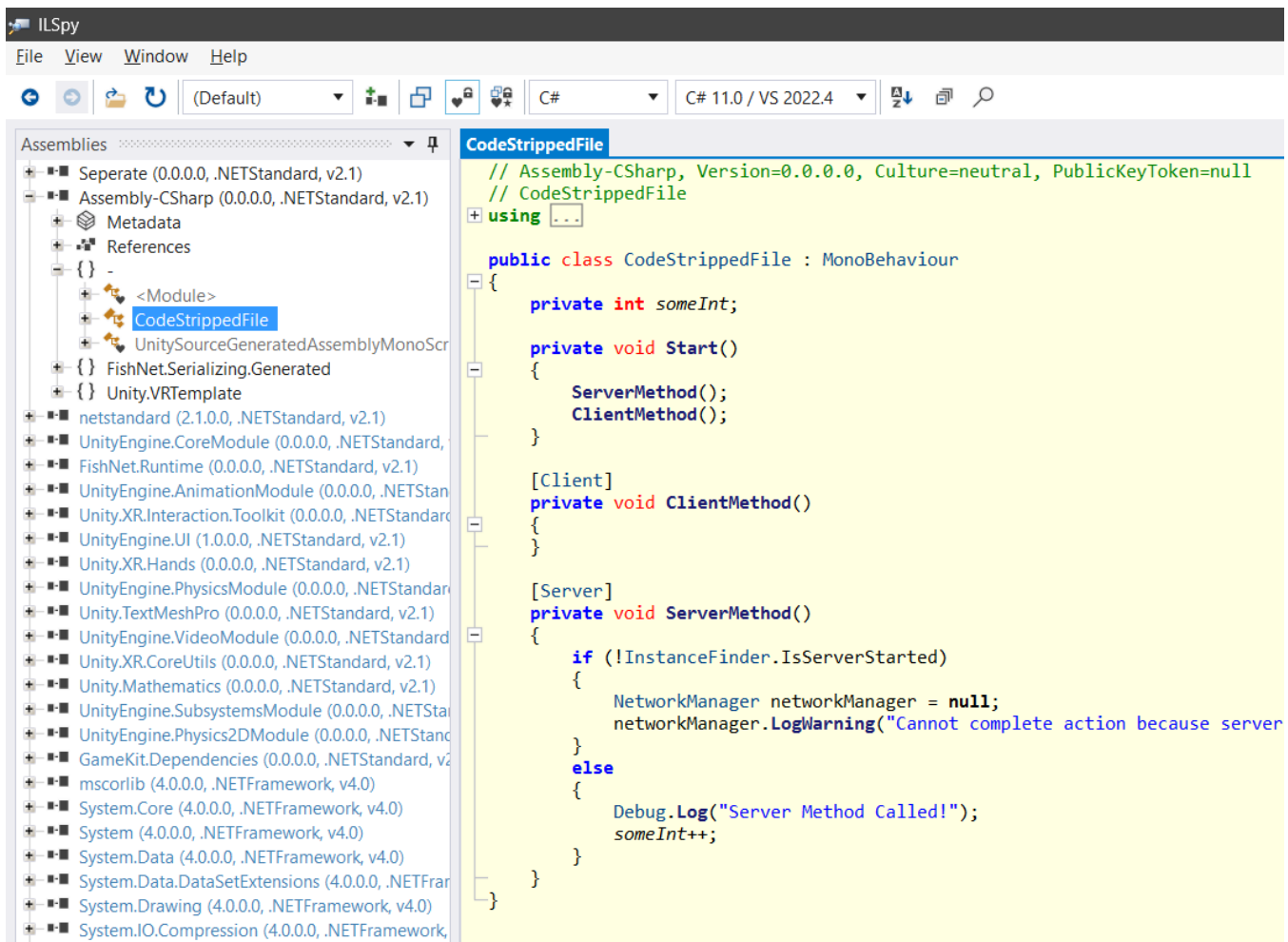
    // Release addressables.
    Addressables.Release(_asyncHandle);
}

```

Important: When using addressables be sure the client has addressable bundles loaded before the server sends spawn messages for the objects. For example: if you are going to load a scene with addressables, ensure the client has already loaded the addressables bundle for that scene before the server adds the client to the scene.

Code Stripping (Pro Feature)





FishNet's code stripping feature allows your server-only code to be automatically stripped from client builds and client-only code to be stripped from server builds.

Why use code stripping?

It helps secure your game server by removing sensitive logic that players shouldn't see. This makes it harder for cheaters to figure out how the server works and keeps sensitive server data hidden.

What gets stripped

FishNet Pro can remove server code from non-server builds and client code from server builds. The code it strips is code within RPCs, code marked with the `Server` or `Client` attributes, as well as server and client only callbacks such as `OnStartServer`/`OnStartClient`.

How to use it

To use FishNet's code stripping you will firstly need to have FishNet Pro installed. You can find details for how to install it here: [Upgrading to FishNet Pro](#).

Next, you can navigate to the FishNet Configuration menu through the Unity toolbar:

(Tools ? Fish-Networking ? Configuration)

On the Configuration page you will see the option to **"Strip Release Builds"**, enable it.

There is now the **"Stripping Type"** option that you can switch between `Redirect` and

`Empty_Experimental.Redirect` will replace the methods and calls with a dummy method that doesn't contain any of the logic, while `Empty_Experimental` will keep the same method, but will strip its method body, leaving it empty. You can use either option you want to.

Now you will need your game to recompile its scripts, you can trigger this by changing something in your code. If your code is in a separate assembly, be sure to trigger that assembly to recompile as well. This is only needed after you change the FishNet stripping settings.

That's all you need to do, you can now build your game, and FishNet will automatically strip the code for you.

Confirming it worked

It's always a good idea to check if the sensitive code was indeed stripped from the builds. For Unity Mono builds you can easily do so using a tool such as ILSpy.

You can download the above-mentioned tool or another of your choosing. Then you'll want to run it and load the chosen assembly in your built game's files. If you find your script, you should see how FishNet stripped its code according to the build.

Custom Serializers Guides

Are useful where an automatic serializer may not be possible, or where you want data to be serialized in a specific manner.

When creating a custom serializer there are a few important things to remember. When you follow the proper steps your custom serializer will be found and used by Fish-Networking. Your custom serializers can also override automatic serializers, but not included ones.

- Your method must be static, and within a static class.
- Writing method names must begin with Write.
- Reading method names must begin with Read.
- The first parameter must be this Writer for writers, and this Reader for readers.
- Data must be read in the same order it is written.

Although Vector2 is already supported, the example below uses a Vector2 for simplicity's sake.

```
// Write each axis of a Vector2.
public static void WriteVector2(this Writer writer, Vector2 value)
{
    writer.WriteSingle(value.x);
    writer.WriteSingle(value.y);
}

// Read and return a Vector2.
public static Vector2 ReadVector2(this Reader reader)
{
    return new Vector2()
    {
        x = reader.ReadSingle(),
        y = reader.ReadSingle()
    };
}
```

Custom serializers are more commonly used for conditional situations where what you write may change depending on the data values. Here is a more complex example where certain data is only written when it's needed.

```

/* This is the type we are going to write.
 * We will save data and populate default values
 * by not writing energy/energy regeneration if
 * the enemy does not have energy. */
public struct Enemy
{
    public bool HasEnergy;
    public float Health;
    public float Energy;
    public float EnergyRegeneration;
}

public static void WriteEnemy(this Writer writer, Enemy value)
{
    writer.WriteBoolean(value.HasEnergy);
    writer.WriteSingle(value.Health);

    // Only need to write energy and energy regeneration if HasEnergy is true.
    if (value.HasEnergy)
    {
        writer.WriteSingle(value.Energy);
        writer.WriteSingle(value.EnergyRegeneration);
    }
}

public static Enemy ReadEnemy(this Reader reader)
{
    Enemy e = new Enemy();
    e.HasEnergy = reader.ReadBoolean();
    e.Health = reader.ReadSingle();

    // If there is energy also read energy values.
    if (e.HasEnergy)
    {
        e.Energy = reader.ReadSingle();
        e.EnergyRegeneration = reader.ReadSingle();
    }

    return e;
}

```

Often when creating a custom serializer you want to use it across your entire project, and all assemblies. Without taking any action further your custom serializer would only be used on the assembly it is written. Presumably, that's probably not what you want.

But making a custom serializer work across all assemblies is very simple. Simply add the `[UseGlobalCustomSerializer]` attribute to the type your custom serializer is for, and done!

```

[UseGlobalCustomSerializer]
public struct Enemy
{
    public bool HasEnergy;
    public float Health;
    public float Energy;
    public float EnergyRegeneration;
}

```

Inheritance Serializers

Another frequently asked question is how to handle serialization for classes which are inherited by multiple other classes. These are often used to allow RPCs to use the base class as a parameter, while permitting other inheriting types to be used as arguments. This approach is similar to [Interface Serializers](#).

Class example

Here is an example of a class you want to serialize, and two other types which inherit it.

```
public class Item : ItemBase
{
    public string ItemName;
}

public class Weapon : Item
{
    public int Damage;
}

public class Currency : Item
{
    public byte StackSize;
}

/* This is a wrapper to prevent endless loops in
 * your serializer. Why this is used is explained
 * further down. */
public abstract class ItemBase {}
```

Using an RPC which can take all of the types above might look something like this.

```
public void DoThing()
{
    Weapon wp = new Weapon()
    {
        ItemName = "Dagger",
        Damage = 50,
    };
    ObsSendItem(wp);
}

[ObserversRpc]
private void ObsSendItem(ItemBase ib)
{
    if (ib is Weapon wp)
        Debug.Log($"Recv: Item name {wp.ItemName}, damage value {wp.Damage}.");
}
```

Creating the writer

Since you are accepting `ItemBase` through your RPC you must handle the different possibilities of what is being sent.

[!WARNING] When using this approach it is very important that you check for the child-most types first. For example: `Weapon` is before `Item`, and so is `Currency`, so those two are checked first. Just as if you had `Melee : Weapon`, then `Melee` would be before `Weapon`, and so on.

```

public static void WriteItembase(this Writer writer, ItemBase ib)
{
    if (ib is Weapon wp)
    {
        // 1 will be the identifier for the reader that this is Weapon.
        writer.WriteByte(1);
        writer.Write(wp);
    }
    else if (ib is Currency cc)
    {
        writer.WriteByte(2);
        writer.Write(cc);
    }
    else if (ib is Item it)
    {
        writer.WriteByte(3);
        writer.Write(it);
    }
}

public static ItemBase ReadItembase(this Reader reader)
{
    byte clsType = reader.ReadByte();
    if (clsType == 1)
        return reader.Read<Weapon>();
    else if (clsType == 2)
        return reader.Read<Currency>();
    else if (clsType == 3)
        return reader.Read<Item>();
    // Unhandled, this would probably result in read errors.
    else
        return null;
}

```

[!TIP] You can still create custom serializers for individual classes in addition to encapsulating ones as shown! If for example you had a custom serializer for Currency then using the code above would use your serializer for Currency rather than the one Fish-Networking generates.

Finally, disclosing why we made the ItemBase class. The sole purpose of ItemBase is to prevent an endless loop in the reader. Imagine if we were able to return only Item and we were also using that as our base. Your reader might look like this:

```

public static Item ReadItem(this Reader reader)
{
    byte clsType = reader.ReadByte();
    if (clsType == 1)
        return reader.Read<Weapon>();
    else if (clsType == 2)
        return reader.Read<Currency>();
    else if (clsType == 3)
        return reader.Read<Item>(); // ? calls ReadItem again ? infinite loop!
    else
        return null;
}

```

The line `return reader.Read<Item>();` is the problem. By calling read on the same type as the serializer you would in result call the ReadItem method again, and then the line return

`reader.Read<Item>( )` again, and then `ReadItem` again — endless loop.
Having a base class (`ItemBase`) which cannot be returned ensures no endless loop.

Interface Serializers

General

Interfaces are very commonly used in most Unity Projects. Since Interfaces are not classes, even if the interface only uses serializable fields, a custom serializer is still needed in order for SyncTypes, and RPCs to serialize them properly over the network.

Serializing the interface's entire class

In most cases you will want to interrogate an Interface as what its class type is, and serialize the entire types class over the network. This allows you to interrogate the interface later on the receiving client/server and have the data match what the sender has at the time it was sent as well.

If the Interface is a NetworkBehaviour you might as well send it as one because serializing them over the network is only sending an ID for the receiving client to look up. Very little network traffic, and you still get all of the data!

Creating the writer

Since Interfaces are not classes you must design the writer to be able to interrogate the class the Interface is, and serialize the class over the network. If an interface can be many types of classes, you will need to account for each class the Interface can be.

```
public static void WriteISomething(this Writer writer, ISomething som)
{
    if(som is ClassA c1)
    {
        // 1 will be the identifier for the reader that this is a ClassA.
        writer.WriteByte(1);
        writer.Write(c1);
    }
    else if(som is ClassB c2)
    {
        // 2 will be the identifier for the reader that this is a ClassB.
        writer.WriteByte(2);
        writer.Write(c2);
    }
}
```

Creating the reader

When reading the interface, we have to read the byte that identifies what class the interface actually is first. Then use the reader to read that class's data, finally casting it as the interface we need.

```

public static ISomething ReadISomething(this Reader reader)
{
    // Gets the byte of what class type we should be reading the next bit of data as.
    byte clsType = reader.ReadByte();

    // Remember we assigned 1 to be ClassA.
    if(clsType == 1)
        return reader.Read<ClassA>();
    // And 2 for ClassB.
    else if(clsType == 2)
        return reader.Read<ClassB>();

    // Fall through, unhandled.
    return default;
}

```

Serializing only the interface's properties

Sometimes you may only want to serialize just the Interface properties over the network, just keep in mind that if you cast it as the Type it actually is on the receiving client, the values of fields not part of the interface will be their default values!

Creating the writer

You still will have to use an identifier to send what class the Interface is, but we will not be sending the entire class over the network — just the Interface properties.

```

public interface ISomething
{
    string Name;
    int Health;
    ushort Level;
}

public static void WriteISomething(this Writer writer, ISomething som)
{
    if(som is CustomClass1 cc1)
        writer.WriteByte(1);
    else if(som is CustomClass2 cc2)
        writer.WriteByte(2);
    // Fall through, indicating unknown type.
    else
        writer.WriteByte(0);

    // Remember: the order data is written is the order it must be read.
    writer.WriteString(som.Name);
    writer.WriteInt32(som.Health);
    writer.WriteUInt16(som.Level);
}

```

Creating the reader

When reading, we will get the class type from the identifier, create a new class, cast the class as the interface, and then assign the custom serialized values to the interface.

```
public static ISomething ReadISomething(this Reader reader)
{
    byte clsType = reader.ReadByte();
    string name = reader.ReadString();
    int health = reader.ReadInt32();
    ushort level = reader.ReadUInt16();

    ISomething som = default;
    if(clsType == 1)
        som = new CustomClass1();
    else if(clsType == 2)
        som = new CustomClass2();

    if(som == default(ISomething))
        return null;

    som.Name = name;
    som.Health = health;
    som.Level = level;

    return som;
}
```


Data Serialization

Serialization and deserialization of data for the purpose of sending it across the network.

Anytime you use a type within a communication, Fish-Networking automatically recognizes you wish to send the type over the network and will create a serializer for it. FishNet will only attempt to automatically serialize public fields and properties. You do not need to perform any extra steps for this process, but if you would like to exclude such fields from being serialized use `[ExcludeSerialization]` above the field.

For example, `Name` and `Level` will be sent over the network but not `Victories`.

```
public class PlayerStat
{
    public string Name;
    public int Level;
    [ExcludeSerialization]
    public int Victories;
}

[ServerRpc]
public void RpcPlayerStats(PlayerStat stats) {}
```

Fish-Networking is also capable of serializing inherited values. In the type `MonsterStat` below, `Health`, `Name`, and `Level` will automatically serialize.

```
public class Stat
{
    public float Health;
}

public class MonsterStat : Stat
{
    public string Name;
    public int Level;
}
```

In very rare cases a data type cannot be automatically serialized; a `Sprite` is a good example of this. It would be very difficult and expensive to serialize the actual image data and send that over the network. Instead, you could store your sprites in a collection and send the collection index, or perhaps you could create a custom serializer.

Tip: FishNet should be able to automatically serialize many basic C# and Unity types as well as classes and structs made with serializable types.

The ExcludeSerialization Attribute

This attribute is part of the `FishNet.CodeGenerating` namespace, and may be used to prevent a field from being serialized by FishNet's automatic serializer and thus sent over the network when using your own types.

```
public class PlayerStats
{
    public float Health;
    public float MoveSpeed;

    /* In this example ControllerIndex is only used
     * for local multiplayer. This data does not need to be sent
     * over the network so it's been marked with ExcludeSerialization. */
    [ExcludeSerialization]
    public int ControllerIndex;
}
```

Features

The following are guides on the many features Fish-Networking provides.

InstanceFinder

There is a lot of useful information you can get from `NetworkBehaviours` but there may be some cases where your script does not inherit from `NetworkBehaviour`. This is where `InstanceFinder` can help you out.

`InstanceFinder` provides quick access to commonly needed references or information. Some examples are: `SceneManager`, `IsClientStarted`, `TimeManager`, and more.

```
public class MyButton : MonoBehaviour
{
    public Image ButtonImage;

    /* It wouldn't make sense to update the UI
    * color on the server since it will have
    * no graphical interface. To save performance
    * we're only going to update color if client.
    * However, since this is a MonoBehaviour class
    * you do not have access to base.IsClientStarted.
    * Instead the InstanceFinder may be used. */
    private void SetColor(Color c)
    {
        if (InstanceFinder.IsClientStarted)
            ButtonImage.color = c;
    }
}
```

Lag Compensation

Also known as collider rollback, this is the act of placing colliders back in time on the server to provide accurate raycast hit detection regardless of client latency.

Projectiles

[Image: untitled]

Which move over time can have lag hidden by accelerating the projectiles for the server and spectators, while allowing the firing client to show the projectile immediately.

Another bonus to this approach is you are not networking the projectile movement each tick, which saves tremendously on bandwidth and performance.

First the local client, or owning client, fires the projectile. The projectile is spawned locally, then the client tells the server to also fire the projectile. The MAX_PASSED_TIME constant is covered in the next code snippet.

```
/// <summary>
/// Projectile to spawn.
/// </summary>
[Tooltip("Projectile to spawn.")]
[SerializeField]
private PredictedProjectile _projectile;

/// <summary>
/// Maximum amount of passed time a projectile may have.
/// This ensures really laggy players won't be able to disrupt
/// other players by having the projectile speed up beyond
/// reason on their screens.
/// </summary>
private const float MAX_PASSED_TIME = 0.3f;

/// <summary>
/// Local client fires weapon.
/// </summary>
private void ClientFire()
{
    Vector3 position = transform.position;
    Vector3 direction = transform.forward;

    /* Spawn locally with 0f passed time.
     * Since this is the firing client
     * they do not need to accelerate/catch up
     * the projectile. */
    SpawnProjectile(position, direction, 0f);

    // Ask server to also fire passing in current Tick.
    ServerFire(position, direction, base.TimeManager.Tick);
}

/// <summary>
/// Spawns a projectile locally.
/// </summary>
private void SpawnProjectile(Vector3 position, Vector3 direction, float passedTime)
{
    PredictedProjectile pp = Instantiate(_projectile, position, Quaternion.identity);
    pp.Initialize(direction, passedTime);
}
```

When the server receives the fire request it will calculate how long it took the client to send the fire message

using the provided tick. Later on the projectile will be accelerated based on this time passed.

You will notice we aren't calling `Spawn` for the projectiles, that is because they aren't `NetworkObjects`, just regular game objects. Do not make the `PredictedProjectile` script a `NetworkBehaviour`, or give it any network components.

After the `passedTime` is calculated, spawn the projectile on the server, then tell spectators to also spawn the projectile; spectators being other clients.

```
/// <summary>
/// Fires on the server.
/// </summary>
/// <param name="position">Position to spawn projectile.</param>
/// <param name="direction">Direction to move projectile.</param>
/// <param name="tick">Tick when projectile was spawned on client.</param>
[ServerRpc]
private void ServerFire(Vector3 position, Vector3 direction, uint tick)
{
    /* You may want to validate position and direction here.
     * How this is done depends largely upon your game so it
     * won't be covered in this guide. */

    // Get passed time. Note the false for allow negative values.
    float passedTime = (float)base.TimeManager.TimePassed(tick, false);

    /* Cap passed time at half of constant value for the server.
     * In our example max passed time is 300ms, so server value
     * would be max 150ms. This means if it took a client longer
     * than 150ms to send the rpc to the server, the time would
     * be capped to 150ms. This might sound restrictive, but that would
     * mean the client would have roughly a 300ms ping; we do not want
     * to punish other players because a laggy client is firing. */
    passedTime = Mathf.Min(MAX_PASSED_TIME / 2f, passedTime);

    // Spawn on the server.
    SpawnProjectile(position, direction, passedTime);

    // Tell other clients to spawn the projectile.
    ObserversFire(position, direction, tick);
}
```

Observers use the same technique to calculate the passed time and spawn the projectile. There are a few small things to note.

First, this RPC is sent to everyone but the owner. The owner does not need to receive the RPC because it already spawned the projectile locally.

Second, the passed time calculation is not limited by half. This is to support the maximum possible passed time. This is not a requirement but is recommended.

```

/// <summary>
/// Fires on all clients but owner.
/// </summary>
[ObserversRpc(ExcludeOwner = true)]
private void ObserversFire(Vector3 position, Vector3 direction, uint tick)
{
    // Like on server get the time passed and cap it. Note the false for allow negative values.
    float passedTime = (float)base.TimeManager.TimePassed(tick, false);
    passedTime = Mathf.Min(MAX_PASSED_TIME, passedTime);

    // Spawn the projectile locally.
    SpawnProjectile(position, direction, passedTime);
}

```

With the projectile spawned all that's left is showing how to use the calculated passed time. You likely noticed the `SpawnProjectile` method was initializing the projectile with some values. Here's what that looks like:

```

/// <summary>
/// Direction to travel.
/// </summary>
private Vector3 _direction;

/// <summary>
/// Distance remaining to catch up. This is calculated from a passed time and move rate.
/// </summary>
private float _passedTime = 0f;

/// <summary>
/// In this example the projectile moves at a flat rate of 5f.
/// </summary>
private const float MOVE_RATE = 5f;

/// <summary>
/// Initializes this projectile.
/// </summary>
/// <param name="direction">Direction to travel.</param>
/// <param name="passedTime">How far in time this projectile is behind the prediction.</param>
public void Initialize(Vector3 direction, float passedTime)
{
    _direction = direction;
    _passedTime = passedTime;
}

```

After initializing with the specified passed time and direction all that's left to accelerate the projectile is a move method. The provided example is a very basic implementation of a move method, while also applying the acceleration.

If there is passed time to apply, then additional delta is added using the `passedTimeDelta` variable.

The percentage applied when assigning the step variable decides how fast your projectile will catch up to the predicted value. In this code I am using 8% of the passed time per Move call. Higher values will result in the projectile catching up faster.


```

/// <summary>
/// Move the projectile each frame. This would be called from Update.
/// </summary>
private void Move()
{
    // Frame delta, nothing unusual here.
    float delta = Time.deltaTime;

    // See if to add on additional delta to consume passed time.
    float passedTimeDelta = 0f;
    if (_passedTime > 0f)
    {
        /* Rather than use a flat catch up rate the
        * extra delta will be based on how much passed time
        * remains. This means the projectile will accelerate
        * faster at the beginning and slower at the end.
        * If a flat rate was used then the projectile
        * would accelerate at a constant rate, then abruptly
        * change to normal move rate. This is similar to using
        * a smooth damp. */

        /* Apply 8% of the step per frame. You can adjust
        * this number to whatever feels good. */
        float step = (_passedTime * 0.08f);
        _passedTime -= step;

        /* If the remaining time is less than half a delta then
        * just append it onto the step. The change won't be noticeable. */
        if (_passedTime <= (delta / 2f))
        {
            step += _passedTime;
            _passedTime = 0f;
        }

        passedTimeDelta = step;
    }

    // Move the projectile using moverate, delta, and passed time delta.
    transform.position += _direction * (MOVE_RATE * (delta + passedTimeDelta));
}

```

Very last, keep in mind these projectiles are fired locally and are not networked. Because of this you may want to perform different actions based on if client or server. The code below demonstrates what a collision event might look like.

```

/// <summary>
/// Handles collision events.
/// </summary>
private void OnCollisionEnter(Collision collision)
{
    /* These projectiles are instantiated locally, as in,
     * they are not networked. Because of this there is a very
     * small chance the occasional projectile may not align with
     * 100% accuracy. But, the differences are generally
     * insignificant and will not affect gameplay. */

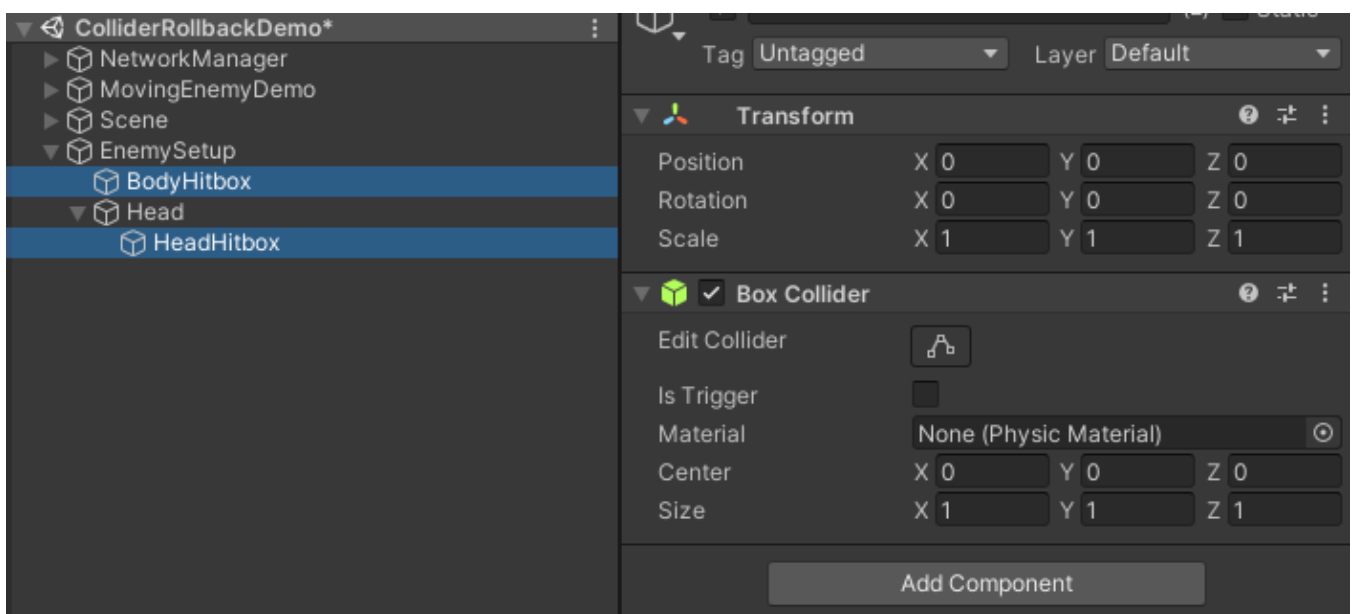
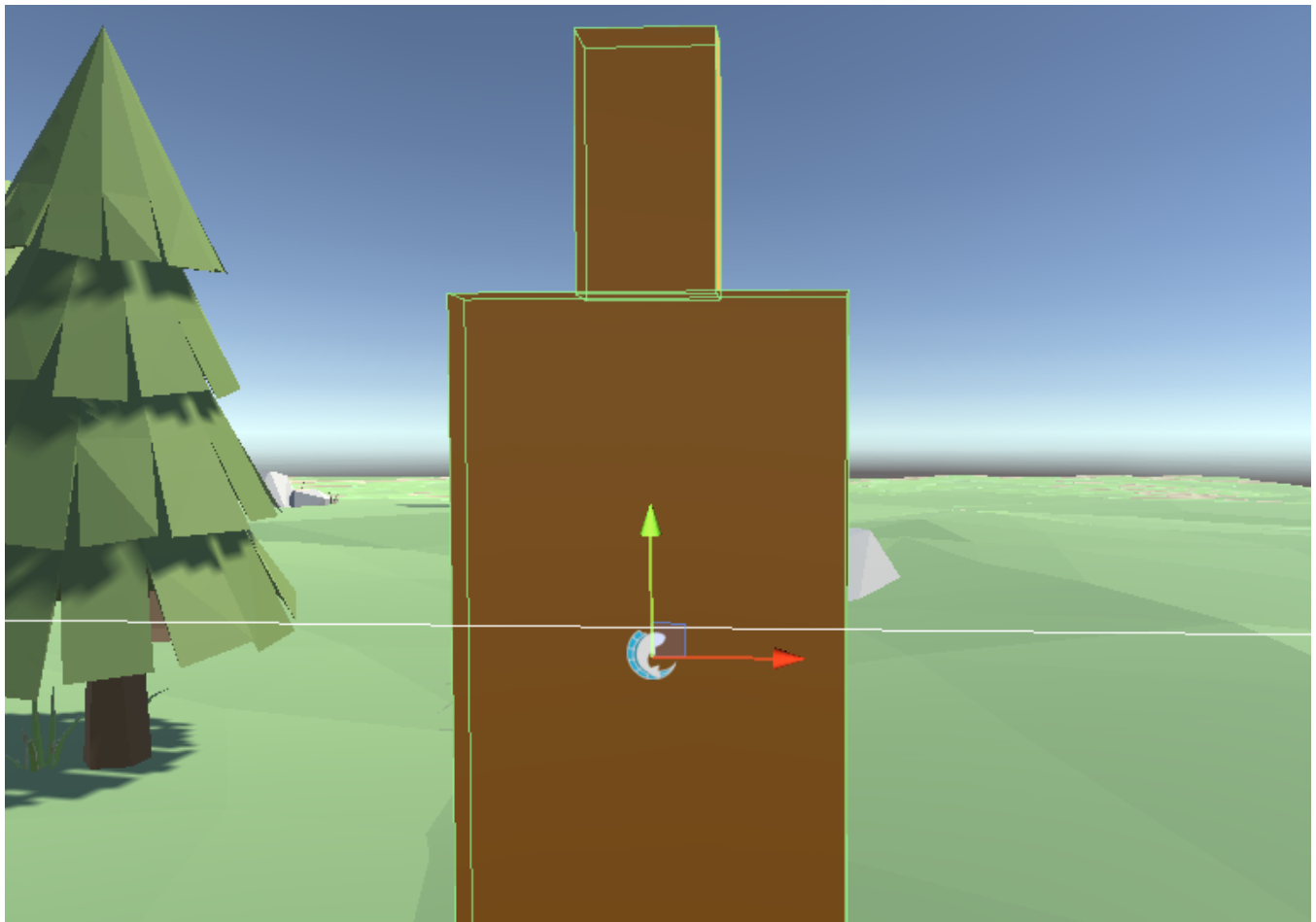
    // If client show visual effects, play impact audio.
    if (InstanceFinder.IsClient)
    {
        // Show VFX.
        // Play Audio.
    }

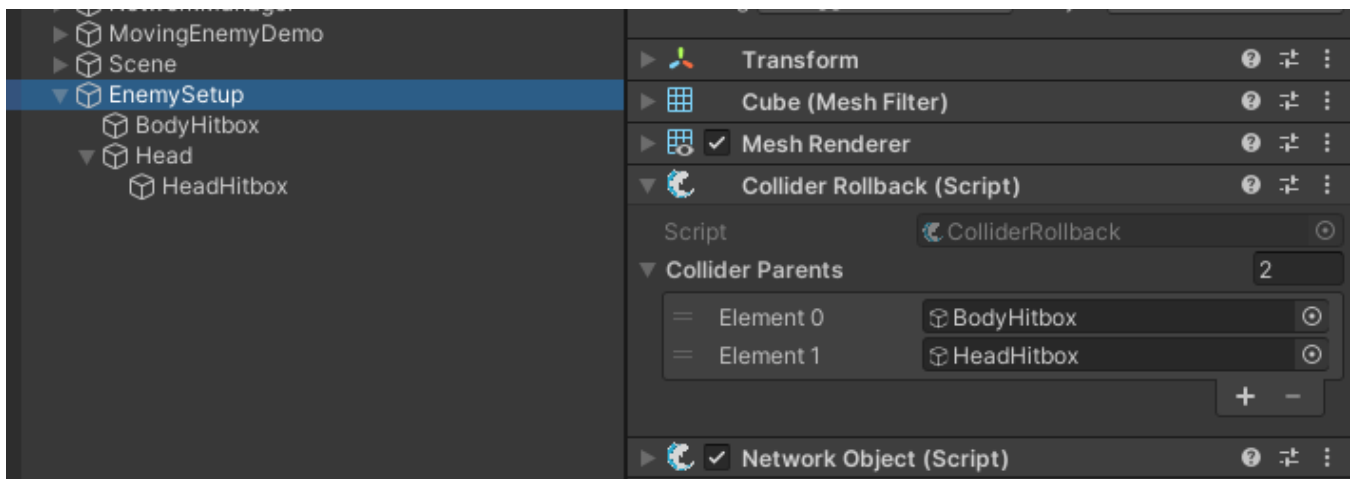
    // If server check to damage hit objects.
    if (InstanceFinder.IsServer)
    {
        PlayerShip ps = collision.gameObject.GetComponent<PlayerShip>();
        /* If a player ship was hit then remove 50 health.
         * The health value can be synchronized however you like,
         * such as a syncvar. */
        if (ps != null)
            ps.Health -= 50f;
    }

    // Destroy projectile (probably pool it instead).
    Destroy(gameObject);
}

```

Raycast





Fish-Networking Pro is required for raycast lag compensation. Over-time projectiles may use another approach and do not require Pro. See Projectiles for more information.

This feature can be setup easily in a few steps.

To begin I am entering the `ColliderRollbackDemo` scene included with Fish-Networking Pro. You do not need to open the scene, but if you wish to follow along visually you may.

Hitboxes

In my scene is an object named `EnemySetup`. The screen capture below shows two boxes on the 'enemy'. One for the body, and one for the head.

Each hitbox must be on a new object. Generally speaking, the hitbox would be a child of whichever component moves. With an actual humanoid rig, you would probably place the head collider on a child beneath the neck bone.

Notice the 'hitbox' objects are children of the moving component, and only have a collider on them. You may use any collider type.

When using a Tick Smoother, ensure that the hitboxes are placed on the "target" objects rather than smoothed objects (the object with the smoother component or one of its children). When a rollback occurs, the server automatically interpolates (lerps) hitboxes between positions, making local smoothing unnecessary. To maintain consistency, set the smoother's Flat Interpolation value to match the Additional Ticks configured in your Rollback Manager.

Next the `ColliderRollback` script must be added somewhere within your prefab or object. For the sake of simplicity I have mine on the root, with the `NetworkObject`.

Once added, specify each object you added as a collider within the **Collider Parents** collection.

To make things easier, place the suffix 'Hitbox' on all your hitbox objects. You can then lock the inspector on your `ColliderRollback` object, search Hitbox, and quickly drag all entries in at once.

You've completed all the steps required for setting up an object.

Rollback Manager

To utilize the lag compensation you must also add the `RollbackManager` script to your `NetworkManager` object. We have a section on the `RollbackManager` component to understand what each field does. Once you review the component settings we can move onto the code.

Performing a rollback

The RollbackManager must know how far back in time to place colliders to obtain accurate hit results. When your client is to fire their weapon you will want to gather the current PreciseTick and include it with your Fire RPC.

```
[Client]
private void Fire()
{
    // Use LastPacketTick to get the best tick alignment.
    PreciseTick pt = base.TimeManager.GetPreciseTick(TickType.LastPacketTick);
    // Call fire on the server.
    ServerFire(pt);
}

[ServerRpc]
private void ServerFire(PreciseTick pt)
{
    // Rollback using the precise tick sent in.
    // Using Physics for 3d rollback, Physics3D for 2d rollback.
    // Both physics types can be used at once.
    base.RollbackManager.Rollback(pt, RollbackManager.PhysicsType.Physics, base.IsOwner);

    // Perform your raycast normally.
    RaycastHit hit;
    if (Physics.Raycast(transform.position, transform.forward, out hit))
    {
    }

    // Return the colliders to their proper positions.
    base.RollbackManager.Return();
}
```

States

Sometimes need to be synchronized between clients and server to prevent desynchronization. This is generally only true when using client-side prediction. However, it is possible this guide could be used outside prediction so please keep that in mind when reading.

Ideally when using prediction the client should run all states locally as they would normally occur, just like the server. For example, if you were to drive a ship into the wall and want to apply a slow, the client and server would apply the state immediately when hitting the wall. In such a scenario the states would most likely exist in the replicate and reconcile data. However, there are cases when only a server is aware of a state, and if the server applies the state before the client does a desynchronization will occur. If this is the design you are intending, consider reworking your logic so clients and server run the same simulation instead; otherwise, continue reading!

A common state that requires synchronizing as example given is something that would impact movement. In this guide I'll be demonstrating on how to apply an EMP.

Since only the server is aware when the state is applied we will perform a check on the server, using the `base.IsServer` check. When the condition is satisfied as by our pretend `ApplyEmp()` method a RPC is sent.

```
private void TimeManager_OnTick()
{
    /* We're going to pretend there is
     * some logic here which happens only on the
     * server resulting in applying a EMP.
     *
     * The EMP is set to apply 50ms in the future.
     * However many ticks used are up to you. The goal is
     * to give clients a chance to get the emp tick
     * before that tick actually runs on their end.
     * This will have clients all run the EMP at roughly
     * the same time, keeping in mind there is sometimes a
     * small margin of error of 1 tick. */
    if (base.IsServer && ApplyEMP())
    {
        uint futureTicks = base.TimeManager.TimeToTicks(0.05f, TickRounding.RoundUp);
        ObserversSetEMPTick(base.TimeManager.LocalTick + futureTicks);
    }
}

// RunLocally is used to the server also sets the emp tick.
[ObserversRpc(RunLocally = true)]
private void ObserversSetEMPTick(uint serverTick)
{
    // Converts the server EMP tick to local tick for this client.
    _empStartTick = base.TimeManager.TickToLocalTick(serverTick);
    // Set end tick. In our example the EMP will last 1 second.
    _empEndTick = _empStartTick + base.TimeManager.TimeToTicks(1f, TickRounding.RoundUp);
}
```

When the RPC is received two fields are set, a tick when to start the EMP and when to end it.

Next in our replicate method if the clients or servers `LocalTick` is within that EMP duration we prevent any further motor movement. As mentioned this example uses prediction but you could apply this to an Update loop as well if not using prediction.

```

// Tick when to start emp.
private uint _empStartTick = uint.MaxValue;
// Tick when to end emp.
private uint _empEndTick = uint.MaxValue;

/* This is an example of a replicate data excluding the extra
 * parameters, given they have no context in this example. */
[Replicate]
private void Replicate(MotorData md)
{
    uint localTick = base.TimeManager.LocalTick;
    /* If localTick is between EMP range then return.
     * It's important to not reset the emp values because
     * we want these to be the same during replays. If you reset
     * values soon as the condition of exiting emp was satisfied
     * then emp would not be properly set during a replay.
     *
     * Note that this logic runs on the server and owner,
     * and if using prediction v2 can run on other clients
     * as well. */
    if (localTick >= _empStartTick && localTick <= _empEndTick)
    {
        /* Since under emp exit method early
         * to not process MotorData. In this example
         * we are using an EMP state, therefor the motor
         * will not work. You'll adjust this to whatever
         * your game needs. */
        return;
    }

    // Normal motor logic here.
}

```

Just like that the client is set to apply the EMP marginally in the future, and as will the server. Yes, this does create a very small delay on when the EMP or state begins but this is often considered acceptable to maintain a reliable simulation. In addition, also as said before, ideally the client will be running the same simulation as the server at all times which would let you avoid this sort of setup.

Network State Events

You can take advantage of numerous available events to stay informed about the current state of the network.

Note: This is not a comprehensive list of available events, but merely some of the most commonly used ones. To view all available ones check out the API pages.

Server Side Events

OnAuthenticationResult

Name: `ServerManager.OnAuthenticationResult` **Namespace:** `FishNet.Managing.Server`
Parameters: `NetworkConnection`, `bool`

This event is called once a client has either been authenticated or failed to authenticate. The parameters are the relevant `NetworkConnection`, and a Boolean representing if that client successfully authenticated or not. This event can be very useful to use to store the `NetworkConnection` of a client that has successfully connected to the server or to send some message to the newly connected client.

OnServerConnectionState

Name: `ServerManager.OnServerConnectionState` **Namespace:** `FishNet.Managing.Server`
Parameters: `ServerConnectionStateArgs`

This event is called when the server's state changes — in other words, immediately when the server starts or stops running. This event is very useful for performing network actions on the server such as loading initial scenes or spawning initial objects.

OnRemoteConnectionState

Name: `ServerManager.OnRemoteConnectionState` **Namespace:** `FishNet.Managing.Server`
Parameters: `NetworkConnection`, `RemoteConnectionStateArgs`

This event is called when a client's state changes with the server — in other words, immediately when a client connects or disconnects from the server. This event is very useful for detecting when a client has disconnected as well as for authentication scripts to handle sending data or checking if the server is full before authenticating a connection.

Client Side Events

OnAuthenticated

Name: `ClientManager.OnAuthenticated` **Namespace:** `FishNet.Managing.Client`

This event is called when the local client has successfully been authenticated with the FishNet server. The client will now have a Client ID and be added to the `ServerManager.Clients` and `ClientManager.Clients` collections. This is a good event to use to know when your client is fully connected to the server and ready to play the game.

OnClientConnectionState

Name: `ClientManager.OnClientConnectionState` **Namespace:** `FishNet.Managing.Client`
Parameters: `ClientConnectionStateArgs`

This event is called when the local client's state changes — in other words, immediately when the client makes contact with the Fish-Networking server or is disconnected from it. This event is useful for detecting

when you are disconnected from the server and when you initially connect. Authentication scripts will often use this event to send initial data to the server so that the server can authenticate the client.

Important: When this event is invoked the client hasn't yet been added to the `ClientManager.Clients` collection.

OnRemoteConnectionState

Name: `ClientManager.OnRemoteConnectionState` **Namespace:** `FishNet.Managing.Client`

Parameters: `RemoteConnectionStateArgs`

This event is called when a remote client's state changes. This event is useful for detecting when another player has connected or disconnected from the game.

Note: This is only available when using `ServerManager.ShareIds`.

Shared Events

OnClientLoadedStartScenes

Name: `SceneManager.OnClientLoadedStartScenes` **Namespace:** `FishNet.Managing.Scened`

Parameters: `NetworkConnection, bool`

Called when a client loads the initial scenes after connecting, for example any global networked scenes. Since this event will be called on the client who loaded the start scenes as well as the server, there is a Boolean parameter which will be `true` if the event was called as the server. This can be used on the host player to prevent logic in this method from running twice. This will invoke even if the `SceneManager` is not used when the client completes fully connecting to the server.

OnPreTick

Name: `TimeManager.OnPreTick` **Namespace:** `FishNet.Managing.Timing`

This event is called right before a network tick occurs, as well as before data is read.

OnTick

Name: `TimeManager.OnTick` **Namespace:** `FishNet.Managing.Timing`

This event is called when a network tick occurs. This can be useful for sending data to the server at a set rate to prevent sending data too frequently and flooding the connection.

OnPostTick

Name: `TimeManager.OnPostTick` **Namespace:** `FishNet.Managing.Timing`

This event is called just after a network tick occurs; physics would have already been simulated if using `PhysicsMode.TimeManager`.

Broadcasts

Allow you to send messages to one or more objects without them requiring a `NetworkObject` component. This could be useful for communicating between objects which are not necessarily networked, such as a chat system.

Like Remote Procedure Calls, broadcasts may be sent reliably or unreliably. Data using broadcasts can be sent either from the client to the server, or server to client(s). Serializers are automatically generated for Broadcasts as well.

Broadcasts must be structures, and implement `IBroadcast`. Below demonstrates what values a chat broadcast may possibly contain.

```
public struct ChatBroadcast : IBroadcast
{
    public string Username;
    public string Message;
    public Color FontColor;
}
```

Since broadcasts are not linked to objects, they must be sent using the `ServerManager`, or `ClientManager`. When sending to the server you will send using `ClientManager`, and when sending to clients, use `ServerManager`.

Tip: The following examples use `InstanceFinder` to get a reference to the `ClientManager` and `ServerManager`, but you can store a reference yourself, use the `NetworkManager`'s public references, or a `NetworkBehaviour`'s.

Here is an example of sending a chat message from a client to the server.

```
public void OnKeyDown_Enter(string text)
{
    // Client won't send their username, server will already know it.
    ChatBroadcast msg = new ChatBroadcast()
    {
        Message = text,
        FontColor = Color.white
    };
    InstanceFinder.ClientManager.Broadcast(msg);
}
```

Sending from the server to client(s) is done very much the same, but you are presented with more options. Here is an example of sending a broadcast to all clients which have visibility of a specific client. This establishes the idea that clientA sends a chat message to the server, and the server relays it to other clients which can see clientA. In this example clientA would also get the broadcast.

```

// When receiving a broadcast on the server, which connection
// sent the broadcast will always be available.
public void OnChatBroadcast(NetworkConnection conn, ChatBroadcast msg, Channel channel)
{
    // For the sake of simplicity we are using observers
    // on conn's first object.
    NetworkObject nob = conn.FirstObject;

    // The FirstObject can be null if the client
    // does not have any objects spawned.
    if (nob == null)
        return;

    // Populate the username field in the received msg.
    // Let us assume GetClientUsername actually does something.
    msg.Username = GetClientUsername(conn);

    // This will send the message to all Observers on nob,
    // and require those observers to be authenticated with the server.
    InstanceFinder.ServerManager.Broadcast(nob, msg, true);
}

```

Given broadcasts are not automatically received on the object they are sent from, you must specify what scripts or objects can receive a broadcast. As mentioned previously, this allows you to receive a broadcast on non-networked objects, but also enables you to receive the same broadcast on multiple objects.

While our example only utilizes one object, this feature could be useful for changing a large number of conditions in your game at once, such as turning off or on lights without having to make them each a networked object.

Listening for a broadcast is much like using events. Below demonstrates how the client will listen for data from the server.

```

private void OnEnable()
{
    // Begins listening for any ChatBroadcast from the server.
    // When one is received the OnChatBroadcast method will be
    // called with the broadcast data.
    InstanceFinder.ClientManager.RegisterBroadcast<ChatBroadcast>(OnChatBroadcast);
}

// When receiving on clients, broadcast callbacks will only have
// the message and channel.
private void OnChatBroadcast(ChatBroadcast msg, Channel channel)
{
    // Pretend to print to a chat window.
    Chat.Print(msg.Username, msg.Message, msg.FontColor);
}

private void OnDisable()
{
    // Like with events it is VERY important to unregister broadcasts
    // when the object is being destroyed (in this case disabled), or when
    // you no longer wish to receive the broadcasts on that object.
    InstanceFinder.ClientManager.UnregisterBroadcast<ChatBroadcast>(OnChatBroadcast);
}

```

As a reminder, a receiving method on the server was demonstrated above. The method signature looked

like this:

```
public void OnChatBroadcast(NetworkConnection conn, ChatBroadcast msg)
```

With that in mind, let's see how the server can listen for broadcasts from clients.

```
private void OnEnable()
{
    // Registering for the server is exactly the same as for clients.
    // Note there is an optional parameter not shown, requireAuthentication.
    // The value of requireAuthentication defaults to true.
    // Should a client send this broadcast without being authenticated
    // the server would kick them.
    InstanceFinder.ServerManager.RegisterBroadcast<ChatBroadcast>(OnChatBroadcast);
}

private void OnDisable()
{
    // There are no differences in unregistering.
    InstanceFinder.ServerManager.UnregisterBroadcast<ChatBroadcast>(OnChatBroadcast);
}
```

If you would like to view a working example of using Broadcast, view the `PasswordAuthenticator.cs` file within the `Demos/Authenticator/Scripts` folder.

Network Communication

Read more about the various methods you can use to communicate over the network!

- [Remote Procedure Calls](#)
- [SyncTypes](#)
- [Broadcasts](#)

Remote Procedure Calls

RPCs are a type of communication that are received on the same object they are sent from. The three types of RPCs are `ServerRpc`, `ObserversRpc`, and `TargetRpc`. RPCs are easy to use and are just like calling another method. Since Remote Procedure Calls are object bound, they must be called on scripts which inherit from `NetworkBehaviour`.

Fish-Net will generate serializers for your types automatically, including arrays and lists. In rare cases Fish-Net may not be able to create a serializer for you. When this occurs you must create a custom serializer.

Tip: Remote Procedure Call methods do not need to have prefixes as shown in the examples, but it's good practice to use them to keep your intentions known.

ServerRpc

A `ServerRpc` allows a client to run logic on the server. The client calls a `ServerRpc` method, and the data to execute that method is sent to the server. To use a `ServerRpc` the client must be active, and the method must have a `ServerRpc` attribute.

```
private void Update()
{
    // If owner and space bar is pressed.
    if (base.IsOwner && Input.GetKeyDown(KeyCode.Space))
        RpcSendChat("Hello world, from owner!");
}

[ServerRpc]
private void RpcSendChat(string msg)
{
    Debug.Log($"Received {msg} on the server.");
}
```

By default only the owner of an object may send a `ServerRpc`. It is however possible to allow `ServerRpcs` to be called by any client, owner or not. This is done by setting `RequireOwnership` to `false` in the `ServerRpc` attribute. You may optionally know which connection is calling the RPC by using a `NetworkConnection` type with a null default as the last parameter. The `NetworkConnection` parameter will automatically be populated with whichever client is calling the `ServerRpc`; you do not need to specify the connection.

```
[ServerRpc(RequireOwnership = false)]
private void RpcSendChat(string msg, NetworkConnection conn = null)
{
    Debug.Log($"Received {msg} on the server from connection {conn.ClientId}.");
}
```

ObserversRpc

`ObserversRpc` allows the server to run logic on clients. Only observing clients will get and run the logic. Observers are set by using the `Network Observer` component. To use `ObserversRpc` add the `ObserversRpc` attribute to a method.

```
private void FixedUpdate()
{
    RpcSetNumber(Time.frameCount);
}

[ObserversRpc]
private void RpcSetNumber(int next)
{
    Debug.Log($"Received number {next} from the server.");
}
```

In some instances you might want the owner of an object to ignore received `ObserversRpc`. To accomplish this set `ExcludeOwner` to `true` on the `ObserversRpc` attribute.

```
[ObserversRpc(ExcludeOwner = true)]
private void RpcSetNumber(int next)
{
    // This won't run on owner.
    Debug.Log($"Received number {next} from the server.");
}
```

You may also have the latest values sent through an `ObserversRpc` automatically send to new clients. This can be useful for values that only change through RPCs. Doing so is done by setting `BufferLast` to `true` on the `ObserversRpc` attribute. The example below will not send to owner, and will also update new joining clients.

```
[ObserversRpc(ExcludeOwner = true, BufferLast = true)]
private void RpcSetNumber(int next)
{
    // This won't run on owner and will send to new clients.
    Debug.Log($"Received number {next} from the server.");
}
```

TargetRpc

Lastly is `TargetRpc`. This RPC is used to run logic on a specific client. You can implement this feature by adding the `TargetRpc` attribute. When sending a `TargetRpc` the first parameter of your method must always be a `NetworkConnection`; this is the connection the data goes to.

```
private void UpdateOwnersAmmo()
{
    /* Even though this example passes in owner, you can send to
    * any connection that is an observer. */
    RpcSetAmmo(base.Owner, 10);
}

[TargetRpc]
private void RpcSetAmmo(NetworkConnection conn, int newAmmo)
{
    // This might be something you only want the owner to be aware of.
    _myAmmo = newAmmo;
}
```

Multi-Purpose Rpc

It is possible to have a single method be both a `TargetRpc` and an `ObserversRpc`. This can be very useful if you sometimes want to send an RPC to all observers, or a single individual. A chat message could

be an example of this.

```
[ObserversRpc][TargetRpc]
private void DisplayChat(NetworkConnection target, string sender, string message)
{
    Debug.Log($"{sender}: {message}."); // Display a message from sender.
}

[Server]
private void SendChatMessage()
{
    // Send to only the owner.
    DisplayChat(base.Owner, "Bob", "Hello world");

    // Send to everyone.
    DisplayChat(null, "Bob", "Hello world");
}
```

Channels

Remote procedure calls can be sent as reliable or unreliable. To utilize this feature add a `Channel` type as the last parameter to your RPC. You may also use your added `Channel` parameter to know which channel the data arrived on.

```
private bool _reliable;

private void FixedUpdate()
{
    // Reliable or unreliable can be switched at runtime!
    _reliable = !_reliable;
    Channel channel = (_reliable) ? Channel.Reliable : Channel.Unreliable;
    RpcTest("Anything", channel);
}

/* This example uses ServerRpc, but any RPC will work.
 * Although this example shows a default value for the channel,
 * you do not need to include it. */
[ServerRpc]
private void RpcTest(string txt, Channel channel = Channel.Reliable)
{
    if (channel == Channel.Reliable)
        Debug.Log("Message received! I never doubted you.");
    else if (channel == Channel.Unreliable)
        Debug.Log($"Glad you got here, I wasn't sure you'd make it.");
}
```

If your RPC is a `ServerRpc` and you are using a `NetworkConnection` with a null default as the last parameter then the `Channel` parameter must be added second to last, just before the `NetworkConnection` parameter.


```

/* Example of using Channel with a ServerRpc that
 * also provides the calling connection. */
[ServerRpc(RequireOwnership = false)]
private void RpcTest(string txt, Channel channel = Channel.Reliable, NetworkConnection sender = null)
{
    Debug.Log($"Received on channel {channel} from {sender.ClientId}.");
}

```

RunLocally

All RPC types may use the `RunLocally` field within the RPC attribute. When `RunLocally` is true the method will send, but also run on the calling side. For example, if a client calls a `ServerRpc` with `RunLocally` true, the `ServerRpc` will be put in the client's buffer to send, then the client will run the method logic locally.

```

// Keeping in mind all RPC types can use RunLocally.
[ServerRpc(RunLocally = true)]
private void RpcTest()
{
    // This debug will print on the server and the client calling the RPC.
    Debug.Log("Rpc Test!");
}

```

Note: Only transports which support unreliable messages may send RPCs as unreliable. If you attempt to send an RPC as unreliable when the transport does not support it the RPC will default to reliable.

DataLength

If you have intentions to send very large amounts of data you may specify the maximum potential size of your data in the RPC attribute. Doing so will use a reserved serializer to prevent resizing and allocations, increasing performance. Every RPC type supports this feature.

```

// This implies we know the data will never be larger than 3500 bytes.
// If the data is larger then a resize will occur resulting in garbage collection.
[ServerRpc(DataLength = 3500)]
private void ServerSendBytes(byte[] data) {}

```

Custom SyncType

With a customized SyncType you can decide how and what data to synchronize, and make optimizations as you see fit.

For example: if you have a data container with many variables you probably don't want to send the entire container when you change it, as a SyncVar would. By making a custom SyncType you can customize the behavior entirely; this is how other SyncTypes work.

```
/* If one of these values change you
 * probably don't want to send the
 * entire container. A custom SyncType
 * is perfect for only sending what is changed. */
[System.Serializable]
public struct MyContainer
{
    public int LeftArmHealth;
    public int RightArmHealth;
    public int LeftLegHealth;
    public int RightLegHealth;
}
```

Custom SyncTypes follow the same rules as other SyncTypes. Internally other SyncTypes inherit from SyncBase, and your type must as well. In addition, you must implement the ICustomSync interface.

```
public class SyncMyContainer : SyncBase, ICustomSync
{
    /* If you intend to serialize your type
     * as a whole at any point in your custom
     * SyncType and would like the automatic
     * serializers to include it then use
     * GetSerializedType() to return the type.
     * In this case, the type is MyContainer.
     * If you do not need a serializer generated
     * you may return null. */
    public object GetSerializedType() => typeof(MyContainer);
}

public class YourClass
{
    private readonly SyncMyContainer _myContainer = new();
}
```

Given how flexible a custom SyncType may be there is not a one-size-fits-all example. You may view several custom examples within your FishNet import under FishNet/Demos/CustomSyncType.

Customizing Behavior

There are settings and attributes unique to SyncTypes which allow various ways of customizing your SyncType.

SyncTypeSettings

SyncTypeSettings can be initialized with any SyncType to define the default settings of your SyncType.

```
// Custom settings are optional.
// This is an example of declaring a SyncVar without custom settings.
private readonly SyncVar<int> _myInt = new();

// Each SyncType has a different constructor to take settings.
// Here is an example for SyncVars. This will demonstrate how to use
// the unreliable channel for SyncVars, and send the value upon any change.
// There are many different ways to create SyncTypeSettings; you can even
// make a const settings and initialize with that!
private readonly SyncVar<int> _myInt = new(new SyncTypeSettings(0f, Channel.Unreliable));
```

Settings can also be changed at runtime. This can be very useful to change behavior based on your game mechanics and needs, or to even slow down send rate as your player count grows.

```
// This example shows in Awake but this code
// can be used literally anywhere.
private void Awake()
{
    // You can change all settings at once.
    _myInt.UpdateSettings(new SyncTypeSettings(...));

    // Or update only specific things, such as SendRate.
    // Change send rate to once per second.
    _myInt.UpdateSendRate(1f);
}
```

Showing in the Inspector

SyncTypes can also be shown in the inspector.

You must first make sure your type is marked as serializable if a container; this is a Unity requirement.

```
// SyncTypes can be virtually any data type. This example
// shows a container to demonstrate the serializable attribute.
[System.Serializable]
public struct MyType { }
```

Next the SyncType must not use the readonly indicator. We require the readonly indicator by default to emphasize you should not initialize your SyncType at runtime.

Below is an example of what **NOT** to do:

```

private SyncVar<int> _myInt = new();

private void Awake()
{
    // This would result in errors at runtime.
    // Do not make a SyncType into a new instance
    _myInt = new();

    // Do not set a SyncType to another instance.
    _myInt = _someOtherDeclaredSyncVar;
}

```

The code above will actually prevent compilation in the editor as our code generators will detect you did not include the readonly indicator. To remove the readonly indicator you must also add the [AllowMutableSyncType] attribute above your SyncType.

```

// This will work and show your SyncType in the inspector!
[AllowMutableSyncType]
[SerializeField] // Be sure to use the SerializeField attribute if not public.
private SyncVar<int> _myInt = new();

```

SyncTypes

SyncTypes are FishNet's built-in mechanism for automatically synchronizing state from the server to clients. They handle serialization and delta compression transparently, so you can declare a field and have it replicate without writing any custom RPC code. This section covers all available SyncTypes — SyncVar, SyncList, SyncHashSet, SyncDictionary, SyncTimer, SyncStopwatch, and Custom SyncTypes — as well as how to customize their behavior.

Syncdictionary

Is an easy way to keep a Dictionary collection automatically synchronized over the network.

SyncDictionary supports all the functionality a normal dictionary would, just as SyncList supports List abilities.

Callbacks for SyncDictionary are similar to SyncList. And like other synchronization types, changes are set immediately before the callback occurs.

```
private readonly SyncDictionary<NetworkConnection, string> _playerNames = new();

private void Awake()
{
    _playerNames.OnChange += _playerNames_OnChange;
}

// SyncDictionaries also include the asServer parameter.
private void _playerNames_OnChange(SyncDictionaryOperation op, NetworkConnection key, string value, bool asServer)
{
    /* Key will be provided for
     * Add, Remove, and Set. */
    switch (op)
    {
        // Adds key with value.
        case SyncDictionaryOperation.Add:
            break;

        // Removes key.
        case SyncDictionaryOperation.Remove:
            break;

        // Sets key to a new value.
        case SyncDictionaryOperation.Set:
            break;

        // Clears the dictionary.
        case SyncDictionaryOperation.Clear:
            break;

        // Like SyncList, indicates all operations are complete.
        case SyncDictionaryOperation.Complete:
            break;
    }
}
```

Warning: If you are using this SyncType with a container, such as a class or structure, and want to modify values within that container, you must set the value dirty. See the example below.

```

[System.Serializable]
private struct MyContainer
{
    public string PlayerName;
    public int Level;
}

private readonly SyncDictionary<int, MyContainer> _containers = new();

private void Awake()
{
    MyContainer mc = new MyContainer { Level = 5 };
    _containers[2] = mc;
}

[Server]
private void ModifyContainer()
{
    MyContainer mc = _containers[2];

    // This will change the value locally but it will not synchronize to clients.
    mc.Level = 10;

    // You may re-apply the value to the dictionary.
    _containers[2] = mc;

    // Or set dirty on the value or key. Using the key is often more performant.
    _containers.Dirty(2);
}

```

SyncHashSet

Is an easy way to keep a HashSet collection automatically synchronized over the network.

Callbacks for SyncHashSet are similar to SyncList. And like other synchronization types, changes are set immediately before the callback occurs.


```

private readonly SyncHashSet<int> _myCollection = new SyncHashSet<int>();

private void Awake()
{
    _myCollection.OnChange += _myCollection_OnChange;
}

private void FixedUpdate()
{
    // You can modify a SyncHashSet as you would any other HashSet.
    _myCollection.Add(Time.frameCount);
}

/* Like SyncVars the callback offers an asServer option
 * to indicate if the callback is occurring on the server
 * or the client. As SyncVars do, changes have already been
 * made to the collection before the callback occurs. */
private void _myCollection_OnChange(SyncHashSetOperation op, int item, bool asServer)
{
    switch (op)
    {
        /* An object was added to the HashSet. Item is
         * the added object. */
        case SyncHashSetOperation.Add:
            break;

        /* An object has been removed from the HashSet. Item
         * is the removed object. */
        case SyncHashSetOperation.Remove:
            break;

        /* The HashSet has been cleared.
         * Item will be default. */
        case SyncHashSetOperation.Clear:
            break;

        /* An entry in the HashSet has been updated.
         * When this occurs the item is removed
         * and added. Item will be the new value.
         * Item will likely need a custom comparer
         * for this to function properly. */
        case SyncHashSetOperation.Update:
            break;

        /* When Complete calls, all changes have been
         * made to the collection. You may use this
         * to refresh information in relation to
         * the changes, rather than doing so
         * after every entry change. All values are
         * default for this operation. */
        case SyncHashSetOperation.Complete:
            break;
    }
}

```

Warning: If you are using this SyncType with a container, such as a class or structure, and want to modify values within that container, you must set the value dirty. See the example below.

```

[System.Serializable]
private struct MyContainer
{
    public string PlayerName;
    public int Level;
}

private readonly SyncHashSet<MyContainer> _containers = new();
private MyContainer _containerReference = new();

private void Awake()
{
    _containerReference.Level = 5;
    _containers.Add(_containerReference);
}

[Server]
private void ModifyPlayer()
{
    // This will change the value locally but it will not synchronize to clients.
    _containerReference.Level = 10;

    // The value must be set dirty to force a synchronization.
    _containers.Dirty(_containerReference);
}

```

SyncList

Is an easy way to keep a List collection automatically synchronized over the network.

Using a SyncList is done the same as with a normal List.

Network callbacks on SyncList do have a little more information than SyncVars. Other non-SyncVar SyncTypes also have their own unique callbacks. The example below demonstrates a SyncList callback.

```

private readonly SyncList<int> _myCollection = new();

private void Awake()
{
    /* Listening to SyncList callbacks are a
     * little different from SyncVars. */
    _myCollection.OnChange += _myCollection_OnChange;
}

private void Update()
{
    // You can modify a SyncList as you would any other list.
    _myCollection.Add(10);
    _myCollection.RemoveAt(0);
    // etc.
}

/* Like SyncVars the callback offers an asServer option
 * to indicate if the callback is occurring on the server
 * or the client. As SyncVars do, changes have already been
 * made to the collection before the callback occurs. */
private void _myCollection_OnChange(SyncListOperation op, int index, int oldItem, int newItem, bool asServer)
{
    switch (op)
    {
        /* An object was added to the list. Index
         * will be where it was added, which will be the end
         * of the list, while newItem is the value added. */
        case SyncListOperation.Add:
            break;

        /* An object was removed from the list. Index
         * is from where the object was removed. oldItem
         * will contain the removed item. */
        case SyncListOperation.RemoveAt:
            break;

        /* An object was inserted into the list. Index
         * is where the object was inserted. newItem
         * contains the item inserted. */
        case SyncListOperation.Insert:
            break;

        /* An object replaced another. Index
         * is where the object was replaced. oldItem
         * is the item that was replaced, while
         * newItem is the item which now has its place. */
        case SyncListOperation.Set:
            break;

        /* All objects have been cleared. Index, oldValue,
         * and newValue are default. */
        case SyncListOperation.Clear:
            break;

        /* When Complete calls, all changes have been

```

```

        * made to the collection. You may use this
        * to refresh information in relation to
        * the list changes, rather than doing so
        * after every entry change. Like Clear,
        * Index, oldItem, and newItem are all default. */
case SyncListOperation.Complete:
    break;
    }
}

```

Warning: If you are using this SyncType with a container, such as a class, and want to modify values within that container, you must set the value dirty. See the example below.

```

private class MyClass
{
    public string PlayerName;
    public int Level;
}

private readonly SyncList<MyClass> _players = new SyncList<MyClass>();

// Call dirty on an index after modifying an entry's field to force a synchronize.
[Server]
private void ModifyPlayer()
{
    _players[0].Level = 10;
    // Dirty the 0 index.
    _players.Dirty(0);
}

```

Structures cannot have their values modified when they reside within a collection. You must instead create a local variable for the collection index you wish to modify, change values on the local copy, then set the local copy back into the collection.

```

[System.Serializable]
private struct MyStruct
{
    public string PlayerName;
    public int Level;
}

private readonly SyncList<MyStruct> _players = new();

[Server]
private void ModifyPlayer()
{
    MyStruct ms = _players[0];
    ms.Level = 10;
    _players[0] = ms;
}

```

Syncstopwatch

Provides an efficient way to synchronize a stopwatch between server and clients.

Like SyncTimer, SyncStopwatch only updates state changes such as start or stopping, rather than each individual delta change.

```
private readonly SyncStopwatch _timePassed = new();
```

Making changes to the stopwatch is very simple, and like other SyncTypes must be done on the server.

```
// All of the actions below are automatically synchronized.

/* Starts the stopwatch while optionally sending a stop
 * message first if stopwatch was already running.
 * If the stopwatch was previously running the time passed
 * would be reset.
 *
 * This would invoke callbacks indicating a stopwatch
 * had stopped, then started again. */
_timePassed.StartStopwatch(true);

/* Pauses the stopwatch and optionally sends the current
 * timer value as it is on the server. */
_timePassed.PauseStopwatch(false);

// Unpauses the current stopwatch.
_timePassed.UnpauseStopwatch();

/* Ends the stopwatch while optionally sending the
 * current value to clients, as it was during the stop. */
_timePassed.StopStopwatch(false);
```

Updating and reading the stopwatch value is much like you would a normal float value.

```
private void Update()
{
    /* Like SyncTimer, SyncStopwatch must be updated
     * with a delta on both server and client. Do not
     * update the delta twice if clientHost. You can update
     * the delta anywhere in your code. */
    _timePassed.Update(Time.deltaTime);

    // Access the current time passed.
    Debug.Log(_timePassed.Elapsed);

    /* You may also see if the stopwatch is paused before
     * accessing elapsed. */
    if (!_timePassed.Paused)
        Debug.Log(_timePassed.Elapsed);
}
```

Like other SyncTypes, you can subscribe to change events for the SyncStopwatch.

```

private void Awake()
{
    _timePassed.OnChange += _timePassed_OnChange;
}

private void OnDestroy()
{
    _timePassed.OnChange -= _timePassed_OnChange;
}

private void _timePassed_OnChange(SyncStopwatchOperation op, float prev, bool asServer)
{
    /* Like all SyncType callbacks, asServer is true if the callback
     * is occurring on the server side, false if on the client side. */

    // Here is an example of performing logic only if starting or stopping.
    if (op == SyncStopwatchOperation.Start || op == SyncStopwatchOperation.Stop)
    {
        // Do logic.
    }

    /* prev, our float, indicates the value of the stopwatch prior to
     * the operation. Remember that you can get the current value using
     * _timePassed.Elapsed */
}

```

SyncTimer

Provides an efficient way to synchronize a timer between server and clients.

Unlike SyncVars, a SyncTimer only updates state changes such as start or stopping, rather than each individual delta change.

SyncTimer is a Custom SyncType, and is declared like any other SyncType.

```
private readonly SyncTimer _timeRemaining = new SyncTimer();
```

Making changes to the timer is very simple, and like other SyncTypes must be done on the server.

```
// All of the actions below are automatically synchronized.

/* Starts the timer with 5 seconds on it.
 *
 * The optional boolean argument will also send
 * a stop event before starting a new timer, only if
 * the previous timer is still running.
 *
 * EG: if a timer was started with 5 seconds and
 * you start a new timer with 2 seconds remaining
 * a StopTimer will be sent with the remaining time
 * of 2 seconds before the timer is started again at
 * 5 seconds. */
_timeRemaining.StartTimer(5f, true);

/* Pauses the timer and optionally sends the current
 * timer value as it is on the server. */
_timeRemaining.PauseTimer(false);

// Unpauses the current timer.
_timeRemaining.UnpauseTimer();

/* Stops the timer early and optionally sends the
 * current timer value as it is on the server. */
_timeRemaining.StopTimer(false);
```

Updating and reading the timer value is much like you would a normal float value.


```

private void Update()
{
    /* Timers must be updated both on the server
    * and clients. This only needs
    * to be done on either/or if clientHost. The timer
    * may be updated in any method. */
    _timeRemaining.Update();

    /* Access the current time remaining. This is how
    * you can utilize the current timer value. When a timer
    * is complete the remaining value will be 0f. */
    Debug.Log(_timeRemaining.Remaining);

    /* You may also see if the timer is paused before
    * accessing time remaining. */
    if (!_timeRemaining.Paused)
        Debug.Log(_timeRemaining.Remaining);
}

```

Like other SyncTypes, you can subscribe to change events for the SyncTimer.

```

private void Awake()
{
    _timeRemaining.OnChange += _timeRemaining_OnChange;
}

private void OnDestroy()
{
    _timeRemaining.OnChange -= _timeRemaining_OnChange;
}

private void _timeRemaining_OnChange(SyncTimerOperation op, float prev, float next, bool asServer)
{
    /* Like all SyncType callbacks, asServer is true if the callback
     * is occurring on the server side, false if on the client side. */

    // Timer has been started with initial values.
    if (op == SyncTimerOperation.Start)
        Debug.Log($"The timer was started with {next} seconds.");

    // Timer has been paused.
    else if (op == SyncTimerOperation.Pause)
        Debug.Log($"The timer was paused.");

    // Timer has been paused and latest server values were sent.
    else if (op == SyncTimerOperation.PauseUpdated)
        Debug.Log($"The timer was paused and remaining time has been updated to {next} seconds.");

    // Timer was unpaused.
    else if (op == SyncTimerOperation.Unpause)
        Debug.Log($"The timer was unpaused.");

    // Timer has been manually stopped.
    else if (op == SyncTimerOperation.Stop)
        Debug.Log($"The timer has been stopped and is no longer running.");

    /* Timer has been manually stopped.
     *
     * When StopUpdated is called, Previous will contain the remaining time
     * prior to being stopped as it is locally. Next will contain the remaining
     * time prior to being stopped as it was on the server. These values
     * often align but the information is provided for your potential needs.
     *
     * When the server starts a new timer while one is already active, and chooses
     * to also send a stop update using the StartTimer(float,bool) option, a
     * StopUpdated is also sent to know previous timer values before starting a new timer.
     */
    else if (op == SyncTimerOperation.StopUpdated)
        Debug.Log($"The timer was stopped at value {next}, previous value was {prev}.");

    // A timer has reached 0f.
    else if (op == SyncTimerOperation.Finished)
        Debug.Log($"The timer has completed!");

    // Complete occurs after all change events are processed.
    else if (op == SyncTimerOperation.Complete)

```

```
    Debug.Log("All timer callbacks have completed for this tick.");  
}
```

Syncvar

SyncVars are the most simple way to automatically synchronize a single variable over the network.

SyncVars are used to synchronize a single field. Your field can be virtually anything: a value type, struct, or class. To utilize a SyncVar you must implement your type as a SyncVar class.

```
public class YourClass : NetworkBehaviour
{
    private readonly SyncVar<float> _health = new SyncVar<float>();
}

/* Any time _health is changed on the server the
 * new value will be sent to clients. */
```

SyncTypes can also be customized with additional options by using the `UpdateSettings` method within your declared SyncTypes, or using the initializer of your SyncType. These options include being notified when the value changes, changing how often the SyncType will synchronize, and more.

Below is a demonstration on sending SyncTypes at a longer interval of at most every 1f, and being notified of when the value changes.

```
private readonly SyncVar<float> _health = new SyncVar<float>(new SyncTypeSettings(1f));

private void Awake()
{
    _health.OnChange += on_health;
}

// This is called when _health changes, for server and clients.
private void on_health(float prev, float next, bool asServer)
{
    /* Each callback for SyncVars must contain a parameter
     * for the previous value, the next value, and asServer.
     * The previous value will contain the value before the
     * change, while next contains the value after the change.
     * By the time the callback occurs the next value had
     * already been set to the field, eg: _health.
     * asServer indicates if the callback is occurring on the
     * server or on the client. Sometimes you may want to run
     * logic only on the server, or client. The asServer
     * allows you to make this distinction. */
}
```

Another common request is achieving client-side SyncVar values. This may be achieved by using a `ServerRpc`.

Important: When modifying SyncVars using client-side, a RPC is sent with every set. If your SyncVar value will change frequently consider limiting how often you set the value client-side to reduce bandwidth usage.

```
// A typical server-side SyncVar.
public readonly SyncVar<string> Name = new SyncVar<string>();

// Create a ServerRpc to allow owner to update the value on the server.
[ServerRpc]
private void SetName(string value) => Name.Value = value;
```

When using client-side SyncVars you may want to consider `ExcludeOwner` in the `SyncVar` `ReadPermissions` to prevent owners from receiving their own updates. In addition, using `WritePermission.ClientUnsynchronized` will allow the client to set the value locally. Lastly, `RunLocally` as `true` in the `ServerRpc` will execute the RPC code on both the sender (client) and the server.

```
// Attributes shown in previous examples can stack but they were removed
// here for simplicity.
private readonly SyncVar<string> Name = new SyncVar<string>(new SyncTypeSettings(
    WritePermission.ClientUnsynchronized, ReadPermission.ExcludeOwner));

// Create a ServerRpc to allow owner to update the value on the server.
// RunLocally = true so the calling client also sets the value locally.
[ServerRpc(RunLocally = true)]
private void SetName(string value) => Name.Value = value;
```

Synchronizing

When changes are made to a SyncType, only the changes are sent. For example, if you have a SyncList of 10 values and add another, only the just added entry will be sent.

Any changes made to SyncTypes in Awake will be performed on server and client without requiring synchronization. This is a great opportunity to add to lists or dictionaries. If you wish values to be synchronized after initializing use OnStartServer.

Important: SyncVar values will be reset before OnDestroy runs on the object, so you won't be able to get them in that method.

By altering the SendRate, you can slow down how often SyncTypes can at most often sync their changes. This can be useful if you make several changes in quick succession and only want the latest change to be sent every couple of seconds, for example. The values are per second, and setting a SendRate of 0f will allow SyncTypes to send changes as often as possible, which is every network tick if your game is running on a non-variable timing mode.

SyncTypes will also automatically generate serializers for custom types.

Note: When SyncTypes are set to use reliable messaging (the default), their changes will arrive on clients in the reverse order that they are defined inside the script. This is due to the way the code generation works.

Host Client Limitations

There is a small limitation with all SyncTypes when running both the client and server in a single build.

While as the host client, the previous value (when applicable) in callbacks will be the current value or null if the asServer parameter is false. This is mostly noticed in SyncVars:

```
// This example assumes you are acting as the host client.
// We will imagine previous value was 10, and next is 20.
private void _mySyncVar_OnChange(int prev, int next, bool asServer)
{
    // If asServer is true then the prev value will correctly be 10,
    // with the next being 20.
    // If asServer is false then the prev value will be 20, as well the
    // next being 20. This is because the value has already been changed
    // on the server, and the previous value is no longer stored.
    DoSomething(prev, next);
}
```

There is a fairly simple way to accommodate this scenario if you plan on using a host client in your game:

```
// This example assumes you are acting as the host client.
// We will imagine previous value was 10, and next is 20.
private void _mySyncVar_OnChange(int prev, int next, bool asServer)
{
    // Only run the logic using the previous value if being called
    // with asServer true, or if only the client is started.
    if (asServer || IsClientOnlyStarted)
        DoSomething(prev, next);
}
```

Networked GameObjects and Scripts

Guides on working with NetworkObjects and NetworkBehaviour scripts, including spawning and despawning objects over the network.

Network Behaviour Guides

[Image: untitled]

[Image: NetworkBehaviour callback flow diagram]

When inheriting from `NetworkBehaviour` you are indicating that your script will utilize the network in some way. Once a `NetworkBehaviour` script is added to an object the `NetworkObject` component will automatically be attached.

`NetworkBehaviours` are essential for Remote Procedure Calls, Synchronizing, and having access to vital network information.

Properties

There are several public properties available within `NetworkBehaviour`, many of which you will use regularly. Most of the properties are only available after the object has been initialized. See the callbacks section for more information on initialization.

Commonly used properties are:

- `IsClientInitialized` — will be true if acting as a client and object is network initialized.
- `IsServerInitialized` — will be true if acting as the server and object is network initialized.
- `IsOwner` — is true if you are a client, and the owner of the object.
- `HasAuthority` — is true if you are the owner, or if server and there is no owner.

To view all properties please visit the `NetworkBehaviour` API.

Tip: When accessing a `NetworkBehaviour` property or method consider using the base keyword to show your intentions. For example, `base.IsOwner`.

Callbacks

Tip: You do not need to include the `base.XYZ()` calls when implementing callbacks.

Like properties, there are a large number of available methods. The methods you will be using commonly are known as callbacks. If you are interested in all available methods view the methods section under API.

Most callbacks have a server and client version, and each has great potential. To use a callback you must inherit from `NetworkBehaviour` and override the callback you wish to use.

```
// Example of implementing an override.
public override void OnStartServer()
{
    // This is run when the server initializes the object.
}
```

The execution order of spawn related callbacks is constant, but some callbacks may occur at any time.

Note: Unity Awake and `OnEnable` callbacks will always occur before any network activity.

OnStartNetwork

In some instances you will need to initialize for both server and client. You can save some code and time by using `OnStartNetwork` instead of the `OnStart` for Client and Server. It's important to remember that `OnStartNetwork` will only call once, even if you are `clientHost`.


```

public override void OnStartNetwork()
{
    /* If you wish to check for ownership inside
     * this method do not use base.IsOwner, use
     * the code below instead. This difference exists
     * to support a clientHost condition. */
    if (base.Owner.IsLocalClient)
        SetupCamera();
}

```

OnStartServer

```

public override void OnStartServer()
{
    /* This callback is performed first. It occurs
     * when the object is initialized on the server.
     * Once called Owner, ObjectId, and much more are
     * already set.
     * OnStartServer is best used for initializing
     * server aspects of your script, such as getting
     * components only the server would need.
     * It can also be useful for setting values based on
     * the current state of your game. If you change synchronized
     * values within this method, such as SyncTypes,
     * those changes will be delivered to clients
     * when this object spawns for them.
     * Eg: perhaps you want to set a player's name, which is
     * a SyncVar. You can do that here and it will be set
     * for clients when the object spawns on their side. */

    /* When using OnStartServer keep in mind that observers have
     * not yet been built for this object. If you were to send an ObserversRpc
     * for example it would not be delivered to any clients. You can however
     * still use an ObserversRpc and set BufferLast to true if you wish
     * clients to get it when the object is spawned for them. Another option
     * is to use OnSpawnServer, displayed below, and send a TargetRpc to the
     * connection which the object is spawning. */
}

```

Note: When clientHost, base.IsOwner will not return true, even with the owner set. This is because IsOwner is a client-side check, and the client has not yet initialized the object. When you need to check IsOwner on OnStartServer use base.Owner.IsLocalClient.

OnOwnershipServer

```

public override void OnOwnershipServer(NetworkConnection prevOwner)
{
    /* This is received when the server is giving ownership
     * to a client. If a client is gaining
     * ownership when the object is spawned OnOwnershipServer
     * will be called immediately after OnStartServer. prevOwner
     * is the connection which just lost ownership. If ownership is changed
     * at runtime this callback also occurs. */
}

```

OnSpawnServer

```

public override void OnSpawnServer(NetworkConnection connection)
{
    /* This callback occurs after a spawn message for this
    * object has been sent out to a client. For example: if
    * this object will be visible to five clients, then this
    * callback will occur five times, where the connection
    * parameter will be for each client.
    * Primarily you will use this callback to send
    * tailored communications to the client
    * the object is being spawned for. */
}

```

OnDespawnServer

```

public override void OnDespawnServer(NetworkConnection connection)
{
    /* OnDespawnServer is similar to OnSpawnServer, except this is
    * called right before an object is despawned for a client.
    * This method can be used to send information to the client
    * before they receive the despawn message. You may even
    * send object orientated communications for this object
    * such as SyncTypes, and RPCs. */
}

```

Note: When the server despawns a `NetworkObject` any pending synchronize changes will be sent out with the despawn message. This ensures clients will get the latest data, even if timed, before the object is despawned for them.

OnStopServer

```

public override void OnStopServer()
{
    /* This is the last callback for server side.
    * OnStopServer will be called immediately before
    * the object is deinitialized. At this time it is
    * too late to send communications specific to this
    * object for clients.
    * This callback may be used for any number of things but
    * could be useful for resetting synchronized values
    * such as SyncTypes when a scene object is despawned. */
}

```

Next are the client callbacks. These callbacks will always occur after the server callbacks, even when acting as host. For example, a callback sequence might look like this: `OnStartServer`, `OnOwnershipServer`, `OnSpawnServer`, `OnStartClient`, `OnOwnershipClient`.

For the most part client callbacks are the same as the server ones, except they occur only if the client connection is started. Only key differences will be included in the descriptions of each callback.

OnStartClient

```

public override void OnStartClient()
{
    /* This is called on each client when the object
    * becomes visible to them. Networked values such as
    * Owner, ObjectId, and SyncTypes will already be
    * synchronized prior to this callback. */
}

```

Note: Any buffered remote procedure calls will occur immediately after `OnStartClient`.

Note: If you are coming from Mirror, instead of using `OnLocalPlayer` use `OnStartClient` with a `base.IsOwner` check.

OnOwnershipClient

```
public override void OnOwnershipClient(NetworkConnection prevOwner)
{
    /* Current owner can be found by using base.Owner. prevOwner
     * contains the connection which lost ownership. Value will be
     * -1 if there was no previous owner. */
}
```

OnStopClient

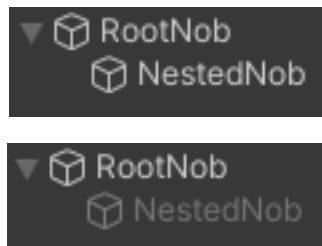
```
public override void OnStopClient()
{
    /* Like the server variant this is called right
     * before the object is deinitialized. It is too
     * late to send communications to the server
     * for this object after this callback has occurred. */
}
```

OnStopNetwork

Like `OnStartNetwork`, this method can be used to save lines of code when needing to deinitialize for both server and client stop. `OnStopNetwork` will also only be called once even when `clientHost`.

```
public override void OnStopNetwork()
{
    /* This will call after OnStopServer
     * and OnStopClient. */
}
```

Networkobjects



NetworkObject

Any `GameObject` with a `NetworkObject` component on it will be considered a "NetworkObject" in these guides going forward. Review the `NetworkObject` component page for details on the various settings for a `NetworkObject`.

When you add a `NetworkBehaviour` component to your prefabs or scene objects, the `NetworkBehaviour` will search for a `NetworkObject` component on the same object, or within parent objects. If a `NetworkObject` is not found then one will be added automatically to the top-most object.

Spawned NetworkObject

`NetworkObjects` that are instantiated and spawned using the `ServerManager.Spawn()` method will be considered a "Spawned `NetworkObject`". The `IsSpawned` property will be marked `true` internally on the `NetworkObject` component.

Scene NetworkObject

Any `NetworkObject` that exists as part of the Scene — never instantiated/spawned into the scene — will be considered a "Scene `NetworkObject`" in these guides going forward. The `IsSceneObject` property will be marked `true` on the attached `NetworkObject` component internally.

Global NetworkObject

Any `NetworkObject` with their `bool IsGlobal` marked `true` either in the inspector or with code will be considered a "Global `NetworkObject`" in the guides going forward.

`GlobalNetworkObjects` will automatically be put into the `DontDestroyOnLoad` scene when instantiated on the server, and when spawned on the clients.

Note: Scene objects cannot be marked as global. All global objects must be instantiated and spawned.

Nested NetworkObject

Any `NetworkObject` that is a child of another `NetworkObject` will be considered a "Nested `NetworkObject`" in these guides going forward. The `IsNested` property will be marked `true` on the attached `NetworkObject` component internally.

Nested NetworkObjects

When a NetworkObject is beneath another NetworkObject it is considered nested. Fish-Networking allows you to nest NetworkObjects both in the scene and prefabs.

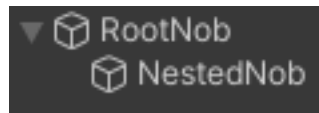
While spawning a root NetworkObject any active nested NetworkObjects will also be spawned. The nested NetworkObjects will experience the same callbacks as root, and share the same information such as IsOwner.

You may also have nested NetworkObjects deactivated in the scene or prefab and spawn them later.

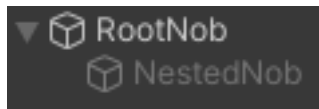
[!WARNING] When a nested NetworkObject is spawned after the root has already been spawned, the owner information is not automatically assumed the same as the root. You must specify if a client should gain ownership while calling Spawn.

Between instantiating the prefab and spawning the root NetworkObject you may make changes to nested NetworkObjects. This includes SyncTypes, and even the active state of the object. These changes are automatically synchronized when the root is spawned.

For example, if my prefab looks like this:



But I change the activate state of *NestedNob* to disabled before server spawning as such:



RootNob will be spawned over the network with NestedNob disabled / not spawned.

```
// An example of changing the enabled state for NestedNob.
public GameObject MyPrefab;

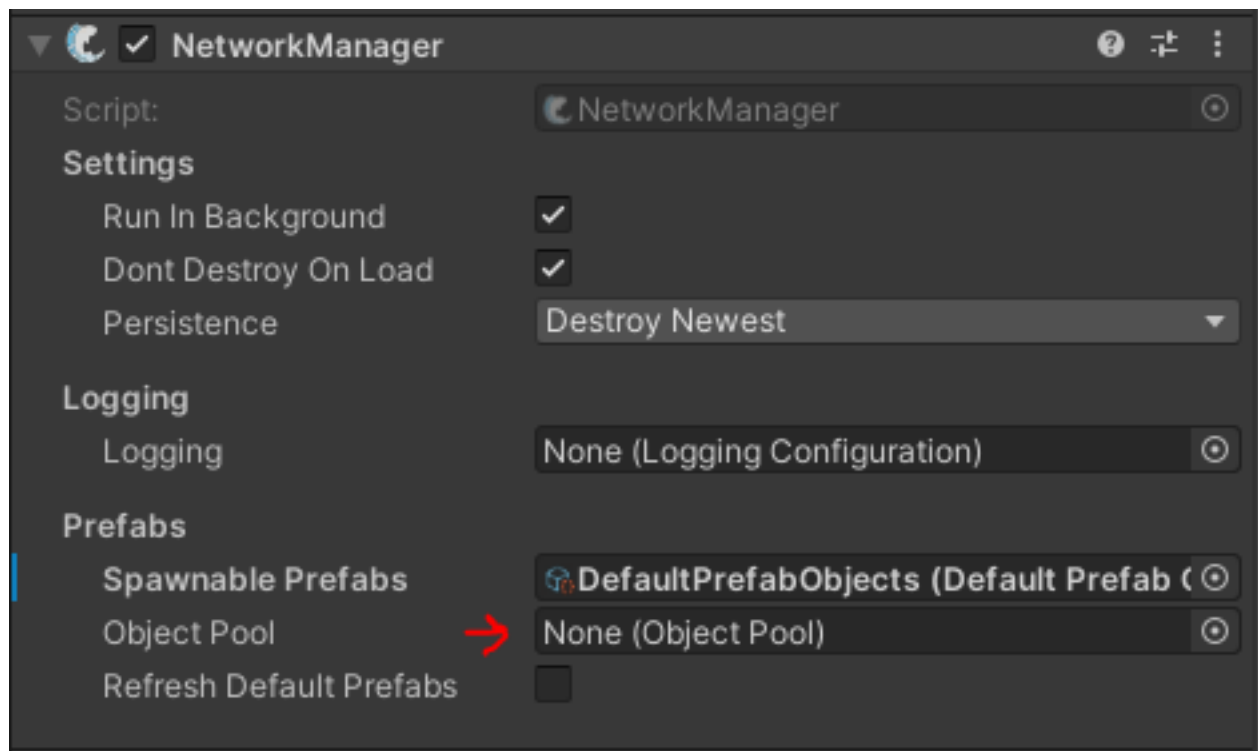
private void SpawnPrefab()
{
    GameObject go = Instantiate(MyPrefab);
    GameObject nestedNob = go.transform.GetChild(0);
    nestedNob.SetActive(false);
    // MyPrefab will spawn with NestedNob disabled for server and all clients
    // until you spawn it at a later time.
    base.Spawn(go);
}
```

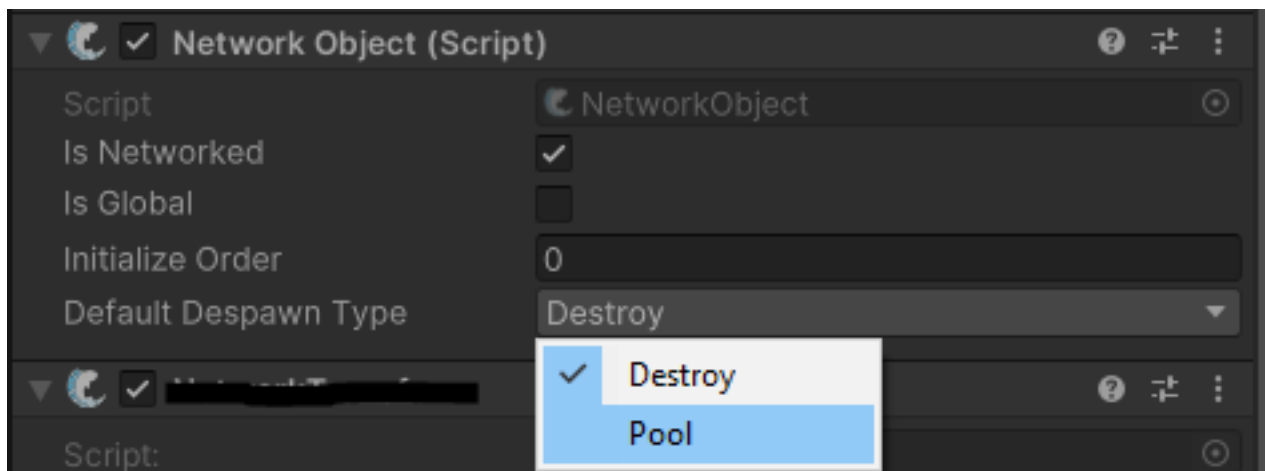
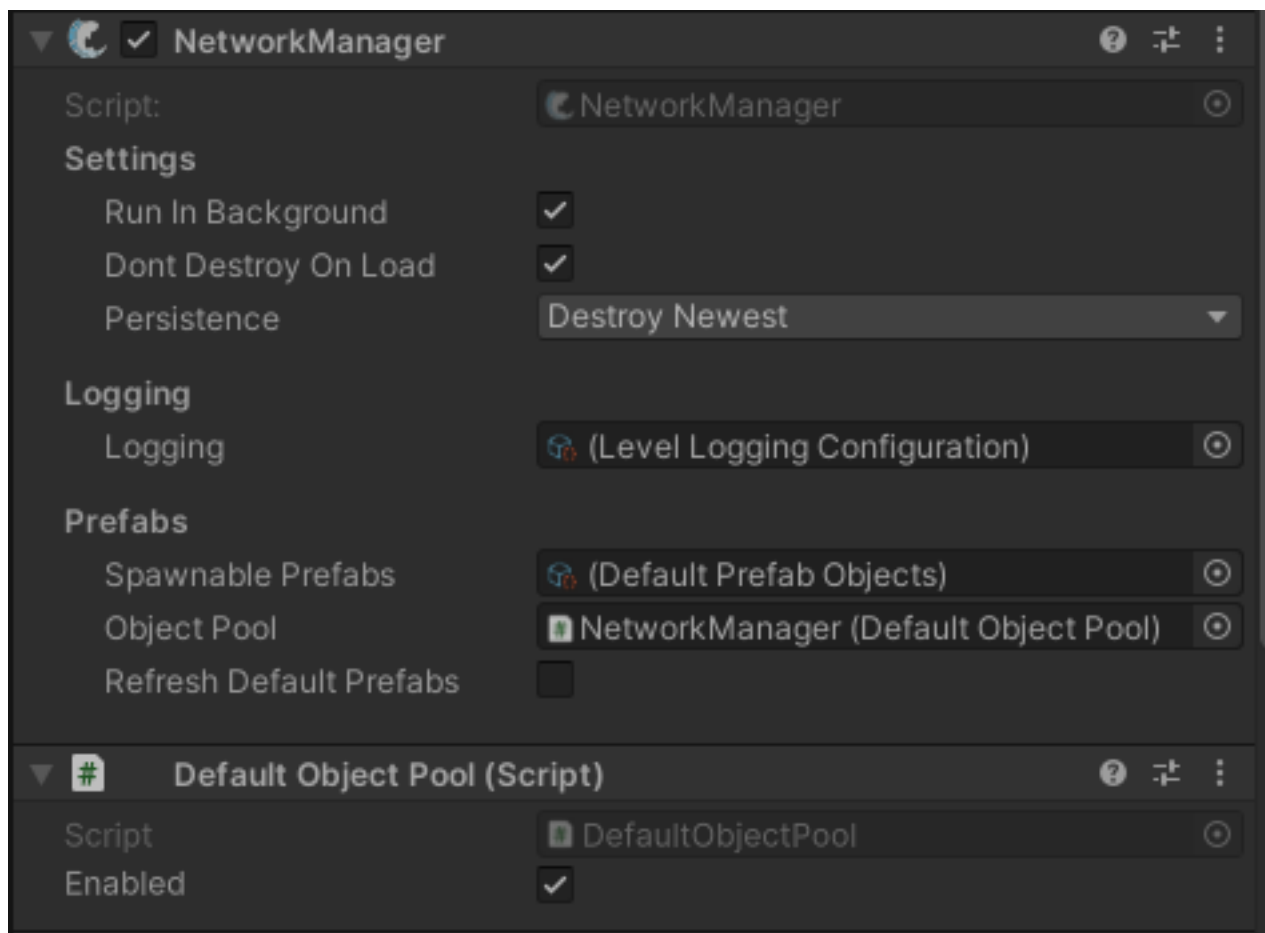
[!WARNING] Nested NetworkObjects may be spawned and despawned but they should not be detached. This feature is being examined for a future update. There is an exception for nested scene NetworkObjects; they may be detached.

Spawning and Despawning

FishNet provides flexible tools for spawning and despawning NetworkObjects at runtime. This section covers the various spawning techniques available, including sending data at spawn time, predicted spawning for client-authoritative games, and using the built-in object pooling system to improve performance.

Object Pooling





Fish-Networking has built-in functionality for Object Pooling that will allow the server and client to keep instances of loaded prefabs in memory for later use. This can potentially provide better spawning performance for clients and server.

General

When despawning spawned **NetworkObjects** using FishNet, instead of destroying the object you may just want to disable it and store it to be used again at a later time. This is what Object Pooling is. FishNet has a default implementation that will allow the objects you have instantiated to be disabled and pooled, instead of being destroyed. This functionality works for clients and server. You can also pre-warm assets for later use,

which is discussed later in this guide.

Note: Scene NetworkObjects do not get added to the Object Pool and are already disabled instead of destroyed when despawned.

Setup

As mentioned on the NetworkManager component page, there is an assignable field labeled **ObjectPool**. You may assign any script which inherits from the `ObjectPool` class. By default when examining the NetworkManager in the Editor nothing will be assigned to this field; however, when you enter play mode the NetworkManager will automatically populate it with the default implementation and attach the script to the NetworkManager GameObject.

Note: By default the object pool is enabled, but your network objects will only use the pool if the default despawn behavior is modified, or through the despawn call. See below for examples to both of these.

Default Despawn Behavior

On the NetworkObject component you can set what the default despawn behavior is for the object where the script is placed. This setting is set to **Destroy** by default, so make sure to switch this over to **Pool** if you want Fish-Networking to automatically use the default object pool.

Manual Despawn Behavior

You can manually change the despawn behavior through code for specific situations.

```
// When calling FishNet's Despawn method from any location, you can pass an enum
// parameter to deviate from the default behavior.
ServerManager.Despawn(nob, DespawnType.Pool);
```

Spawning NetworkObjects

When using the object pool you will want to retrieve NetworkObjects from it prior to network spawning them. Doing so will pull from the pool rather than instantiate new objects.

```
// There are many overrides which allow for a variety of information.
// You can use GameObjects, NetworkObjects, PrefabIds, CollectionIds,
// spawn positions, and more.
NetworkObject nob = NetworkManager.GetPooledInstantiated(...);

// Spawn normally.
ServerManager.Spawn(nob);
```

If you are certain you do not wish to use the object pool on a specific object you can still use the code above and simply set the **Default Despawn Type** to **Destroy** on the NetworkObject, or instantiate and spawn normally.

Pre-Warming the ObjectPool

If you want to manually store NetworkObjects to the ObjectPool prior to needing them at runtime you may do so through the NetworkManager API.

Here is a very basic implementation of pre-warming the ObjectPool:

```

[SerializeField] private NetworkObject _nobPrefab;

private void Start()
{
    /// <summary>
    /// Instantiates a number of objects and adds them to the pool.
    /// </summary>
    /// <param name="prefab">Prefab to cache.</param>
    /// <param name="count">Quantity to spawn.</param>
    /// <param name="asServer">True if storing prefabs for the server collection.</param>
    InstanceFinder.NetworkManager.CacheObjects(_nobPrefab, 100, IsServer);
}

```

Custom Implementation

FishNet allows you to implement your own method of object pooling. First create your own class inheriting from the `ObjectPool` class. Place your new class component in your scene, typically directly on the `NetworkManager` object. Then assign your component to the **ObjectPool** field on the `NetworkManager`.

Predicted Spawning

Important: Currently predicted spawns cannot be nested during the spawning process. This limitation is planned to be removed in a future version.

The term predicted spawning also means predicted despawning, where the client may despawn the object locally while awaiting a server response.

Global Settings

By default predicted spawning is disabled. To enable predicted spawning you must adjust settings on your `ServerManager`. If you do not already have a `ServerManager` then you must add one to your `NetworkManager` object to change settings.

The enabled state of this feature can be set by checking **Allow Predicted Spawning** on the `ServerManager`. You may also adjust the **Reserved Object Ids** value. To learn more about these visit the `ServerManager` component section.

Settings Per Object

Even with predicted spawning enabled globally, `NetworkObjects` must be set to also allow predicted spawning or despawning. This ensures that only the objects you specify may use this feature, allows you to limit predicted spawning capabilities for the object, and lets you use custom code to further validate or control predicted spawning.

For prefabs or scene objects you wish to allow predicted spawning on, add the `PredictedSpawn` component to them.

Tip: You can inherit the `PredictedSpawn` component for overrides to customize predicted spawning.

Using Predicted Spawning

Once enabled globally and per your desired objects, predicted spawning and despawning may be used on clients as though they were the server. All of the same spawn and despawn methods work, including even specifying who may own the object when spawned.

Spawn Payloads

NetworkBehaviours have support for writing and reading extra data and sending it along with the spawn payloads for the network object. This can be quite useful for syncing an initial state of the object.

To use this, you need to override the NetworkBehaviour's WritePayload and ReadPayload methods.

Important: When writing and reading, the order and amount of data written and read is important and MUST be the same.

Write Payload

The first parameter of the WritePayload method needs to be a NetworkConnection; this represents which client the spawn is going to happen for and is provided by FishNet when it calls the method.

The second parameter should be of the type Writer; this will represent a reference to the Writer being used and you will use it to add data to the payload.

```
public override void WritePayload(NetworkConnection connection, Writer writer)
{
    // Writes the string stored in the class variable _playerName to the spawn payload.
    writer.WriteString(_playerName);

    // Writes the integer 42 to the payload.
    writer.WriteInt32(42);
}
```

Read Payload

The first parameter of the ReadPayload method needs to be a NetworkConnection; this represents which client the spawn is happening for and is provided by FishNet when it calls the method.

The second parameter should be of the type Reader; this will represent a reference to the Reader being used and you will use it to get data out of the payload.

```
public override void ReadPayload(NetworkConnection connection, Reader reader)
{
    // Reads a string from the payload. Since a string was written first, it must be read
    first too.
    _playerName = reader.ReadStringAllocated();

    // Reads an integer from the payload.
    _playerScore = reader.ReadInt32();
}
```

More Advanced Example

```

// SpawnPayloadExample.cs
public class SpawnPayloadExample : NetworkBehaviour
{
    private ArraySegment<byte> _cachedPayload = ArraySegment<byte>.Empty;

    public override void WritePayload(NetworkConnection connection, Writer writer)
    {
        /* If you are using predicted spawning and want to write the
        * payload if you are the spawning client, this might help! */
        bool isPredictedSpawner = NetworkObject.PredictedSpawner.IsLocalClient;

        /* It is worth noting that the server will still call WritePayload when
        * spawning the object for clients, even if a predicted spawn.
        * Should a client send a predicted spawn the server will receive ReadPayload
        * for the predicted spawn, and the server will also be given an opportunity to
        * send a payload to other clients which are not the predicted spawner. If you wis
h to
        * forward a payload from a predicted spawner cache the values locally, and
        * send them here. */
        if (isPredictedSpawner)
        {
            // As the predicted spawner let's write our own payload!
        }
        // If not the predicted spawner, we are the server sending a payload to other clie
nts.
        else
        {
            // Write our cached payload, or don't, send whatever you want!
            // Check if not empty first.
            if (_cachedPayload != ArraySegment<byte>.Empty)
                writer.WriteArraySegmentAndSize(_cachedPayload);
        }
    }

    public override void ReadPayload(NetworkConnection connection, Reader reader)
    {
        /* You can also check if the payload is coming from the predicted spawner.
        *
        * ReadPayload would be first called on the server, and if the connection IsValid,
we know
        * that the connection is a predicted spawner, as that is the only time connection
would be
        * valid on the server. */
        bool isFromPredictedSpawner = IsServerStarted && connection.IsValid;

        /* Another way using a similar workflow to WritePayload. */
        bool isFromPredictedSpawner2 = IsServerStarted && connection == NetworkObject.Pred
ictedSpawner;

        /* When the connection is not valid, as noted in the parameter comments, this mean
s
        * the server is the sender. */
        bool isFromServer = !connection.IsValid;

        /* If you're the server and wish to cache a payload received from a predicted spaw
ner,
        * this is the place to do it. We are caching into an ArraySegment named _cachedPa

```

```

yload --
    * see WritePayload on how to forward this to other clients. */
    if (isFromPredictedSpawner)
    {
        int readerStart = reader.Position;
        // Do all your reads as you would.
        int readAmount = reader.Position - readerStart;

        // Let's use a pooled array so we don't allocate.
        byte[] payload = ArrayPool<byte>.Shared.Rent(readAmount);

        /* Copy into payload array from where the reader started, up to
         * the calculated readAmount. */
        Array.Copy(reader.GetBuffer(), sourceIndex: readerStart, payload, destinationIndex: 0, readAmount);
        _cachedPayload = new(payload, 0, readAmount);

        // /* You will probably want to return the rented array when OnStop callbacks
        occur. */
        // if (_cachedPayload != ArraySegment<byte>.Empty)
        //     ArrayPool<byte>.Shared.Return(_cachedPayload.Array);
    }
}

```

Spawning

Information about how to spawn and despawn objects on the network.

For objects to exist over the network they must have a `NetworkObject` component on them, and must be spawned using the server. To spawn an object it must first be instantiated, and then passed into a `Spawn` method. There are a variety of `Spawn` methods to access for your convenience, all of which provide the same outcome.

Important: Networked addressable prefabs must be registered with the `NetworkManager` differently. See the [Addressables](#) guide for more information on this.

Spawning without an owner is done by passing in `null` for the owner, or simply leaving the argument out.

```
GameObject go = Instantiate(_yourPrefab);
InstanceFinder.ServerManager.Spawn(go, null);

GameObject go = Instantiate(_yourPrefab);
InstanceFinder.ServerManager.Spawn(go);
```

Spawning with Ownership

```
GameObject go = Instantiate(_yourPrefab);
InstanceFinder.ServerManager.Spawn(go, ownerConnection);
```

You may also access the `spawn` method within any script that inherits `NetworkBehaviour`, or by accessing the `NetworkObject`.

```
GameObject go = Instantiate(_yourPrefab);
base.Spawn(go, ownerConnection); // networkBehaviour.
// or
networkObject.Spawn(go, ownerConnection); // referencing a NetworkObject.
```

Despawning can be accessed the same ways as spawning — through a `NetworkBehaviour` script, a reference `NetworkObject`, or through the `ServerManager` directly.

```
base.Despawn(); // networkBehaviour. Despawns the NetworkObject.
networkObject.Despawn(); // referencing a NetworkObject.
InstanceFinder.ServerManager.Despawn(gameObject); // through ServerManager.
```

When despawning you may also choose to pool the object rather than destroy it. There are optional parameters available to change this behavior.

```
base.Despawn(DespawnType.Pool); // pools the object instead of destroying it.
```

You can check if an object is spawned at any time using the `IsSpawned` property within `NetworkBehaviour`, or `NetworkObject`.

Scene Objects are spawned and despawned like instantiated objects, except you pass in the reference of the already instantiated/placed scene object. A scene object becomes disabled rather than destroyed when despawned.

Custom Conditions

A simple guide on how to write a custom observer condition.

Sometimes you may have unique requirements for an observer condition. When this is the case you can easily create your own `ObserverCondition`. The code below comments on how to create your own condition.


```

// The example below does not have many practical uses
// but it shows the bare minimum needed to create a custom condition.
// This condition makes an object only visible if the connection's
// ClientId matches the serialized value, _id.

// Make a new class which inherits from ObserverCondition.
// ObserverCondition is a ScriptableObject, so also create an asset
// menu to create a new ScriptableObject of your condition.
[CreateAssetMenu(menuName = "FishNet/Observers/ClientId Condition", fileName = "New Client
Id Condition")]
public class ClientIdCondition : ObserverCondition
{
    /// <summary>
    /// ClientId a connection must be to pass the condition.
    /// </summary>
    [Tooltip("ClientId a connection must be to pass the condition.")]
    [SerializeField]
    private int _id = 0;

    private void Awake()
    {
        // Awake can optionally be used to initialize values based on serialized
        // data. The source file of DistanceCondition is a good example
        // of where Awake may be used.
    }

    /// <summary>
    /// Returns if the object which this condition resides should be visible to connection
    .
    /// </summary>
    /// <param name="connection">Connection which the condition is being checked for.</par
am>
    /// <param name="currentlyAdded">True if the connection currently has visibility of th
is object.</param>
    /// <param name="notProcessed">True if the condition was not processed. This can be us
ed to skip processing for performance. While output as true this condition result assumes
the previous ConditionMet value.</param>
    public override bool ConditionMet(NetworkConnection connection, bool currentlyAdded, o
ut bool notProcessed)
    {
        notProcessed = false;

        // When true is returned it means the connection meets
        // the condition requirements. When false, the
        // connection does not and will not see the object.

        // Will return true if connection Id matches _id.
        return (connection.ClientId == _id);
    }

    /// <summary>
    /// Type of condition this is. Certain types are handled differently, such as Timed wh
ich are checked for changes at timed intervals.
    /// </summary>
    /* Since clientId does not change, a Normal condition type will work.
    * See API on ObserverConditionType for more information on what each
    * type does. */

```

```
        public override ObserverConditionType GetConditionType() => ObserverConditionType.Normal;  
    }  
}
```

You can get an idea of the flexibility of a condition by exploring the source of other premade conditions.

Area of Interest (Observer System)

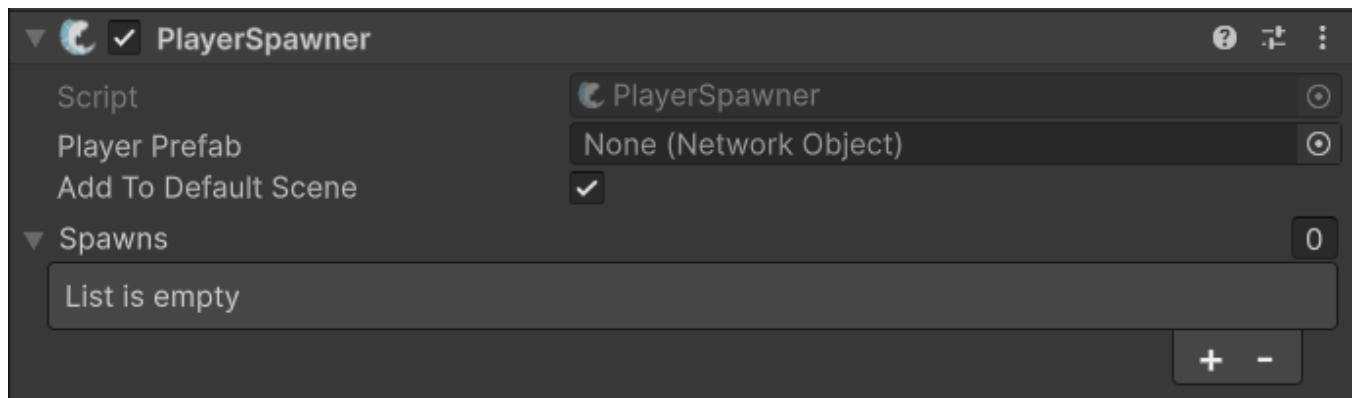
Fish-Networking features an advanced area-of-interest system that controls which clients receive updates about specific objects.

An observer is a client which can see an object and use communications for the object. You may control which clients can observe an object by using the `NetworkObserver` and/or `ObserverManager` components.

If a client is not an observer of an object, then the object will not activate, and the client will not receive network messages or callbacks for that object. Should the object be a scene object then it will remain disabled on the client until they become an observer of it. If the object is instantiated, then the client will simply not instantiate the object until after becoming an observer.

The observer system is designed to work out of the box for new developers. When it comes time to customize how clients observe objects, the observer system additionally offers a large amount of flexibility — there are many condition types, and you may also create your own.

FishNet comes with a `NetworkManager` prefab which contains the recommended minimum components to begin working on a new project. Within that prefab is the `ObserverManager` with an included `Scene Condition`. If you have not familiarized yourself with the `ObserverManager` and condition types, please do so now.



Common Issue: Scene Objects Not Spawning for Clients

A common problem new developers encounter is scene network objects not being enabled for clients. This occurs when the client is not considered part of the scene where the object resides, and the scene condition is preventing that object from spawning for the client. The `NetworkManager` prefab contains a `PlayerSpawner` script which adds the player to the current scene; this makes the client an observer for objects in that scene. This requires a player object to be spawned. Should you have made your own `NetworkManager` object or removed the `PlayerSpawner` script you will need to add the client to the scene you wish the client to be an observer of.

When encountering such an issue you may of course also remove the `ObserverManager` or scene condition from the `ObserverManager`, but this is not recommended as objects in other scenes will attempt to spawn for clients which do not occupy such scenes. Alternatively, you may add the client to the scene where the objects reside.

You can use a script such as the following on an object in the scene. Be careful not to place it on a potentially `DontDestroyOnLoad` object such as the `NetworkManager`.

```

using FishNet;
using FishNet.Connection;
using UnityEngine;

public class InitialSceneObserver : MonoBehaviour
{
    private void Start()
    {
        NetworkManager networkManager = InstanceFinder.NetworkManager;
        if (networkManager != null)
            networkManager.SceneManager.OnClientLoadedStartScenes += OnClientLoadedStartScenes;
    }

    private void OnDestroy()
    {
        NetworkManager networkManager = InstanceFinder.NetworkManager;
        if (networkManager != null)
            networkManager.SceneManager.OnClientLoadedStartScenes -= OnClientLoadedStartScenes;
    }

    private void OnClientLoadedStartScenes(NetworkConnection conn, bool asServer)
    {
        if (!asServer)
            return;

        if (!conn.Scenes.Contains(gameObject.scene))
            InstanceFinder.NetworkManager.SceneManager.AddConnectionToScene(conn, gameObject.scene);
    }
}

```

Under the assumption you removed the `PlayerSpawner` and/or are not using `SceneManager.AddOwnerToDefaultScene` or the method above, you must load the client into the scene using the `SceneManager`. Clients are only considered networked into scenes when those scenes are loaded using the `SceneManager`. Clients may become part of a scene by loading a scene globally or by loading a scene for a specific client (connection). See the `SceneManager` section for more information on how to manage networked client scenes and understand the difference between global and connection scenes.

Modifying Conditions

Several conditions may be modified at run-time. What can be modified for each condition may vary — view the API to see what each condition exposes.

To change properties on a condition you must access the condition through the `NetworkObserver` component.

```
// Below is an example of modifying the distance requirement
// on a DistanceCondition. This line of code can be called from
// any NetworkBehaviour. You may also use nbReference.NetworkObserver...
base.NetworkObserver.GetObserverCondition<DistanceCondition>().MaximumDistance = 10f;
```

All conditions can be enabled or disabled. When a condition is disabled its requirements are ignored, as if the condition does not exist. This can be useful for temporarily disabling condition requirements on objects.

```
// The OwnerOnlyCondition is returned and disabled.
// This allows all players to see the object, rather than just the owner.
ObserverCondition c = base.NetworkObject.NetworkObserver.GetObserverCondition<OwnerOnlyCon
dition>();
c.SetIsEnabled(false);

// Even though we are returning ObserverCondition type, it could be cast to
// OwnerOnlyCondition.
```

Ownership

What is Ownership?

Ownership is a core concept in Fish-Net, determining which client has control over an object.

Every `NetworkObject` in Fish-Net can be assigned a client to own it, or the object can have no assigned owner. The server is exclusively responsible for changing ownership of objects, even though it cannot own an object itself.

When a client owns an object, they are considered its rightful user, capable of actions like moving a character or using a weapon. Ownership can also be temporarily granted for interactions with world objects, such as using a turret.

Ownership checks are essential for ensuring proper control over objects. Common scenarios include:

- Validating if a client can manipulate an object.
- Checking ownership status for player-related values like name or score.
- Ensuring only the owner executes specific network calls (e.g., `ServerRpcs`).

Assigning Ownership

Ownership can be granted in several ways:

Spawning with Ownership: When creating an object, ownership can be assigned to a specific connection.

```
GameObject go = Instantiate(_yourPrefab);
InstanceFinder.ServerManager.Spawn(go, ownerConnection);
```

Changing or Adding Ownership: If an object is already spawned, ownership can be modified at any time. Previous owners are replaced with the new owner.

```
networkObject.GiveOwnership(newOwnerConnection);
```

Removing Ownership: Ownership can be revoked from a client when needed.

```
networkObject.RemoveOwnership();
```

Tip: `ownerConnection` and `newOwnerConnection` above represent a `NetworkConnection` that will be set as the new owner.

Checking Ownership

Ownership can be verified via the following `NetworkObject` or `NetworkBehaviour` properties:

```
base.IsOwner;        // Returns true if the local client owns the object.
base.Owner;           // Retrieves the current owner NetworkConnection.
base.IsController;    // True if the local client owns the object or is the server with no as
signed owner.
```

Simple Movement Example

A client must own an object to execute certain actions. Below is an example of movement logic restricted to the object's owner:

```

void Update()
{
    // The client's game instance will have several player objects in it if there
    // are multiple players playing, but he will only own one of them and should
    // only move the one that he owns.
    if (base.IsOwner)
    {
        float hor = Input.GetAxisRaw("Horizontal");
        float ver = Input.GetAxisRaw("Vertical");
        transform.position += new Vector3(hor, 0f, ver);
    }
}

```

When paired with a Network Transform set to Client Authoritative, this movement will be relayed to the server and synchronized across all clients.

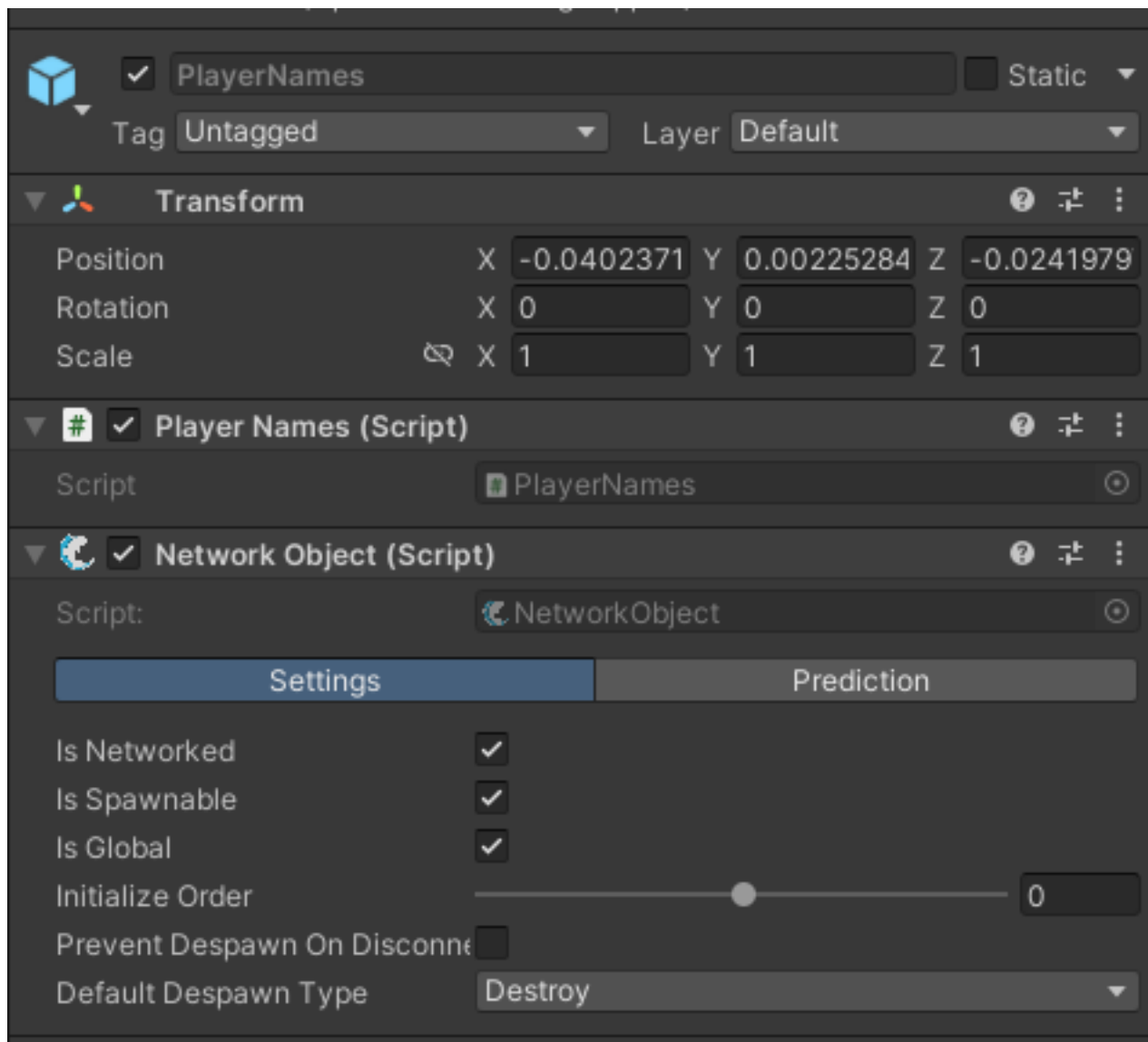
Transferring Ownership

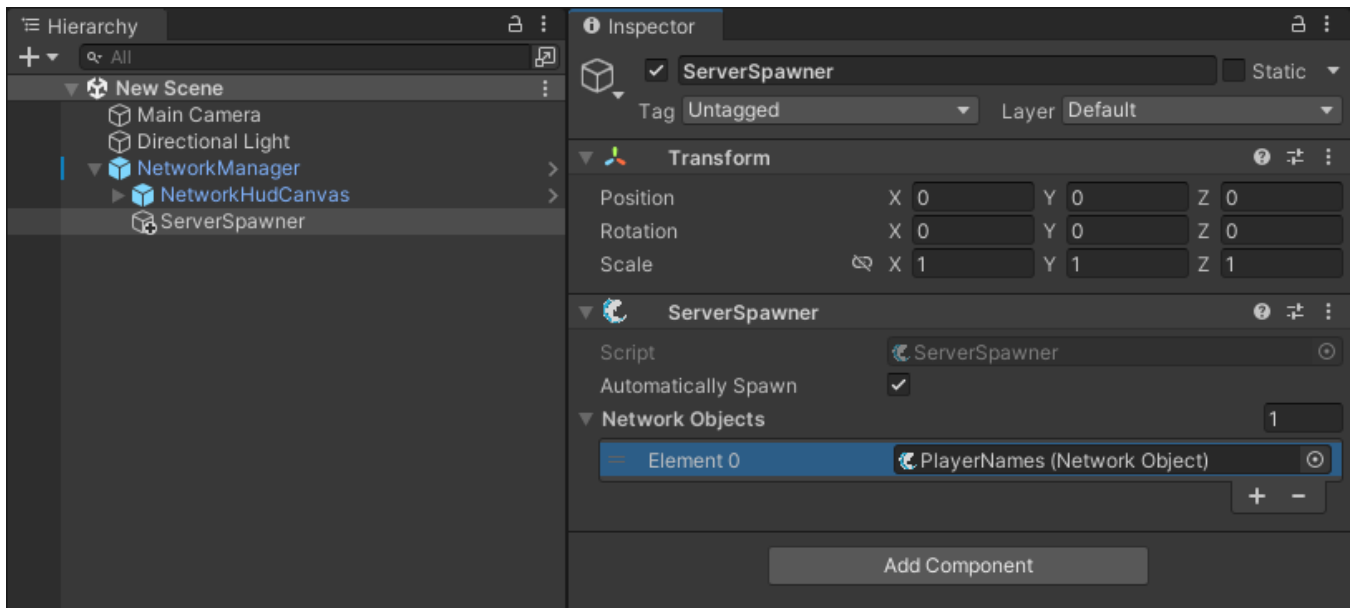
Only the server can assign, transfer, or remove ownership. Typically, ownership is granted when spawning an object.

Automatic Ownership Assignment: The `PlayerSpawner` script (found in the `NetworkManager` prefab) ensures the spawned player object is owned by its respective client.

Immediate Ownership: In cases where a client needs immediate ownership — such as controlling a turret without delay — the `PredictedOwner` component can be used. This component can be extended for customized logic.

Using Ownership To Read Values





Learn how to store values for clients and read them on unlinked objects by reading owner information.

Note: This guide requires Share Ids to be enabled on the ServerManager for clients to read values. Share Ids is enabled by default, and does not reveal any sensitive information to clients.

We are going to demonstrate how to assign a display name to each client, and display that name on player objects.

Prerequisite: This guide assumes you already have a NetworkManager object in your scene.

PlayerNames

First make a new script on an empty scene object and name it PLayerNames; this script needs to inherit NetworkBehaviour.

After adding the script to your scene object a NetworkObject component will automatically be added to the same object. On the NetworkObject enable **Is Global**, make this object a prefab, then delete it from your scene.

Tip: Notice that on the NetworkObject we also set the Initialize Order to -128. Doing so ensures that this NetworkObject will initialize before any other object does, which promises the OnStart callbacks will execute before other scripts. This step is most likely not needed, but given this is more-or-less a managing script for player names, giving it execution priority is good practice.

We are now going to populate the PLayerNames script with the following code.

```

using System;
using FishNet.Connection;
using FishNet.Object;
using FishNet.Object.Synchronizing;
using FishNet.Transporting;
using Random = UnityEngine.Random;

public class PlayerNames : NetworkBehaviour
{
    /* Since this is a syncType it will automatically be synchronized to clients
    * whenever it updates, and when clients spawn this object! */

    /// <summary>
    /// Called when a player name is updated.
    /// </summary>
    public event Action<NetworkConnection, string> OnPlayerNameChanged;

    /// <summary>
    /// Names of all connected clients.
    /// </summary>
    public readonly SyncDictionary<NetworkConnection, string> Names = new();

    private void Awake()
    {
        Names.OnChange += NamesOnOnChange;
    }

    public override void OnStartNetwork()
    {
        // Register this to the NetworkManager so it can be found easily by any script!
        NetworkManager.RegisterInstance(this);
    }

    public override void OnStartServer()
    {
        ServerManager.OnRemoteConnectionState += ServerManagerOnOnRemoteConnectionState;
    }

    public override void OnStopServer()
    {
        ServerManager.OnRemoteConnectionState -= ServerManagerOnOnRemoteConnectionState;
    }

    public override void OnStopNetwork()
    {
        // Unregister to clean up.
        if (NetworkManager != null)
            NetworkManager.UnregisterInstance<PlayerNames>();
    }

    private void ServerManagerOnOnRemoteConnectionState(NetworkConnection conn, RemoteConn
ectionStateArgs args)
    {
        // If disconnecting remove from the dictionary.
        if (args.ConnectionState == RemoteConnectionState.Stopped)
        {
            Names.Remove(conn);
        }
    }
}

```

```

    }
    // If connecting then add.
    else if (args.ConnectionState == RemoteConnectionState.Started)
    {
        /* When a player connects assign them a random number
         * as their name. */
        string randomName = Random.Range(1000, 999999).ToString();
        Names.Add(conn, randomName);
    }
}

/// <summary>
/// Called whenever the Names collection updates.
/// </summary>
private void NamesOnOnChange(SyncDictionaryOperation op, NetworkConnection key, string
value, bool asServer)
{
    // If an add or modify then invoke.
    if (op == SyncDictionaryOperation.Add || op == SyncDictionaryOperation.Set)
        OnPlayerNameChanged?.Invoke(key, value);
}

/// <summary>
/// Allows a client to call this RPC, updating their name.
/// </summary>
[ServerRpc(RequireOwnership = false)]
public void ServerSetName(string newName, NetworkConnection caller = null)
{
    // Caller will never be null; the server will assign it automatically when a clien
t calls this since RequireOwnership is false.
    Names[caller] = newName;
}
}

```

The above code snippet will give players a random name when they connect, and allow clients to change their name by calling the SetName RPC.

Automatically Spawning PlayerNames

After you have made the prefab select your scene NetworkManager, make a child object named ServerSpawner, and add the script ServerSpawner. You may place this script anywhere in your scene or game, but for simplicity we're going to nest it beneath the NetworkManager. After you add the script, insert your newly created PlayerNames prefab into the **Network Objects** field and ensure **Automatically Spawn** is enabled.

Displaying Player Names

Next we are going to make a very simple script which changes the text value on a TextMeshPro component. This is a very simple example which might go on the player's character. You could use similar scripts for chat names, and more.

Make a new script named CharacterName. Add the following:

```

using FishNet.Connection;
using FishNet.Object;
using TMPPro;
using UnityEngine;

public class CharacterName : NetworkBehaviour
{
    /// <summary>
    /// Text box used to display the name of this character.
    /// </summary>
    [Tooltip("Text box used to display the name of this character.")]
    [SerializeField]
    private TextMeshProUGUI _text;

    // Cached value for unsubscribing to save a little perf.
    private PlayerNames _playerNames;

    public override void OnStartClient()
    {
        // If owner is not set then do not get the name, as this does not belong to a client.
        if (!base.Owner.IsValid)
            return;

        // Get the PlayerNames instance to read this character's name (the player name).
        _playerNames = base.NetworkManager.GetInstance<PlayerNames>();

        // If cannot be found exit method; this shouldn't ever happen.
        if (!_playerNames.Names.TryGetValue(base.Owner, out string theName))
            return;

        _text.text = theName;

        // Also listen for updates.
        _playerNames.OnPlayerNameChanged += PlayerNamesOnOnPlayerNameChanged;
    }

    public override void OnStopClient()
    {
        // Unsubscribe from events to clean up.
        if (_playerNames != null)
            _playerNames.OnPlayerNameChanged -= PlayerNamesOnOnPlayerNameChanged;
    }

    /// <summary>
    /// Called when a player name changes after initially being set, or when added for the first time.
    /// </summary>
    private void PlayerNamesOnOnPlayerNameChanged(NetworkConnection conn, string theName)
    {
        // If the name being changed is not for this owner then do not update anything.
        if (conn != base.Owner)
            return;

        // Set new name.
        _text.text = theName;
    }
}

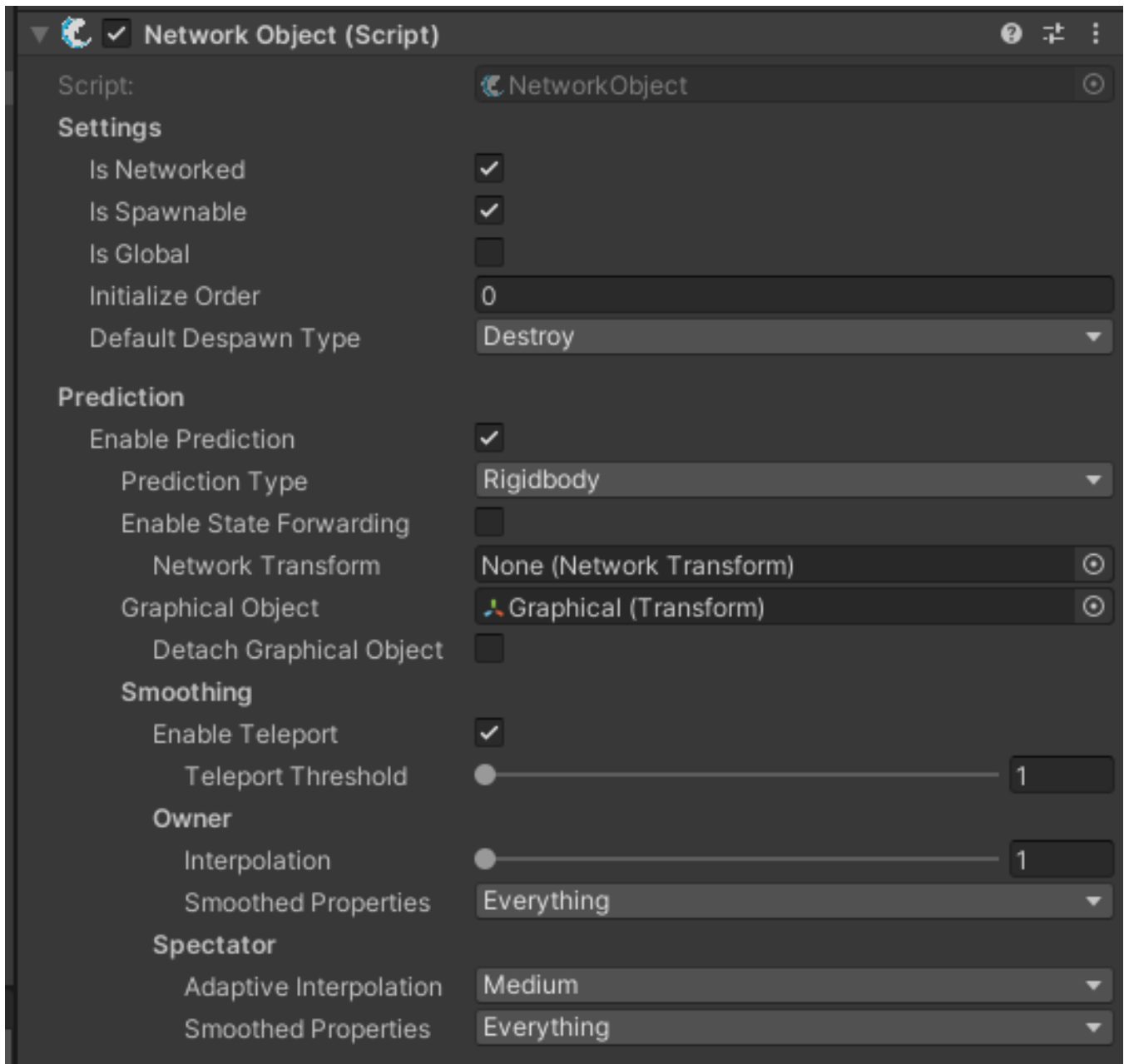
```

```
}
```

Now when an object spawns containing the script above, the `_text` field will be updated to the owner's player name. In addition, if the owner changes their name at any time, the text will be updated again.

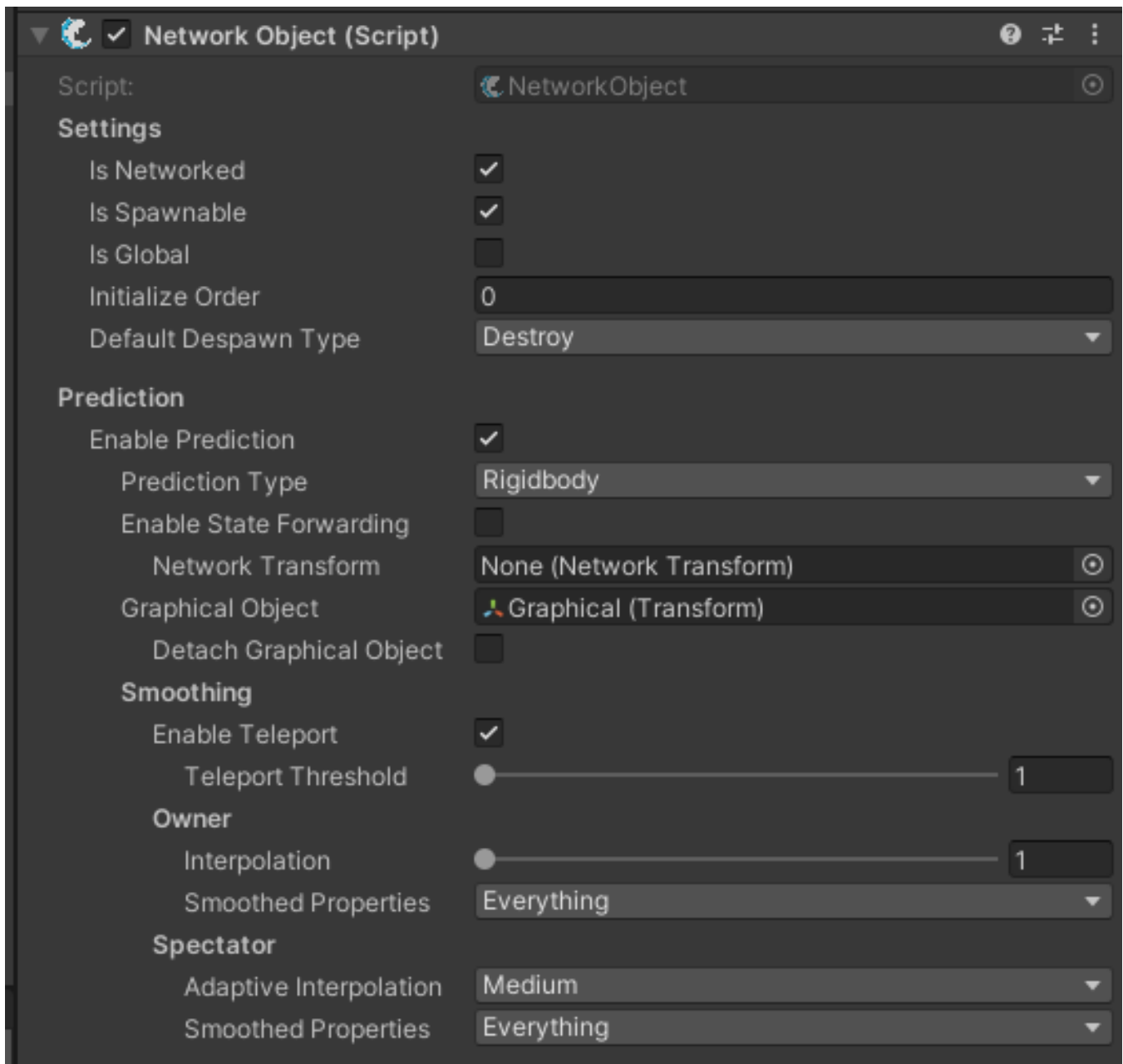
Tip: Notice how we make use of `NetworkManager.RegisterInstance`, `UnregisterInstance`, and `GetInstance` in this guide. This is a very useful feature for accessing global networked scripts.

Configuring NetworkObject



Setting up the NetworkObject inspector for prediction is mandatory for using prediction methods in scripts belonging to the NetworkObject.

To begin you must first choose to **Enable Prediction**. Next you will set the **Prediction Type**. If you are using a Rigidbody or Rigidbody2D set the prediction type accordingly. **Other** is used for non-physics such as character controllers.



State Forwarding

State forwarding will allow the same inputs to run on all clients as they do on the server. This can be useful if you want all clients and server to run the same input-based logic, similar to if the client or server owns the object. State forwarding is more CPU intensive as it means a state buffer must be kept, and the object must reconcile to make corrections and re-run past states.

Even with the overhead, state forwarding is often the most preferred approach because you gain the ability to run code identical to how the client and server ran it, resulting in a more reliable simulation and potentially easier coding approach.

When state forwarding is disabled, only the owner and server will run the inputs, and only the owner will keep the buffer for objects they own. This also means you must forward any information to clients that is essential to displaying actions on non-owned objects — such as using a `NetworkAnimator` to relay

animations, or using RPCs to send gun fire audio. Movement is also not forwarded, so you may want to attach a `NetworkTransform` and specify it within the **NetworkTransform** field.

Graphical Object

This is the object which holds your graphics for the `NetworkObject`. By graphics, this means anything which holds visual representation that you would likely want smoothed between ticks, as well as corrections from any potential de-synchronizations.

When using a graphical object it's very important to remember that you do not want any components on or beneath it that could negatively be affected by smoothing, such as a capsule to move your player. Typically your colliders and triggers which affect gameplay or transform the `NetworkObject` should be on the same `GameObject` as your `NetworkObject` — or at the very least, not within the graphical object.

Detaching the graphical object is a supplemental feature which might be useful depending on your setup. The default is keeping the graphical object as a child. When attached, the graphical object rolls back to its transform properties after the tick, then smooths to the transform properties it had after the tick. This allows lower tick rates with smooth transform updates rather than everything stepping at the tick rate delta.

However, keeping the graphical object attached could be problematic for certain animation setups or cameras. Third party assets often do not expect transforms to teleport back and then smooth to their destination, and in result you might see unexpected behavior. By detaching the graphical object it is no longer rolled back after the tick and simply exists in world space moving towards the proper goal over the duration of a tick. In most scenarios detaching would likely be better, but there is always a chance it's not right for you — hence the default of staying attached.

Smoothing

The smoothing options are only present when a graphical object is set.

There are certain cases where you may want to customize smoothing to control what is actually smoothed versus what you want to control yourself. That's where the **Smoothed Properties** come in. You will notice there is a smoothed properties setting for if you are the owner of the object, or a spectator (not owner). In a number of cases developers want to self-smooth things such as rotation while letting FishNet handle position and scale. In such a scenario you would just untick rotation from the smoothed properties.

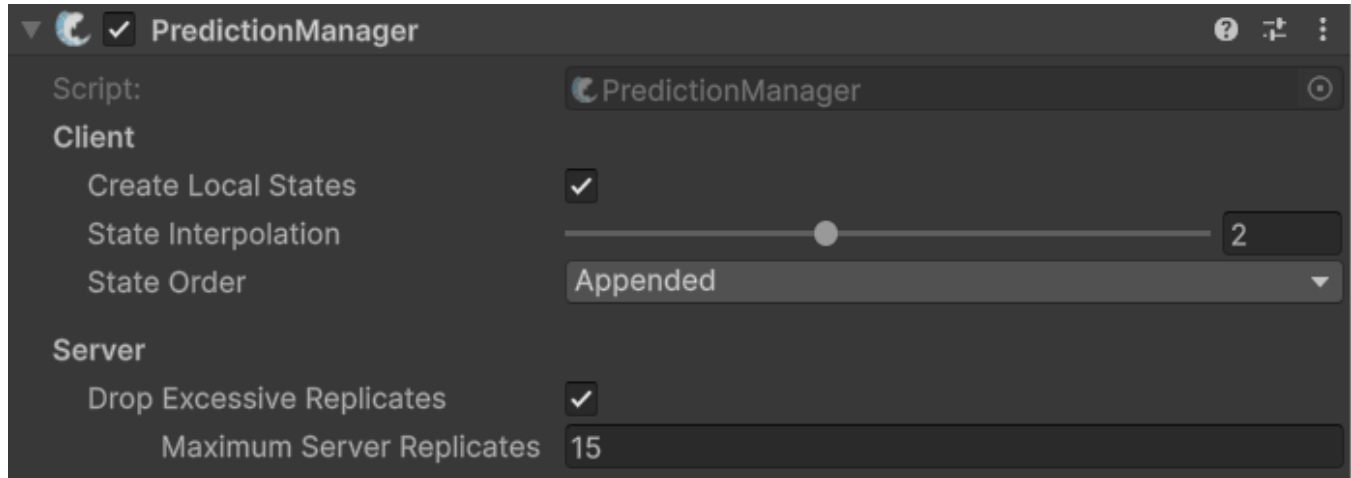
Adaptive Interpolation under Spectator options determines the level of interpolation on spectated objects. A lower value means less interpolation — and in result a larger chance that a de-synchronization will be more visually apparent. A higher value means more interpolation and much less chance of seeing the effects of a de-synchronization. The adaptive interpolation API is also exposed, letting you tweak it at runtime.

Configuring PredictionManager

[Image: untitled]

The PredictionManager is responsible for global prediction settings and other prediction related information.

You do not necessarily need to add it to your NetworkManager object, but doing so allows you to alter default settings.



Interpolation

Depending on your game type you may want to adjust the default client and server interpolation. In short, more interpolation means more resilience against network instability at the cost of larger delays between running actions.

Having more interpolation also means reducing the chances of having to predict data when it is expected to be known. There are a variety of ways to know if data is confirmed, predicted, or in the future; this topic is covered later.

Client interpolation indicates how many ticks inputs from the server (and other clients) are held before they are run. For casual games an interpolation of 2–3 may be desired to drastically improve the likelihood that inputs will always be available to run. This will add a delay to when those inputs are run, so perhaps for fast-paced games a value of 1 would be better.

Server interpolation is much the same but typically should be a lower value. This is how much of a buffer the server tries to hold for inputs, resulting in the server not running them after a number of ticks equal to the specified interpolation.

Excessive Replicate Dropping

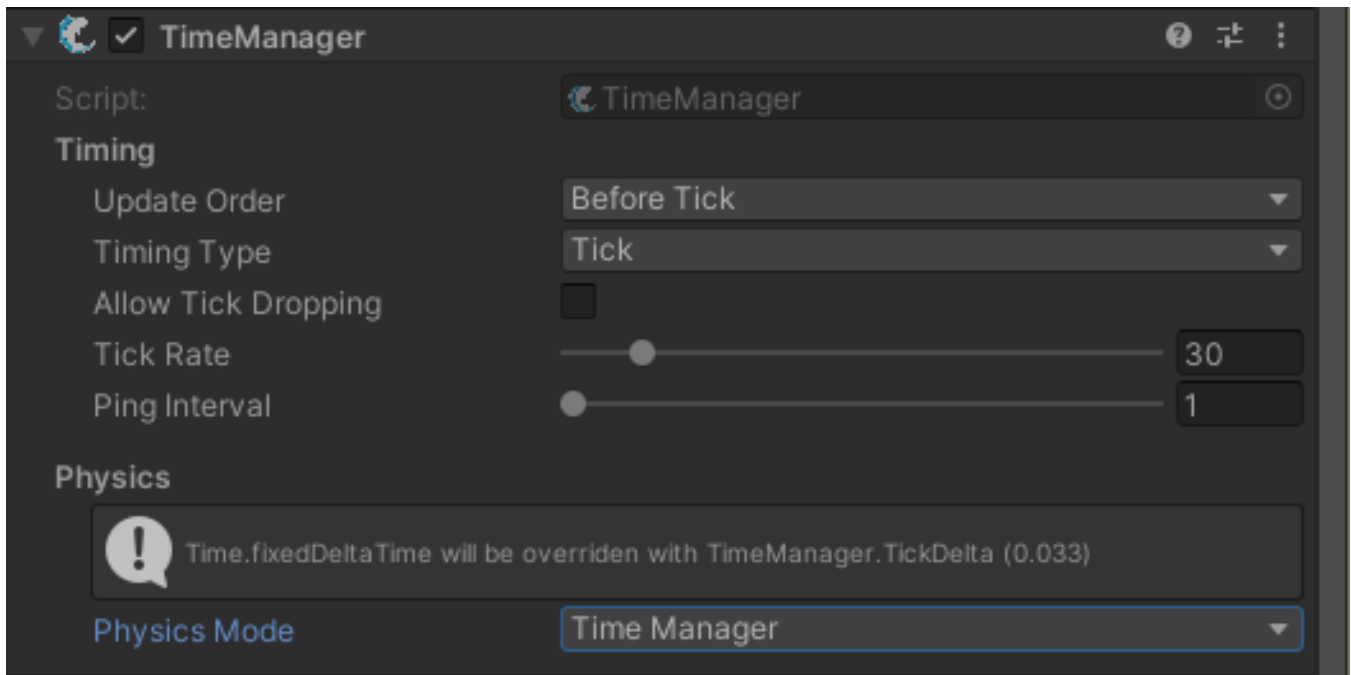
Typically speaking the server will never have more than its Server Interpolation + 1 in queue. However, if the client is having network issues and is sending inputs in bursts, the queue could for example go from 0 to 5 if 5 client inputs came through at once.

When the server queued inputs exceed the maximum it will begin to drop old values. This protects the server against an allocation attack and also prevents cheating by the client trying to send extra inputs.

You may disable dropping of excessive replicates but this opens your game up to cheating, as multiple replicates will be run per tick on the server to consume extras. There is still a generous hard-coded value of

the maximum amount of replicates which may be queued, to protect from allocation attacks.

Configuring TimeManager

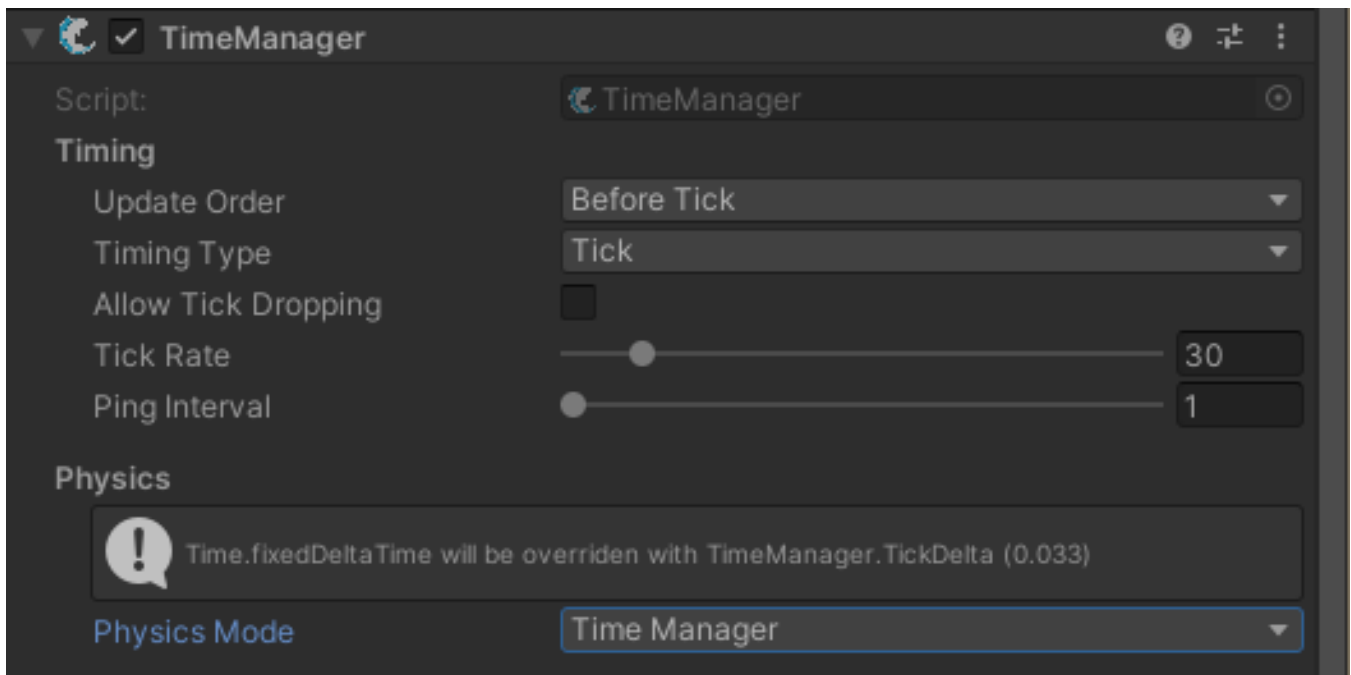


Very little of the TimeManager has to be configured for prediction.

When using prediction it is essential that Fish-Networking's timing system is used. By default Unity's timing is used, but this can be changed on the TimeManager component.

Add the TimeManager component to your NetworkManager if it does not already exist.

Once added, change the **Physics Mode** to **Time Manager**, and you are done.



Advanced Controls

This guide supplements the basic prediction guide by showing how to introduce more complexities to your controls.

Be sure to review the previous guides in this section before reviewing this page.

Guide goal

Implementing additional features into your prediction is much like you would code in a single player game, only remembering to reconcile anything that could de-synchronize the prediction.

In this guide a pseudo ground check before a jump will be added, as well as a sprint function.

Sprinting and ground checks

First the `ReplicateData` needs to be updated to contain our sprint action, which will rely on a stamina mechanic. Not much changed other than we added the `Sprint` boolean and set it using the constructor.

```
public struct ReplicateData : IReplicateData
{
    public bool Jump;
    public bool Sprint;
    public float Horizontal;
    public float Vertical;

    public ReplicateData(bool jump, bool sprint, float horizontal, float vertical) : this(
)
    {
        Jump = jump;
        Sprint = sprint;
        Horizontal = horizontal;
        Vertical = vertical;
    }

    private uint _tick;
    public void Dispose() { }
    public uint GetTick() => _tick;
    public void SetTick(uint value) => _tick = value;
}
```

After updating the `ReplicateData` we need to poll for the sprint key when creating the data, like you would in most games.

We are re-using methods from our previous guides so much of this should be familiar.

```

private ReplicateData CreateReplicateData()
{
    if (!base.IsOwner)
        return default;

    // Build the replicate data with all inputs which affect the prediction.
    float horizontal = Input.GetAxisRaw("Horizontal");
    float vertical = Input.GetAxisRaw("Vertical");
    /* Sprint if left shift is held.
    * You do not necessarily have to perform security checks here.
    * For example, it was mentioned sprint will rely on stamina, we
    * are not checking the stamina requirement here. You certainly could
    * as a precaution but this is only building the replicate data, not where
    * the data is actually executed, which is where we want
    * the check. */
    bool sprint = Input.GetKeyDown(KeyCode.LeftShift);
    ReplicateData md = new ReplicateData(_jump, sprint, horizontal, vertical);
    _jump = false;

    return md;
}

```

Declare a stamina float in your class.

```

// Current stamina for the player.
private float _stamina;

```

Now use our new Sprint bool and stamina field to apply sprinting within the replicate method.

```

[Replicate]
private void RunInputs(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
Channel channel = Channel.Unreliable)
{
    float delta = (float)base.TimeManager.TickDelta;

    // Regenerate stamina at 3f per second.
    _stamina += (3f * delta);

    // How much it cost to use sprint per delta.
    // This causes sprint to use stamina twice as fast
    // as the stamina recharges.
    float sprintCost = (6f * delta);

    Vector3 forces = new Vector3(data.Horizontal, 0f, data.Vertical) * _moveRate;

    // If sprint is held and enough stamina exist then multiple forces.
    if (data.Sprint && _stamina >= sprintCost)
    {
        // Reduce stamina by cost.
        _stamina -= sprintCost;
        // Increase forces by 30%.
        forces *= 1.3f;
    }

    /* You should check for any changes in replicate like we do
    * with stamina. Recall how it was said checking stamina when
    * gathering the inputs is not so important, but doing so in the replicate
    * is what grants server authority, as well makes prediction function
    * properly with corrections and rollbacks. */

    /* Now check if to jump. IsGrounded() does not exist, we're going to
    * pretend it uses a raycast or overlap to check. */
    if (data.Jump && IsGrounded())
    {
        Vector3 jmpFrc = (Vector3.up * _jumpForce);
        PredictionRigidbody.AddForce(jmpFrc, ForceMode.Impulse);
    }

    // Rest of the code remains the same.
}

```

Warning: If a value can affect your prediction do not store it outside the replicate method, unless you are also reconciling the value. An exception applies if you are setting the value inside your replicate method. This is a very important detail to remember, and is discussed further below.

Reconciling only a rigidbody state is very simple.

```

[Reconcile]
private void ReconcileState(ReconcileData data, Channel channel = Channel.Unreliable)
{
    // Call reconcile on your PredictionRigidbody field passing in
    // values from data.
    PredictionRigidbody.Reconcile(data.PredictionRigidbody);
}

```

If you are using multiple rigidbodies you at the very least need to reconcile their states as well. You can do so quickly by adding a RigidbodyState for each rigidbody to your reconcile. If you are also

applying forces to these rigidbodies be sure to use PredictionRigidbody with them, and reconcile the PredictionRigidbody instead of RigidbodyState.

Changes to reconcile

Because objects can reconcile to previous states it's fundamental to also reconcile any values stored outside the replicate method. Imagine if you had 10f stamina, enough to sprint, and did so successfully on the server and owner. After your sprint you only had 1f stamina, not enough to sprint further.

If you were to reconcile without resetting stamina to its previous values then you would still be at 1f stamina after reconciling. Your replayed inputs, which previously allowed the sprint, would not sprint because you now lacked the needed stamina. In result of this, you would have a de-synchronization which would most likely be seen as jitter.

Including more variables in your prediction is fortunately easy enough. All you have to do is update your reconcile to include states or your new values or variables.

Added to our ReconcileData structure is a Stamina float.

```
public struct ReconcileData : IReconcileData
{
    public PredictionRigidbody PredictionRigidbody;
    public float Stamina;

    public ReconcileData(PredictionRigidbody pr, float stamina) : this()
    {
        PredictionRigidbody = pr;
        Stamina = stamina;
    }

    private uint _tick;
    public void Dispose() { }
    public uint GetTick() => _tick;
    public void SetTick(uint value) => _tick = value;
}
```

Then of course we must include the current value of stamina within our created reconcile data.

```
public override void CreateReconcile()
{
    ReconcileData rd = new ReconcileData(PredictionRigidbody, _stamina);
    ReconcileState(rd);
}
```

Very last you must utilize the new reconcile data to reset the stamina state.

```
[Reconcile]
private void ReconcileState(ReconcileData data, Channel channel = Channel.Unreliable)
{
    PredictionRigidbody.Reconcile(data.PredictionRigidbody);
    _stamina = data.Stamina;
}
```

With minor additions to the code you now have an authoritative ground check as well as a stamina driven sprint.

Controlling an Object

Learn how to create a predicted object that the owner or server can control.

Data structures

Implementing prediction is done by creating a replicate and reconcile method, and making calls to the methods accordingly.

Your replicate method will take inputs you wish to run on the owner, server, and other clients if using state forwarding. This would be any input needed for your controller such as jumping, sprinting, movement direction, and could even include other mechanics such as holding a fire button.

The reconcile method takes a state of the object after a replicate is performed. This state is used to make corrections in the chance of de-synchronizations. For example, you may send back health, velocity, transform position and rotation, so on.

It's also worth mentioning if you are going to allocate in your structures it could be beneficial to utilize the Dispose callback, which will run as the data is being discarded.

Here are the two structures containing basic mechanics for a rigidbody.


```

public struct ReplicateData : IReplicateData
{
    public bool Jump;
    public float Horizontal;
    public float Vertical;

    public ReplicateData(bool jump, float horizontal, float vertical) : this()
    {
        Jump = jump;
        Horizontal = horizontal;
        Vertical = vertical;
    }

    private uint _tick;
    public void Dispose() { }
    public uint GetTick() => _tick;
    public void SetTick(uint value) => _tick = value;
}

public struct ReconcileData : IReconcileData
{
    // PredictionRigidbody is used to synchronize rigidbody states
    // and forces. This could be done manually but the PredictionRigidbody
    // type makes this process considerably easier. Velocities, kinematic state,
    // transform properties, pending velocities and more are automatically
    // handled with PredictionRigidbody.
    public PredictionRigidbody PredictionRigidbody;

    public ReconcileData(PredictionRigidbody pr) : this()
    {
        PredictionRigidbody = pr;
    }

    private uint _tick;
    public void Dispose() { }
    public uint GetTick() => _tick;
    public void SetTick(uint value) => _tick = value;
}

```

Learn more about using PredictionRigidbody.

Preparing to call prediction methods

Typically speaking you would want to run your replicate (or inputs) during OnTick. When you send the reconcile depends on if you are using physics bodies or not.

When using physics bodies, such as a rigidbody, you would send the reconcile during OnPostTick because you want to send the state after the physics have simulated your replicate inputs. See the TimeManager API for more details on tick and physics event callbacks.

Non-physics controllers can also send in OnTick, since they do not need to wait for a physics simulation to have the correct outcome after running inputs.

The code below shows which callbacks and API to use for a rigidbody setup.

You may need to modify move and jump forces depending on the shape, drag, and mass of your rigidbody.

```

// How much force to add to the rigidbody for jumps.
[SerializeField]
private float _jumpForce = 8f;
// How much force to add to the rigidbody for normal movements.
[SerializeField]
private float _moveForce = 15f;

// PredictionRigidbody is set within OnStart/StopNetwork to use our
// caching system. You could simply initialize a new instance in the field
// but for increased performance using the cache is demonstrated.
public PredictionRigidbody PredictionRigidbody;
// True if to jump next replicate.
private bool _jump;

private void Awake()
{
    PredictionRigidbody = ObjectCaches<PredictionRigidbody>.Retrieve();
    PredictionRigidbody.Initialize(GetComponent<Rigidbody>());
}

private void OnDestroy()
{
    ObjectCaches<PredictionRigidbody>.StoreAndDefault(ref PredictionRigidbody);
}

public override void OnStartNetwork()
{
    base.TimeManager.OnTick += TimeManager_OnTick;
    base.TimeManager.OnPostTick += TimeManager_OnPostTick;
}

public override void OnStopNetwork()
{
    base.TimeManager.OnTick -= TimeManager_OnTick;
    base.TimeManager.OnPostTick -= TimeManager_OnPostTick;
}

```

Calling prediction methods

For our described demo, below is how you would gather input for your replicate and reconcile methods.

Update is used to gather inputs which are only fired for a single frame. Ticks do not occur every frame, but rather at the interval of your TickDelta, much like FixedUpdate works. While the code below only uses Update for single frame inputs there is nothing stopping you from using it for held inputs as well.

```

private void Update()
{
    if (base.IsOwner)
    {
        if (Input.GetKeyDown(KeyCode.Space))
            _jump = true;
    }
}

```

OnTick will now be used to build our replicate data. A separate method of CreateReplicateData is not needed to create the data but is done to organize our code better.

When attempting to create the replicate data we return with default if not the owner of the object. Server

receives and runs inputs from the owner so it does not need to create datas, and when clients do not own an object they will get the input for it from the server, as forwarded by other clients if using state forwarding. When not using state forwarding default should still be used in this scenario, but clients will not run replicates on non-owned objects. You can also run inputs on the server if there is no owner; using `base.HasAuthority` would probably be best for this. See [Checking Ownership](#) for more information.

```
private void TimeManager_OnTick()
{
    RunInputs(CreateReplicateData());
}

private ReplicateData CreateReplicateData()
{
    if (!base.IsOwner)
        return default;

    // Build the replicate data with all inputs which affect the prediction.
    float horizontal = Input.GetAxisRaw("Horizontal");
    float vertical = Input.GetAxisRaw("Vertical");
    ReplicateData md = new ReplicateData(_jump, horizontal, vertical);
    _jump = false;

    return md;
}
```

Now implement your replicate method. The name may be anything but the parameters shown are required. The first is what we pass in, the remainder are set at runtime. Although, you absolutely may change the default channel used in the parameter or even at runtime.

For example, it could be beneficial to send an input as reliable if you absolutely want to ensure it's not dropped due to network issues.

```

[Replicate]
private void RunInputs(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
Channel channel = Channel.Unreliable)
{
    /* ReplicateState is set based on if the data is new, being replayed, ect.
    * Visit the ReplicationState enum for more information on what each value
    * indicates. At the end of this guide a more advanced use of state will
    * be demonstrated. */

    // Be sure to always apply and set velocities using PredictionRigidbody
    // and never on the rigidbody itself; this includes if also accessing from
    // another script.
    Vector3 forces = new Vector3(data.Horizontal, 0f, data.Vertical) * _moveRate;
    PredictionRigidbody.AddForce(forces);

    if (data.Jump)
    {
        Vector3 jmpFrc = new Vector3(0f, _jumpForce, 0f);
        PredictionRigidbody.AddForce(jmpFrc, ForceMode.Impulse);
    }

    // Add gravity to make the object fall faster. This is of course
    // entirely optional.
    PredictionRigidbody.AddForce(Physics.gravity * 3f);

    // Simulate the added forces.
    // Typically you call this at the end of your replicate. Calling
    // Simulate is ultimately telling the PredictionRigidbody to iterate
    // the forces we added above.
    PredictionRigidbody.Simulate();
}

```

On non-owned objects a number of replicates will arrive as `ReplicateState.Created`, but will contain default values. This is our `PredictionManager.RedundancyCount` feature working. This is normal and indicates that the client or server had gracefully stopped sending states as there is no new data to send. This can be useful if you are Predicting States.

Now the reconcile must be sent to clients to perform corrections. Only the server will actually send the reconcile but be sure to call `CreateReconcile` no matter if client, server, owner or not; this is to future proof an upcoming feature. Unlike our `CreateReplicateData` method, using `CreateReconcile` is not optional.

```

private void TimeManager_OnPostTick()
{
    CreateReconcile();
}

// Create the reconcile data here and call your reconcile method.
public override void CreateReconcile()
{
    // We must send back the state of the rigidbody. Using your
    // PredictionRigidbody field in the reconcile data is an easy
    // way to accomplish this. More advanced states may require other
    // values to be sent; this will be covered later on.
    ReconcileData rd = new ReconcileData(PredictionRigidbody);
    // Like with the replicate you could specify a channel here, though
    // it's unlikely you ever would with a reconcile.
    ReconcileState(rd);
}

```

Reconciling only a rigidbody state is very simple.

```

[Reconcile]
private void ReconcileState(ReconcileData data, Channel channel = Channel.Unreliable)
{
    // Call reconcile on your PredictionRigidbody field passing in
    // values from data.
    PredictionRigidbody.Reconcile(data.PredictionRigidbody);
}

```

When reconciling an object with a `CharacterController`, ensure you disable the `CharacterController` component immediately before setting its position and re-enable it afterwards. Because this controller uses a custom physics engine, skipping this toggle will prevent the internal state from refreshing, resulting in a synchronization error between the physics and the transform. As an example:

```

_characterController.enabled = false;
transform.localPosition = reconcileData.Position;
_characterController.enabled = true;

```

Creating Code

This guide provides examples of creating prediction code, understanding important aspects of the code, and leveraging it for your game.

- [Controlling an Object](#)
- [Non-Controlled Object](#)
- [Understanding ReplicateState](#)
- [Advanced Controls](#)

Non-Controlled Object

Many games will require physics bodies to be networked, even if not controlled by players or the server.

These objects can also work along-side the new state system by adding a basic prediction script on them.

It's worth noting that you can also 'control' non-owned objects on the server by using `base.HasAuthority`. This was discussed previously [here](#).

Sample script

Below is a full example script to synchronize a non-controlled rigidbody. Since the rigidbody is only reactive, input polling is not needed. Otherwise you'll find the data structures are near identical to the ones where we took input.

It is strongly recommended to review the [controlling objects guide](#) for additional notes in understanding the code below.

```

public class RigidbodySync : NetworkBehaviour
{
    // Replicate structure.
    public struct ReplicateData : IReplicateData
    {
        // The uint isn't used but Unity C# version does not
        // allow parameter-less constructors we something
        // must be set as a parameter.
        public ReplicateData(uint unused) : this() {}

        private uint _tick;
        public void Dispose() { }
        public uint GetTick() => _tick;
        public void SetTick(uint value) => _tick = value;
    }

    // Reconcile structure.
    public struct ReconcileData : IReconcileData
    {
        public PredictionRigidbody PredictionRigidbody;

        public ReconcileData(PredictionRigidbody pr) : this()
        {
            PredictionRigidbody = pr;
        }

        private uint _tick;
        public void Dispose() { }
        public uint GetTick() => _tick;
        public void SetTick(uint value) => _tick = value;
    }

    // Forces are not applied in this example but you
    // could definitely still apply forces to the PredictionRigidbody
    // even with no controller, such as if you wanted to bump it
    // with a player.
    private PredictionRigidbody PredictionRigidbody;

    private void Awake()
    {
        PredictionRigidbody = ResettableObjectCaches<PredictionRigidbody>.Retrieve();
        PredictionRigidbody.Initialize(GetComponent<Rigidbody>());
    }

    private void OnDestroy()
    {
        ResettableObjectCaches<PredictionRigidbody>.StoreAndDefault(ref PredictionRigidbody);
    }

    // In this example we do not need to use OnTick, only OnPostTick.
    // Because input is not processed on this object you only
    // need to pass in default for RunInputs, which can safely
    // be done in OnPostTick.
    public override void OnStartNetwork()
    {
        base.TimeManager.OnPostTick += TimeManager_OnPostTick;
    }
}

```



```

    }

    public override void OnStopNetwork()
    {
        base.TimeManager.OnPostTick -= TimeManager_OnPostTick;
    }

    private void TimeManager_OnPostTick()
    {
        RunInputs(default);
        CreateReconcile();
    }

    [Replicate]
    private void RunInputs(ReplicateData md, ReplicateState state = ReplicateState.Invalid
, Channel channel = Channel.Unreliable)
    {
        // If this object is free-moving and uncontrolled then there is no logic.
        // Just let physics do it's thing.
    }

    // Create the reconcile data here and call your reconcile method.
    public override void CreateReconcile()
    {
        ReconcileData rd = new ReconcileData(PredictionRigidbody);
        ReconcileState(rd);
    }

    [Reconcile]
    private void ReconcileState(ReconcileData data, Channel channel = Channel.Unreliable)
    {
        // Call reconcile on your PredictionRigidbody field passing in
        // values from data.
        PredictionRigidbody.Reconcile(data.PredictionRigidbody);
    }
}

```

Understanding ReplicateState

Being familiar with what each state means will help you fine-tune your gameplay on spectated objects.

If you need a refresher on what each state means see our API or simply navigate to the `ReplicateState` enum in your code editor.

Each state is a flag value; a replicate state may contain multiple flags. There are numerous extensions you may use to check if a state contains certain values.

You can view all built-in state extensions within the `ReplicateState` file.

Invalid

An invalid `ReplicateState` should never occur. This would imply internally Fish-Networking failed to properly set the state.

Ticked

Server and clients use this flag. This flag will be set if the data tick has run outside a reconcile, such as from user code within `OnTick`. Ticked can exist during a replay/reconcile, but only if the data had first run outside the replay/reconcile.

Replayed

Only clients will use this flag. The replayed flag is set if data is being run during a reconcile.

The server is always considered right and never has to correct data, so it never reconciles or replays inputs.

Created

Server and client use this flag. A created flag indicates that the data was created by controller, such as owner or the server if no owner. The created flag will not be present if the controller has not sent updates, such as if they are not taking any action.

State examples

Below are examples of some possible states and what they mean.

```
// You will see this value if the data is being replayed, it previously ran outside the
// reconcile, and data is created by controller.
state = (ReplicateState.Replayed | ReplicateState.Ticked | ReplicateState.Created);

// When the state is only Replayed then the data is not Created, and the tick on data
// has not occurred outside the reconcile yet. This is what we often refer to as a
// future state.
state = ReplicateState.Replayed;

// When a state is Ticked only it indicates that the data is being run outside a
// reconcile, and that the controller has not sent data for this particular tick.
state = ReplicateState.Ticked;
```

For more possible flag examples see the state extensions (the `ReplicateState` file).

Predicting States in Code

If your game is fast-paced and reaction based, even if not using physics, predicting states can be useful to *predict* the next inputs you'll get from the server before you actually get them.

[!INFO] Both the server and client can predict inputs. The server may predict inputs to accommodate for clients with unreliable connections. Clients would predict inputs on objects they do not own, or as we often call them "spectated objects".

By predicting inputs you can place objects in the future of where you know them to be, and even make them entirely real-time with the client. Keep in mind however, when predicting the future, if you guess wrong there will be a de-synchronization which may be seen as jitter when it's corrected in a reconcile.

Below is an example of a simple replicate method.

```
// What data does is irrelevant in this example.
// We're only interested in how to predict a future state.
[Replicate]
private void RunInputs(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
Channel channel = Channel.Unreliable)
{
    float delta = (float)base.TimeManager.TickDelta;
    transform.position += new Vector3(data.Horizontal, 0f, data.Vertical) * _moveRate * delta;
}
```

Before we go any further you must first understand what each ReplicateState is. These change based on if input is known, replaying inputs or not, and more. You can check out the [ReplicateState API](#) which will explain thoroughly. You can also find this information right in the source of FishNet.

If you've read through ReplicateState and do not fully understand them please continue reading as they become more clear as this guide progresses. You can also visit us on Discord for questions!

Covered in the ReplicateStates API: CurrentCreated will only be seen on clients if they own the object. When inputs are received on spectated objects clients run them in the reconcile, which will have the state ReplayedCreated. Clients will also see ReplayedFuture and CurrentFuture on spectated objects. A state ending in Future essentially means input has not been received yet, and these are the states you could predict.

Let's assume your game has a likeliness that players will move in the same direction regularly enough. If a player was holding forward for three ticks the input would look like this:

```
(data.Vertical == 1)
(data.Vertical == 1)
(data.Vertical == 1)
```

But what if one of the inputs didn't arrive, or arrived late? The chances of inputs not arriving at all are pretty slim, but arriving late due to network variance is extremely common. If perhaps an input did arrive late the values may appear as something of this sort:

```
(data.Vertical == 1)
(data.Vertical == 1)
(data.Vertical == 0) // Didn't arrive here, but will arrive late next tick.
(data.Vertical == 1) // This was meant to arrive the tick before, but arrived late.
```

Because of this interruption the player may seem to move forward twice, pause, then forward again. Realistically to help cover this up you will have interpolation on your graphical object as shown under the

prediction settings for NetworkObject. The PredictionManager also offers QueuedInputs which can give you even more of a buffer. For the sake of this guide though we're going to pretend both of those didn't get the job done, and you need to account for the late input.

Below is a simple way to track and use known inputs to create predicted ones.

```
private ReplicateData _lastCreatedInput = default;

[Replicate]
private void RunInputs(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
Channel channel = Channel.Unreliable)
{
    // If inputs are not known, you could predict all the way into CurrentFuture,
    // which would be real-time with the client. Though the more you predict
    // in the future the more you are likely to miss predict.
    if (state.IsFuture())
    {
        uint lastCreatedTick = _lastCreatedInput.GetTick();
        // If it's only been 2 ticks since the last created input then run the
        // logic below. This essentially means if the last created tick was 100,
        // this logic would run if the future tick was 102 or less.
        uint thisTick = data.GetTick();
        if ((data.GetTick() - lastCreatedTick) <= 2)
        {
            // We do not necessarily want to predict all states.
            // For example, it probably wouldn't make sense to predict
            // multiple jumps in a row. In this example only the movement
            // inputs are predicted.
            data.Vertical = _lastCreatedInput.Vertical;
        }
    }
    // If created data then set as lastCreatedInput.
    else if (state == ReplicateState.ReplayedCreated)
    {
        // If ReplicateData contains fields which could generate garbage you
        // probably want to dispose of the lastCreatedInput before replacing it.
        _lastCreatedInput.Dispose();
        // Assign newest value as last.
        _lastCreatedInput = data;
    }

    float delta = (float)base.TimeManager.TickDelta;
    transform.position += new Vector3(data.Horizontal, 0f, data.Vertical) * _moveRate * de
lta;
}
```

[!WARNING] If your ReplicateData allocates, do not forget to dispose of the lastCreatedInput when the network is stopped.

Using States in Code

Future states

You will see the term 'in the future' or 'future state' used frequently when working with prediction. When in the future it's not possible to know the data from the controller, which is why we call it the *future*. When using client-prediction, as a client you are always moving in real-time, before even knowing the server's current state. Due to this, you will not know other clients' or server states until they are forwarded to you, and during that time of unknowing we consider the object to be in the future.

The future is where you gain the opportunity to predict future input from the controller. Or, you can simply prevent future movement entirely; uses vary depending on your needs. We'll cover this more later on this page.

[!INFO] You will never be in the future on objects you control, given you only replay inputs up to what you created and never beyond.

The created flag

When a state is not created the data will be default. This often catches a lot of developers off guard as they might expect to see continual input from the controller, such as if the controller is always holding a movement key.

A common use case is updating the object's animator only when data is known.

```
[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    // Left/right movement.
    float horizontal = data.Horizontal;

    // Only update the animator if data is created. Do not update the animator
    // if not created as this will cause the animator to switch between having input
    // and a default value.
    if (state.ContainsCreated())
        _myAnimator.SetFloat("Horizontal", horizontal);
}
```

[!WARNING] If you are using State Order -> Inserted on the PredictionManager then Created will only ever be set on spectated objects during a reconcile. Since states on spectated objects are inserted into the replicate history they will run during reconciles rather than outside of reconciles. See the PredictionManager guide for more information on State Orders.

Ticked

As mentioned before, Ticked indicates the data has run outside a reconcile. The state can be Ticked as well as Replayed, which means it ran outside a reconcile previously but is currently running again during a replay/reconcile.

Ticked and not replayed can often be used to perform one-time actions, such as showing visual effects like jumping. You probably wouldn't want to play jump audio when the jump first occurs, as well as every time the input replays during a reconcile.

```
[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    if (data.Jump)
    {
        DoJump();
        // If ticked and not replayed then also play jump audio.
        if (state.ContainsTicked() && !state.ContainsReplayed())
            PlayJumpAudio();
    }
}
```

Preventing future state logic and movement

The replayed state is most commonly used to predict the future, or as opposition, limit the future.

Limiting future velocities is where the replayed flag is typically used the most. This keeps the object out of future prediction, which can limit real-time reflection of the object, but also prevents excessive corrections or movement snapping.

Below is a basic example showing jumping and moving without going too into depth of move rates. On a spectated object without any future checks, during a replay the object will jump and then continue to snap upward as you replay into the future (beyond data you could possibly know due to latency).

[!TIP] This behavior can be difficult to explain, but is easy to see. Try our character controller prediction demo with and without an IsFuture check.

```
[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    // Exit the method early to prevent going into the future, which would
    // result in the controller snapping upward very fast when replaying a jump.
    if (state.IsFuture())
        return;

    // Set vertical velocity to jump up.
    if (data.Jump)
        _verticalVelocity = 10f;

    Vector3 movement = new(0f, _verticalVelocity, 0f);
    // Reduce vertical velocity to begin falling, but prevent it from going too low.
    _verticalVelocity -= (float)base.TimeManager.TickDelta;
    _verticalVelocity = Mathf.Max(_verticalVelocity, -5f);

    _characterController.Move(movement);
}
```

The example above shows a very easy implementation of preventing future movement on a character controller, but rigidbodies are a little different. With rigidbodies, even if you exit the method early preventing additional added velocities, the rigidbody still carries existing velocities — this is because physics simulates with every tick, replayed or not, regardless of if replicate runs.

Here's an example **without** preventing future movement on a rigidbody.

[!INFO] The code below shows use of PredictionRigidbody. You do not need to understand how the component works to understand this example.

```
[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    float horizontal = data.Horizontal;
    Vector3 movement = new(horizontal, 0f, 0f);
    _predictionRigidbody.AddVelocity(movement);
    _predictionRigidbody.Simulate();

    // Nothing in this code prevents the rigidbody from moving into the future.
}
```

Preventing future movement on rigidbodies is a few more lines of code, but still pretty easy. We're going to use the `RigidbodyPauser` reference on the `NetworkObject` to pause the rigidbody when in the future.

```
[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    bool canChangePause = !base.IsOwner && !base.IsServerStarted;
    if (state.IsFuture())
    {
        if (canChangePause)
            base.NetworkObject.RigidbodyPauser.Pause();
    }
    else
    {
        if (canChangePause)
            base.NetworkObject.RigidbodyPauser.Unpause();

        float horizontal = data.Horizontal;
        Vector3 movement = new(horizontal, 0f, 0f);
        _predictionRigidbody.AddVelocity(movement);
        _predictionRigidbody.Simulate();
    }
}
```

As explained above, pausing a rigidbody can cause objects to pass through it due to the pauser making the rigidbody kinematic. Here's a different technique which instead simply zeros out velocities when in the future.

```

[Replicate]
private void MovePlayer(ReplicateData data, ReplicateState state = ReplicateState.Invalid,
    Channel channel = Channel.Unreliable)
{
    // If in the future then zero out velocities and exit to prevent input logic.
    // Using this approach will allow objects to still collide with the rigidbody.
    if (state.IsFuture())
    {
        _myPredictionRigidbody.Velocity(Vector3.Zero);
        _myPredictionRigidbody.AngularVelocity(Vector3.Zero);
        return;
    }

    float horizontal = data.Horizontal;
    Vector3 movement = new(horizontal, 0f, 0f);
    _predictionRigidbody.AddVelocity(movement);
    _predictionRigidbody.Simulate();
}

```


Custom Comparers

Fish-Networking generates comparers for prediction data to perform internal optimizations, but on occasion certain types cannot have comparers automatically generated.

You may see an error in the console about a type requiring a custom comparer. For example, generics and arrays specifically must have a custom comparer provided.

```
/* For example, this will create an error stating
 * byte[] needs a custom comparer. */
public struct MoveData : IReplicateData
{
    public Vector2 MoveDirection;
    public byte[] CustomData;
    // rest omitted..
}
```

While a comparer could be made automatically for the type `byte[]`, we still require you to create your own as we may not know exactly how you want to compare these types. The code below compares byte arrays by iterating every byte to check for mismatches. Given how often prediction data sends, this could potentially burden the processor.

```
[CustomComparer]
public static bool CompareByteArray(byte[] a, byte[] b)
{
    bool aNull = (a is null);
    bool bNull = (b is null);

    // Both are null.
    if (aNull && bNull)
        return true;
    // One is null, other is not.
    if (aNull != bNull)
        return false;
    // Not same lengths, cannot match.
    if (a.Length != b.Length)
        return false;

    // Both not null and same length, compare bytes.
    int length = a.Length;
    for (int i = 0; i < length; i++)
    {
        // Differs.
        if (a[i] != b[i])
            return false;
    }

    // Fall through, if here everything matches.
    return true;
}
```

The above code is a working example of how to create a custom comparer, but it may not be the most ideal comparer for your needs; this is why we require you to make your own comparer for such types.

Creating your own comparer is simple. Make a new static method with any name and boolean as the return type. Decorate the method with the `[CustomComparer]` attribute. There must also be two parameters,

each being the type you want to compare. The method logic can contain whichever code you like.

Prediction

Is the act of server-authoritative actions while allowing clients to move in real-time without delay.

- [What Is Client-Side Prediction](#)
- [Configuring PredictionManager](#)
- [Configuring TimeManager](#)
- [Configuring NetworkObject](#)
- [Offline Rigidbodies](#)
- [Interpolations](#)
- [Creating Code](#)
- [Custom Comparer](#)
- [PredictionRigidbody](#)
- [Using NetworkCollider](#)

Interpolations

Both the `PredictionManager` and `NetworkObject` have interpolation values, but with separate objectives.

`PredictionManager` interpolation is not the same as `NetworkObject` interpolation.

Interpolation on the `PredictionManager` holds a number of states in queue as interpolation, while `NetworkObject` interpolation handles graphical smoothing after running states.

A prediction interpolation of 0 — be it client or server setting — means that states would be run as soon as they are received. Most games will not use 0 interpolation as it's generally best to have at least 1 state in queue to compensate for latency. For every 1 interpolation added it's like adding latency to when spectated states are run, equal to $(\text{interpolation} * \text{TickDelta})$.

For smoothing graphical objects beneath your `NetworkObject` see `NetworkTickSmoother`. This component allows you to configure how the object is interpolated for owner and spectators.

Under the scenario your `TickSmoother` graphical had a set spectator interpolation of 2 and your `PredictionManager` an interpolation of 1, the spectated graphical object would take 2 ticks to get to its goal after the `PredictionManager` ran the state 1 interpolation later than receiving it.

Offline Rigidbodies

In some cases you will want the player to be able to interact with non-networked rigidbodies; these require a special component.

While prediction is in use, if you have a rigidbody object in your game that is not synchronized with the network it must contain an `OfflineRigidbody` component. Please review the `OfflineRigidbody` component page for more information on its uses.

PredictionRigidbody

This class provides accurate simulations and re-simulations when applying outside forces, most commonly through collisions.

Using PredictionRigidbody is very straight forward. In short, you move from applying force changes to the rigidbody onto the PredictionRigidbody instance instead.

View the Creating Code guide for using PredictionRigidbody in your replicate and reconcile methods.

Using outside the script

As mentioned above you should always apply forces to the PredictionRigidbody rather than the Rigidbody directly. Our first guides demonstrate how to do this within the replicate method, as well how to reconcile using the PredictionRigidbody, but do not show how to add forces from outside scripts, such as a bumper in your game.

There is virtually no complexity to adding outside forces other than remembering to add them again, to the PredictionRigidbody and not the Rigidbody.

The example below is what it might look like if using a trigger on a world object to repel the player.

Important: For triggers and collisions to work properly with prediction you must use our NetworkTrigger/NetworkCollision components. Otherwise, due to a Unity limitation, such interactions would not work. You can learn more about those components [here](#).

Fun fact: Fish-Networking is the only framework that has the ability to simulate Enter/Exit events with prediction; not even Fusion does this!

```
private void NetworkTrigger_OnEnter(Collider other)
{
    // Add upward impulse when hitting this trigger.
    if (other.TryGetComponent<RigidbodyPlayer>(out rbPlayer))
        rbPlayer.PredictionRigidbody.AddForce(Vector3.up, ForceMode.Impulse);

    // Do not call Simulate on the PredictionRigidbody here. That should only be done
    // within the replicate method.
}
```

There is a fair chance a number of controllers will want to set velocities directly as well.

PredictionRigidbody does support this. We encourage you to review the PredictionRigidbody API for all available functionality.

Our last example demonstrates setting velocities directly.

You would still call PredictionRigidbody.Simulate() regardless of how velocities are set.

```
float horizontal = data.Horizontal;
Vector3 velocity = new Vector3(data.Horizontal, 0f, 0f) * _moveRate;
PredictionRigidbody.Velocity(velocity);
```

Using NetworkColliders

Using each NetworkCollider component is the same, and can be used very similar to Unity callbacks.

When using client-side prediction the NetworkCollider components are designed to accurately dispatch Enter, Stay, and Exit callbacks for collisions and triggers.

All events have the same name across each component: NetworkCollider, NetworkCollider2D, NetworkTrigger, and NetworkTrigger2D. What is returned in each event naturally will vary, depending on which component you use. For example, the NetworkCollision component will return Collider for what was intersected, while NetworkCollision2D will return Collider2D.

Our OnStay callback does not return Collision as you would expect with the Unity version. Due to Unity limitations we are unable to provide a Collision callback without hotpath allocations. If you require Collision to be returned please use the Unity OnStay callback instead; this will function properly with prediction.

Callbacks

OnEnter is invoked when this collider has entered the bounds of another collider.

OnStay is invoked when this collider remains in contact with another.

OnExit is invoked when this collider has exited the bounds of another collider.

Keep in mind each callback description varies if you are using Collision, Trigger, or their 2D counterparts. None-the-less, they behave just like Unity callbacks.

Usage

This example script is placed on the player object. Our script demonstrates subscribing to all three events and using the OnEnter event to play a sound, as well applying forces to 'this' player.

```

// We are assuming this script inherits NetworkBehaviour.
[SerializeField]
private AudioSource _hitSound;

private NetworkCollision _networkCollision;

private void Awake()
{
    // Get the NetworkCollision component placed on this object.
    // You can place the component anywhere you would normally
    // use Unity collider callbacks!
    _networkCollision = GetComponent<NetworkCollision>();

    // Subscribe to the desired collision event
    _networkCollision.OnEnter += NetworkCollisionEnter;
    _networkCollision.OnStay += NetworkCollisionStay;
    _networkCollision.OnExit += NetworkCollisionExit;
}

private void OnDestroy()
{
    // Since the NetworkCollider is placed on the same object as
    // this script we do not need to unsubscribe; the callbacks
    // will be destroyed with the object.
    // But if your NetworkCollider resides on another object you
    // likely will want to unsubscribe to your events as well as shown.
    if (_networkCollision != null)
    {
        _networkCollision.OnEnter -= NetworkCollisionEnter;
        _networkCollision.OnStay -= NetworkCollisionStay;
        _networkCollision.OnExit -= NetworkCollisionExit;
    }
}

private void NetworkCollisionEnter(Collider other)
{
    // Only play the sound effect when not currently reconciling.
    // If you were to play the sound also during reconciliations
    // the audio would execute each reconcile, each tick, until the
    // player was no longer reconciling into the collider.
    if (!base.PredictionManager.IsReconciling)
        _hitSound.Play();

    // Always apply velocity to this player on enter, even if reconciling.
    PlayerMover pm = GetComponent<PlayerMover>();
    // For this example we are pushing away from the other object.
    Vector3 dir = (transform.position - other.gameObject.transform).normalized;
    pm.PredictionRigidbody.AddForce(dir * 50f, ForceMode.Impulse);
}

private void NetworkCollisionStay(Collider other)
{
    // Handle collision stay logic
}

private void NetworkCollisionExit(Collider other)
{
}

```



```
    // Handle collision exit logic (e.g., deactivate visual effects)  
}
```

You may be wondering what is `PredictionRigidbody` and why did we apply forces to that instead of the `Rigidbody` directly. `PredictionRigidbody` (and `2D`) is a mandatory but still simple way to apply forces to predicted rigidbodies. You can learn more about its usage [here](#).

What Is Client-Side Prediction

Client-Side Prediction allows clients to perform actions in real-time while maintaining server authority.

Client-side prediction is a technique used to move in real-time on clients, providing responsive actions, while also ensuring such actions cannot be cheated. From here on, we will refer to client-side prediction as CSP.

During your development you may also hear the term "server authoritative movement". CSP is a form of server authoritative movement, but they are not the same.

As mentioned, CSP allows the client to move in real-time while also ensuring they cannot cheat. Some server authoritative movement approaches will ensure the client cannot cheat but do so by moving on the server only and relaying the results to clients. While both work, the latter of moving on the server and then relaying will result in the client having a delay based on their latency.

Having such a delay would be unfair to those with higher latency and could ruin the experience for players if your game is expected to have responsive movement. This is why having CSP built into Fish-Networking is so great!

Addressables

This guide assumes you know how to load and unload addressable scenes without Fish-Networking. Below describes how to use your addressables scene logic with the Fish-Networking SceneManager.

There are a variety of ways to load and unload addressable scenes; because of this you are provided an easy way to create your own logic for accessing addressables. To begin using addressables with the SceneManager create a new script, and inherit from **DefaultSceneProcessor**.

[!INFO] LoadOptions and UnloadOptions have an addressables boolean you may utilize to know if your scene change is using addressables.

The DefaultSceneProcessor script inherits from SceneProcessorBase to implement the default scene changing functionality. Your custom script inherits from DefaultSceneProcessor because often only some of the logic needs to be changed, and this allows you to only override the differences.

As mentioned, what you need to modify may vary, but most developers find themselves overriding the following methods.

```
public abstract void BeginLoadAsync(string sceneName, LoadSceneParameters parameters);
public abstract void BeginUnloadAsync(Scene scene);
public abstract bool IsPercentComplete();
public abstract float GetPercentComplete();
public abstract IEnumerator AsyncIsDone();
```

Each method contains XML documentation to provide you a better description.

After you have completed your implementation of DefaultSceneProcessor add your newly created component to your NetworkManager object.

If the *SceneManager* component is not added to the NetworkManager object yet, add that as well. Drag your DefaultSceneProcessor implementation into the *SceneProcessor* field on the SceneManager.

You are all set now! Whenever Fish-Networking loads or unloads a scene your processor will be used and you can control the logic.

Custom Scene Processors

You can create a custom scene processor to handle how a scene is loaded/unloaded.

FishNet's SceneManager uses Unity's methods under the hood; you can override its functionality to add or change how it functions. This can be useful when using addressables or for making custom loading screens.

To start, simply create a script that inherits from `DefaultSceneProcessor`, add it to your network manager object, and then in the SceneManager component on the network manager, assign it as the **Scene Processor**.

Addressables

You will need to make a custom scene processor if you want your game to make use of addressable networked scenes.

Loading Screen

You can use a custom scene processor to easily hook up your loading screen into FishNet's networked scene loading.

Scene Management

Fish-Networking comes with a powerful scene manager tool that enables you to synchronize networked scenes with minimal effort, while also exposing a lot of powerful options.

General

The SceneManager component has many scene features to support your multiplayer needs. The links below will take you to the different guides for each feature that the SceneManager has to offer.

Tip: Visit the API to see the public items exposed to the user in the SceneManager.

Sub Pages

Automatic Online and Offline Scenes

If you simply want an online scene to be loaded when the network starts and a different one to be loaded when it stops, then you can do that with a single component.

Scene Events

"Scene Events" are the invoked events that happen along the loading and unloading method.

Scene Data

The data classes that are needed for the various features to function.

Loading Scenes

Information on how to "load" scenes and the options available to the user while loading.

Unloading Scenes

Information on how to "unload" scenes and the options available to the user while unloading.

Scene Stacking

"Scene Stacking" is the ability for server or host to load multiple instances of a scene at once, usually with different observers in each scene.

Scene Caching

"Scene Caching" is the ability for the server to keep a scene loaded when either all clients have unloaded that scene, or stopped observing that scene.

Scene Visibility

"Scene Visibility" guide offers details of using the "Scene Condition" with the ObserverManager, and how to manage observers in a scene.

Persisting NetworkObjects

"Persisting NetworkObjects" is the ability to keep a network object's state when loading and unloading scenes.

Custom Scene Processors

FishNet has the ability for users to create their own Custom Scene Processor for loading and unloading scenes. Using Addressables, for example, would need a Custom Scene Processor created.

Automatic Online And Offline Scenes

Using the `DefaultScene` component to automatically manage simple scene setups.

If you're looking for a simple way to manage scene transitions when your network starts or stops, FishNet has an easy way to manage it without any custom code. The `DefaultScene` component is a ready-made solution that automatically loads your designated online scene when the network starts, and your offline scene when the network shuts down.

This is perfect for developers who want a plug-and-play setup without writing custom scene logic.

Setup Instructions

To get started:

- Add the `DefaultScene` component to your `NetworkManager` `GameObject`.

- In the Inspector, assign your **Offline Scene** and **Online Scene**.

- Make sure the two scenes are different — using the same scene for both will prevent the component from functioning properly.

Loading Scenes

{% embed url="https://www.youtube.com/watch?index=3&list=PLkx8oFug638qBthd3nF98zAtOdI8oGb&v=PhWKZsL0Za4" %} Loading scenes video guide {% endembed %}

General

This guide will go over how to setup and load new scenes, how to load into existing scenes, how to replace scenes, and advanced info on what happens behind the "scenes" during a load.

{% hint style="success" %} Having a simple online and offline scene is very to do through the built-in DefaultScene component. You can read more about using it [here](#). {% endhint %}

{% hint style="info" %} Loading Scenes globally or by connection will add the specified clients connection as an **Observer** to the scenes if utilizing the **ObserverManagers** Scene Condition, so globally will add all clients as Observers, and by connection will only add the connections specified. - See **Scene Visibility** for more info. {% endhint %}

{% hint style="warning" %} In the Examples below, all SceneManager calls are done inside a NetworkBehaviour class, that is why you can get reference to SceneManager by doing base.SceneManager.\ If you would like a reference outside of a NetworkBehaviour consider using FishNet's **InstanceFinder**.SceneManager. {% endhint %}

<details>

<summary>Setup</summary>

Before calling the SceneManager's Load Scene functions you will need to setup the load data to tell the **SceneManager** how you want it to handle the scene load.

SceneLookupData

SceneLookupData is the class used to specify what scene you want the **SceneManager** to load. You do not need to create the lookup data manually but can instead use the SceneLoadData constructors which will create the SceneLookupData automatically.

SceneLoadData

When loading a scene in any way, you must pass in an instance of a **SceneLoadData** class into the load methods. This class provides the scene manager all of the info it needs to load the scene or scenes properly.

The constructors available for **SceneLoadData** will automatically create the **SceneLookupData** needed for the SceneManager to handle if you are loading a new scene, or an existing instance of one.

</details>

<details>

<summary>Loading New Scenes</summary>

Scenes can be loaded globally or for any number of specified clients, including none (which will only load the scene on the server).

{% hint style="info" %} Loading new Scenes can only be done by Name, you **cannot** use Handles or Scene References. {% endhint %}

Global Scenes

- Global Scenes can be loaded by calling LoadGlobalScenes() in the SceneManager.

- When loaded globally, scenes will be loaded for all current, and future clients.

```
SceneLoadData sld = new SceneLoadData("Town");
base.SceneManager.LoadGlobalScenes(sld);
```

Connection Scenes

Connection Scenes follow the same principle but have a few method overloads.

- You can load scenes for a single connection, multiple connections at once, or load scenes only on the server in preparation for connections.
- When loading by connection only the connections specified will load the scenes.
- You can add additional connections into a scene at any time.

```
SceneLoadData sld = new SceneLoadData("Main");

// Load scenes for a single connection.
NetworkConnection conn = base.Owner;
base.SceneManager.LoadConnectionScenes(conn, sld);

// Load scenes for several connections at once.
NetworkConnection[] conns = new NetworkConnection[] { connA, connB };
base.SceneManager.LoadConnectionScenes(conns, sld);

// Load scenes only on the server. This can be used to preload scenes
// that you don't want all players in.
base.SceneManager.LoadConnectionScenes(sld);
```

Loading Multiple Scenes

- Whether loading globally or by connection, you can load more than one scene in a single method call.
- When loading multiple scenes in one call, the NetworkObjects you put into **Moved NetworkObjects** will be moved to the first valid scene in the list of scenes you tried to load. See Persisting NetworkObjects for more info about keeping NetworkObjects across scenes.

```
// Loading Multiple Connections into Multiple Scenes
string[] scenesToLoad = new string[] { "Main", "Additive" };
NetworkConnection[] conns = new NetworkConnection[] { connA, connB, connC };

SceneLoadData sld = new SceneLoadData(scenesToLoad);
base.SceneManager.LoadConnectionScenes(conns, sld);
```

</details>

<details>

<summary>Loading Existing Scenes</summary>

If the scene is already loaded on the server, and you want to load clients into that instance of the scene. Most likely you will want to lookup that scene by scene reference, or handle to make sure you are getting the exact scene you need.

If you load the scene by name, it will load the connections into the first scene found with that name. If you are utilizing **Scene Stacking**, then there may be multiple scenes loaded with the same name. So be alert when loading into existing scenes by name.

You can load clients into scenes that have no other clients in them if you are utilizing **Scene Caching** - the ability to keep a scene loaded with its current state on the server when all clients leave the scene.

Getting References to a Loaded Scene

Here are a few ways to get reference to the scenes that you already loaded using FishNet's **SceneManager** .

By Event:

```
// Manage your own collection of SceneReferences/Handles
// Customize how you want to manage you scene references so it's easy
// for you to find them later.
List<Scene> ScenesLoaded = new();

public void OnEnable()
{
    InstanceFinder.SceneManager.OnLoadEnd += RegisterScenes;
}

public void RegisterScenes(SceneLoadEventArgs args)
{
    // Only register on the server
    if (!args.QueueData.AsServer) return;

    // if you know you only loaded one scene you could just grab index [0]
    foreach(var scene in args.loadedScenes)
        ScenesLoaded.Add(scene);
}

public void OnDisable()
{
    InstanceFinder.SceneManager.OnLoadEnd -= RegisterScene;
}
```

By Connection:

```
// NetworkConnections have a list of Scenes they are currently in.
int clientToLookup;
InstanceFinder.ServerManger.Clients[clientToLookup].Scenes;
```

By SceneManager.SceneConnections:

```
// SceneManager keeps a Dictionary of all connection scenes as the key
// and the client connections that are in that scene as the value.
NetworkConnection conn;
Scene sceneNeeded;

// Get the scene you need with foreach or use LINQ to filter your conditions.
foreach(var pair in SceneManager.SceneConnections)
{
    if (pair.Value.Contains(conn))
        sceneNeeded = pair.Key;
}
```

Using Reference to Load Into Existing Instance

Use the methods above to get the reference or handle of a scene, and use that reference or handle to load a client into an existing scene.

```

scene sceneReference;
NetworkConnection[] conns = new() {connA,connB};

// by reference
SceneLoadData sld = new(sceneReference);
base.SceneManager.LoadSceneScenes(conns, sld);

// by handle
SceneLoadData sld = new(sceneReference.handle);
base.SceneManager.LoadSceneScenes(conns, sld);

```

</details>

<details>

<summary>Replacing Scenes</summary>

Fishnet gives the ability to replace scenes that are already loaded on the clients with the new requested scenes to load.

To Replace Scenes, you will set the ReplaceScene Option in the SceneLoadData

Replaced scenes will be unloaded before the new scenes are loaded.

Replacing Scenes by Default will replace scenes on both the server and clients. If you would like the server to keep the scene loaded and only replace the scene on the clients - see [**Scene Caching**](#) for more details.

Replace None:

This is the default method when loading, it will ignore the replace options and load the scene in normally.

Replace All:

This will replace all scenes currently loaded in unity, even ones not managed by FishNet's SceneManager.

```

// Replace All Option.
SceneLoadData sld = new SceneLoadData("DungeonScene");
sld.ReplaceScenes = ReplaceOption.All;

// This will replace all Scenes loaded by FishNet or outside of FishNet like Unity,
// and load "DungeonScene"
SceneManager.LoadScene(sld);

```

Replace Online Only:

This will replace only scenes managed by the SceneManager in FishNet.

```

// Replace Online Only Option.
SceneLoadData sld = new SceneLoadData("DungeonScene");
sld.ReplaceScenes = ReplaceOption.OnlineOnly;

// This will replace only scenes managed by the SceneManager in FishNet.
SceneManager.LoadScene(sld);

```

</details>

<details>

<summary>Advanced Info</summary>

Behind the "Scenes"

The [**SceneManager**](#) Class has very detailed XML comments on how the load process works in detail, if you need to troubleshoot the scene load process, these comments will help you understand the flow of how

a scene loads.

Events

Make sure to check out the ****Scene Events**** that you can subscribe to in order to have better control over your game.

More Control

If you want more control over how FishNet loads and unloads scenes, you can create a custom scene processor to override FishNet's functionality. Learn more about that here.

</details>

Loading Scenes

Instructions for loading networked scenes with FishNet, both on the server and on clients.

General

This guide will go over how to setup and load new scenes, how to load into existing scenes, how to replace scenes, and advanced info on what happens behind the scenes during a load.

Tip: Having a simple online and offline scene is very easy to do through the built-in `DefaultScene` component.

Note: Loading scenes globally or by connection will add the specified client's connection as an Observer to the scenes if utilizing the `ObserverManager`'s Scene Condition — so globally will add all clients as Observers, and by connection will only add the connections specified. See [Scene Visibility](#) for more info.

Note: In the examples below, all `SceneManager` calls are done inside a `NetworkBehaviour` class, which is why you can get a reference to `SceneManager` by doing `base.SceneManager`. If you would like a reference outside of a `NetworkBehaviour`, consider using `InstanceFinder.SceneManager`.

Setup

Before calling the `SceneManager`'s load scene functions you will need to set up the load data to tell the `SceneManager` how you want it to handle the scene load.

SceneLookupData — The class used to specify what scene you want the `SceneManager` to load. You do not need to create the lookup data manually but can instead use the `SceneLoadData` constructors which will create the `SceneLookupData` automatically.

SceneLoadData — When loading a scene in any way, you must pass in an instance of a `SceneLoadData` class into the load methods. This class provides the scene manager all of the info it needs to load the scene or scenes properly.

Loading New Scenes

Scenes can be loaded globally or for any number of specified clients, including none (which will only load the scene on the server).

Note: Loading new scenes can only be done by name; you cannot use handles or scene references.

Global Scenes

Global scenes can be loaded by calling `LoadGlobalScenes()` in the `SceneManager`. When loaded globally, scenes will be loaded for all current and future clients.

```
SceneLoadData sld = new SceneLoadData("Town");
base.SceneManager.LoadGlobalScenes(sld);
```

Connection Scenes

Connection scenes follow the same principle but have a few method overloads. You can load scenes for a single connection, multiple connections at once, or load scenes only on the server in preparation for connections. When loading by connection, only the connections specified will load the scenes.

```

SceneLoadData sld = new SceneLoadData("Main");

// Load scenes for a single connection.
NetworkConnection conn = base.Owner;
base.SceneManager.LoadConnectionScenes(conn, sld);

// Load scenes for several connections at once.
NetworkConnection[] conns = new NetworkConnection[] { connA, connB };
base.SceneManager.LoadConnectionScenes(conns, sld);

// Load scenes only on the server. This can be used to preload scenes
// that you don't want all players in yet.
base.SceneManager.LoadConnectionScenes(sld);

```

Loading Multiple Scenes

Whether loading globally or by connection, you can load more than one scene in a single method call. When loading multiple scenes in one call, the `NetworkObjects` you put into `MovedNetworkObjects` will be moved to the first valid scene in the list of scenes you tried to load.

```

// Loading multiple connections into multiple scenes.
string[] scenesToLoad = new string[] { "Main", "Additive" };
NetworkConnection[] conns = new NetworkConnection[] { connA, connB, connC };
SceneLoadData sld = new SceneLoadData(scenesToLoad);
base.SceneManager.LoadConnectionScenes(conns, sld);

```

Loading Existing Scenes

If the scene is already loaded on the server and you want to load clients into that instance of the scene, most likely you will want to look up that scene by scene reference or handle to make sure you are getting the exact scene you need.

If you load the scene by name, it will load the connections into the first scene found with that name. If you are utilizing Scene Stacking, then there may be multiple scenes loaded with the same name.

Getting References to a Loaded Scene

By Event:

```

List<Scene> ScenesLoaded = new();

public void OnEnable()
{
    InstanceFinder.SceneManager.OnLoadEnd += RegisterScenes;
}

public void RegisterScenes(SceneLoadEventArgs args)
{
    // Only register on the server.
    if (!args.QueueData.AsServer) return;

    foreach (var scene in args.LoadedScenes)
        ScenesLoaded.Add(scene);
}

public void OnDisable()
{
    InstanceFinder.SceneManager.OnLoadEnd -= RegisterScenes;
}

```

By Connection:

```

int clientToLookup;
InstanceFinder.ServerManager.Clients[clientToLookup].Scenes;

```

By SceneManager.SceneConnections:

```

// SceneManager keeps a Dictionary of all connection scenes as the key
// and the client connections that are in that scene as the value.
NetworkConnection conn;
Scene sceneNeeded;

foreach (var pair in SceneManager.SceneConnections)
{
    if (pair.Value.Contains(conn))
        sceneNeeded = pair.Key;
}

```

Using Reference to Load Into Existing Instance

```

Scene sceneReference;
NetworkConnection[] conns = new[] { connA, connB };

// By reference.
SceneLoadData sld = new(sceneReference);
base.SceneManager.LoadConnectionScenes(conns, sld);

// By handle.
SceneLoadData sldByHandle = new(sceneReference.handle);
base.SceneManager.LoadConnectionScenes(conns, sldByHandle);

```

Replacing Scenes

FishNet gives the ability to replace scenes that are already loaded on the clients with the new requested scenes to load. To replace scenes, set the `ReplaceScenes` option in the `SceneLoadData`. Replaced scenes will be unloaded before the new scenes are loaded.

Replace None — This is the default method when loading; it will ignore the replace options and load the

scene normally.

Replace All — This will replace all scenes currently loaded in Unity, even ones not managed by FishNet's SceneManager.

```
SceneLoadData sld = new SceneLoadData("DungeonScene");
sld.ReplaceScenes = ReplaceOption.All;
// This will replace all scenes loaded by FishNet or outside of FishNet,
// and load "DungeonScene".
SceneManager.LoadScene(sld);
```

Replace Online Only — This will replace only scenes managed by the SceneManager in FishNet.

```
SceneLoadData sld = new SceneLoadData("DungeonScene");
sld.ReplaceScenes = ReplaceOption.OnlineOnly;
// This will replace only scenes managed by the SceneManager in FishNet.
SceneManager.LoadScene(sld);
```

Advanced Info

The SceneManager class has very detailed XML comments on how the load process works in detail. If you need to troubleshoot the scene load process, these comments will help you understand the flow.

Make sure to check out the Scene Events that you can subscribe to in order to have better control over your game.

If you want more control over how FishNet loads and unloads scenes, you can create a custom scene processor to override FishNet's functionality.

Persisting NetworkObjects

General

The options available to keep NetworkObjects persisting across scenes depends on the type of NetworkObject.

Spawned NetworkObjects

Spawned NetworkObjects that do not fall into the other categories below can persist between scenes by moving them while loading into the next scene. The SceneLoadData that is passed into the load method of the SceneManager has an array you can populate with all of the spawned NetworkObjects you want to send to the newly loaded scene.

The SceneManager will handle the objects correctly for you and also log a warning if you try to send a GameObject that is not allowed.

Important: If you load multiple scenes in one method call and you are moving network objects using SceneLoadData, the moved NetworkObjects will move into the first valid scene requested.

Example

```
// Create an array of NetworkObjects
// that are not Scene, Global, or Nested NetworkObjects.
NetworkObject[] objectsToMove = new NetworkObject[] { object1, object2, object3 };

// Assign this array to the SceneLoadData before you load a scene.
SceneLoadData sld = new SceneLoadData("NewScene");
sld.MovedNetworkObjects = objectsToMove;

// FishNet will handle the rest after loading!
SceneManager.LoadScene(sld);
```

Scene NetworkObjects

Scene NetworkObjects currently cannot persist across scenes; it is a limitation with the way Unity and FishNet were designed. You cannot mark them as IsGlobal, or put them into the "DontDestroyOnLoad" scene. If you would like a scene NetworkObject to persist across scenes it is recommended to remove them and use the other options available on this page.

Global NetworkObjects

Global NetworkObjects work similarly to how a normal GameObject would when put into the DontDestroyOnLoad (DDOL) scene. When loading and unloading scenes, global NetworkObjects will stay in the DDOL scene on both the server and client, persisting their state. No extra steps needed.

To make a NetworkObject global, just mark the IsGlobal boolean true on the NetworkObject component.

Tip: Clients are typically always an observer of the DDOL scene, so global objects may make more sense for manager-type GameObjects rather than GameObjects that have meshes — but you are not limited to this.

Nested NetworkObjects

Unity will not allow nested GameObjects to be moved into other scenes. However, FishNet will automatically

detect if you are trying to send a `NestedNetworkObject` and send the root of the object instead!

Scene Caching

General

When loading a scene you can specify in the `SceneLoadData` whether to `AutomaticallyUnload` the scene when all clients unload or leave the scene as an observer. When unloading the scene you can override the `AutomaticallyUnload` option by setting the `ServerUnloadMode`.

Examples

```
// When Loading Scenes.
SceneLoadData sld = new SceneLoadData("MainScene");
sld.Options.AutomaticallyUnload = false;

// When Manually Unloading Scenes.
// Whatever you set the ServerUnloadMode to here will override the AutomaticallyUnload
// setting you used when loading the scene earlier.
SceneUnloadData sud = new SceneUnloadData("MainScene");
sud.Options.Mode = ServerUnloadMode.KeepUnused;
```

Host Behaviour

In situations where the host's server needs to keep a scene loaded, but the host's client was requested to unload that scene — instead of unloading, the host's client will be removed from the scene using the observer system, updating the client's scene visibility. As host, the server and client share the same instance of loaded scenes and `GameObjects`. If it were to actually unload a scene from the host client, it would also unload on the server.

Warning: A Scene Condition must be set in the `ObserverManager` for you to utilize the ability to have the server keep a scene loaded while the host client does not see the objects in that scene.

Important: This usually means that every `GameObject` with a mesh will have to have a `NetworkObject` attached if you want the host client not to visualize the scene.

Scene Data

Sub-Pages

- [SceneLookupData](#)
- [SceneLoadData](#)
- [SceneUnloadData](#)

SceneLoadData

General

Loading scenes of all types depend upon SceneLoadData. The SceneLoadData class contains information about what scenes to load, how to load them, which objects to move to new scenes, and more. You can view the SceneLoadData API [here](#).

Default values

```
SceneLoadData sld = new SceneLoadData()
{
    PreferredActiveScene = null,
    SceneLookupDatas = new SceneLookupData[0],
    MovedNetworkObjects = new NetworkObject[0],
    ReplaceScenes = ReplaceOption.None,
    Params = new LoadParams()
    {
        ServerParams = new object[0],
        ClientParams = new byte[0]
    },
    Options = new LoadOptions()
    {
        AutomaticallyUnload = true,
        AllowStacking = false,
        LocalPhysics = LocalPhysicsMode.None,
        ReloadScenes = false, // Obsolete, Coming Soon.
        Addressables = false
    }
};
```

<details>

<summary>PreferredActiveScene</summary>

Preferred Active Scene will allow you to choose what scene will be active on the server and/or client. You can choose to have the chosen SceneLookupData apply to both the server and client or separate ones for each.

If left with the default value of null, the first valid scene loaded will become the ActiveScene.

</details>

<details>

<summary>SceneLookupDatas</summary>

This Array is populated with scenes you would like to load, depending on the parameters you pass into the SceneLoadData when constructed.

See [**Loading Scenes**](#) for examples.

</details>

<details>

<summary>MovedNetworkObjects</summary>

Instantiated NetworkObjects can be moved when loading new scenes, such as if you want to move a player to a different scene as you load the new scene. You may include an array of NetworkObjects to move to the

new scenes. NetworkObjects within this array will be moved to the first scene specified in SceneLookupData. \ \ See **Persisting NetworkObjects** for more details on what type of NetworkObjects you are allowed to move.

</details>

<details>

<summary>ReplaceScenes</summary>

Like the Unity SceneManager when loading a single scene, ReplaceScenes allows you to replace currently loaded scenes with new ones. There are a variety of options to use. \ \ See **Replacing Scenes** Section of Loading Scenes guide for more details and examples.

</details>

<details>

<summary>Params</summary>

Params are an optional way to assign data to your scene loads/unloads. This data will be available within **Scene Events**, Information used in Params can be useful for storing information about the scene load/unload and referencing it later when the scene load/unload completes.

ServerParams

ServerParams are only included on the server side, and are not networked. It is an array of objects, meaning you can send anything you want. However when accessing the Params through event args, you will have to cast the object to the data you want.

ClientParams

ClientParams is a byte array which may contain anything, and will be sent to clients when they receive the load scene instructions. Clients can access the *ClientParams* within the scene change events.

</details>

<details>

<summary>Options</summary>

You may further enhance how you load/unload scenes with Options.

AutomaticallyUnload

- When *set to true* scenes will be unloaded automatically on the server when there are no more connections present. This is the default behaviour.
- When set to false the scene will remain if connections leave the scene unexpected, such as being disconnected.
- However, discussed in UnloadSceneData, this behavior can be overridden using the UnloadOptions of UnloadSceneData.
- Only scenes loaded for connections will be automatically unloaded when emptied.
- Global scenes can only be unloaded using ReplaceScenes or by calling unload on them.

AllowStacking

- When *AllowStacking* remains false the SceneManager will not stack scenes in your SceneLoadDatas.
- If true, then scenes can be stacked (loaded multiple times).
- In the SceneLookupData section it was mentioned that if a Scene reference or handle is specified then the SceneManager will favor loading a scene using a scene handle. When you would like to load

connections into the same stacked scene over multiple load calls, you will populate your SceneLookupDatas by Scene reference or handle.

- See **Scene Stacking** for more detail and examples

LocalPhysics

- LocalPhysics is a Unity property that lets you determine how physics are simulated in your scenes.
- Generally, if you are stacking scenes you will want to set a LocalPhysics mode so that stacked scenes do not collide with each other.

Addressables

- *Addressables* is only used as a reference and performs no additional functionality.
- You may set this value to know if a scene is loading using addressables, without having to create Params.

</details>

Scenelookupdata

General

Important: When creating SceneLoadData or SceneUnLoadData there are provided constructors that automatically create SceneLookupData for you. Most likely you will be using these constructors and will not be creating a separate SceneLookupData.

When you specify a scene by reference or handle, the SceneManager will prefer to look up that scene using the scene handle. This is important information when using Scene Stacking. Looking up a scene by handle or object reference will place connections in the scene specified, but when using scene names, the server will create a new scene instance for the specified connections and place them into that scene. The described behavior only applies when loading scenes over multiple load calls; such as if you call LoadConnectionScene twice, each with its own connection.

Default Values

Note: A Scene Handle is an integer that Unity generates for a scene instance when it is loaded. You can use it to specify an instance of a scene that is already loaded, but not for loading a new instance of that scene.

```
// SceneLookupData default values
SceneLookupData slud = new SceneLookupData()
{
    // If Handle is greater than 0 then it will ignore Name and use Handle
    // to look up the scene.
    Handle = 0,

    // If Handle is set to 0, then Name is used to look up the scene instead.
    Name = null
};
```

See Loading Scenes and Unloading Scenes for implementation.

SceneUnloadData

General

When unloading scenes information on what to unload is constructed within the SceneUnloadData class. SceneUnloadData is very similar to SceneLoadData. The API for SceneUnloadData can be found [here](#).

Default values

```
SceneUnloadData sud = new SceneUnloadData()
{
    SceneLookupDatas = new SceneLookupData[0],
    Params = new UnloadParams()
    {
        ServerParams = new object[0],
        ClientParams = new byte[0]
    },
    Options = new UnloadOptions()
    {
        Mode = ServerUnloadMode.UnloadUnused,
        Addressables = false
    }
};
```

<details>

<summary>SceneLookupDatas</summary>

This Array is populated with the scenes you want to unload, depending on the parameters you pass into the SceneUnloadData when constructed.

See [**Unloading Scenes**](#) for examples.

</details>

<details>

<summary>Params</summary>

Params are an optional way to assign data to your scene loads/unloads. This data will be available within [**Scene Events**](#), Information used in Params can be useful for storing information about the scene load/unload and referencing it later when the scene load/unload completes.

ServerParams

ServerParams are only included on the server side and are not networked. It is an array of objects, meaning you can send anything you want. However when accessing the Params through event args, you will have to cast the object to the data you want.

ClientParams

ClientParams is a byte array which may contain anything. It will be sent to clients when they receive the load scene instructions. Clients can access the *ClientParams* within the scene change events.

</details>

<details>

<summary>Options</summary>

Like with Options in loading, the UnloadOptions offer additional settings when unloading.

Mode

These values will override the `AutomaticallyUnload` Option that was used when loaded the scene. If you set *AutomaticallyUnload* to false but specified *ServerUnloadModes.UnloadUnused* then the scene would be unloaded when emptied.

ServerUnloadModes.UnloadUnused

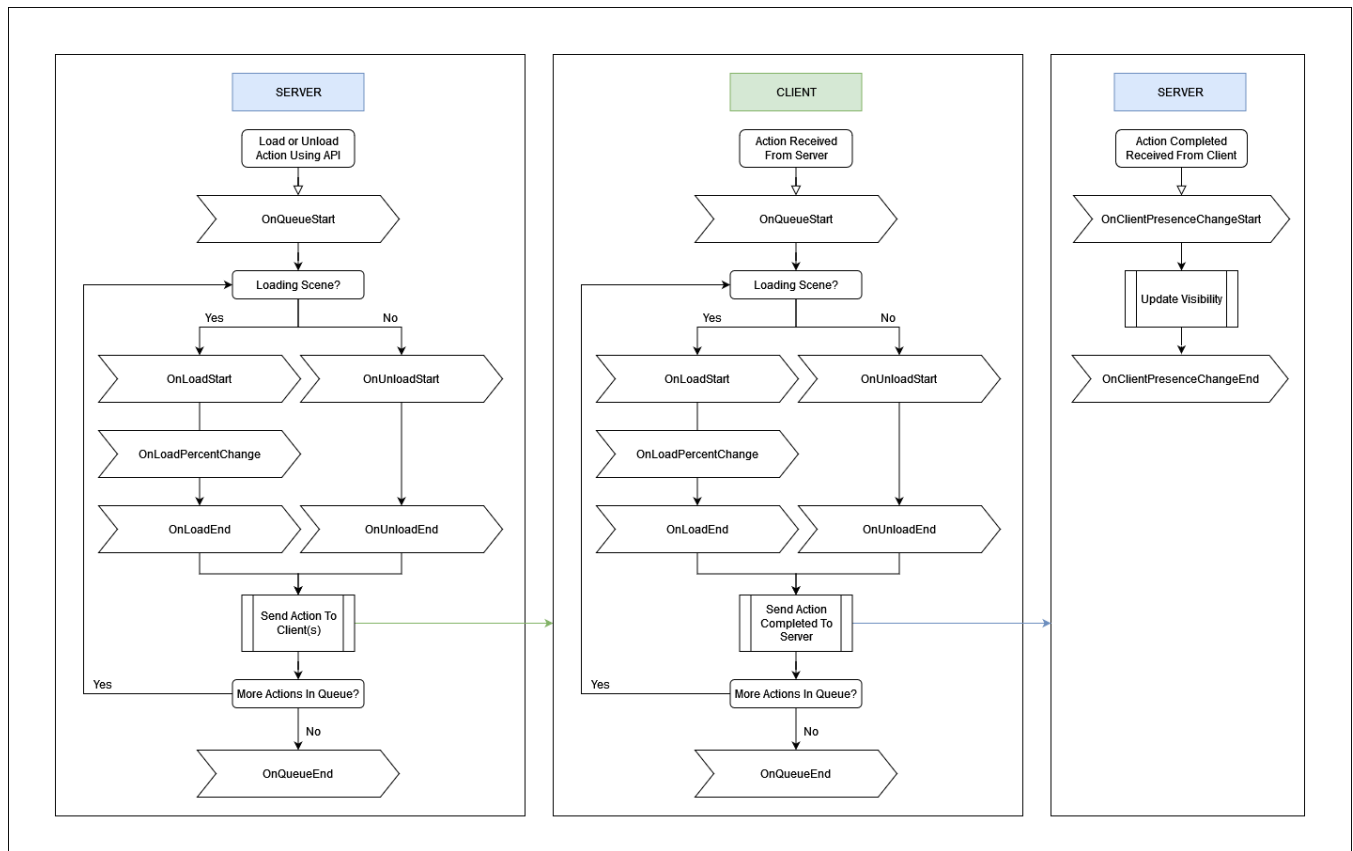
- This is the default setting which will only unload a scene which is no longer used.

ServerUnloadModes.KeepUnused

- This option will keep the scene loaded on the server if all clients have been removed. See **Scene Caching** for more details

</details>

Scene Events



General

There are a variety of events within SceneManager to help with your development. The order in which events are invoked is reliable.

Note: Client and server process queues and events exactly the same. **OnClientPresenceChange** is a special exception, as discussed below.

Events

OnClientLoadedStartScenes

This event calls every time a client loads the starting scenes for the first time. The start scenes would be any global scenes you have set, or if no global scenes are set then the scene you are entering play into. The `PlayerSpawner.cs` example demonstrates usage of the `OnClientLoadedStartScenes` event. Included is the connection (or client) which loaded the scenes, and a boolean indicating `true` if the callback is on the server side or `false` if on the client side.

The `NetworkConnection` class also has a similar event `OnLoadedStartScenes` if you'd like to know when only a specific connection has loaded the start scenes.

OnQueueStart, OnQueueEnd

`OnQueueStart` and `OnQueueEnd` are called when a scene change queue occurs. Start will only call if a scene has successfully begun to load or unload. The queue may process any number of scene events. For

example: if a scene is told to unload while a load is still in progress, then the unload will be placed in the queue. `OnQueueEnd` will invoke after both the load and unload have completed.

OnLoadStart, OnUnloadStart

`OnLoadStart` and `OnUnloadStart` occur before a queue entry is processed. These will call for every entry in a queue. For example: if you call `SceneManager.LoadGlobalScenes()` twice in a row, `OnQueueStart` will invoke once, and `OnLoadStart` will invoke twice. `OnLoadStart` will be called when a scene load begins, and `OnUnloadStart` when an unload begins.

Both the Load and Unload events contain a structure which has a field labeled `QueueData`. `QueueData` holds information about the queue entry. The provided data is exposed for your convenience.

OnLoadPercentChange

Only available when loading scenes is the `OnLoadPercentChange` event. This event contains information about what percentage of your scenes have loaded for the queue entry. Like the Start events, `QueueData` is provided. You will also be supplied a `float Percent`, indicating the total progress of your scene load. This can be useful to show loading screens, or perform a variety of initialization tasks.

OnLoadEnd

The `OnLoadEnd` event is called after all scenes for the queue entry have been loaded. This event will only invoke after the scenes have fully loaded, and after the active scene has been set if applicable. Like the `OnLoadStart` event, `QueueData` is provided. Additionally, `LoadedScenes` and `SkippedSceneNames` are included. `LoadedScenes` provides Scene references to which scenes were loaded. `SkippedSceneNames` contains strings of scenes which were not loaded; this generally occurs if the scene is already loaded.

Important: You may want to spawn objects for players once a scene is loaded, but it's best to do that once the specific client has started observing the scene to prevent race conditions. Thus it's better to use one of the `OnClientPresenceChange` events or `OnSpawnServer` from a `NetworkBehaviour` in the scene.

OnUnloadEnd

After scene unloading has occurred, `OnUnloadEnd` is invoked. This event contains `QueueData` and `UnloadedSceneHandles`. As before, `QueueData` contains information about the queue entry. `UnloadedSceneHandles` is a collection occupied with the handles of which scenes were unloaded.

OnClientPresenceChangeStart, OnClientPresenceChangeEnd

These two events are only available on the server and indicate when a client is being added to a scene, or being removed from a scene. Both events are only invoked after the client has fully loaded or unloaded the scene. The `Start` variant is invoked before the observer status has been updated for the client, and `End` after being updated. To clarify, if a client joins a scene `OnClientPresenceChangeStart` will call before the client has visibility of any networked content in the scene, and `OnClientPresenceChangeEnd` after the client has gained visibility.

Both events contain the same structure which has the following information: `Scene`, `Connection`, and `Added`. `Scene` is which scene the client is being added or removed from. `Connection` is the `NetworkConnection` for the client. Lastly, `Added` will be `true` if being added to the scene, or `false` if being removed.

Scene Stacking

General

A good example of stacking scenes is having dungeon instances on a single server. If two clients went into the same dungeon, each client will load their own copy of that dungeon and have individual `GameObjects` and state for those scenes. The server however has two instances of that scene loaded at once. The server having those two instances of the same scene loaded is called "Scene Stacking". Stacked scenes typically have different clients observing each instance of the scene.

Stacking Scenes

Loading into a New Stacked Scene

To stack scenes you must set `AllowStacking` to `true` in your `SceneLoadData`'s `Options`. To create a new instance of a stacked scene the `SceneLookupData` must be populated by using the scene name.

Note: Global scenes cannot be stacked!

```
// Select connections to load into new stacked scene.
NetworkConnection[] conns = new NetworkConnection[] { connA, connB };

// You must use the scene name to stack scenes!
SceneLoadData sld = new SceneLoadData("DungeonScene");

// Set AllowStacking option to true.
sld.Options.AllowStacking = true;

// Decide if you want separate physics for the scene.
sld.Options.LocalPhysics = LocalPhysicsMode.Physics3D;

// Load the scene via connections; you cannot stack global scenes.
base.SceneManager.LoadConnectionScenes(conns, sld);
```

Loading into an Existing Stacked Scene

If you were to load two connections into a scene by scene reference or handle they will be added to the same scene, regardless of whether `AllowStacking` is `true` or not. This is identical to the examples given in the Loading Scenes guide on how to load into existing scenes.

Separating Physics

You may want to separate physics while stacking scenes. This ensures that the stacked scenes' physics do not interact with each other. Set the `LocalPhysics` option in the `SceneLoadData`.

- `LocalPhysicsMode.None` — Default option. Scene physics will collide with other scenes in this state.
- `LocalPhysicsMode.Physics2D` — A local 2D physics scene will be created and owned by the scene.
- `LocalPhysicsMode.Physics3D` — A local 3D physics scene will be created and owned by the scene.

If you are using separate physics scenes and want to also simulate physics within them you must do so manually. Below is a script you can place in your stacked physics scenes to simulate physics alongside the default physics scenes.

```

using FishNet.Object;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// Synchronizes scene physics if not the default physics scene.
/// </summary>
public class PhysicsSceneSync : NetworkBehaviour
{
    [SerializeField] private bool _synchronizePhysics2D;
    [SerializeField] private bool _synchronizePhysics;

    private static HashSet<int> _synchronizedScenes = new HashSet<int>();

    public override void OnStartNetwork()
    {
        int sceneHandle = gameObject.scene.handle;
        if (_synchronizedScenes.Contains(sceneHandle))
            return;

        _synchronizePhysics = (gameObject.scene.GetPhysicsScene() != Physics.defaultPhysicsScene);
        _synchronizePhysics2D = (gameObject.scene.GetPhysicsScene2D() != Physics2D.defaultPhysicsScene);

        if (_synchronizePhysics || _synchronizePhysics2D)
        {
            _synchronizedScenes.Add(sceneHandle);
            base.TimeManager.OnPrePhysicsSimulation += TimeManager_OnPrePhysicsSimulation;
        }
    }

    public override void OnStopNetwork()
    {
        if (_synchronizePhysics || _synchronizePhysics2D)
        {
            _synchronizedScenes.Remove(gameObject.scene.handle);
            base.TimeManager.OnPrePhysicsSimulation -= TimeManager_OnPrePhysicsSimulation;
        }
    }

    private void TimeManager_OnPrePhysicsSimulation(float delta)
    {
        if (_synchronizePhysics)
            gameObject.scene.GetPhysicsScene().Simulate(delta);

        if (_synchronizePhysics2D)
            gameObject.scene.GetPhysicsScene2D().Simulate(delta);
    }
}

```

Scene Visibility

General

You can control if clients become an observer of `GameObjects` in a scene that they are in. This is possible if you set your `ObserverManager` to include a `Scene Condition`. The `Scene Condition` ensures that `NetworkObjects` — both scene and spawned — are only visible to players in the same scene as the object. In most cases you will want to add a `NetworkObserver` component with a scene condition to your networked objects.

Note: When encountering an error about being unable to find a `NetworkObject` or `RPCLink` during a scene change, you likely forgot to add the scene condition.

Managing Visibility

Initial Scene Load

When a client is loading into the game for the first time, the first scene/offline scene loaded is done so by the `Unity SceneManager`. This means that clients were not automatically added to the scene when connected, which also means they do not have visibility of that scene.

Once a client connects to a host or server, if you use the `PlayerSpawner` component FishNet provides on the `NetworkManager`, there is logic in that component that will add the client to the default scene on the player object's spawn.

Warning: If you decided not to use the `PlayerSpawner` component provided on the `NetworkManager` in FishNet, you will either have to load a client into a scene using FishNet's `SceneManager`, or call `SceneManager.AddOwnerToDefaultScene()` on an object that you gave that client ownership of.

Adding Client Connections to Scenes

When loading a scene globally or by connection, the `SceneManager` will automatically place that connection into the scene. Once added, that client will be able to view all `NetworkObjects` that are part of that scene.

You can add client connections to scenes manually if the scene is already loaded on the client by using `SceneManager.AddConnectionsToScene()`.

Note: Manually adding and removing client connections is recommended for power users only.

Removing Client Connections from Scenes

When unloading a scene from a client, the server will automatically remove the client connection from the scene.

If you are the host, and you unload the host's client from a scene, it will only remove the host from the scene and the host's client will lose visibility. See `Host Behaviour` in the `Scene Caching` guide for more details.

You can remove a client manually from a loaded scene by using `SceneManager.RemoveConnectionsFromScene()`.

Unloading Scenes

Instructions for how to unload networked scenes in FishNet.

General

Unloading scenes is the same as loading scenes, except you call `Unload` rather than `Load`. Scenes can be unloaded by connection, or globally. When unloaded globally, scenes will be unloaded for all players. When unloading by connection, only the connections specified will unload the scenes.

Unloading Scenes

Global Scenes

Global scenes can be unloaded by calling `UnloadGlobalScenes()` in the `SceneManager`. You cannot unload a global scene with the `UnloadConnectionScenes()` method.

```
SceneUnloadData sud = new SceneUnloadData("Town");
base.NetworkManager.SceneManager.UnloadGlobalScenes(sud);
```

Connection Scenes

Connection scenes follow the same principle, but have a few method overloads. You can unload scenes for a single connection, multiple connections at once, or unload scenes on the server.

The server will only unload a connection scene on itself if all connections have been unloaded from that scene.

If you wish to unload a scene and all connections, get all the connections in the scene and call `unload` with those connections. `SceneManager.SceneConnections` holds all online scenes and the connections in them.

If you wish to keep a scene loaded on the server when unloading all connections, see [Scene Caching](#).

```
SceneUnloadData sud = new SceneUnloadData(new string[] { "Main", "Additive" });

// Unload scenes for a single connection.
NetworkConnection conn = base.Owner;
base.NetworkManager.SceneManager.UnloadConnectionScenes(conn, sud);

// Unload scenes for several connections at once.
NetworkConnection[] conns = new NetworkConnection[] { connA, connB };
base.NetworkManager.SceneManager.UnloadConnectionScenes(conns, sud);

// Unload scenes only on the server.
base.NetworkManager.SceneManager.UnloadConnectionScenes(sud);
```

Advanced Info

The `SceneManager` class has very detailed XML comments on how the unload process works in detail. If you need to troubleshoot the scene unload process, these comments will help you understand the flow.

Make sure to check out the [Scene Events](#) that you can subscribe to for better control over your game.

Executing On Server Or Client

How you can run certain code on only the server, or only a client, or only on a host.

Overview

Very often your code will need to know if it's running on a server or a client, or both at the same time. There are several ways of knowing this, for example, code executed directly from client or server only events or methods, checking compiler directives to see the type of game build, marking methods to only allow running on a specific instance, or simply checking a Boolean value.

Checking a Boolean

You can easily check if the current code is running on the server with a Boolean check. FishNet's `NetworkBehaviour` and `NetworkObject` classes have easy to use properties for this exact reason. They are also accessible from the `NetworkManager` directly.

- `IsClientStarted` — detects if the instance running the code is a client (it could also be a server).
- `IsClientOnlyStarted` — detects if the instance running the code is a client only and not a server.
- `IsServerStarted` — detects if the instance running the code is the server (it could also be a client).
- `IsServerOnlyStarted` — returns whether the instance running the code is a server only and not a client.

When checking from a `NetworkBehaviour` or on a `NetworkObject` you will often want to know if the code is running on the client or server side **and** the `NetworkObject` has already been initialized on the network and can be safely used for network functionality, such as RPCs or network properties. You can do that using the `IsServerInitialized`, `IsServerOnlyInitialized`, `IsClientInitialized`, and `IsClientOnlyInitialized` properties.

Tip: Instead of using something like `IsClientInitialized && IsServerInitialized` each time you want to check if the code is running on the host, you can simply check `IsHostInitialized`. There are `Started` and `Only` variations of this as well.

Simple example showing how a `NetworkBehaviour` could prevent code from running on the server:

```
// Player.cs
using FishNet.Object;

public class Player : NetworkBehaviour
{
    public void OnStruck()
    {
        if (!IsClientInitialized)
            return;

        // Play visual effect and sounds only on a client.
    }
}
```

Method Attributes

There are a variety of attributes which can be used to ensure a method only runs on the appropriate game instance. Unlike properties, attributes are `NetworkBehaviour`-independent and can be used in any type of class or struct.

Tip: These methods are also used for Fish-Networking Pro's automatic code stripping. Importantly, methods marked with the `Server` attribute will have their method body stripped from non-server builds.

Client Attribute

Placing a `[Client]` attribute above a method ensures that the method cannot be called unless the local client is active and the `NetworkObject` is initialized (the latter of which can be disabled by enabling the `UseIsStarted` option). There are additional properties which may be added to the attribute for additional functionality, such as for disabling the warning that would be logged if you try to call the method with this and it doesn't pass the check.

Simple example showing the `Client` attribute to prevent the method from running when not a client:

```
[Client]
private void ShowUI()
{
    // This code will only run on a client, otherwise it will print a warning.
}
```

If this method were called with no client active a warning would be printed and the method would not run. You can change the logging type using the `Logging` property to change whether it should log a warning at all or not. The default value is `LoggingType.Warning`.

It's also possible to allow only the owner of the object to call the method by setting `RequireOwnership` to `true`; this value is `false` by default.

Finally, you can use the `UseIsStarted` property to cause this to ignore the initialized state of the `NetworkObject` and check if the client alone is started.

```
// The server does not need to play VFX; only play VFX if the client is active.
[Client(Logging = LoggingType.Off, RequireOwnership = true, UseIsStarted = true)]
private void PlayVFX()
{
    // Play VFX here...
}
```

Server Attribute

The `[Server]` attribute provides similar features of the `Client` variant, except it will not allow the method to run if the server is not active and the `NetworkObject` is initialized (the latter of which can be disabled by enabling the `UseIsStarted` option). There are additional properties which may be added to the attribute for additional functionality, such as for disabling the warning that would be logged if you try to call the method with this and it doesn't pass the check.

Like the `Client` attribute, a warning will be thrown if this is called while the server is not active. The warning can be changed or disabled by adjusting the `Logging` property.

You can use the `UseIsStarted` property to cause this to ignore the initialized state of the `NetworkObject` and only check if the server is started.

```
// The server would validate hit results from a client.
[Server(Logging = LoggingType.Off, UseIsStarted = false)]
private void ValidateHit()
{
    // Hit validation code here that will only run on the server.
}
```

Server and Client Identification

Find out how to know if your code is executing on the server or client, as well as how to identify clients from each other.

Networkconnections

A look at how Fish-Networking distinguishes clients from one another.

NetworkConnections

A `NetworkConnection` in FishNet is a core object that represents a single connected client within the networking system. As soon as a client connects to the server, a `NetworkConnection` is created to represent it, perform actions on it, and gather information about that specific client.

Purposes

- **Client representation:** Each connected player/client has a corresponding `NetworkConnection` instance. This object tracks the client's identity, state, and the objects it owns.
- **Ownership and authority:** `NetworkConnection` tracks which networked objects (such as player avatars) the client owns, enabling authority checks and state management.
- **Authentication and state:** It manages authentication state, and notes if it's disconnecting.
- **Scene tracking:** It is used to track which networked scenes the client has loaded into.
- **Events:** It provides events for actions like gaining or losing ownership of objects and loading start scenes.
- **Custom data:** Developers can associate custom data with a connection for gameplay or server logic. (This data is not automatically synchronized across the network.)
- **Disconnection:** Provides methods to disconnect the client, either immediately or after sending pending data.

Important Fields and Properties

ClientId — One important field is the `ClientId`. This is a unique ID for the `NetworkConnection` and it is used in a few other places in Fish-Networking. If a `ClientId` is not set, it defaults to -1, and generally `ClientIds` increment from 0 until the maximum integer value before reusing old ones from the beginning.

FirstObject — FishNet doesn't have a forced Player Game Object, but you can get the first object owned by a client with the `FirstObject` property. You can also set it with the `SetFirstObject` method, which can be useful if you have the object destroyed and re-instantiated.

Objects — This field is a `HashSet` of all the network objects owned by this connection. It is available to this connection and server.

Scenes — These are the networked scenes this connection is considered a part of.

CustomData — As mentioned above, this is a user-definable object for storing arbitrary data on the connection. It is not synced at all.

Tip: Many more useful fields, properties, and methods are available and can be found in the API page.

Where to Get Them and Where to Use Them

How Is It Created/Used?

Each client has a `NetworkConnection` representing itself; this can be accessed with `ClientManager.Connection`, or `LocalConnection` from within a `NetworkBehaviour`.

The server maintains a dictionary of `NetworkConnection` instances for all connected clients; this is

available at `ServerManager.Clients`. It is also available on clients under the `ClientManager.Clients`. You should use the `ServerManager` version if the instance is running as a server. The dictionary key is the Client ID and the value is the `NetworkConnection`.

`NetworkObjects` can have an owner, which is a `NetworkConnection` that has authority over the object. From the `NetworkObject` or `NetworkBehaviour` you can get the `NetworkConnection` with the `Owner` property.

When calling `TargetRpc`s you will need to provide a `NetworkConnection` as the first argument of the method; this represents which client you will send the RPC to.

Transports

Some transports offer features additional to what is configured within the inspector. Within this section are guides on using unique features of each official transport.

- [Multipass](#)

Multipass

▼ # Transport Manager (Script) ? ⚙ ⋮

Script

TransportManager

Transport

NetworkManager (Multipass)

▼ # Multipass (Script) ? ⚙ ⋮

Script

Multipass

▼ Transports

Size

2

Element 0

🌀 NetworkManager (Yak)

Element 1

🌀 NetworkManager (Tugboat)

▼ 🌀 Yak (Script) ? ⚙ ⋮

Script

🌀 Yak

▼ 🌀 Tugboat (Script) ? ⚙ ⋮

Script

🌀 Tugboat

Channels

Unreliable MTU

1023

Server

Attack Response Type

Warn And Kick

Port

7770

Maximum Clients

4095

Client

Client Address

localhost

Misc

Timeout

15

Tip: You may want to access specific transports directly. You can do so by getting a reference to Multipass and calling `multipassReference.GetTransport<Type>()`. The Transports collection is also publicly accessible within Multipass.

274

Setup

Like other transports, to use Multipass it must be added to a `GameObject` — generally your `NetworkManager` object. After adding Multipass you must assign it as the transport to use within `TransportManager`. If `TransportManager` is not added to your `NetworkManager` automatically, you may need to add the component first. Once Multipass is added and specified in your `TransportManager`, add whichever other transports you wish to support. Drag the added reference of each transport to **Transports** under Multipass.

Important: Be sure that each transport within Multipass is listening on a different port.

Client

A client may only use one transport within Multipass, while the server may listen on all the transports at once.

For client actions to work you must specify which transport to use for the client.

```
// This can be done easily using the TransportManager.
Multipass mp = transportManager.GetTransport<Multipass>();

// In this example if the build is a WebGL build
// then use Bayou, otherwise use Tugboat.
#ifdef UNITY_WEBGL && !UNITY_EDITOR
    mp.SetClientTransport<Bayou>();
#else
    mp.SetClientTransport<Tugboat>();
#endif
```

The example above shows using Bayou and Tugboat, but you may of course use your own logic for any number of transports.

Once a client transport is set you may perform functions normally as if you were using only one transport. An example of this would be `ClientManager.StartConnection()`.

Server

As covered in the component settings, **GlobalServerActions** will execute server actions on all transports specified within Multipass. If you were to call `ServerManager.StartConnection()` while `GlobalServerActions` is true then the server would start on all transports; otherwise an error will be thrown.

```
// Start all server transports. This requires GlobalServerActions to be true.
serverManagerReference.StartConnection();
```

You may also access functions on a single transport by specifying the index of which transport to use.

```
// Start the connection on index 0.
Multipass mp = TransportManager.GetTransport<Multipass>();
// The true parameter indicates you are starting a server.
mp.StartConnection(true, 0);
```

Beyond the Basics

Where to go from here?

After having gone through the previous guides, it's recommended that you read the following sections to get a better understanding of how Fish-Networking works and its most commonly used features.

Important Features of FishNet

High-Level Overview

Get a high-level overview of multiplayer games with FishNet.

Networked GameObjects and Scripts

Learn about Networking GameObjects and Scripts.

Server and Client Identification

Find out how to determine if code is running on the server or client.

Network Connections

Understand how FishNet identifies different clients.

Ownership

Discover what owning a NetworkObject means and how clients can do so.

Spawning

Get to grips with spawning objects across the network.

Synchronized Variables

Learn about FishNet's automatically synchronized variables and how to use them!

Remote Procedure Calls

Find out about how to call methods remotely.

Data Serialization

Take a look at how FishNet can automatically serialize your data to send across the network.

Network State Events

Get information about the most commonly used network events, such as players connecting and the network starting or stopping.

Scene Management

Learn about managing scenes while the network is connected.

More Feature Guides

Explore guides for even more FishNet features, including Client-side Prediction, Area of Interest, Transports, Addressables, and Lag Compensation on the [Features](#) page.

Moving On and Getting Help

After reading through those sections you are encouraged to jump in and start making projects using FishNet! There are some available demos on specific Fish-Networking features included in the FishNet asset itself — these can be a great tool for learning more about the specific features.

If you run into errors or issues you can check out the [Troubleshooting](#) guides to find solutions to common

problems. You are also welcome to join the official Discord community and ask questions and learn from others there: <https://discord.gg/Ta9HgDh4Hj>

Now go create something amazing!

Connecting to Remote Devices

Learn how to connect clients to a remote FishNet server, allowing players on different machines to join your game.

Once you have a basic networked game set up and can connect as a host, the next crucial step for true multiplayer is enabling clients to connect to a server running on a different machine. This guide will walk you through the process of connecting to remote devices.

Step 1 — Identifying the Server's IP Address

For a client to connect to a remote server, it needs to know the server's IP address.

Local Network (LAN): If the server and client are on the same local network (e.g., two computers in the same house), you'll need the server machine's local IP address (e.g., 192.168.1.100). You can usually find this through your operating system's network settings.

Public Network (Internet): If clients are connecting over the internet, you'll need the server's public IP address. This is often the IP address of the router that the server machine is connected to. You might need to set up port forwarding on the server's router to allow incoming connections to the game server's port.

Info: For testing on a local network, you can often find your machine's IP address by opening a command prompt (Windows) and typing `ipconfig`, or a terminal (macOS/Linux) and typing `ifconfig` or `ip a`. Look for the IPv4 address associated with your active network adapter.

Step 2 — Configuring the Client to Connect

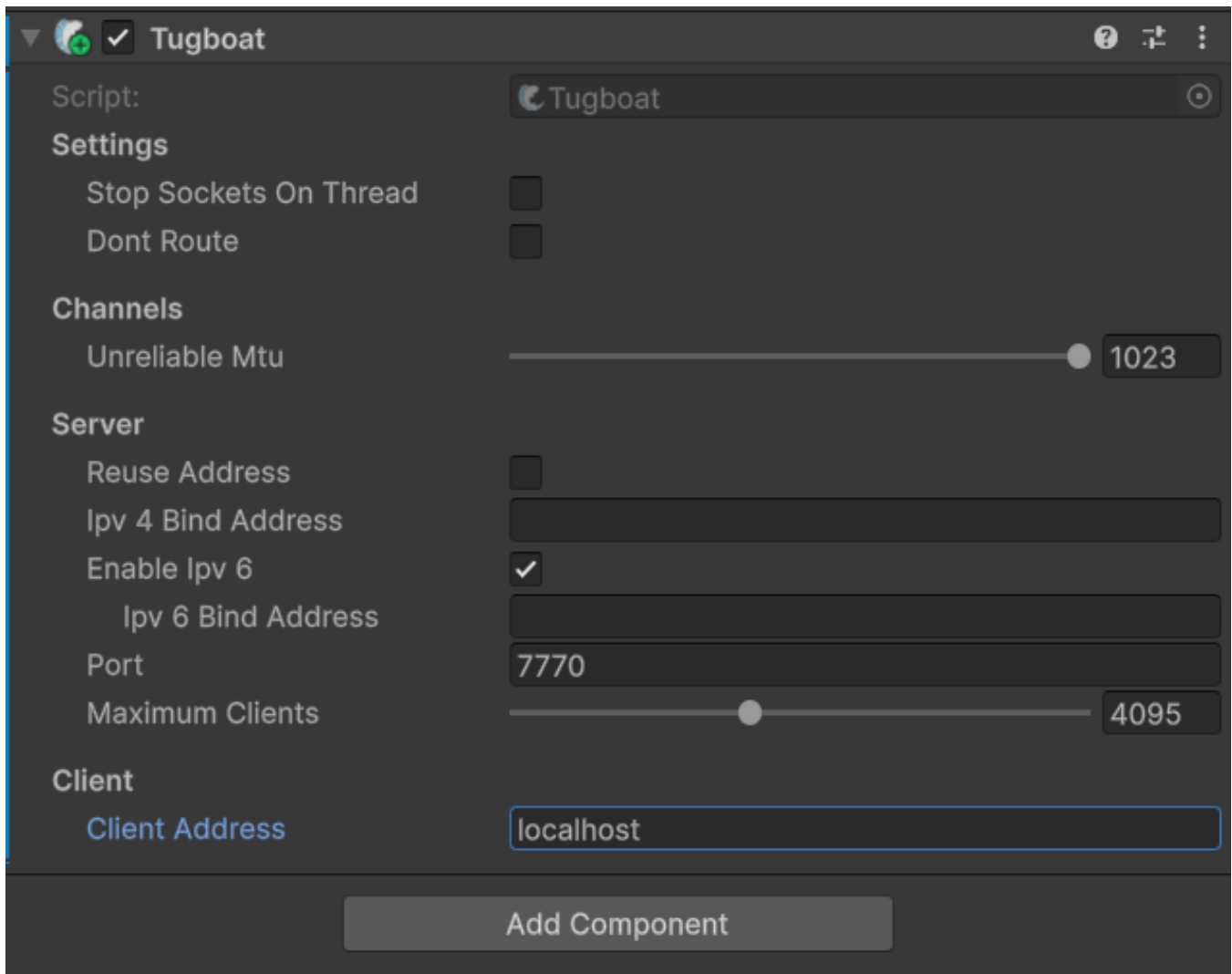
To connect as a client to a remote server, you'll need to manually set the server's address.

Locate the NetworkManager — In your Unity scene, select the NetworkManager game object.

Select or add the Transport Component — In the Inspector, locate the Transport component (e.g., **Tugboat** if you're using the default). If there is none, add **Tugboat** now.

Set the Address — Find the **Client Address** field within the Transport component. Enter the IP address of your remote server here.

Set the Port (if different) — The default port for FishNet is 7777. If your server is configured to use a different port, update the **Port** field accordingly.



Info: The NetworkHudCanvas is a convenient debugging tool. In a finished game, you would typically create your own UI for players to enter an IP address and connect.

Step 3 — Building and Testing the Connection

To test connecting to a remote device, you'll either need to build your game or set up your Unity editor with your project on the other devices.

Build the Client — Go to **File > Build Settings....** Ensure your scene is added to "Scenes In Build". Select your target platform and click **Build**.

Start the Server — Launch one instance of your game on the machine that will act as the server. If it's a dedicated server build, the FishNet server will start automatically. Otherwise, start it with the NetworkHudCanvas **Server** button or Autostart option.

Launch the Client — On a separate machine, launch the client build you just created.

Connect — If you disabled auto-start, click the **Start Client** button from the NetworkHudCanvas in the client build.

If configured correctly, the client should connect to the server, and you should see network-related messages in both the server and client consoles indicating a successful connection.

Step 4 — Troubleshooting Common Connection Issues

If your client fails to connect, consider these common issues:

Incorrect IP Address: Double-check that the IP address entered in the Transport component is correct for the server machine. Also make sure you entered it in the **Client Address** field and not one of the **Server Bind Address** fields.

Firewall: Ensure that the firewall on the server machine is not blocking incoming connections on the game's port (default 7777). You may need to create an inbound rule to allow traffic.

Port Forwarding (for Internet connections): If connecting over the internet, the server's router needs to be configured to forward the game's port to the server machine's local IP address. Without this, external clients cannot reach your server.

Server Not Running: Verify that the server instance of your game is actually running and has successfully started its network services.

Different Game Versions: Ensure both the server and client are running the exact same version of your game build. Mismatched versions can cause connection failures.

Network Congestion/Latency: On public networks, high latency or packet loss can sometimes lead to connection timeouts.

Info: If you are trying to test over the Internet, it may be easier and safer to use an application to create a virtual LAN instead of port forwarding your router.

Getting Connected

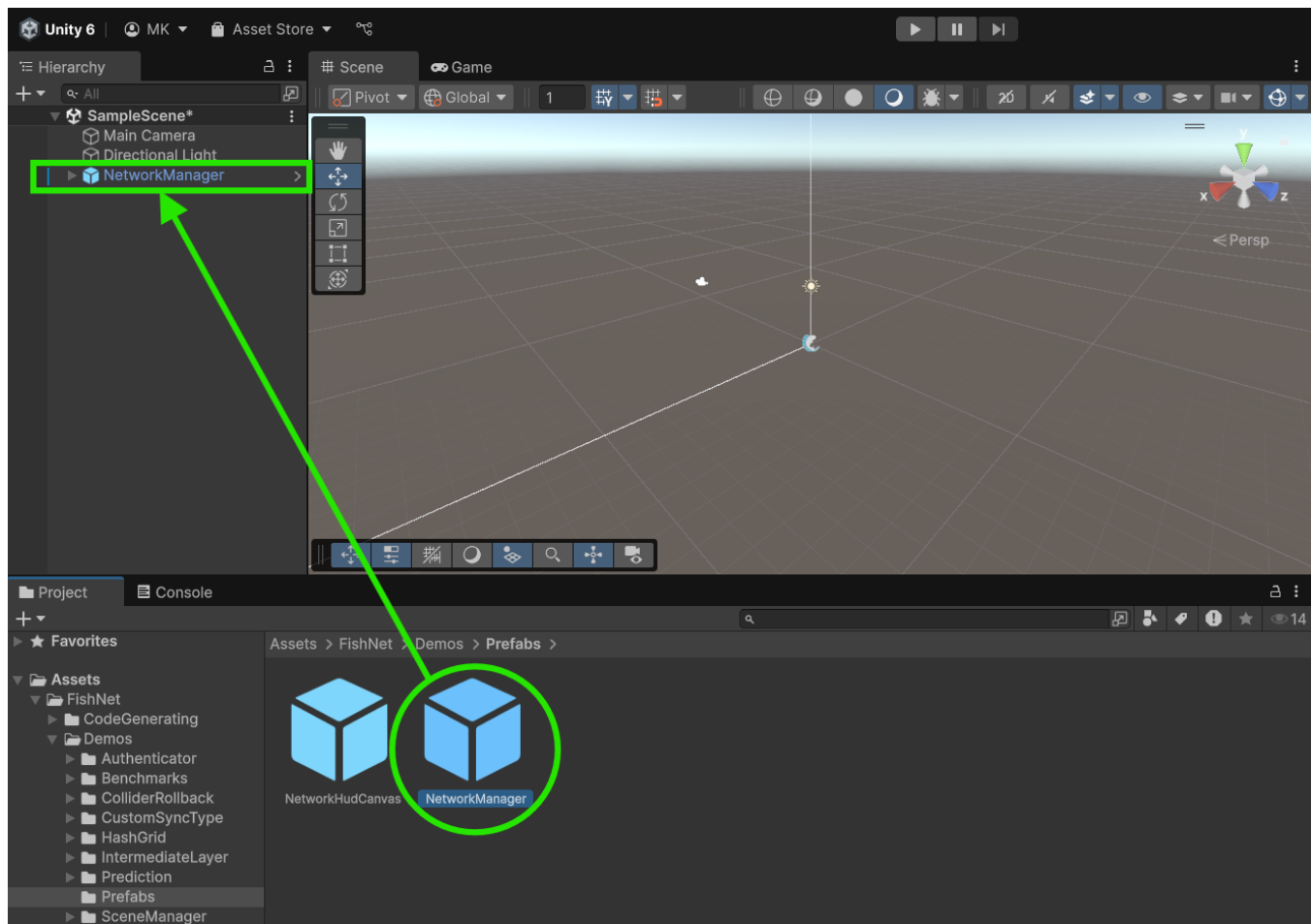
The first step to networking is connecting over the network. Learn about that process here!

Step 1 — Creating the NetworkManager

The NetworkManager is a central component of FishNet that manages the networking lifecycle, including establishing connections, hosting sessions, and handling client-server communication. For convenience, FishNet has a ready-made NetworkManager prefab that you can use.

You can find the prefab located in the FishNet directory under Demos/Prefabs/NetworkManager.prefab. Drag and drop this NetworkManager prefab into your Unity scene.

Info: If you imported FishNet from the git URL instead of the asset store or .unitypackage file, then the location of the FishNet directory will be Packages/FishNet: Networking Evolved instead of Assets/FishNet.

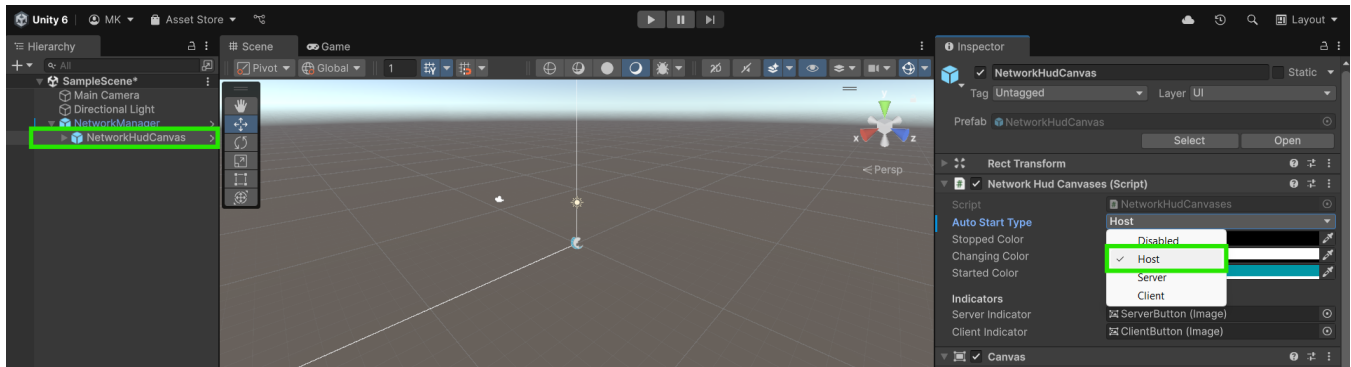


Step 2 — Configuring FishNet to Auto Start

The **NetworkHudCanvas** is a user interface provided by the example NetworkManager designed for quick testing of network connections. It displays buttons to start as a client and/or server. It also allows you to automatically start as a server, client, or host as soon as the NetworkManager loads in the game.

Expand the NetworkManager's fold-out icon to see its child game objects and select the **NetworkHudCanvas** game object. Find the **Network Hud Canvases** component on the object and set the **Auto Start Type** to **Host**.

This configuration will ensure that the application automatically starts as a host (a server and a client at the same time) when the scene is loaded.



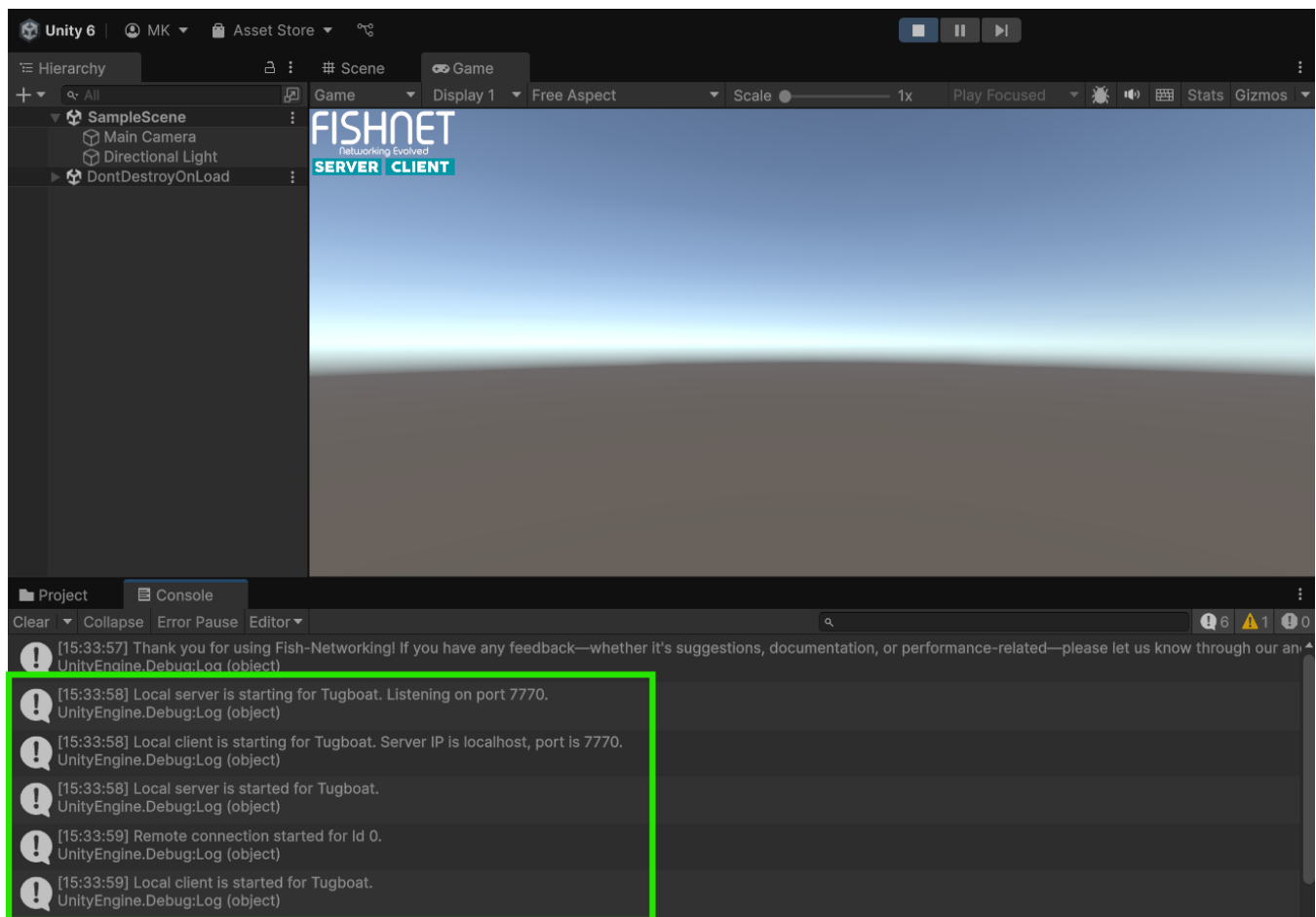
Step 3 — Testing the Connection

Once everything is set up, press the **Play** button in Unity's Editor to launch your scene.

The two **Server** and **Client** buttons in the top left of the game window should turn blue to indicate that the server and client are both active.

Info: If the buttons do not look like the example image then it just means your project is using Unity's new input system package; this is not a problem and you can simply check if the server and client are started by the button names being "Stop Server" and "Stop Client" respectively.

Additionally the console will log some network-related messages. Successful connection messages should appear, indicating that the host and client are running properly.



Tip: You will see the following warning: "Player prefab is empty and cannot be spawned for connection 0." This is fine and will be dealt with in the next section.

By following these steps, you should now have a functional network setup in your Unity project using FishNet. This guide provides the foundation for more complex networking workflows, such as syncing objects and managing connections.

Getting Started

Whether you're new to network programming or transitioning to Fish-Networking for your latest project, this guide will provide the essential foundation you need to get started smoothly.

What to Expect

This section introduces key features of Fish-Networking that you'll use frequently in your development journey. You'll find dedicated resources to deepen your understanding and step-by-step guides tailored for common first-time tasks.

By following this guide, you'll gain insight into setting up your project, getting your clients connected to your server, and leveraging core networking capabilities.

If you're ready to dive in, let's get started on building robust networked applications with Fish-Networking!

Installing Fish-Networking

Step-by-step instructions for installing Fish-Networking into your Unity project.

Info: If after importing the package you get a message asking to automatically update the API, select either one of the **Yes** options.

The Unity Asset Store

Installing via the Unity Asset Store is the easiest method and ensures you get a stable release:

Open your Unity project.

Go to **Window > Asset Store** or visit the [Unity Asset Store](#) online.

Search for **FishNet**.

Click **Add to My Assets** and then **Open in Unity**.

In Unity, go to **Package Manager** (Window > Package Manager).

Select **My Assets**, find FishNet, and click **Download** and then **Import**.

Tip: Recommended for most users who want a stable release without version control setup.

Unity Package File

Use this method if you have a .unitypackage file, for example downloaded from [GitHub Releases](#), from the [Fish-Networking website](#), or shared by a team member.

Download the FishNet .unitypackage file.

In Unity, go to **Assets > Import Package > Custom Package...**

Select the downloaded .unitypackage file.

Click **Import** to bring all the FishNet files into your project.

Info: Best for offline installs or specific package versions not yet on the Asset Store.

Git URL (via Unity Package Manager)

This method keeps FishNet up to date via a Git repository. Ideal if you want the latest updates or to contribute to development.

Open your Unity project.

Go to **Window > Package Manager**.

Click the **+** button in the top-left corner.

Select **Add package from Git URL...**

Enter the following URL:

```
https://github.com/FirstGearGames/FishNet.git?path=Assets/FishNet
```

Click **Add** to install.

Warning: Ensure you have the [Git client](#) (minimum version 2.14.0) installed on your system and added to the PATH system environment variable.

Installing FishNet Pro

Installing FishNet Pro can be done in the same way as the steps listed above.

If you need more details, you can find complete steps for purchasing and installing FishNet Pro at these pages:

- [FishNet Pro, Projects, and Support](#) — Details about what FishNet Pro is, the available paid Projects, and Professional Support.

Fix for Errors After Importing

If you receive the following errors after importing FishNet, make sure you don't have **Netcode for GameObjects** inside your project. The files it uses will clash with FishNet's files and you will need to remove it first and then import FishNet again.

```
Assets\FishNet\CodeGenerating\ILCore\ILCoreHelper.cs(18,13): error CS0246: The type or namespace name 'PostProcessorAssemblyResolver' could not be found (are you missing a using directive or an assembly reference?)
Assets\FishNet\CodeGenerating\ILCore\ILCoreHelper.cs(24,50): error CS0246: The type or namespace name 'PostProcessorReflectionImporterProvider' could not be found (are you missing a using directive or an assembly reference?)
```

Moving Your Player Around

Get your player objects moving around and synchronized!

Now that your player object is properly spawned and prepared, it's time to get it moving. In this section, we'll implement a simple client-authoritative movement script and utilize FishNet's `NetworkTransform` component to effortlessly synchronize that movement across all connected devices.

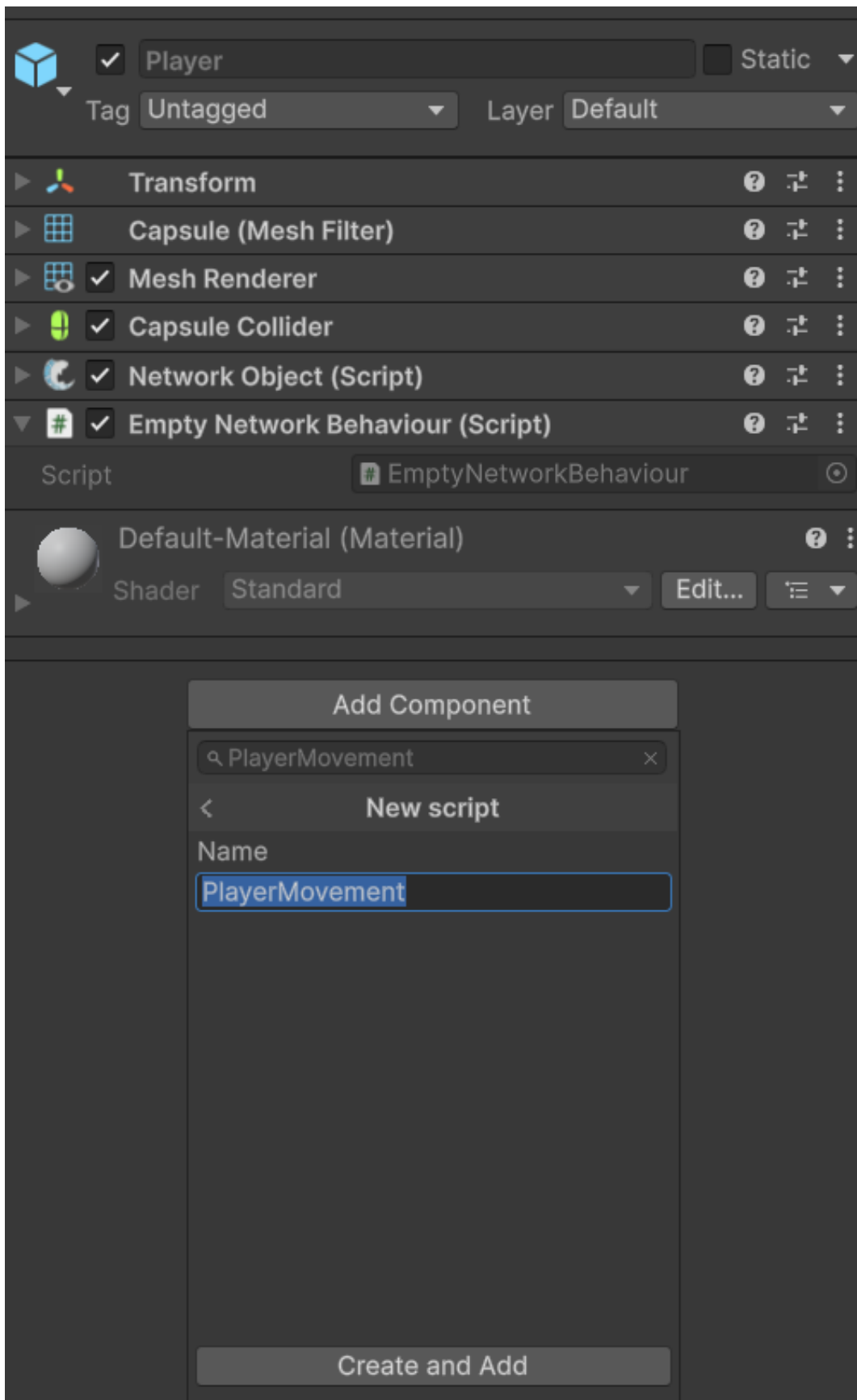
Client-Authoritative Movement

For many games, particularly those with real-time player input, client-authoritative movement is the most straightforward approach. This means the client directly controls its own player object and then informs the server (and other clients) about its position and rotation.

Warning: Client-authoritative movement makes it easier for the client to cheat with speed hacks, teleportation, and other movement related hacks, but it is a lot easier to implement and understand.

Step 1 — Creating a Player Movement Script

Select your Player prefab in the Project window. In the Inspector, click on **Add Component** and search for **New Script**. Name the script `PlayerMovement` and click **Create and Add**.



Step 2 — Editing the Script

Double-click the `PlayerMovement` script to open it in your code editor. Replace the default code with the following:

```
using FishNet.Object;
using UnityEngine;

// Inherit from NetworkBehaviour instead of MonoBehaviour
public class PlayerMovement : NetworkBehaviour
{
    public float MoveSpeed = 5f;

    private void Update()
    {
        // Only run this code on the object the local client owns.
        // This prevents us from moving other players' objects.
        if (!IsOwner) return;

        float horizontal = Input.GetAxis("Horizontal");
        float vertical = Input.GetAxis("Vertical");
        Vector3 moveDirection = new Vector3(horizontal, 0f, vertical);

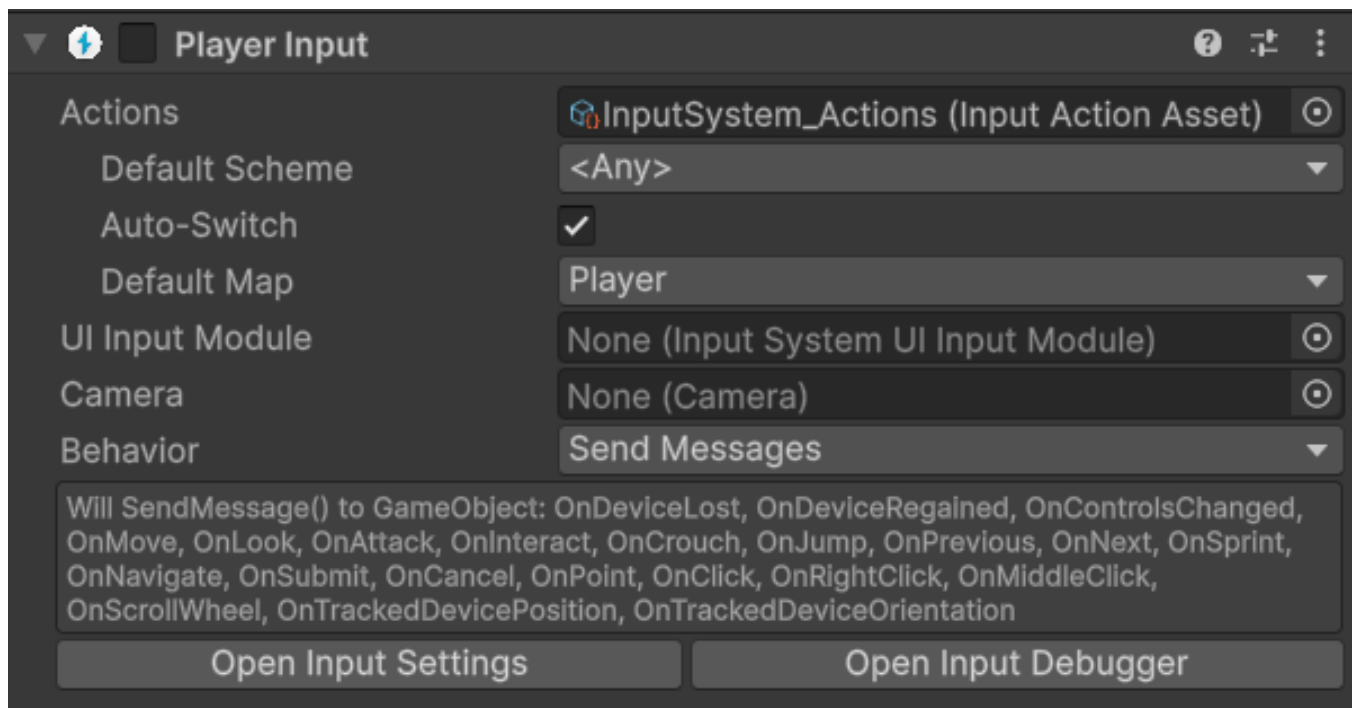
        if (moveDirection.magnitude > 1f)
            moveDirection.Normalize();

        transform.position += MoveSpeed * Time.deltaTime * moveDirection;
    }
}
```

Unlike typical single-player scripts that use `MonoBehaviour`, this script inherits from `NetworkBehaviour`. This allows for direct access to the `IsOwner` field. Although you could still check ownership via the `NetworkObject` component on a `MonoBehaviour`, `NetworkBehaviour` provides significant advantages — it enables RPCs, `SyncVars`, and specialized override methods (akin to Unity's `Start`, `Awake`, and `OnDestroy` but for networked objects).

Since there will be multiple player game objects in the game, we need to determine which one is "our" local player and only move that with our input. The `IsOwner` guard clause handles this.

If using Unity's [New Input System](#), you should add the `Player Input` component to your `Player Prefab` and be sure to **disable it** by default.



Then use the following script instead:

```

using FishNet.Object;
using UnityEngine;
using UnityEngine.InputSystem;

// Inherit from NetworkBehaviour instead of MonoBehaviour
public class PlayerMovement : NetworkBehaviour
{
    public float MoveSpeed = 5f;
    private Vector2 _currentMovementInput;

    public override void OnStartClient()
    {
        if (IsOwner)
            GetComponent<PlayerInput>().enabled = true;
    }

    public void OnMove(InputValue value)
    {
        _currentMovementInput = value.Get<Vector2>();
    }

    public void Update()
    {
        // Only run this code on the object the local client owns.
        if (!IsOwner) return;

        Vector3 moveDirection = new Vector3(_currentMovementInput.x, 0f, _currentMovementInput.y);

        if (moveDirection.magnitude > 1f)
            moveDirection.Normalize();

        transform.position += MoveSpeed * Time.deltaTime * moveDirection;
    }
}

```

We use the `NetworkBehaviour OnStartClient` callback to enable the `PlayerInput` component when `IsOwner` is true. This method runs when the network object is initialized on the network and only on the client side.

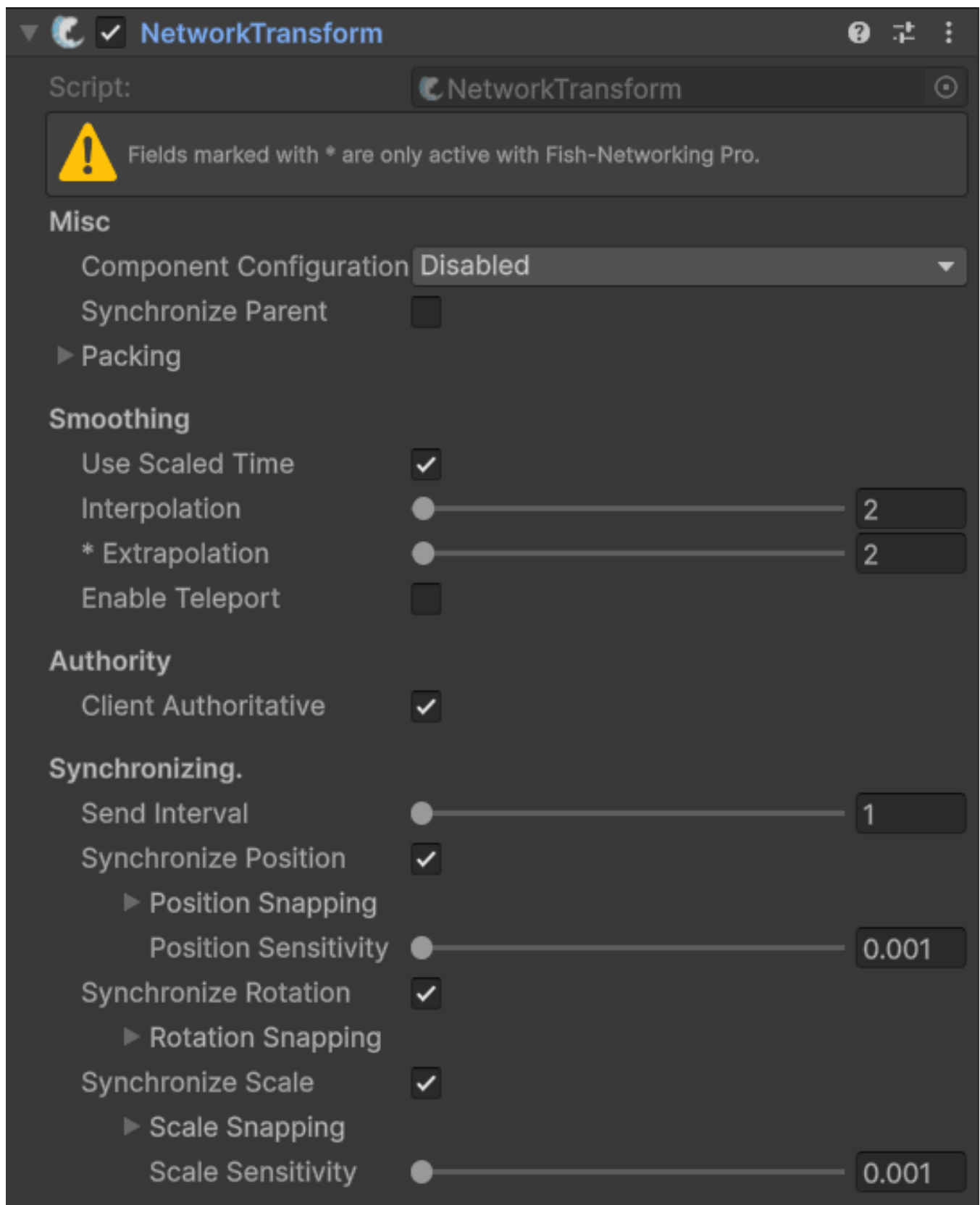
Info: You might wonder why we can't use `Awake` or `Start` here — `FishNet` doesn't always have enough time to link the object on the network and assign its owner before those methods execute.

Step 3 — Synchronizing the Movement

Now we have code that will allow clients to move only their respective player object, but nothing is yet synchronized over the network. To fix this, we will use `Fish-Networking`'s built-in **NetworkTransform** component.

Open your `Player Prefab` and add the **NetworkTransform** component to it.

The most important settings right now are **Client Authoritative** and **Synchronize Position** — both should be enabled. This means positional changes made by the owner client will automatically be synchronized to the server and then to all other clients.



Step 4 — Testing the Player Movement

Save your scene and press the **Play** button in Unity's Editor. Your player capsule should now be visible. Use the W, A, S, D keys or arrow keys to move your player around the scene.

To test multiplayer movement, build and run the game (or use multiple editor windows). You should then be able to control your player in the editor, and the second instance will control its own player, seeing both players move independently.

[Image: Player movement synchronized over the network]

Preparing Your Player

Spawn an object over the network to represent each client's player.

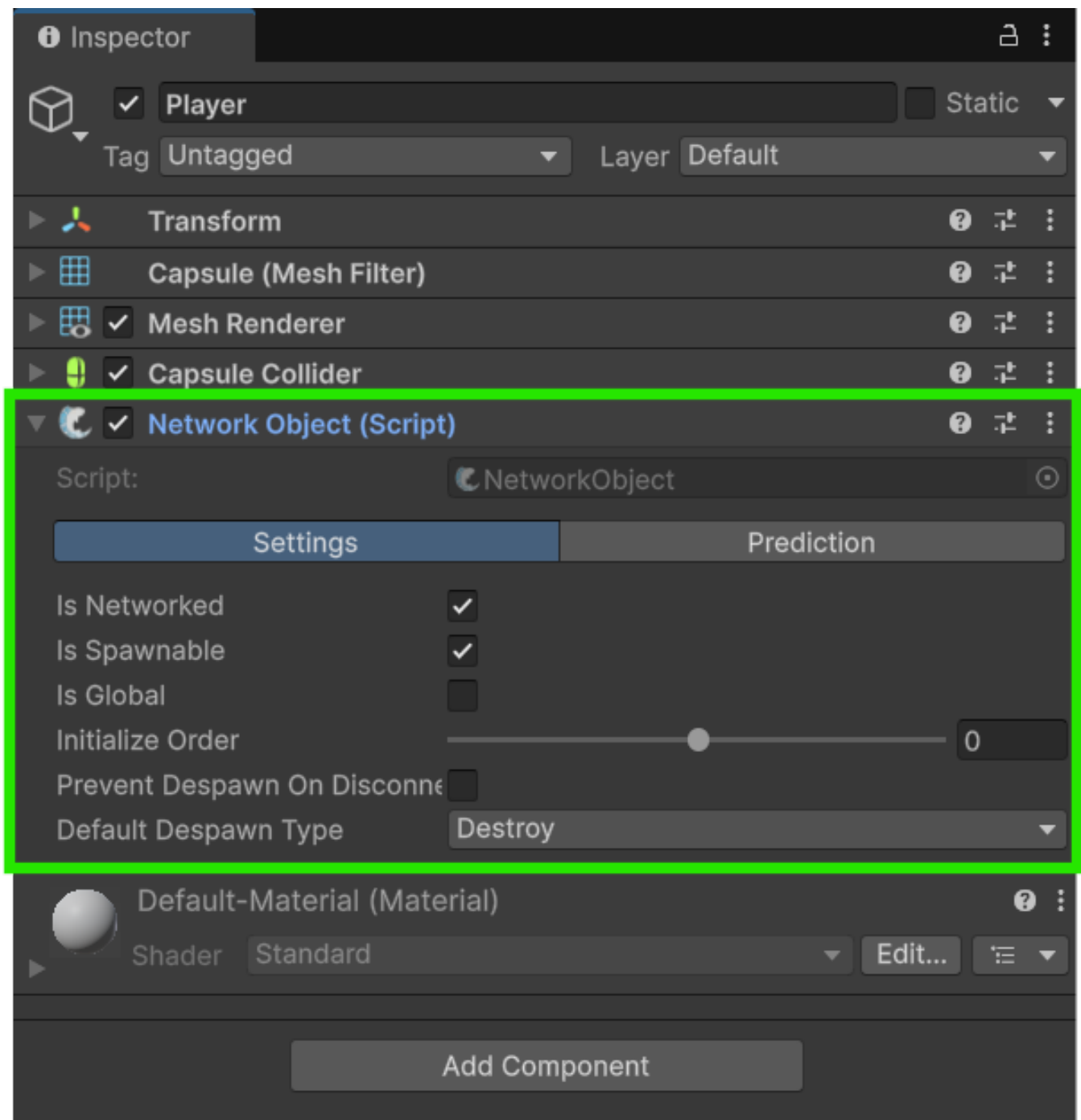
For many games you will want to have a player object for each client. This guide will walk you through the process step by step.

Tip: Before You Begin — ensure that you're not in Play Mode in the Unity Editor. Making changes during Play Mode will not persist after exiting.

Step 1 — Create a Player Game Object

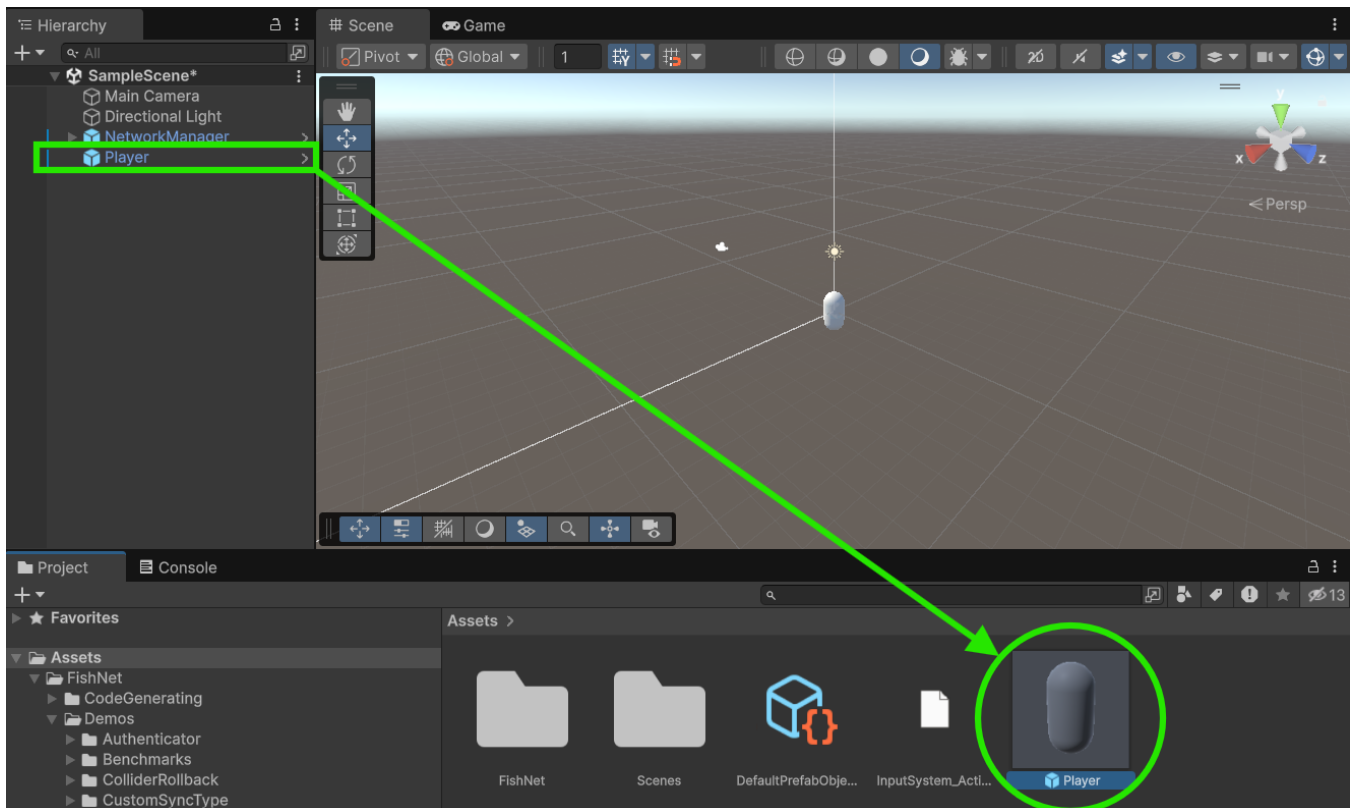
In the Unity Scene Hierarchy, create a new 3D object of type **Capsule**. Select the capsule in the Hierarchy and rename it to **Player**. In the Inspector, click on **Add Component** and search for **NetworkObject**. Add the **NetworkObject** component to the capsule.

Info: The **NetworkObject** component is required for linking a game object over the network. It enables the object to synchronize its state across clients and the server.

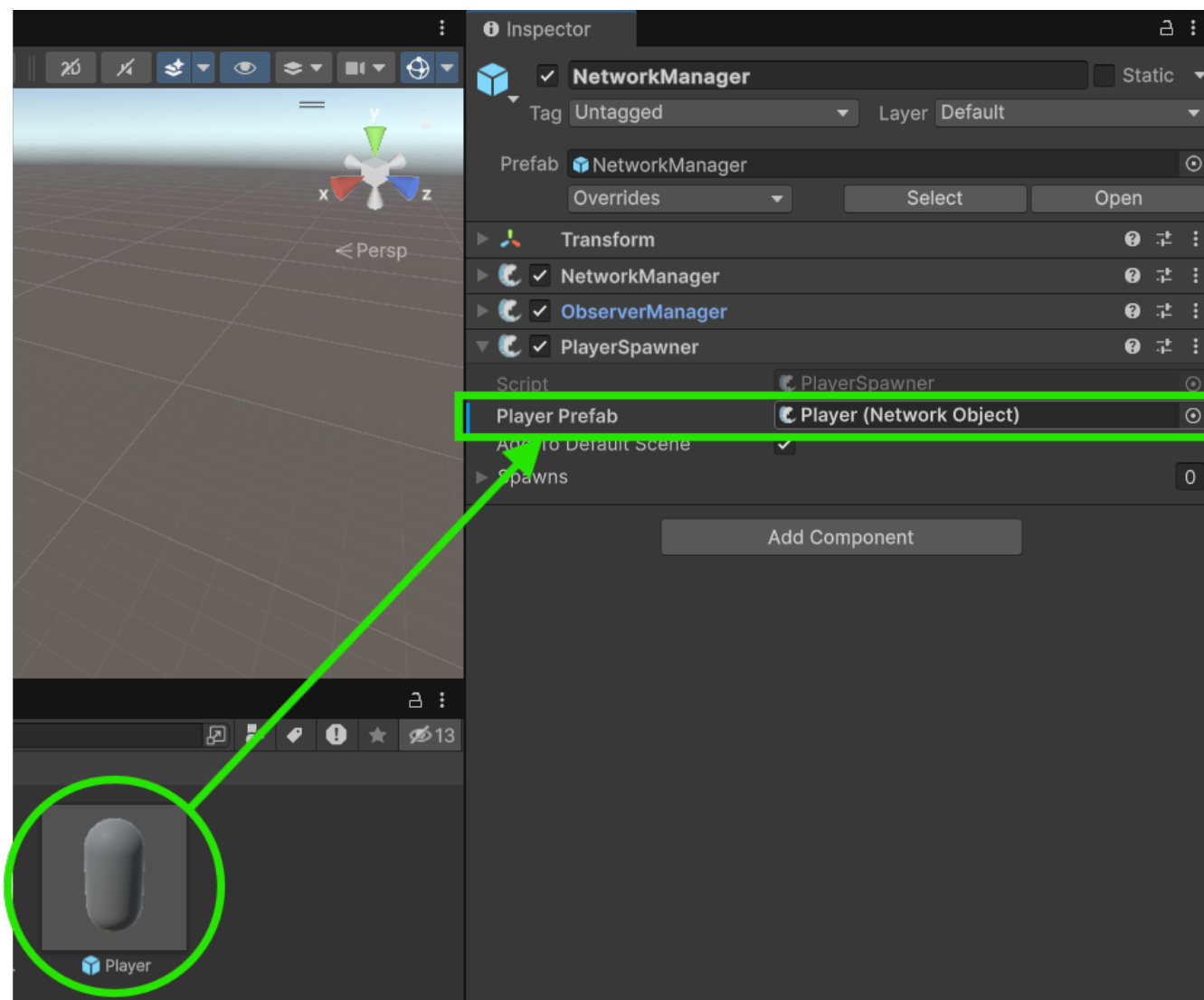


Step 2 — Setup the Player Object for Spawning

Drag the player game object into the project window to turn it into a prefab, then delete the game object from the scene.

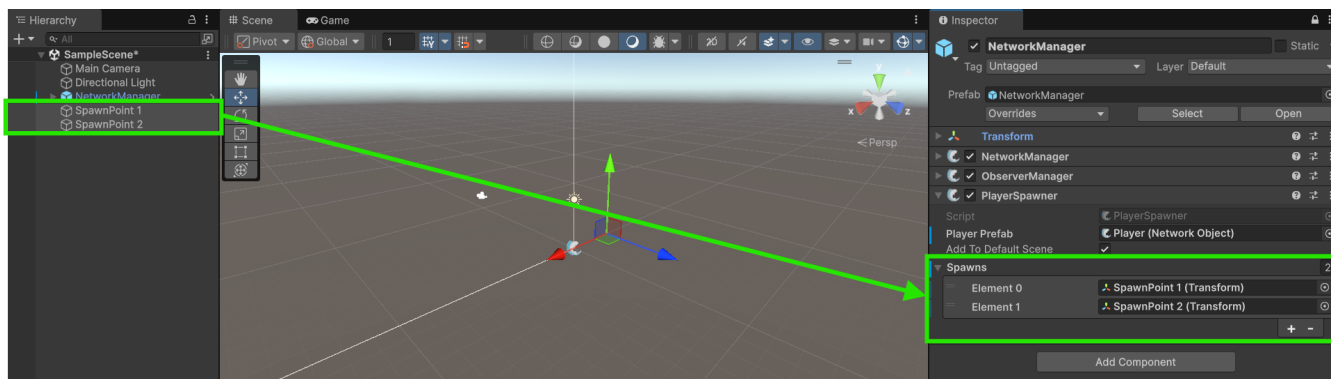


Select the **NetworkManager** in the scene and assign your new Player prefab into the **Player Prefab** field on the **PlayerSpawner** component.



Step 3 — Add Spawn Points

Create two empty Game Objects in the scene to act as spawn points. Position them where you'd like players to spawn on the map. Add these Game Objects as Transform references to the PlayerSpawner component in the **Spawns** list field.



Info: The PlayerSpawner uses these transforms to determine spawn locations, choosing them in order

from top to bottom for each subsequent player and beginning again after reaching the final spawn point. If no spawn points are specified, the spawner defaults to using the prefab's transform properties for placement.

Step 4 — Test the Player Spawning

Save the scene and press the **Play** button in the Unity Editor. As soon as the server and client start you should see a capsule spawning as the player object at the location of the first spawn point.

To see another player spawn you can build and run the game, which will then automatically connect as a client to the editor. You should then see a second player capsule spawn for the second player at the second spawn point.

Alternatively, you can launch a second Unity Editor instance by using Unity's [Multiplayer Play Mode](#) package or a third party package such as [ParrelSync](#). Find out more about these options and how to use them here: [Testing with Multiple Editors](#).

Info: Don't worry if you encounter an error in a second game window stating: "Server failed to start. This usually occurs when the specified port is unavailable, be it closed or already in use." This occurs because multiple game instances are attempting to start as a server on the same machine using the same port. Only the first instance will successfully start as a server and the rest will instead start as clients only, thus this error is safe to ignore for local testing.

Basic Setup with Cinemachine

Managing a Cinemachine Camera in multiplayer.

This guide will show you one way you can set up your player camera for multiplayer when you are using the Cinemachine package.

Step 1 — Installing Cinemachine

If you haven't done so already, install [Cinemachine](#) through the [Unity Package Manager](#).

Step 2 — Adding the Cinemachine Brain

Add the **Cinemachine Brain** component to the **Main Camera** game object in the scene.

Step 3 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be our container for the Cinemachine Camera.

Step 4 — Adding the Cinemachine Camera

Add the **Cinemachine Camera** component to the newly created **CameraHolder** game object.

Step 5 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab.

```
using FishNet.Object;
using Unity.Cinemachine;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnStartClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private CinemachineCamera _cinemachineCamera;

    public override void OnStartClient()
    {
        _cinemachineCamera.enabled = IsOwner;
    }
}
```

This script uses the `OnStartClient` override method to enable or disable the Cinemachine Camera component on the player objects depending on if they are our local player or not.

Step 6 — Assign Your References

Select the **Player Camera** component in your Player Prefab and drag the **Camera Holder** game object into the **Cinemachine Camera** field.

Step 7 — Test in Game

With all that set, you should be able to run the game and see how the camera is controlled by only your local player.

Setting Up a Camera

How to set up a camera for your local player in a multiplayer game.

Instantiating a Local Camera

Using a Camera Prefab to instantiate your local player's camera.

Using the Scene Camera

Using the camera in the scene for your local player.

Basic Setup with Cinemachine

Managing a Cinemachine Camera in multiplayer.

Instantiating a Local Camera

Using a Camera Prefab to instantiate your local player's camera.

With this method of camera management we will instantiate a camera from a prefab we create and assign it to our local player object when it spawns in.

Step 1 — Creating a Camera Prefab

Drag the **Main Camera** in the scene into your Project Window to create a prefab out of it. You can now delete the Main Camera from the Scene Hierarchy.

Step 2 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be where we instantiate and position the Camera object.

Step 3 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab. We will use it to instantiate our Camera Prefab once our player spawns in.

```
using FishNet.Object;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnStartClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private Camera _cameraPrefab;
    [SerializeField] private Transform _cameraHolder;

    // This method will run on the client once this object is spawned.
    public override void OnStartClient()
    {
        // Since this will run on all clients that this object spawns for
        // we need to only instantiate the camera for the object we own.
        if (IsOwner)
            Instantiate(_cameraPrefab, _cameraHolder.position, _cameraHolder.rotation, _cameraHolder);
    }
}
```

This script uses the OnStartClient override method from NetworkBehaviour to instantiate the camera prefab for our local player.

Step 4 — Assign Your References

Select the **Player Camera** component in your Player Prefab and add the Camera Prefab to the **Camera Prefab** field. Also select the CameraHolder game object in the **Camera Holder** field.

Step 5 — Test in Game

With all that set, you should be able to run the game and see how the camera is created for only your local player.

Overview

Instantiating a Local Camera

Using a Camera Prefab to instantiate your local player's camera.

With this method of camera management we will instantiate a camera from a prefab we create and assign it to our local player object when it spawns in.

Step 1 — Creating a Camera Prefab

Drag the **Main Camera** in the scene into your Project Window to create a prefab out of it. You can now delete the Main Camera from the Scene Hierarchy.

Step 2 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be where we instantiate and position the Camera object.

Step 3 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab. We will use it to instantiate our Camera Prefab once our player spawns in.

```
using FishNet.Object;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnStartClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private Camera _cameraPrefab;
    [SerializeField] private Transform _cameraHolder;

    // This method will run on the client once this object is spawned.
    public override void OnStartClient()
    {
        // Since this will run on all clients that this object spawns for
        // we need to only instantiate the camera for the object we own.
        if (IsOwner)
            Instantiate(_cameraPrefab, _cameraHolder.position, _cameraHolder.rotation, _cameraHolder);
    }
}
```

This script uses the OnStartClient override method from NetworkBehaviour to instantiate the camera prefab for our local player.

Step 4 — Assign Your References

Select the **Player Camera** component in your Player Prefab and add the Camera Prefab to the **Camera Prefab** field. Also select the CameraHolder game object in the **Camera Holder** field.

Step 5 — Test in Game

With all that set, you should be able to run the game and see how the camera is created for only your local player.

Using the Scene Camera

Using the camera in the scene for your local player.

With this method our local player will take control of the Main Camera it finds in the scene.

Step 1 — The Main Camera Tag

Before we start we need to ensure our camera object in the scene is tagged as **"Main Camera"**.

Step 2 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be where we parent and position the Camera object.

Step 3 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab.

```
using FishNet.Connection;
using FishNet.Object;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnOwnershipClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private Transform _cameraHolder;

    // This method is called on the client after gaining or losing ownership of the object.
    // We could have used OnStartClient instead, but using OnOwnershipClient
    // means this will work for a player object we don't initially own but are given ownership to later.
    public override void OnOwnershipClient(NetworkConnection prevOwner)
    {
        if (Camera.main == null) return;

        // If we are the new owner, take control of the camera by parenting it
        // and moving it to our camera holder.
        if (IsOwner)
        {
            Camera.main.transform.SetPositionAndRotation(_cameraHolder.position, _cameraHolder.rotation);
            Camera.main.transform.SetParent(_cameraHolder);
        }
    }
}
```

This script uses the `OnOwnershipClient` override method to take control of the camera as soon as the local client gets ownership of the player object.

Step 4 — Assign Your References

Select the **Player Camera** component in your Player Prefab and select the CameraHolder game object in the **Camera Holder** field.

Step 5 — Test in Game

With all that set, you should be able to run the game and see how the camera from the scene is controlled by only your local player.

Basic Setup with Cinemachine

Managing a Cinemachine Camera in multiplayer.

This guide will show you one way you can set up your player camera for multiplayer when you are using the Cinemachine package.

Step 1 — Installing Cinemachine

If you haven't done so already, install [Cinemachine](#) through the [Unity Package Manager](#).

Step 2 — Adding the Cinemachine Brain

Add the **Cinemachine Brain** component to the **Main Camera** game object in the scene.

Step 3 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be our container for the Cinemachine Camera.

Step 4 — Adding the Cinemachine Camera

Add the **Cinemachine Camera** component to the newly created **CameraHolder** game object.

Step 5 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab.

```
using FishNet.Object;
using Unity.Cinemachine;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnStartClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private CinemachineCamera _cinemachineCamera;

    // This method is called on the client after the object is spawned in.
    public override void OnStartClient()
    {
        // Simply enable our local cinemachine camera on the object if we are the owner.
        _cinemachineCamera.enabled = IsOwner;
    }
}
```

This script uses the `OnStartClient` override method to enable or disable the Cinemachine Camera component on the player objects depending on if they are our local player or not.

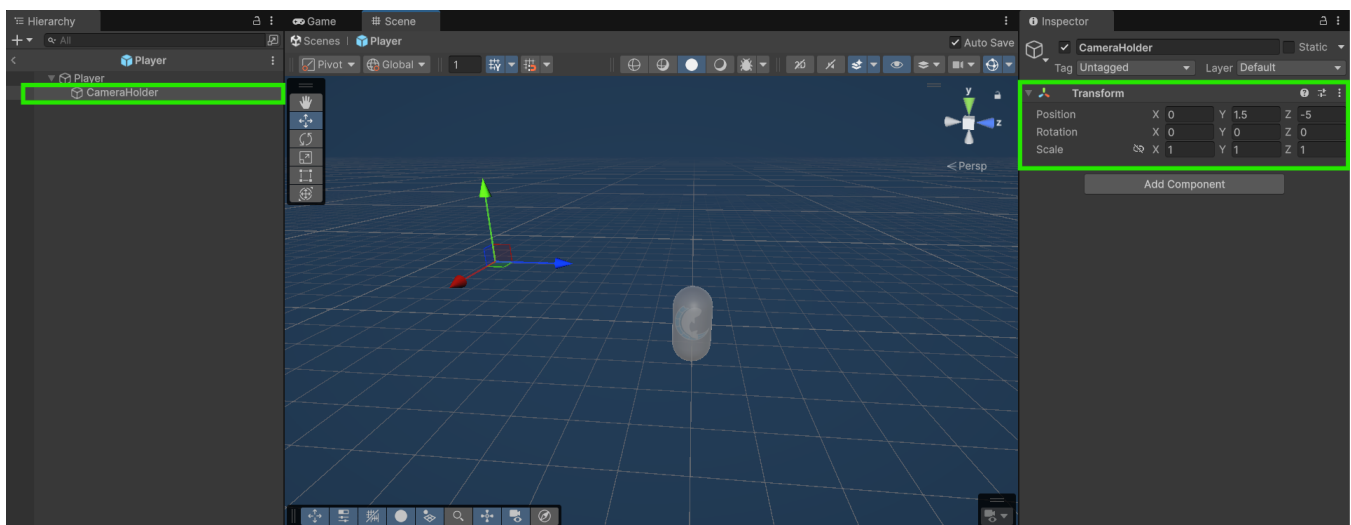
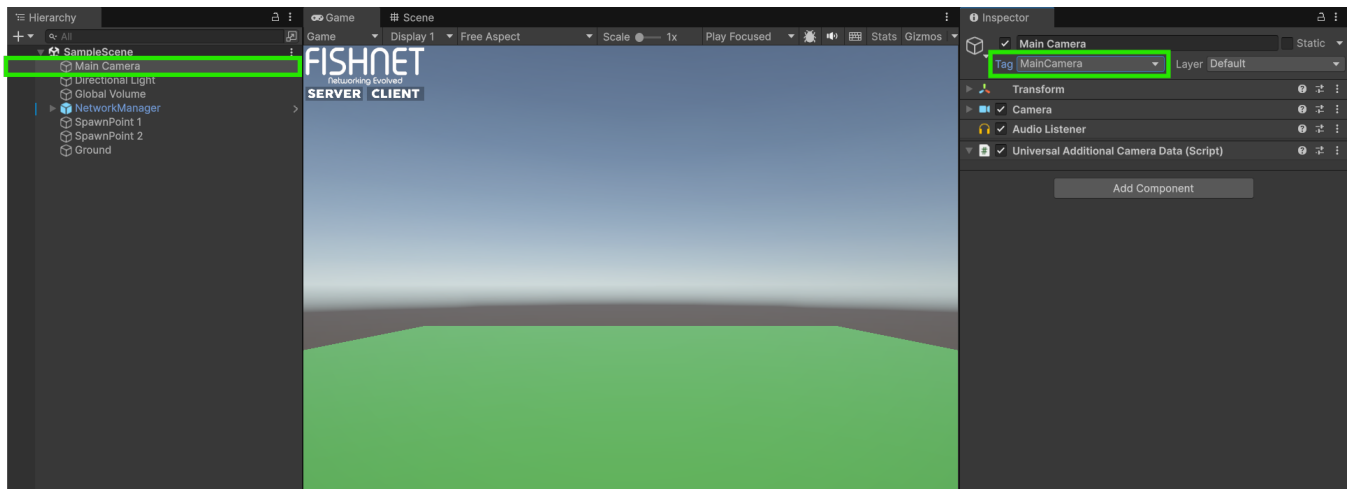
Step 6 — Assign Your References

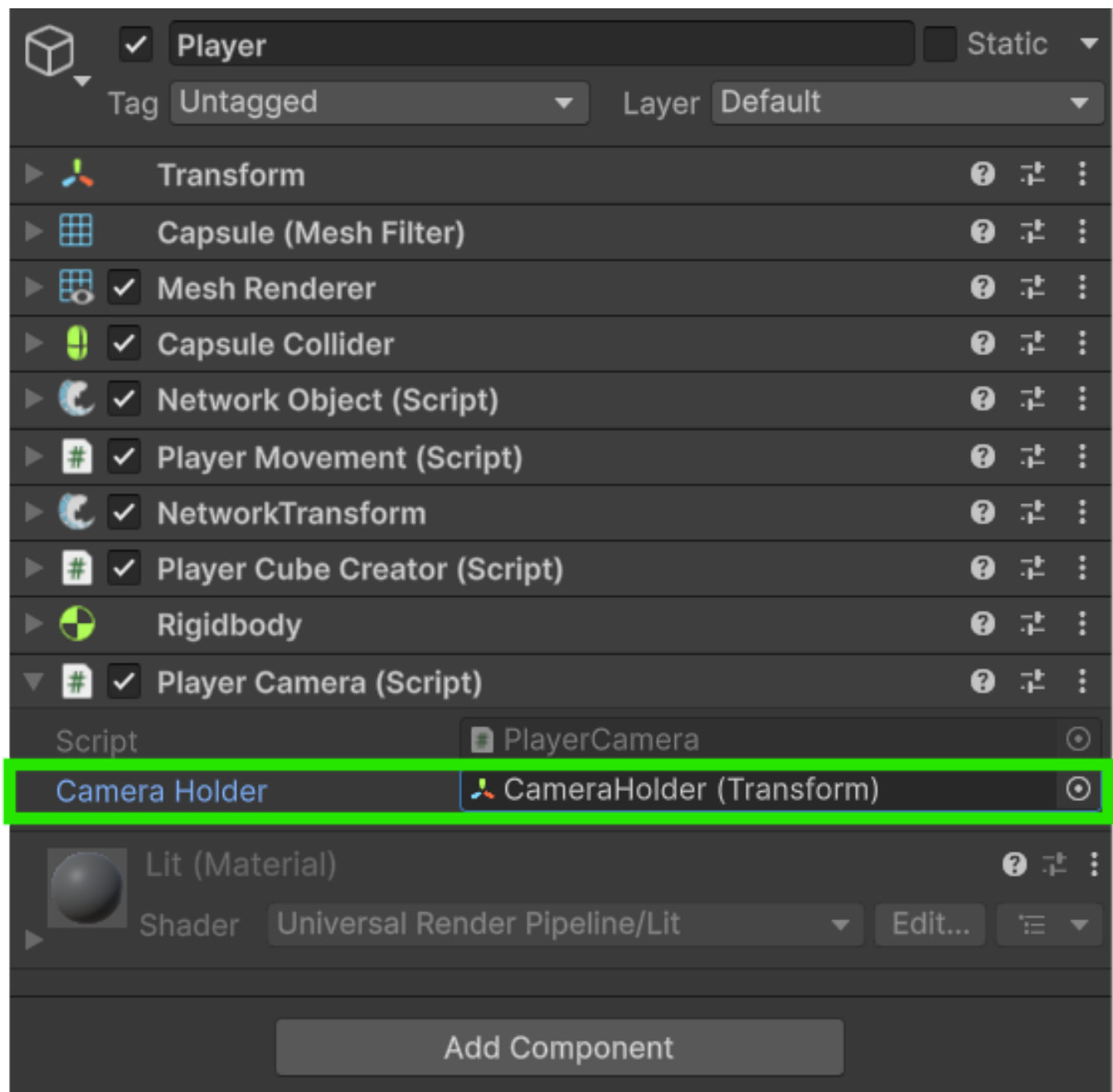
Select the **Player Camera** component in your Player Prefab and drag the **Camera Holder** game object into the **Cinemachine Camera** field.

Step 7 — Test in Game

With all that set, you should be able to run the game and see how the camera is controlled by only your local player.

Using the Scene Camera





[Image: untitled]

Using the camera in the scene for your local player.

With this method our local player will take control of the Main Camera it finds in the scene.

Step 1 — The Main Camera Tag

Before we start we need to ensure our camera object in the scene is tagged as "Main Camera".

Step 2 — Giving the Player a Camera Holder

Create an empty Game Object on your **Player Prefab** and position it where you'd like. This will be where we parent and position the Camera object.

Step 3 — Writing a PlayerCamera Script

Add the following PlayerCamera script to the Player Prefab.

```
using FishNet.Connection;
using FishNet.Object;
using UnityEngine;

// This script will be a NetworkBehaviour so that we can use the OnOwnershipClient override.
public class PlayerCamera : NetworkBehaviour
{
    [SerializeField] private Transform _cameraHolder;

    public override void OnOwnershipClient(NetworkConnection prevOwner)
    {
        if (Camera.main == null) return;

        if (IsOwner)
        {
            Camera.main.transform.SetPositionAndRotation(_cameraHolder.position, _cameraHolder.rotation);
            Camera.main.transform.SetParent(_cameraHolder);
        }
    }
}
```

This script uses the OnOwnershipClient override method to take control of the camera as soon as the local client gets ownership of the player object.

Step 4 — Assign Your References

Select the **Player Camera** component in your Player Prefab and select the CameraHolder game object in the **Camera Holder** field.

Step 5 — Test in Game

With all that set, you should be able to run the game and see how the camera from the scene is controlled by only your local player.

Spawning and Despawning Items

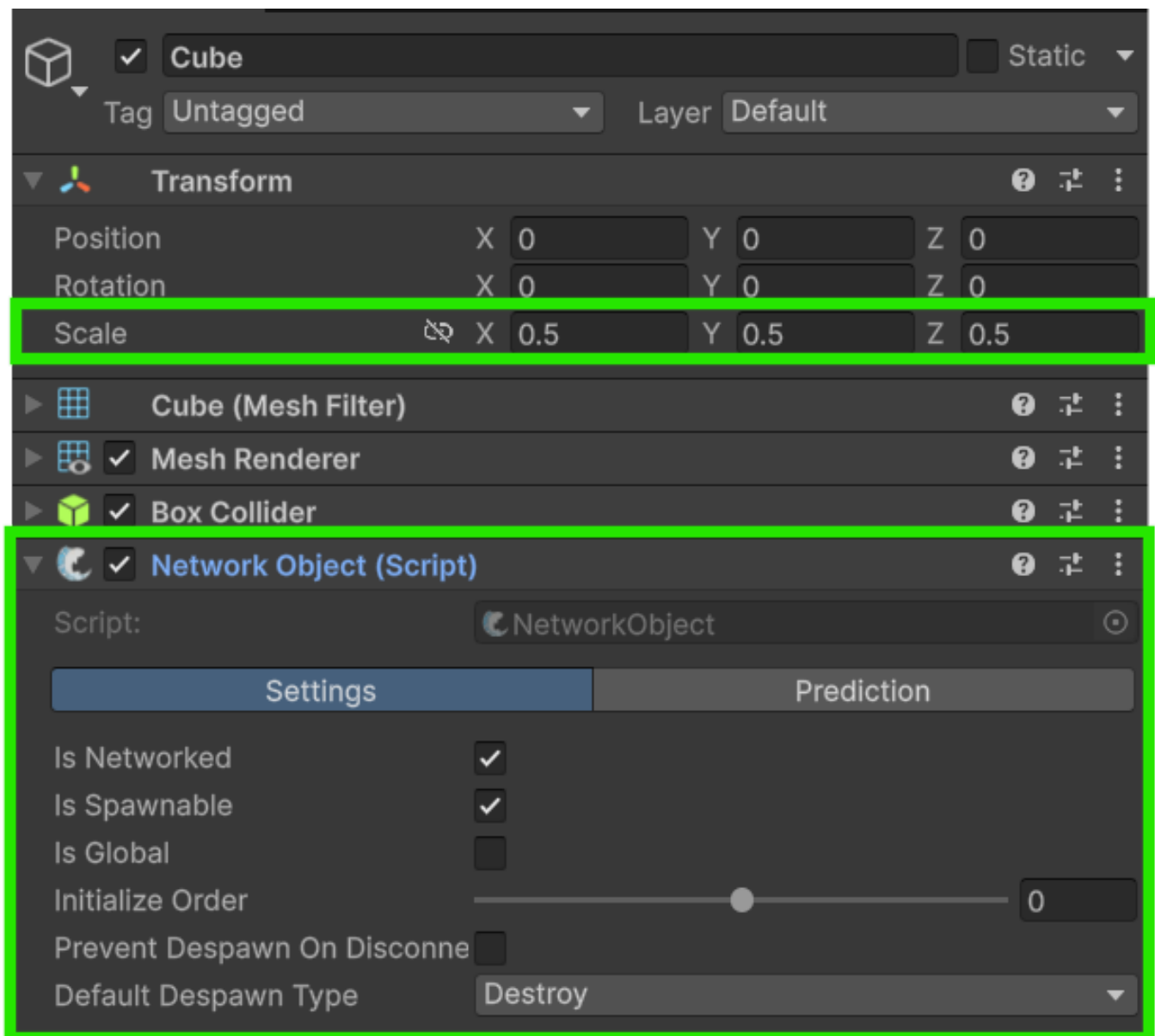
Learn about spawning and despawning by having your players throw cubes around!

Now that we have players moving around, let's learn how to spawn and despawn NetworkObjects.

While you can Instantiate and Destroy regular Game Objects, Network Objects need to be **Spawned** and **Despawnd** — this tells FishNet to synchronize the action over the network. FishNet stores all network object prefabs in a collection called the **Spawnable Prefabs**. You can see this and customize it on the NetworkManager.

Step 1 — Creating the Item Network Object Prefab

Create a **Cube** in the scene hierarchy and add a **NetworkObject** component to it. Optionally decrease its scale by half so it doesn't look too large. Then drag it into the project window to turn it into a prefab; you can destroy the original game object in the scene hierarchy now.



Step 2 — Creating a Script to Spawn the Cube

Now create a script called `PlayerCubeCreator` and add it to your **Player Prefab**. This script is going to be responsible for spawning our new cube item across the network.

```
using FishNet.Object;
using UnityEngine;

public class PlayerCubeCreator : NetworkBehaviour
{
    public NetworkObject CubePrefab;

    private void Update()
    {
        // Only the local player object should perform these actions.
        if (!IsOwner) return;

        if (Input.GetButtonDown("Fire1"))
            SpawnCube();
    }

    // We are using a ServerRpc here because the Server needs to do all network object spawning.
    [ServerRpc]
    private void SpawnCube()
    {
        NetworkObject obj = Instantiate(CubePrefab, transform.position, Quaternion.identity);
        Spawn(obj); // NetworkBehaviour shortcut for ServerManager.Spawn(obj);
    }
}
```

Info: This may be a great time to read the page about RPCs, such as the `ServerRpc` we use here.

Because this script contains a `ServerRpc`, it will also need to be a `NetworkBehaviour`. The `Update` method listens for the user to press the `Fire1/Attack` button (typically the left mouse button) and then calls `SpawnCube`.

Understanding `SpawnCube`:

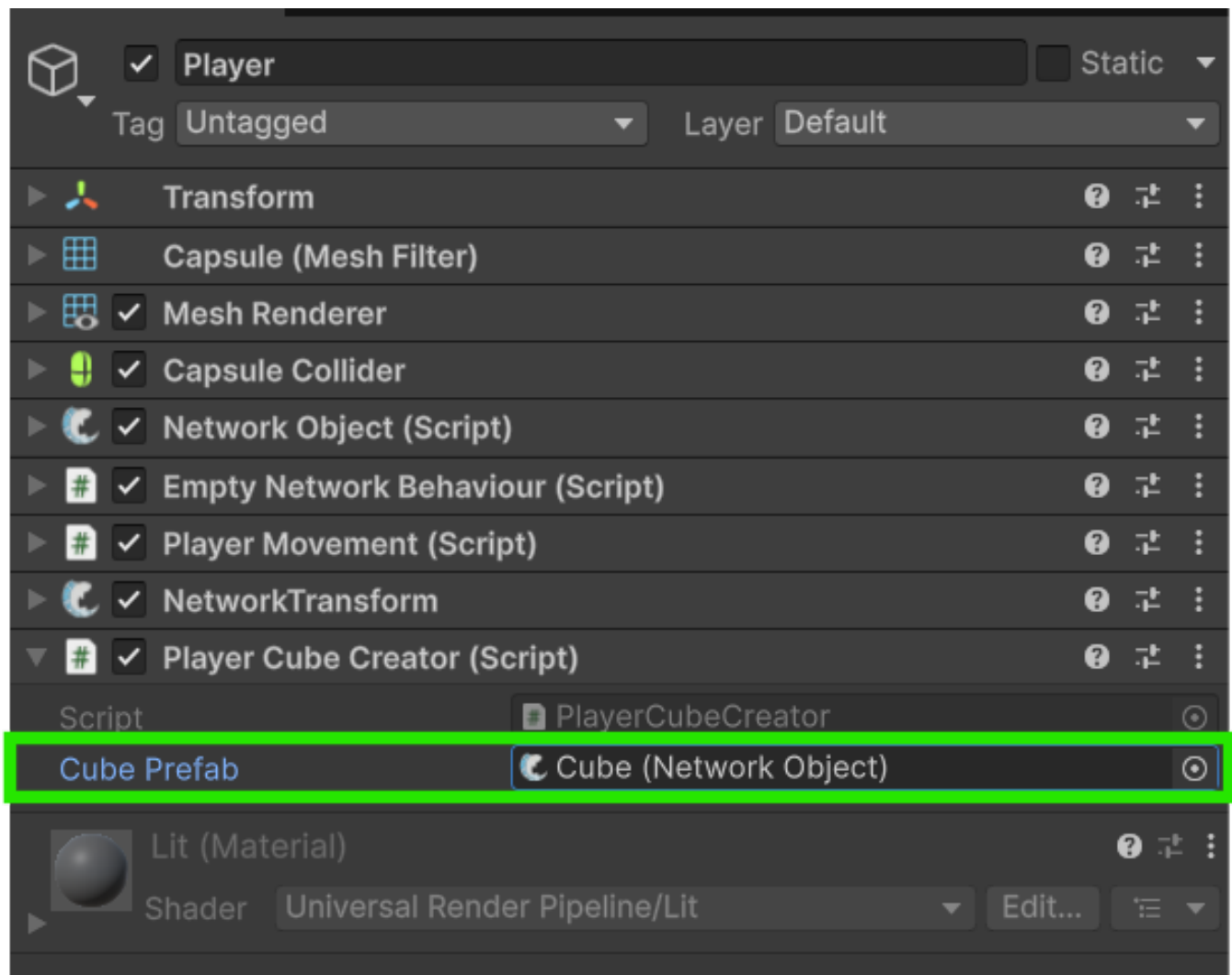
The `[ServerRpc]` attribute means this method is intended to be called by a client, but it will execute only on the server. The client sends a message to the server, requesting this method to be run.

`Spawn(obj)` is the most critical line for networked objects. Calling `Spawn` on the `NetworkObject` tells FishNet's `ServerManager` to "spawn" this object over the network — the server will now instruct all currently connected clients (and any clients that connect later) to instantiate their own replica of this object.

Info: You can read the more in-depth page about [Spawning](#) here.

Step 3 — Assigning the Prefab to Your Script

Select your **Player Prefab** and assign your Cube Prefab to the **Cube Prefab** field in your newly created Player Cube Creator script.



Info: You may have noticed that an **Empty Network Behaviour** component was automatically added to your game objects with a **NetworkObject** component. This happens because FishNet requires every **NetworkObject** to also have a **NetworkBehaviour** on it, and it will automatically add an empty one if it doesn't detect any. You can safely remove this empty component as you have added other **NetworkBehaviour** components, but it's not a problem either way.

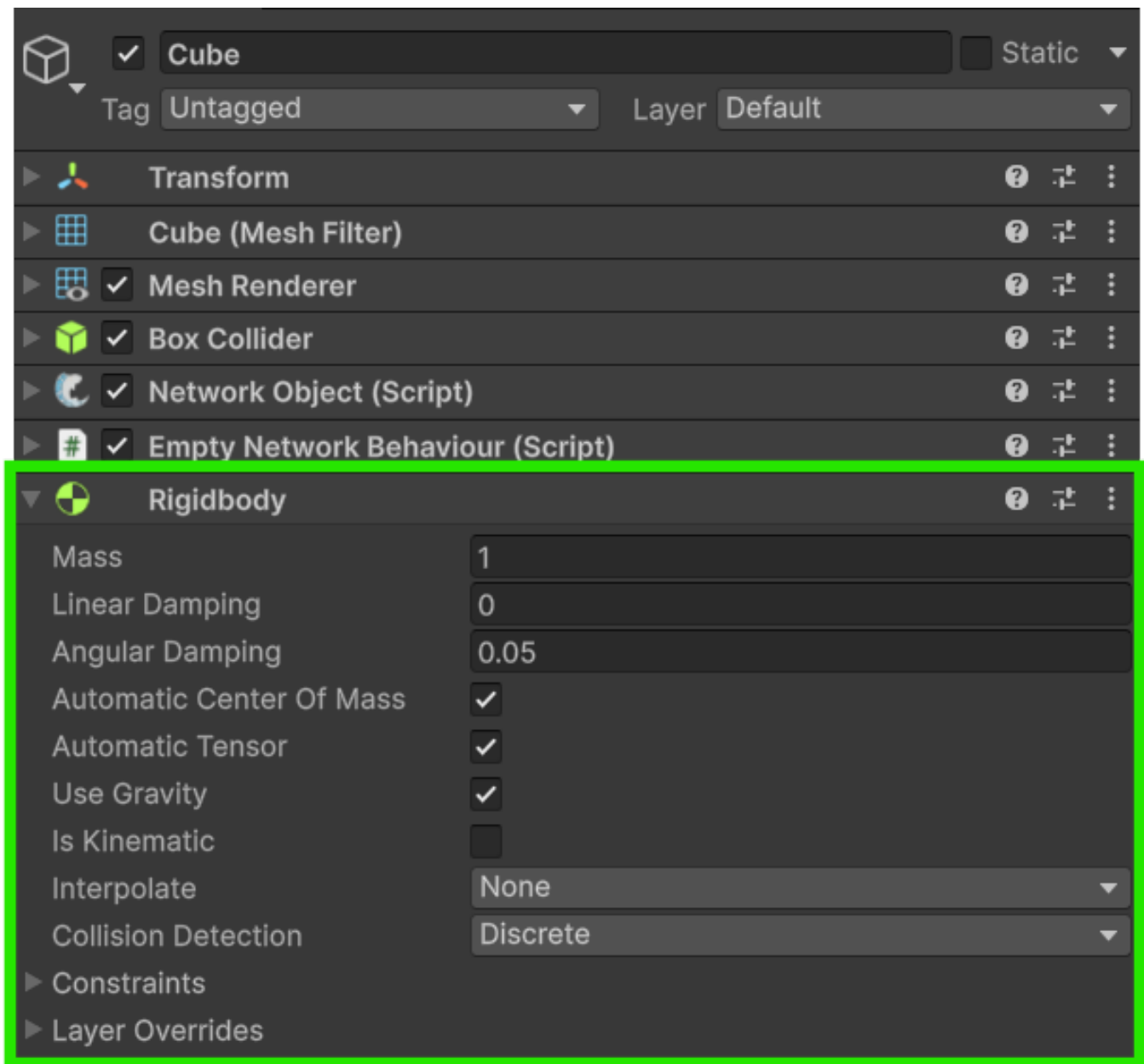
Step 4 — Test If the Spawning Works

Launch a couple instances of your game and see if you can run around and spawn cube items by pressing the Fire1/Attack button. The cubes should be visible on all devices connected to each other.

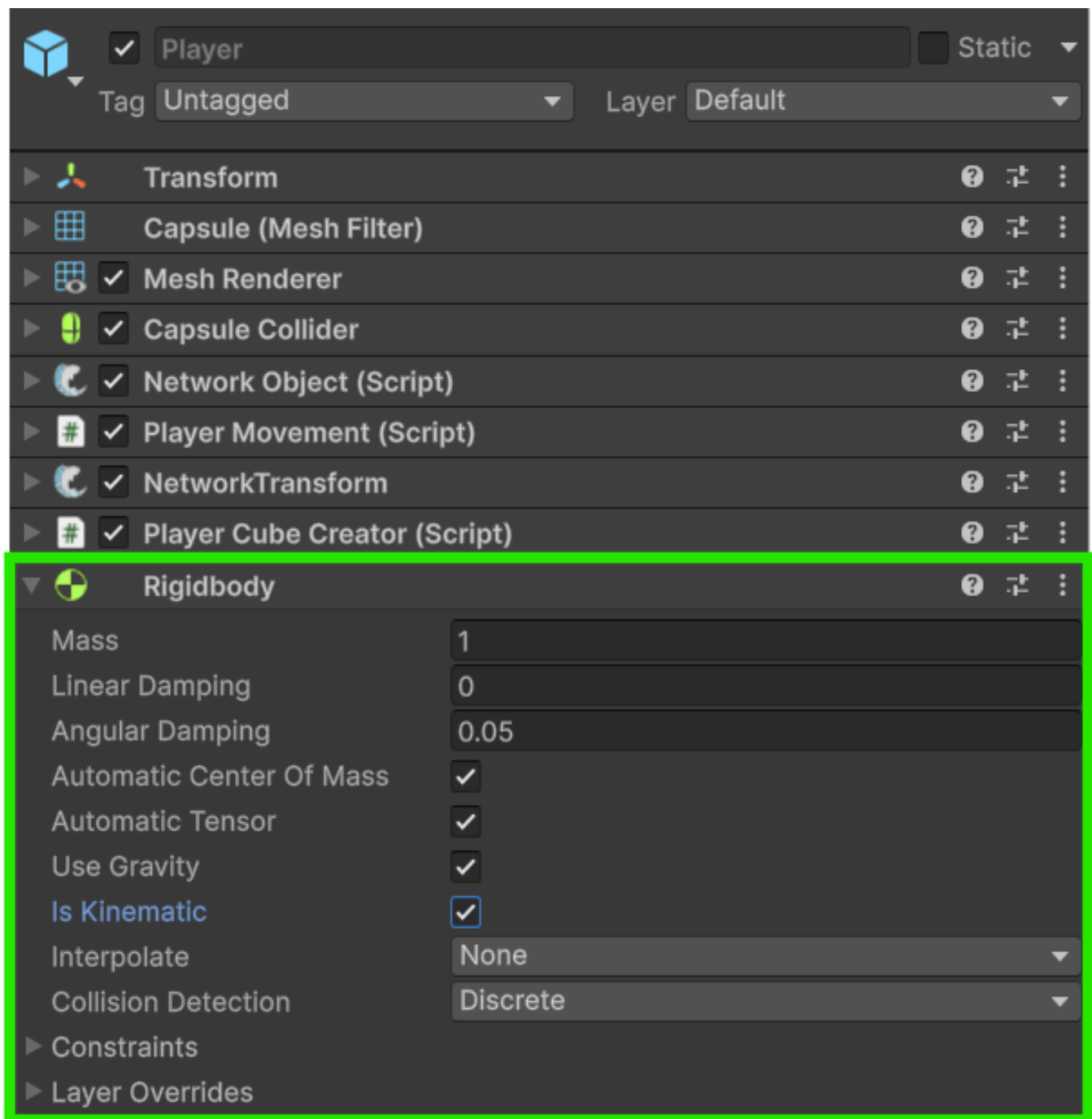
[Image: Testing cube spawning across two instances]

Step 5 — Add Some Physics

Static cubes are boring! Let's add a **Rigidbody** component to the Cube Prefab to enable basic physics interactions with them.



Now the cubes should roll around and collide with the ground. Also add a **Rigidbody** to the Player and set it to **Is Kinematic**.



Step 6 — Test If the Physics Works

If you launch the game now, the cubes should have basic physics working.

[Image: Players spawning physics cubes]

You may notice the cubes' positions don't always sync up after they are moved around — that's because we haven't done any positional syncing besides the initial position syncing that FishNet automatically did when we called `Spawn`.

Info: Networked Rigidbodies are a complex topic, better suited to a different guide. For a really basic setup, you can add a **NetworkTransform** to your Cube Prefab and set its **Component Configuration**

to **Rigidbody**.

Step 7 — Script to Despawn Cubes on a Timer

Now that we can Spawn cubes successfully, let's Despawn them after a few seconds so that we don't end up with too many objects at once.

Create a new script called `DespawnAfterTime`:

```
using FishNet.Object;
using System.Collections;
using UnityEngine;

public class DespawnAfterTime : NetworkBehaviour
{
    public float SecondsBeforeDespawn = 3f;

    public override void OnStartServer()
    {
        StartCoroutine(DespawnAfterSeconds());
    }

    private IEnumerator DespawnAfterSeconds()
    {
        yield return new WaitForSeconds(SecondsBeforeDespawn);
        Despawn(); // NetworkBehaviour shortcut for ServerManager.Despawn(gameObject);
    }
}
```

This script uses the `OnStartServer` `NetworkBehaviour` override method to start a Coroutine which will Despawn the object. `OnStartServer` will run on the server when the object is initialized with the network.

`Despawn` needs to be called on the server, and it will destroy the game object locally as well as on all clients automatically for you.

Step 8 — Assigning the Script and Testing

Open your **Cube Prefab** and add the `DespawnAfterTime` script to the object.

Launch the game again and observe the objects being Despawned successfully on the server and all clients after a few seconds.

[Image: Cubes despawning after a few seconds]

Using SyncVars to Sync Colors

Synchronizing color with synchronized variables!

We've got quite a few things synchronized now, but how might you go about synchronizing a specific variable in one of your scripts? **SyncVars** are one answer! A SyncVar is a FishNet generic type that can be used within NetworkBehaviours that automatically synchronize their value from the server to all clients.

Let's liven up the colors in our game and synchronize them with SyncVars.

Step 1 — Creating a Script to Sync Color

Create a new script called SyncMaterialColor. This will be used to synchronize the Material color of our MeshRenderers.

```
using FishNet.Object;
using FishNet.Object.Synchronizing;
using UnityEngine;

[RequireComponent(typeof(MeshRenderer))]
public class SyncMaterialColor : NetworkBehaviour
{
    public readonly SyncVar<Color> Color = new SyncVar<Color>();

    private void Awake()
    {
        Color.OnChange += OnColorChanged;
    }

    private void OnColorChanged(Color previous, Color next, bool asServer)
    {
        GetComponent<MeshRenderer>().material.color = Color.Value;
    }
}
```

This simple script has a Color variable of the type SyncVar<Color>. It needs to be set to readonly, but don't worry, we can still get and set its Value.

Info: Setting your SyncVar as readonly will cause it to be hidden from the Unity Inspector. You can read about a workaround for this in the SyncVar documentation.

In Awake we subscribe to the SyncVar's OnChange event. This event is very useful as it will be invoked as soon as the SyncVar changes, even on a client when the server changes it and it's synced.

OnColorChanged is our method that we subscribed to the OnChange event. This method simply gets the MeshRenderer and sets its material color to the SyncVar's Color Value, thus updating the visuals to match on all devices.

Tip: You may want to read more about SyncVars and the other available SyncTypes that FishNet has, such as SyncLists. You can find those pages in the [Features](#) section.

Step 2 — Add the Script Component

Add your newly created SyncMaterialColor script to your **Cube Prefab**. The script won't currently do anything unless we change the Color SyncVar in it, so let's do that next.

Step 3 — Give the Cubes Random Colors

Let's give the cubes some color as soon as we instantiate them.

Reopen the `PlayerCubeCreator.cs` script and make the following addition after the call to `Instantiate` and before the call to `Spawn`:

```
obj.GetComponent<SyncMaterialColor>().Color.Value = Random.ColorHSV();
```

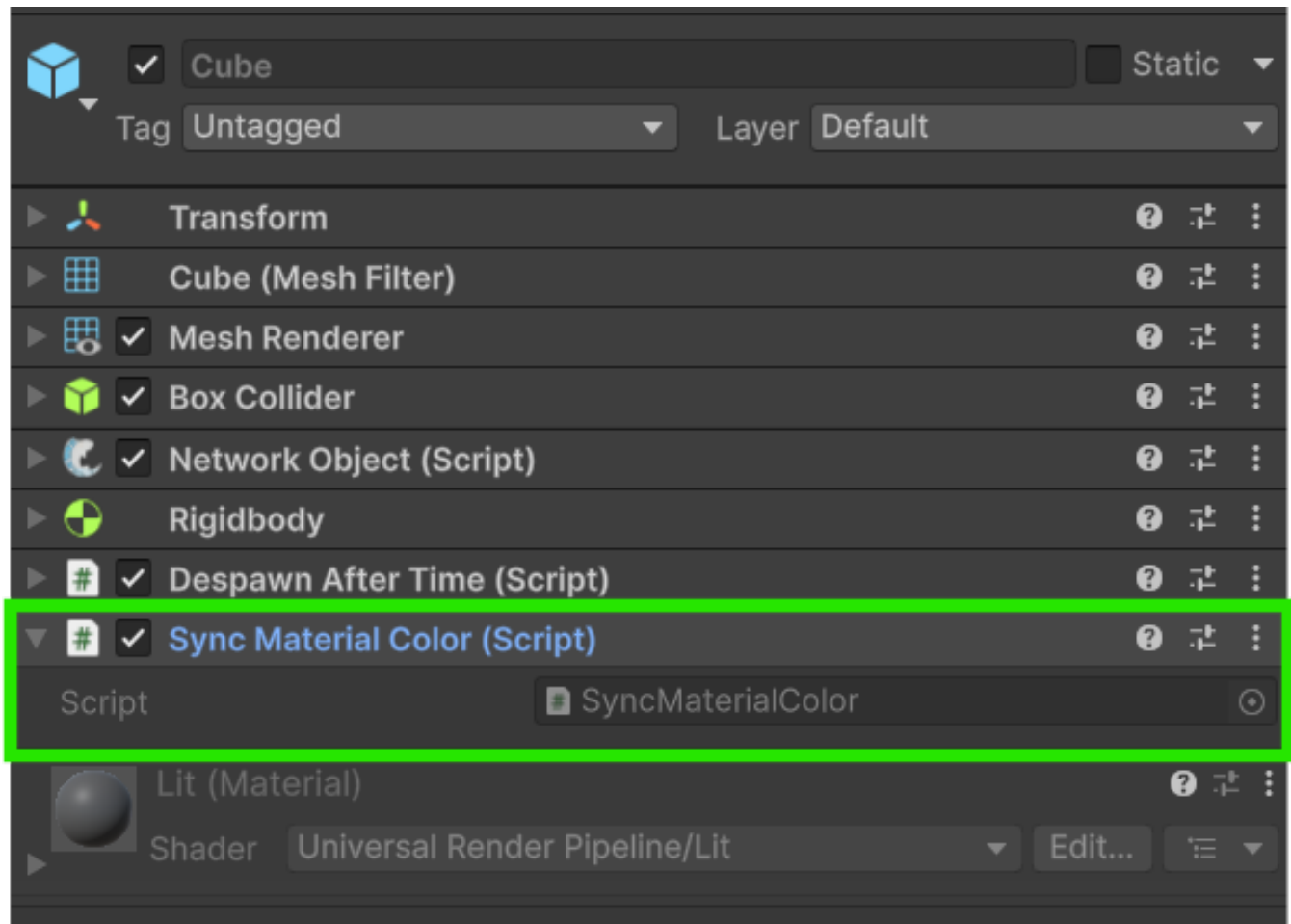
Here is the complete updated `SpawnCube` method:

```
[ServerRpc]
private void SpawnCube()
{
    NetworkObject obj = Instantiate(CubePrefab, transform.position, Quaternion.identity);
    obj.GetComponent<SyncMaterialColor>().Color.Value = Random.ColorHSV();
    Spawn(obj); // NetworkBehaviour shortcut for ServerManager.Spawn(obj);
}
```

This line of code gets the `SyncMaterialColor` component and sets the `SyncVar Value` to a random color as soon as it's instantiated. Once `Spawn` is called on the next line, FishNet will spawn this object for all clients with the color `SyncVar` state synced.

Step 4 — Test the Synchronized Cubes

Now all you need to do is run your game again and see if the cubes spawn with a random color and if that color is synchronized across the network.



[Image: Synchronized cube colors across clients]

Fundamentals

To understand FishNet, it's helpful to grasp a few fundamental concepts.

A Networked Game

At its heart, a networked game using Fish-Networking can be thought of as a single-player game running on multiple machines (or even multiple instances of the game running on the same machine, which is very useful for development). The magic of multiplayer comes from FishNet's ability to synchronize various parts of your game's code and state across these machines, and to send custom messages back and forth to trigger actions or update information. This ensures that all players experience a consistent and interactive world.

The code you write behaves the same as it does in a regular game — code in `Update` will run each frame, `Awake` triggers as soon as a script is being loaded, etc. But FishNet adds additional features such as **SyncVars**, which you will be able to set on the game instance that is acting as the server and they will automatically have their value synced to all connected client game instances. FishNet also has **Remote Procedure Calls**, which are methods that you call on one game instance and FishNet will execute the method on a different game instance instead of executing it locally.

To enable these features, FishNet needs to be able to identify game objects and scripts across the network so that it can know which game object and specific script on that game object you called an RPC or changed a SyncVar on. FishNet does this by assigning IDs to selected `GameObjects` that you give a `NetworkObject` component to. It also assigns IDs to any component that inherits from `NetworkBehaviour`, which is a `MonoBehaviour` that supports RPCs, SyncVars, and useful network methods and properties.

For `NetworkObjects` that can be instantiated during run-time, FishNet needs to store a prefab of them and assign an ID to the prefab so that it can instantiate that same one across the network when you spawn it on the server. This is handled with a prefab collection scriptable object called the **Spawnable Prefabs**.

The Server and Client

FishNet is capable of running as a server and/or as a client, and they can be started and stopped independently of one another. When you start the server, FishNet will set various fields such as `IsServerStarted` as `true` in that game instance that is running as the server. The same applies when you start a client; when both are running FishNet will also set values such as `IsHostStarted`. These fields can be used to determine if the code is running in the game instance acting as a server, client, or both.

Key Concepts

NetworkManager — This is the central hub of your FishNet application. It manages the networking lifecycle, including starting and stopping servers and clients, handling connections, and overseeing the spawning of networked objects.

NetworkObject — Any `GameObject` in your Unity scene that needs to be linked across the network must have a `NetworkObject` component attached. This component assigns a unique network ID and manages the object's networked lifecycle, ensuring its existence and state are consistent across all connected clients.

NetworkBehaviour — Similar to Unity's `MonoBehaviour`, `NetworkBehaviour` is the base class for all scripts that need to directly contain RPCs, SyncTypes, or `NetworkBehaviour` methods. Scripts inheriting from `NetworkBehaviour` can access useful properties like `NetworkManager`, `IsServerStarted`, `IsSpawned`, `IsOwner`, as well as be directly referenced across the network.

Ownership and Observers — FishNet provides a clear system for defining ownership of NetworkObjects, typically assigned to the client that created or controls the object. Observers are clients that are "observing" a particular NetworkObject, meaning they receive updates about its state. This allows for efficient data transmission, sending updates only to relevant clients.

Remote Procedure Calls (RPCs) — RPCs are a core mechanism for communication between clients and the server. They allow you to call methods on a specific NetworkBehaviour instance that execute on a remote machine's linked version of that instance. Any script can call an RPC, even regular MonoBehaviour s, ScriptableObjects, or other classes.

SyncVars — These are powerful features for automatically synchronizing variable values and complex data structures (like lists or dictionaries) across the network. When a SyncVar's value changes on the server, it's automatically replicated to all observing clients.

High-Level Overview





A general overview of networking using FishNet.

?? Fundamentals

Learn the core concepts and basic principles of our system.

? Networking Models

Dive deep into the various networking architectures and designs.

? Terminology

Understand the key terms and definitions used across our documentation.

? Transports

Discover the different methods and protocols for data transfer.

Networking Models

This page discusses the various networking models and architectures used in games and what Fish-Networking uses.

Client-Server Architecture

The client-server model is the most widely used approach in multiplayer networking. In this model:

- **Server** — The authoritative instance of the game. It manages game logic, synchronizes game state, validates player actions, and handles communication between clients.
- **Client** — A player's instance of the game. It sends input to the server and receives updates about the game state.

There are two main types of servers:

Dedicated Server

The server runs as its own process, separate from any player. It's more secure and scalable, commonly used in competitive or large-scale games.

As shown in the dedicated server model, the clients (players) are all connected directly to the server and communicate with it instead of with each other.

[Image: Client-Server (Dedicated) Model]

Host (Listen-Server)

The server and one of the clients run in the same process. This is common in smaller games or development environments.

In a listen-server model all clients connect directly with the server, but unlike the dedicated server, the listen server is also acting as a client (player).

[Image: Listen-Server Model]

? FishNet supports both dedicated and listen-server models out of the box.

Peer-to-Peer (P2P)

In peer-to-peer networking, each player (or peer) connects directly to others without a central server. This reduces latency and server costs but comes with challenges:

- Poor security and cheating prevention
- Difficult to synchronize game state reliably
- NAT traversal issues

P2P is rarely used in modern real-time multiplayer games, especially competitive ones, due to these drawbacks. FishNet does not use P2P, as it is designed around the client-server model for reliability and security.

[Image: Peer-to-Peer Model]

?? Very often a listen server setup is referred to as P2P. FishNet does support a listen server.

Relay Servers

Sometimes clients are unable to connect directly to the server due to NAT (Network Address Translation) or firewall restrictions. A relay server acts as a middleman to forward traffic between clients and the server.

Relay servers:

- Allow connectivity when direct connections fail
- Introduce additional latency
- May be used as a fallback or for web-based games

[Image: Relay Server Model]

? Fish-Networking supports multiple third-party relay services including Unity's Relay, Edgegap's Distributed Relay, Photon Realtime, Epic Online Services' Relay Service, and Steam Datagram Relay.

Host Migration

When using a listen server, the host leaving the game would cause all players to leave the game. Host migration is a feature for maintaining game sessions in the event the current host disconnects or experiences network issues. It enables a new host to be elected from the remaining connected players, minimizing disruption and ensuring the game session can continue seamlessly.

?? Fish-Networking doesn't yet have built-in host migration, but it is a feature that may be added in the future.

Communicating

There are a variety of ways to send communications between server and clients. Below are several terms you will become familiar with when wanting to send data.

SyncTypes

SyncTypes reside on objects and are data-driven variables or collections. SyncTypes send at adjustable intervals. When a SyncType is modified on an object the changes are automatically sent from the server to clients. Clients will receive the changes locally on the same object. A great example is a health variable — you may update the health variable as a player takes damage, and the new values will be sent to clients.

This basic example shows a simple integer variable that will automatically be synchronized to all clients whenever the server changes it.

```
using FishNet.Object;
using FishNet.Object.Synchronizing;

public class Player : NetworkBehaviour
{
    private readonly SyncVar<int> _health = new SyncVar<int>(100);

    public void StepOnLego()
    {
        if (!IsServerInitialized) return;
        _health.Value -= 10;
    }
}
```

Remote Procedure Calls

Remote Procedure Calls are another object-bound communication type. While SyncTypes are used to synchronize variables, RPCs allow you to run logic on server and clients. RPCs are not limited to an interval like states — RPCs can be sent immediately on the next network tick or as soon as possible. They are required to be within a NetworkBehaviour class.

This is a simple example that shows a Server Remote Procedure Call. The method should be called from a client and the body will not execute locally — instead, the arguments will be sent across the network and the method body with those arguments will run remotely on the server.

```
using FishNet.Object;

public class Player : NetworkBehaviour
{
    [ServerRpc]
    private void UpdatePlayerName(string newName)
    {
        print($"Player {OwnerId} has updated their name to {newName}");
    }
}
```

Broadcasts

Unlike SyncTypes, broadcasts are not object-bound. Broadcasts can be used for any number of tasks but are more commonly preferred for communicating data groups between server and clients. Broadcasts can

be received and sent from anywhere in your code and do not need to be within a `NetworkBehaviour` class as RPCs do.

This example shows how a client could send a broadcast to the server.

```
using FishNet;
using UnityEngine;

public class ChatSystem : MonoBehaviour
{
    public void SendChatMessage(string text)
    {
        ChatBroadcast msg = new ChatBroadcast()
        {
            Message = text,
            FontColor = Color.white
        };
        InstanceFinder.ClientManager.Broadcast(msg);
    }
}
```

?? For all of the above communication methods, two channels are supported: **Reliable** and **Unreliable**. Data sent reliably is guaranteed to arrive and be processed in the order it was sent. Data sent unreliably uses less bandwidth, but can occasionally arrive out of order, or not at all.

?? Some features of Fish-Networking internally use eventual consistency — an example of this are unreliable `SyncVars` or the `NetworkTransform` component. These features use the unreliable channel to send data to consume less bandwidth and provide better performance, but you can still use them knowing that even if data is dropped or arrives out of order the features will eventually resolve the desynchronizations automatically.

Terminology

The following pages give some commonly used terminology for Fish-Networking as well as general networking terms you are likely to encounter.

[Server](#), [Client](#), [Host](#) [Communicating](#) [Miscellaneous](#)

Miscellaneous

There are several commonly used terms which you may encounter throughout your development experience with FishNet.

Scene Object

A scene object is an object which is placed in a scene's hierarchy during edit-time. Scene objects are objects which may be spawned over the network, but do not require instantiating as they are already placed in the scene.

Instantiated Object

An instantiated object is an object which was instantiated at run-time, and is not a scene object. For example, `Instantiate(yourPrefab)`.

Predicted Object

An object which utilizes the prediction system. Predicted features generally run on the client and server at the same time to allow real-time gameplay interactions. These types of behaviors require more work but a proper prediction setup will prevent cheating.

Client Authoritative

Often means an object is controlled by the client and results are sent to the server without validation. Client authoritative coding is easier to understand but may allow players to cheat more easily. An example of this is a `NetworkTransform` component with `Client Authoritative` set to `true` — the client controls the object locally and the server updates to the client's values.

Server Authoritative

Values are controlled or verified by the server, such as `SyncTypes`. Predicted objects are server authoritative.

Server, Client, Host

Ownership and Controller

Ownership is when a specific client owns the object, while controller is whichever entity controls the object. By our terms, only a client can own an object, but either the server or a client may be the controller.

When a client owns an object they are the "controller" and are able to send remote procedure calls without disabling authority checks, among other tasks such as generating prediction data.

A controller is the server if the object is not owned by any client, but again, the client becomes the controller when they own the object.

Server

Server is an instance of your project which clients connect to. A server allows clients to interact with each other and will often be responsible for important aspects of your game such as score keeping, world spawning, and more.

Local Server

When a server is started the build or instance running the server is referred to as the local server.

Remote Client

A remote client is a term only applicable when running as the server. A remote client is a player connection established with the server. It's possible for a remote client to be the same as a local client. The main difference is the local client refers to the player-side logic while a remote client refers to the server's connection with the client.

Client

A client connects to the server, whether the server be local or online. A client is the player for your game. A client experiences the gameplay while the server synchronizes data between clients.

Local Client

Local client refers to the client controlling their game. If you join the server as a player, you are the local client.

Remote Server

When the local client connects to a server, they are connecting to a remote server. It's possible the remote server is also the local server if the client is running as host.

Host

When behaving as a host you are both the client and the server. An individual can become host without having to compile additional executables, nor having to provide any special access to their computer. Host is ideal for speedy development.

Transports

Transports control how data is sent, received, and handled over the network.

The Transport is the low-level layer responsible for sending and receiving raw data over the network. It abstracts away the details of TCP/UDP sockets and handles packet delivery, reliability, ordering, and more.

FishNet uses events internally to plug into transport messages. Although it would be unlikely you would need to access such messages, they are available to you for your development as well.

There are many transports available — some are maintained by the Fish-Networking team and others maintained by the community.

Transport	Description
Tugboat	Uses LiteNetLib. The default recommended transport for most games.
Bayou	WebGL-compatible transport for browser-based games.
Yak	Offline transport for local/offline gameplay without network overhead.
Multipass	Enables using multiple transports simultaneously.
FishySteamworks	Steam networking via Steamworks.NET.
FishyFacepunch	Steam networking via Facepunch.Steamworks.
FishyEOS	Epic Online Services relay and P2P transport.
FishyUnityTransport	Unity's official transport layer (e.g. for Relay).
FishyRealtime	Photon Realtime relay transport.
FishyWebRTC	WebRTC transport for browser-to-browser connectivity.
CanoeWebRTC	Community WebRTC transport alternative.

Branding

We have a variety of images you may choose from for branding Fish-Networking.

- You may not alter the colors nor design on the images provided.
- You may resize images but not skew them such as by changing their aspect ratio.

Branding Pack

Business Support

We offer a variety of business plans for those that wish exceptional support or priority on their needs. All business tiers come with life-time access to Pro updates, via github account.

Please contact FirstGearGames@gmail.com or FirstGearGames#0001 on Discord prior to purchasing plans, or for any questions you may have.

To subscribe to or view plan benefits please visit <https://github.com/sponsors/FirstGearGames>.

Development

Fish-Networking is under constant improvement, receiving regular updates containing new features and bug fixes.

Making suggestions

The best way to make suggestions is to post on our [GitHub Discussions](#) page. Once posted you'll be able to follow comments and progress.

Reporting bugs

We appreciate all bug reports that our users may have, and make a strong effort to resolve them. We have a guide for how to post bug reports on our GitHub [here](#).

Making code contributions

We welcome all kinds of code contributions to Fish-Networking, including bug fixes, new features, and improvements. We have bounties for any serious exploits that can be found and also miscellaneous bounties for other features. If you have something you'd like to work on, please head over to the [open-development](#) channel in our [Discord community](#) and have a chat with us. We also welcome pull requests on our [GitHub Repository](#).

Making documentation improvements

Our documentation is open-source, and we encourage our users to contribute content and fix any mistakes that it may contain. You can do so directly on the [Documentation GitHub Repository](#).

If you don't want to make changes you can also suggest documentation improvements on our [GitHub Discussions](#) page.

Creating guides, tutorials, samples, and add-ons

If you have a desire to create guides, tutorials, samples, or add-ons for FishNet, it would be most appreciated by us and the community. You can share such creations with the community directly in our Discord server, but you are also welcome to post them on YouTube, Itch.io, or any other site you want to.

If you want to use the FishNet branding in your creations, you can find it here: [Branding](#)

Features

Info: For a more exact feature comparison of various networking solutions please see this [comparison chart](#).

Performance and reliability are very important to us, but so is including great features to make your experience easier, and better. The last thing we want to do is create limitations; we support all features Mirror does, and then some.

Below is a comparison of unique and enhanced features, with more to come.

Features unique to Fish-Networking

Update Stability Other solutions do not guarantee that their API will not change between releases, making it difficult and dangerous to update for fixes or features. Fish-Networking prides itself on it's No-Break Promise, ensuring updates do not require alteration of user code, keeping you on-top with the best to offer.

Built-in Addressables Support Unlike other networking solutions, Fish-Networking has addressables support included within core features. Addressable scenes are easily implemented with our network SceneManager, and addressable prefabs are supported as well.

Network Balancing NetworkManagers are the center of your network environment. They contain other managers, and handle communications between the client and server. Fish-Networking allows an infinite number of NetworkManagers, allowing clients at once to connect to multiple different servers, to have each handle certain tasks or worlds. Various NetworkManagers may also be used to fit different game modes, transports, platforms, and a number of other tasks.

Local Remote Calls Fish-Networking allows remote calls to run not only on their intended destination, but also on the caller as well. Typically you would have to copy logic into multiple methods to accomplish such a task, but Fish-Networking allows it to be done in one method. This drastically reduces potential errors in code, lines of code, and improves ease of use.

Multi-purpose Remote Calls Target and Observers remote calls can be used interchangeable, in the same method. This allows you to use a single method to send data to one client or multiple clients. Like Local Remote Calls, this creates a more reliable code-base and improves the user experience.

Buffered Remote Calls Often the last value of a remote call must be sent to newly joined clients. Fish-Networking is one of few solutions that offer this ability.

Client-side Prediction Client-side prediction primarily focuses on ensuring players in your game cannot cheat a variety of movement types. This feature is an absolute must for any competitive game. Fish-Networking is the only free solution to provide built-in client-side prediction with only a few lines of code. Also, included are a variety of components for desynchronization smoothing, and rigidbody prediction.

Lag Compensation (pro feature) Another very important feature for precision based gaming is lag compensation. This is the act of rolling back colliders in time to where a client had seen them; this ensures accurate hit registration. This technique is applicable to several genre types but is most commonly seen in shooter games.

Automatic Code Stripping (pro feature) Code stripping helps protect your game server by removing sensitive logic that the player should not be aware of.

Large Packet Handling In some scenarios you may need to send large amounts of data either to the client or server. Tests show Fish-Networking was able to effectively send anywhere from a few, to a few

hundred megabytes over the network. Other solutions such as Mirror and even Fusion throw errors on the same job.

Reliable Order of Operations Race conditions are possibly the worst kind of problems to solve, and it's especially frustrating when those race conditions are caused by someone else's software. Competitors such as Mirror have shown to have a variety of race conditions, such as networked objects referencing one-another before they actually exist. Fish-Networking ensures references cannot go missing, and that the reliability of operations and callbacks are consistent in all scenarios.

Child Network Components Child Network Components allow for easier design and better code organization by using components such as Network Objects or Network Behaviours on child objects. Mirror for example requires all networked components to be placed on the root object, which could quickly clutter your work flow.

Custom SyncTypes Custom SyncTypes allow you to create logic on how something may be updated over the network. For example: SyncVars, SyncList, SyncDictionaries are all custom SyncTypes included within Fish-Networking. If you would prefer a more hands-on/tailored experience you may create your own Custom SyncType using the exposed framework.

Dual Projects Fish-Networking has limited support for dual projects. We recommend using a single project and utilize server defines along with our automatic code stripping feature.

Single, Additive, Stacked Scene Management Scene management is essential for offline and online games. While other solutions such as Mirror, Fusion, and Netcode are limited to basic single scene management out of the box, Fish-Networking provides built-in logic to control separation of clients through single, additive, and even stacked scenes. This is a powerful tool for world streaming, dungeon instances, lobbies and more.

Automatic Prefab Detection Networking solutions such as Mirror require you to specify and organize networked object prefabs. This is a time consuming task which often requires dragging every prefab into a collection, or looking up prefabs by string before you may spawn them over the network. Fish-Networking strongly believes in ease-of-use, and part of that is having your networked prefabs automatically detected by our framework. Changes made to networked objects are automatically known and configured by Fish-Networking without requiring additional steps from the user.

Built-in, or Custom Object Pool Included is the ability to automatically reset and pool network objects. Objects can be set to pool at runtime, or be pre-configured on the NetworkObject. Pooled objects automatically have their network values reset, including all SyncTypes on the object. Fish-Networking comes with a basic object pool but fully supports any object pool of your choice.

Zero Hotpath Allocations Allocations can massively affect both server and client performance. While Mirror and Netcode allocate garbage regularly, Fish-Networking does not, ensuring better scalability and all-around performance.

Offline Networked Objects In some instances an object may need to only exist locally, rather than over the network, and sometimes both. Fish-Networking allows users to default objects to being local only so that the objects network functionality is unused. In addition, these objects are intelligently synchronized over the network when the user shows such intent.

Features which do more in Fish-Networking

Serializables Fish-Networking supports more object types to be serialized than any other free solution. Dictionaries, scriptable objects, nullables and more can be automatically serialized in Fish-Networking. Solutions which have existed longer, such as Mirror, still require manual serialization of many types which Fish-Networking handles automatically.

Area of Interest System Area of interest, often called AOI, controls which clients receive what information. AOI not only can prevent cheating by disallowing information clients get, but also drastically improve bandwidth and performance of the server by not sending unnecessary information to clients. Both Fish-Networking and Mirror support AOI, however in Mirror you may only use one AOI system at a time. For example, you can separate clients by scenes in Mirror, but you cannot also separate them by distance at the same time. In result if you have a larger world players will see each other at all times. Fish-Networking not only streamlines setting up the AOI system but allows multiple conditions to run at once, which would resolve the just mentioned Mirror problem. Fish-Networking's AOI system can also be easily customized with your own logic.

Reliable and Unreliable Remote Calls Several networking solutions support reliable and unreliable remote calls, but Fish-Networking simplifies the process for users. In Mirror if you wish to have a remote call which can send both reliable and unreliable you must duplicate your logic into multiple methods, one for reliable and one for unreliable. Fish-Networking again saves on lines of code and risk of error by allowing a single remote call method to run both reliably and unreliably. You can change the reliability effortlessly at runtime as well.

SyncType Flexibility SyncTypes are another very common feature found in just about every high level library. However, very few solutions offer flexibility on how SyncTypes can be configured. Mirror for example can only send SyncTypes reliably, cannot adjust the interval per SyncType, has limited callbacks, and cannot control who receives the values of each SyncType. Fish-Networking on the other hand may send reliable or unreliable with eventual consistency. SyncType intervals can be different for each SyncType. Callbacks can occur on both the server and client. Each SyncType can be specified to go to only the owner, or all clients.

Tick Based Tick based timings allow accurate simulation for client-side prediction, smooth transform replication, and a lot more. Fish-Networking has a strong and efficient tick based core to provide a smoother experience for both the developer and their players. Older networking solutions such as Mirror are not tick based, which results in sloppy simulations as well the inability to create a proper client-side prediction system.

Broadcasts and Messages Fish-Networking supports broadcasts, interchangeably called messages. Broadcasts and messages are data which can be sent over the network without requiring a networked object receiver. These are useful for objects which you don't want networked but would still like to be able to communicate with, such as a door in your scene. Fish-Networking enhances this communication type by allowing multiple objects to utilize the same message. For example, in Fish-Networking all your scene doors could listen to the same message for updates; Mirror on the other hand only one door could listen to updates while you would have to create additional broadcast or message types for every single added door.

Original Mirror, Netcode, and several other networking solutions are based off from previous works, some which are deprecated. Fish-Networking is made original from the ground up, with no prior solution's limitations to bog it down.

Anti-Singleton Design Most networking solutions, even newer ones such as Netcode, use a singleton design pattern. Fish-Networking does not utilize singleton classes for it's managers, but still provides an easy way to access networking data from anywhere.

XML Documentation An often unconsidered part of any project are the XML comments for it's API. Netcode, Mirror, and even Unity's official API have several parts which are not fully commented internally nor through the XML. Fish-Networking provides complete XML coverage with well thought-out descriptions to make developing easier without constantly have to jump through the source code or visit external sources.

Benchmark Setup

Everything looks good on paper but dedicated benchmarks can show you what to really expect.

Setup

- New scene with Camera and directional light.
- 1 object per player connection.
- Object moves and rotates randomly.
- Object has a client authoritative NetworkTransform with 100ms send rate.
- Object sends two ServerRPCs a second.
- Server sends an ObserverRPC for every ServerRPC it receives.
- Every client is it's own executable.
- Server tick rate is set to 10 (every 100ms).

Gathering results

Performance results are determined by setting a server at it's highest possible frame rate and comparing frame time as more variables or clients are introduced.

Bandwidth results are discovered by using tools which come with the operating system, and averaging bandwidth per second.

Scripts

These are the scripts created specifically to benchmark and ensure test are performed equally.

MovingAverage.cs

```

using FishNet.Documenting;
using System;
using UnityEngine;

namespace FishNet.Managing.Timing
{
    [APIExclude]
    public class MovingAverage
    {
        #region Public.
        /// <summary>
        /// Average from samples favoring the most recent sample.
        /// </summary>
        public float Average { get; private set; }
        #endregion

        /// <summary>
        /// Next index to write a sample to.
        /// </summary>
        private int _writeIndex;
        /// <summary>
        /// Collected samples.
        /// </summary>
        private float[] _samples;
        /// <summary>
        /// Number of samples written. Will be at most samples size.
        /// </summary>
        private int _writtenSamples;
        /// <summary>
        /// Samples accumulated over queue.
        /// </summary>
        private float _sampleAccumulator;

        public MovingAverage(int sampleSize)
        {
            if (sampleSize < 0)
            {
                sampleSize = 0;
            }
            else if (sampleSize < 2)
            {
                if (NetworkManager.StaticCanLog(Logging.LoggingType.Warning))
                    Debug.LogWarning("Using a sampleSize of less than 2 will always return
the most recent value as Average.");
            }
            _samples = new float[sampleSize];
        }

        /// <summary>
        /// Computes a new windowed average each time a new sample arrives
        /// </summary>
        /// <param name="newSample"></param>
        public void ComputeAverage(float newSample)
        {
            if (_samples.Length <= 1)
            {
                Average = newSample;
            }
        }
    }
}

```

```

        return;
    }

    _sampleAccumulator += newSample;
    _samples[_writeIndex] = newSample;

    // Increase writeIndex.
    _writeIndex++;
    _writtenSamples = Math.Max(_writtenSamples, _writeIndex);
    if (_writeIndex >= _samples.Length)
        _writeIndex = 0;

    Average = _sampleAccumulator / _writtenSamples;

    /* If samples are full then drop off
     * the oldest sample. This will always be
     * the one just after written. The entry isn't
     * actually removed from the array but will
     * be overwritten next sample. */
    if (_writtenSamples >= _samples.Length)
        _sampleAccumulator -= _samples[_writeIndex];
    }
}

```

DisplayPerformance.cs

```

using UnityEngine;
using FishNet;
using FishNet.Managing.Timing;

public class DisplayPerformance : MonoBehaviour
{
    public int TargetFrameRate = 60;
    private float _nextDisplayTime = 0f;
    private MovingAverage _average = new MovingAverage(3);
    private uint _frames = 0;

    private void Update()
    {
#if UNITY_SERVER
        Application.targetFrameRate = TargetFrameRate;
        _frames++;
        if (Time.time < _nextDisplayTime)
            return;

        _average.ComputeAverage(_frames);
        _frames = 0;
        // Update display twice a second.
        _nextDisplayTime = Time.time + 1f;

        double avgFrameRate = _average.Average;
        // Performance lost.
        double lost = avgFrameRate / (double)TargetFrameRate;
        lost = (1d - lost);

        // Replace this with the equivalent of your networking solution.
        int clientCount = InstanceFinder.ServerManager.Clients.Count;
        Debug.Log($"Average {lost.ToString("0.###")} performance lost with {clientCount} c
lients.");
#elif UNITY_EDITOR
        // Max out editor frames to test client side scalability.
        Application.targetFrameRate = 9999;
#else
        /* Limit client frame rate to 15
         * so your computer doesn't catch fire when opening
         * hundreds of clients. */
        Application.targetFrameRate = 15;
#endif
    }
}

```

SimulatePlayer.cs

```

using FishNet.Object;
using UnityEngine;

public class SimulatePlayer : NetworkBehaviour
{
    private float _nextMoveUpdate = 0f;
    private float _nextRpc = 0f;
    private Vector3 _posGoal;
    private Quaternion _rotGoal;

    private void Update()
    {
        if (!base.IsOwner)
            return;

        transform.position = Vector3.MoveTowards(transform.position, _posGoal, Time.deltaTime * 3f);
        transform.rotation = Quaternion.RotateTowards(transform.rotation, _rotGoal, Time.deltaTime * 20f);

        if (Time.time > _nextRpc)
        {
            _nextRpc = Time.time + 0.5f;
            ServerRpc();
        }

        if (Time.time > _nextMoveUpdate)
        {
            bool rotate = (Random.Range(0f, 1f) <= 0.5f);
            bool x = (Random.Range(0f, 1f) <= 0.5f);
            bool y = (Random.Range(0f, 1f) <= 0.5f);
            bool z = (Random.Range(0f, 1f) <= 0.5f);

            if (!x && !y && !z)
                x = true;

            transform.position = _posGoal;
            transform.rotation = _rotGoal;

            float xPos = (x) ? Random.Range(-30f, 30f) : transform.position.x;
            float yPos = (y) ? Random.Range(-30f, 30f) : transform.position.y;
            float zPos = (z) ? Random.Range(-30f, 30f) : transform.position.z;

            _nextMoveUpdate = Time.time + 2f;
            _posGoal = new Vector3(xPos, yPos, zPos);

            if (rotate)
                _rotGoal = Quaternion.Euler(
                    Random.Range(-80f, 80f),
                    Random.Range(-80f, 80f),
                    Random.Range(-80f, 80f)
                );
        }
    }

    [ServerRpc]
    private void ServerRpc()

```

```
{  
    ObserversRpc();  
}  
  
[ObserversRpc]  
private void ObserversRpc() { }  
}
```


Fish-Networking vs Mirror

Danger: These benchmarks are very old. Server outbound bandwidth metrics have changed. As of the date 2023/08/26 Mirror is believed to be approximately 5% more efficient in bandwidth, and Fish-Networking about 90-95% more efficient in bandwidth. Example: If Mirror previously sent 100mB/s they are now expected to send 95mB/s. If Fish-Networking previously sent 100mB/s it would now send 5mB/s.

Versions

- Fish-Networking v1.4.3
- Mirror Networking v66.0.3

Results

Hardware

- Windows Server 2019
- AMD EPYC 7402P 24-Core (1 SLICE/CORE USED!)
- 2.00 GB RAM

100 CCU Scaling

- FishNet Server lost 1.5% of it's performance.
- FishNet Client ran at 1350 FPS.
- Mirror Server lost 29.5% of it's performance.
- Mirror Client ran at 791 FPS.

200 CCU Scaling

- FishNet Server lost 7.4% of it's performance.
- Mirror Server lost 83.2% of it's performance.
- Clients were untested.

100 CCU Bandwidth

- FishNet Server per second sent 7.3mB (adjusted 0.55mB) and received 1.1mB.
- Mirror Server per second sent 31.8mB (adjusted 30.2mB) and received 3.2mB.

200 CCU Bandwidth

- FishNet Server per second sent 29.9mB (adjusted 2.24mB) and received 2.2mB.
- Mirror Server per second sent 94.3mb (adjusted 89.6mB) and received 4.8mB.

Spawned Network Objects (idle)

Objects	Fish-Networking	Mirror
0	1.5ms	1.5ms
500	1.5ms	1.9ms
1000	1.5ms	2.2ms
2000	1.5ms	2.9ms

Results Summary

- **Client FPS:** FishNet clients achieved 70% more FPS than Mirror.
- **Bandwidth:** FishNet used 67-78% less bandwidth than Mirror.
- **Server Performance (200 CCU):** FishNet retained 92.6% server performance, while Mirror retained only 16.8% performance.
- **Server Performance (4000 idle objects):** FishNet retained 100% server performance, while Mirror retained only 36% performance.

Performance

Prior to Fish-Networking we developed assets for other solutions, such as Mirror Networking. These assets have proven to perform 4 times faster, and use little as 13% the bandwidth of Mirror's internal components. Unfortunately, third party assets can only go so far. This is strongly in part why Fish-Networking was made.

Fish-Networking has been designed with performance and bandwidth in mind. Features are regularly benchmarked for client and server performance, as well bandwidth consumption; these metrics are important while deploying. Better performance allows your project to run on cheaper hardware, and enables a higher concurrent user count. By using less bandwidth Fish-Networking reduces your server bills.

Bandwidth

Networked Objects (moving)

In most games moving objects will contribute the majority of used bandwidth. Fish-Networking's bandwidth consumption competes aggressively against modern paid solutions, and does better than any other free solution.

Remote Procedure Calls

Remote procedure calls, shortened to RPCs, are commonly placed second in bandwidth consumption. RPCs are used to communicate between the server and client. Fish-Networking uses only two bytes per RPC, while other free solutions use a minimum of 23 bytes.

CPU

Another very important aspect is how well the networking solution scales. A better scaling solution ensures it will run on less expensive hardware, and get you closer to your MMO goals.

Network Object Count (idle)

It's not uncommon for frameworks to lose performance as more networked objects are spawned. Fish-Networking retains nearly 100% of it's performance regardless of how many objects are spawned.

Concurrent Users

Fish-Networking retains a large amount of performance with hundreds of users updating every tick. Other networking solutions have shown to slow down drastically as more concurrent users join the server.

Client Experience

With more objects, users, or activity even clients can be affected by a poorly designed networking solution. Fish-Networking has scored roughly 70% more frames per second on clients than other free solutions, such as Mirror.

Unity Compatibility

Fish-Networking's compatibility with recent Unity versions.

Unity Version	FishNet Support
Unity 6+	All features work, fully supported.
Unity 2023.2+	All features work, fully supported.
Unity 2022.3+	All features work, fully supported.
Unity 2021.3+	All features work, fully supported.
Unity 2020 and earlier	Not supported.

Legal Restrictions

There are no restrictions for personal and commercial use. Fish-Networking does not limit concurrent users. For the full license please see <https://github.com/FirstGearGames/FishNet/blob/main/LICENSE.md>.

There are no fees for using Fish-Networking free. There are optional paid features, projects, and enhanced support tiers; more information on these may be found [here](#). Any versions of Fish-Networking, free and Pro, are yours to keep.

For information on sharing Fish-Networking Pro please review the license link above.

Info: You are not required nor will ever be asked to share any profits from your work. Fish-Networking is funded through community support.

Pro, Projects, and Support

Learn how to access a variety of enhancements for your Fish-Networking experience. We aim to provide reasonable pricing ranging from smaller indie developers to large studios.

FishNet Pro

Some features are only available in Fish-Networking Pro. All business support tiers also grant a complimentary lifetime Pro license.

Tip: We have an extremely flexible sharing license for Fish-Networking Pro, allowing teams and classrooms to work on a single purchase. You can [view that license here](#).

Features for Pro:

- **Lag Compensation** — The act of rolling back colliders in time to where a client had seen them; this ensures accurate hit registration. Most commonly seen in shooter games.
- **Automatic Code Stripping** — Helps protect your game server by removing sensitive logic that the player should not be aware of. Far as we know, Fish-Networking is the only solution with the ability to remove server code from clients, and client code from the server.
- **Extrapolation, NetworkTransform** — When enabled the NetworkTransform will extrapolate as necessary to create a smoother experience for unstable connections.
- **Synchronized Parameters, NetworkAnimator** — Customize which parameters are synchronized over the network on the NetworkAnimator.
- **Yak** — A transport which provides an entirely offline experience using your multiplayer code. When combined with Multipass (available to free users), you may run your game offline or online.

Paid projects

Projects are completed Unity projects which act as a template or learning opportunity.

Lobby and Worlds

Lobby and Worlds is designed to use a single server for a lobby containing your players, as well create games on the same server using rooms created by your players. Each room is isolated from other players, both visually and in network traffic.

Many aspects of the lobby can be customized to your needs. Lobby and Worlds allows you to design your game normally in it's own scenes, while the project takes care of everything else.

Features include:

- Loading any number of game scenes, multiple times; think dungeon instances!
- Sign in system.
- Lobby to create and join rooms.
- Join rooms after start, optional.
- Password rooms.
- Ready up system, optional.
- Kick players.
- Player limits.

- Customizable lobby logic and demo game.

FPS Land

Warning: FPS Land is not yet available for Fish-Networking V4. We are in the process of rewriting FPS Land to better take advantage of newer FishNet features. Upon release, FPS Land V4 will be available to anyone that joined Pro tier from January 31st 2024 and onward, even if you are not currently a Pro subscriber.

Info: FPS Land for Fish-Networking V3 is still available for download.

This project is a full server authoritative demo on how you may get started on creating a FPS game with Fish-Networking.

Key points are:

- Full server authoritative: movement, sound, firing, reloading, animations, etc.
- Four weapon types: rifle, pistol, knife, and grenades.
- Lag compensation for weapons such as rifles.
- Lag compensation projectiles, such as grenades.
- Picked up items.
- Movement modifiers from weapon weight and walking.
- Pet friendly goats.

Support tier options

All business support tiers also grant a complimentary lifetime Pro license.

Free Tier Support

- Discord: Multiple channels of support, with support provided by our community and helpers.
- Unity Forum: support provided by our community and helpers.
- Website(s): Documentation, Guides, and Videos available from our Team and Community.

Priority Tier Support

- Everything in Free Tier.
- Discord: Additional access to post in our Priority Support channel. This channel uses threads to keep each question individualized and sorted. The questions asked in Priority Support are prioritized by our helpers.
- Projects: Access to projects that are completed Unity projects which act as a template or learning opportunity.

There are multiple priority tier options. Support quality is equal for all tiers. Priority tiers 2+ receive enhanced visibility within our Discord community to show off your support.

Business Support

- Everything in Free Tier.
- Everything in Priority Tier.
- Additional Enterprise Features — please view our [monthly tiers on GitHub](#) to see what we offer per tier. For more information you may also email firstgargames@gmail.com, or contact FirstGearGames on Discord.

Purchasing

Danger: Please read the following very carefully to ensure you understand it fully. If you have questions please [contact staff on Discord](#).

There are two types of purchases: Fish-Networking Pro, and Projects.

Fish-Networking Pro can be purchased from [GitHub](#), [Patreon](#), or the [Unity Asset Store](#).

At this time Projects can only be purchased through GitHub and Patreon. Once FPS Land V4 is complete, projects will be added to the asset store as well.

Please consider making purchases through GitHub as our preferred platform, or Patreon. When purchased through GitHub or Patreon more of your support goes towards the developers and community helpers. In addition, GitHub and Patreon offer extra bonuses as a thank you!

GitHub and Patreon perks

Info: Becoming a supporter is designed to be indie-dev and budget friendly.

When obtaining Fish-Networking Pro through GitHub or Patreon you will have access to both Pro and Projects.

You must make a one-time payment of \$10 for your initial downloads. Afterwards you're welcome to switch to \$2/month for updates. You may leave the updates tier and join us again at any time.

When on any \$10 tier or higher you will gain access to Priority Support, this includes your initial purchase; see support tier options for more information on this.

Info: Business support tiers are only available on GitHub.

Lifetime benefits:

Tip: You can get bonus Priority Support months on GitHub! Our one-time \$60 GitHub option provides lifetime access to Pro and Project downloads, as well 8 total months of Priority Support, rather than 6 months of Priority Support when purchased monthly.

- Lifetime updates to both Pro and Projects can be obtained by supporting over-time, or by a one-time payment.
- Lifetime updates to projects are received after your total contributing meets \$35.
- Lifetime updates to Fish-Networking Pro are received after your total contribution meets \$60.
- Should you sponsor \$200 at once or over-time, you will also gain lifetime access to Priority Support.

Info: All higher tiers provide access to any content beneath them. For example: If you have access to Fish-Networking Pro, you also have access to Projects.

Unity Asset Store perks

Asset store purchases provide lifetime updates to Fish-Networking Pro.

You can also gain access to Priority Support with your asset store purchase! Your priority support months begin only after you claim your perks on our Discord; see After Purchasing please.

Info: The amount of months received are directly related to what goes back into the project after Unity's fees. If you purchase Fish-Networking Pro at \$50 you will receive 4 months of Priority Support, and 2 months at \$25.

After purchasing

Danger: Please be certain you read Purchasing thoroughly.

If you purchased through GitHub or Patreon your downloads will be available on our [website](#).

If you purchased Fish-Networking Pro on the asset store, please use the asset store to download Fish-Networking Pro.

Info: Keep in mind that you may change your support tier at anytime when joining us through GitHub or Patreon.

Do not forget your Discord roles!

You can claim supporter roles on our Discord no matter what purchase technique you use, but as previously said GitHub offers the most benefits per dollar. Roles show off that you're a supporter, and are crucial for gaining you access to our priority help channels if you're on a Priority Support tier.

Business tiers are unique in that they receive dedicated channels. Please contact FirstGearGames on Discord to have your support channel created.

How to claim your role:

- **GitHub Instructions:** log into your downloads on our website. After logging in you will be shown a temporary Id above downloads. Copy this Id and direct message our Discord bot Sharky with it.
- **Patreon Instructions:** <https://support.patreon.com/hc/en-us/articles/212052266-Getting-Discord-access>
- **Asset Store Instructions:** Join our Discord and message Sharky your purchase invoice number.

More information

For more detailed instructions of purchasing and installing FishNet Pro, checkout this guide: [Upgrading to FishNet Pro](#)

What Is FishNet?

Overview

Fish-Networking (FishNet) is an original free versatile networking solution for Unity, built from the ground up, offering more features than any other free solution.

FishNet is server authoritative by design by allowing the use of dedicated servers, but does permit users to act as server and client, for faster development and testing. **There are no CCU caps or CCU paywalls in FishNet** — you can also use any server host you want to, including hosting the server yourself or letting your players do it.

Any kind of network topology is supported through the Transport system. Transports can use a variety of technologies to allow communications between server, client, and even third parties.

High-level API allows you to quickly access the ability to synchronize states, logic, objects, and more, without needing to get your hands dirty. Low-level functionality is also available via included events or inheritance.

No-break promise

Developing projects can take a lot of time, and updating your networking solution along the way is often inevitable. Fish-Networking promises to not release any breaking API or behavior changes between major versions. Major releases will occur no more frequent than every six months, unless **absolutely necessary**.

When breaks do occur we will do our best to keep the changes simple. We also have our Break Solutions section which will describe planned breaks, the next major release, and how to remedy breaks for each version.

Long-Term support

Fish-Networking is the only solution to offer free LTS. We will be using a unique but effective approach at creating LTS releases. Rather than the standard expectations of being locked into a version for long-term support, FishNet is providing what we refer to as 'Release' and 'Development' switches. Any version of Fish-Networking which ends in R supports switching between Release and Development features — for example: 3.10.7R.

With Fish-Networking, Long-Term support does not mean being stuck on older versions! Whenever a change or new feature becomes available you may disable it at any time. Disabling an upcoming change will suspend the changes and allow FishNet to operate on the proven stable version of the same feature. This allows you to stay on the latest releases to get the latest tech and bug fixes without worrying about each update breaking your project.

Tip: To toggle between beta features simply use the Fish-Networking menu in engine, choose Beta, and turn on or off each feature to your liking.

External links

[Fish-Networking GitHub](#)

[Asset Store](#)

[Community Discord](#)

Community made resources

There are many third-party add-ons, integrations, and resources that can aid you through your development. Many assets also have direct support for FishNet and work well together with it.

[Asset Integrations](#) [Community Resources](#)

Contributing documentation

We are now taking documentation contributions! If you'd like to include your own guide, or make edits to the current documentation, you can do so on the documentation [GitHub](#).

Moving to Fish-Networking

Helpful tips for how to move your project from another solution to FishNet.

Moving to Fish-Networking will vary depending on which previous networking solution you are coming from. With other solutions changing their API so regularly it may be difficult to keep transition information up to date but we will try our best.

If you encounter any questions we welcome you on our Discord, where our helpful community can get you started.

Leaving Mirror Networking

Always backup before making large changes. Having another copy of your project to reference during the upgrade is a great idea!

The majority of these changes can be done by using quick replace within your IDE!

Upgrade tool

An upgrade tool is provided to help transition from Mirror to Fish-Net. The tool will replace several Mirror components with the like ones for Fish-Net. Note that while the components will be replaced the settings of your components will not be preserved. For best results keep a copy of your original project for reference until switching over.

Currently supported components are: `NetworkIdentity`, `NetworkProximityChecker`, `NetworkTransform/Child`, `NetworkAnimator`, `NetworkSceneChecker`.

Additional components from FirstGearGames Mirror Assets/Projects are supported:

`FlexNetworkTransform/Child`, `FastProximityChecker`, `FlexNetworkAnimator`, `FlexSceneChecker`.

The upgrade tool requires scripting defines to function. These defines should be removed after running the upgrade tool. - `MIRROR` — for Mirror components. - `FGG_ASSETS` — for FirstGearGames assets. - `FGG_PROJECTS` — for FirstGearGames projects.

Before running the upgrade tool you will likely need to remove a few files. If using FirstGearGames Mirror assets or projects, delete any of the demo folders included for those assets and projects.

In addition, any scripts requiring component `NetworkIdentity` will need to have the declaring line removed.

```
// Replace the following with an empty string.  
[RequireComponent(typeof(NetworkIdentity))]  
// The hotkey in Visual Studio is Ctrl + Shift + H.
```

Component changes

Mirror	Fish-Networking	Notes
Interest Management	ObserverManager (see also: NetworkObserver)	Both support multiple conditions
NetworkIdentity	NetworkObject	

Changing out components can be a tedious task. Save time by using **Window -> Fish-Networking -> Upgrading -> From Mirror -> Replace Components**. This will swap out most components for you. Mirror and Fish-Networking must be within your project to run the operation mentioned. Afterwards you

may remove Mirror.

NetworkBehaviour property renames

Each `IsServer`, `IsClient`, etc option has two varieties: `Initialized` and `Started`. **Initialized** means the object has been marked as ready for network use by the side (e.g.: client or server). **Started** means the client or server socket is connected, but does not mean the object has been initialized for network use yet. When converting from Mirror you will most likely want to use `Initialized`.

Mirror	Fish-Networking	Notes
<code>`hasAuthority`</code>	<code>`IsOwner`</code>	
<code>`isLocalPlayer`</code>	removed, use <code>`IsOwner`</code> instead	
<code>`isClient`</code>	<code>`IsClientInitialized`</code>	
<code>`isClientOnly`</code>	<code>`IsClientOnlyInitialized`</code>	
<code>`isServer`</code>	<code>`IsServerInitialized`</code>	
<code>`isServerOnly`</code>	<code>`IsServerOnlyInitialized`</code>	
<code>`connectionToClient`</code>	<code>`Owner`</code>	Can be used on client
<code>`connectionToServer`</code>	<code>`LocalConnection`</code>	
<code>`netId`</code>	<code>`ObjectId`</code>	

NetworkBehaviour callbacks

Callbacks such as `OnStartClient` for the most part are the same in Fish-Networking. Some of the callbacks do provide additional information which is not available within Mirror's callbacks. There are also several new callbacks to use. For a complete guide on using Fish-Networking callbacks see the Network Behaviour Guides page.

Remote Procedure Call renames

See Remote Procedure Calls for enhanced features and usage related to RPCs.

Mirror	Fish-Networking	Notes
<code>`[Command]`</code>	<code>`[ServerRpc]`</code>	
<code>`[TargetRpc]`</code>	<code>`[TargetRpc]`</code>	First parameter must be a <code>`NetworkConnection`</code>
<code>`[ClientRpc]`</code>	<code>`[ObserversRpc]`</code>	

RPCs within Fish-Networking have many advantages over Mirror; see Remote Procedure Calls for more information.

Manager renames

Mirror uses several singletons while Fish-Networking uses an anti-singleton design. There are still easy ways to access similar features. All of the same access can be returned from the `NetworkObject` component, as well as by using `InstanceFinder`.

All of these may also be found within the `NetworkManager`.

Mirror	Fish-Networking	Notes
--------	-----------------	-------

<code>`NetworkManager`</code>	<code>`NetworkManager`</code>	Unchanged
<code>`NetworkServer`</code>	<code>`ServerManager`</code>	
<code>`NetworkClient`</code>	<code>`ClientManager`</code>	
	<code>`TransportManager`</code>	Fish-Networking only
	<code>`TimeManager`</code>	Fish-Networking only
	<code>`SceneManager`</code>	Fish-Networking only

Switching from Mirror.NetworkManager

The NetworkManager in Mirror features several overrides for state changes, such as when clients connect, disconnect, server start or stop, scene changes, and more.

Fish-Networking instead uses sealed managers with events. To mimic the NetworkManager of Mirror create a new MonoBehaviour and place it on your NetworkManager object. Instead of using overrides subscribe to your needed events from each appropriate Fish-Networking manager.

```
// Mirror example.
// Called when a remote client connects.
public override OnServerConnect(NetworkConnectionToClient conn) {}

// FishNet example.
// Called when a remote client state changes.
networkManager.ServerManager.OnRemoteConnectionState += YourCallback;
```

NetworkManager callback equivalents

Mirror	Fish-Networking
<code>`ServerChangeScene`</code> / <code>`OnServerChangeScene`</code> / <code>`OnServerSceneChanged`</code>	<code>`SceneManager.OnLoadEnd`</code>
<code>`ClientChangeScene`</code> / <code>`OnClientChangeScene`</code> / <code>`OnClientSceneChanged`</code>	<code>`SceneManager.OnClientPresenceChangeEnd`</code> — See SceneManager guide for more information
<code>`OnServerReady`</code>	<code>`SceneManager.OnClientLoadedStartScenes`</code>
<code>`OnServerConnect`</code> / <code>`OnServerDisconnect`</code>	<code>`ServerManager.OnRemoteConnectionState`</code>
<code>`OnServerAddPlayer`</code> / <code>`OnClientNotReady`</code>	
<code>`OnStartHost`</code> / <code>`OnStopHost`</code>	Not needed
<code>`OnStartServer`</code> / <code>`OnStopServer`</code> / <code>`OnServerError`</code>	<code>`ServerManager.OnServerConnectionState`</code>
<code>`OnStartClient`</code> / <code>`OnStopClient`</code> / <code>`OnClientConnect`</code> / <code>`OnClientDisconnect`</code> / <code>`OnClientError`</code>	<code>`ClientManager.OnClientConnectionState`</code>

Automatic online/offline scenes

To mimic the Online and Offline scene setting in the Mirror NetworkManager you can add the DefaultScene component to your network manager object and fill it in with your desired scenes.

Prefabs

FishNet has some helpful prefabs that can make development and testing quicker. They are located inside the `/FishNet/Demos/Prefabs` directory and can be dragged directly into your scene to be made use of.

NetworkHudCanvas

The NetworkHudCanvas prefab can be used to show a simple Heads-Up Display in your game allowing you to quickly start and stop the FishNet server and client.

It also allows you to set an **Auto Start Type**, which will dictate how FishNet behaves as soon as the object is loaded in the game.

<figure><figcaption><p>Setting the Auto Start Type to

Host</p></figcaption></figure>

{% hint style="info" %} This object is most commonly used for debugging and development. {% endhint %}

NetworkManager

The NetworkManager prefab contains a basic [NetworkManager](#) with a [NetworkHudCanvas](#) to provide you with a quick setup for using FishNet in your game.

You can simply add this object into your scene and use the **Server** and **Client** buttons in play mode to start FishNet as a server, client, or both. You can also setup the [autostart options](#) ^[^1] on the NetworkHudCanvas child object.

The components this object contains are the NetworkManager, [ObserverManager](#) (with a [Scene Observer Condition](#)), and the PlayerSpawner.

^[^1]: Autostart options allow FishNet to begin as a client, server, both, or none as soon as the object is loaded in the game.

ScriptableObjects

Fish-Networking includes quite a few ScriptableObjects that you can use to customize the behavior of specific features. There are default ones included with FishNet, but you can also create your own from the ***Assets ? Create ? FishNet*** toolbar menu.

<figure><figcaption><p>Create FishNet asset
menu</p></figcaption></figure>

LevelLoggingConfiguration

Description

The **Level Logging Configuration** scriptable object simply allows you to customize Fish-Networking's logging. Once created, it can be selected on the NetworkManager component. There are four logging levels available for the various build and editor types. **Off** will permit no logs, **Error** will only permit direct error logs, **Warning** will allow error and warning logs, and **Common** will allow all logging, including errors, warnings, and informational logs.

Settings

<figure><figcaption><p>Default settings</p></figcaption></figure>

:gear: Is Enabled

This is simply a toggle to enable and disable all of Fish-Networking's logs.

:gear: Add Local Tick

This option allows you to prepend the network's local tick when log was made onto the log message. This can be quite handy for precise debugging.

:gear: Add Timestamps

This allows you to prepend to all logs the local date and time when the log was made.

:gear: Enable Timestamps In Editor

Since Unity's console already includes the time that logs are sent at, it can be handy to hide FishNet's Timestamps in the editor and only display them in builds.

:gear: Development Logging

This controls the level of logs sent while the game is a development build or editor.

:gear: GUI Logging

This controls the level of logging to use when the game is built regularly, as opposed to a headless (non GUI) build.

:gear: Headless Logging

This controls the logging level for headless game builds.

DistanceCondition

Description

The **Distance Condition** ensures that an object is only observed if it is at least a specified minimum distance away from all objects owned by a client. The Distance Condition is a Timed Condition^[^1].

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

:gear: Maximum Distance

The maximum distance that any object owned by a client must be within an object to observe it. If all of the client's owned NetworkObjects are farther than this distance, the object will not be visible to them.

:gear: Hide Distance Percent

This determines an additional buffer zone beyond the maximum distance before an object is hidden. The purpose of the "Hide Distance Percent" is to prevent objects from flickering in and out of visibility when a client is near the maximum distance threshold. It creates a smoother transition for hiding objects by adding a percentage-based distance buffer. As an example, if the **Maximum Distance** is set to 100f and the **Hide Distance Percent** is 0.5 (50%), the object will remain visible until the client is 150f units away. This ensures objects do not disappear immediately as they cross the initial visibility boundary.

[^1]: A Timed Condition is evaluated periodically rather than only when certain conditions change.

GridCondition

Description

The **Grid Condition** is similar to the [DistanceCondition](#) and evaluates to true if clients are within specified distance of the object. The Grid Condition is however less accurate but more performant; it is also a [Timed Condition](#)^[^1]. This condition requires you to place a [HashGrid](#) on or beneath your [NetworkManager](#) object.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

[^1]: A Timed Condition is evaluated periodically rather than only when certain conditions change.

HostOnlyCondition

Description

The **Host Only Condition** evaluates to true when the player is a client host. Any connection which is not client host will fail this evaluation. You can use this observer condition to only show objects to the host player, either permanently or temporarily. This condition is not a timed condition.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

ObserverConditions

ObserverConditions in Fish-Networking define specific conditions under which clients receive updates about a NetworkObject's state. These conditions help optimize network traffic by ensuring that only relevant updates are transmitted, as well as prevent client's from cheating by only sending relevant data to individual clients.

MatchCondition

Description

The **Match Condition** evaluates to true when players or objects share the same match^[^1]. Both owned and non-owned objects can be added to matches. Objects or players not added to matches will have their data synchronized with everyone, unless prevented by another condition. See the [MatchCondition api](#) for more information on usage. This condition is not a timed condition.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left" data-full-width="false"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

[^1]: A *match* can mean a lot of different things, but in this case it is basically a custom tag you can give objects and players to separate them from observing one another.

OwnerOnlyCondition

Description

The **Owner Only Condition** evaluates to true when a player owns the object. An owner condition will make an object only visible to its owner. If there is no owner, the object will not be visible to any clients. This can be very useful for sharing information with only the owner client, for example inventory data or a player's hand of cards, which shouldn't be shared with other clients to prevent cheating and wasted bandwidth.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

SceneCondition

Description

The **Scene Condition** evaluates to true if the client shares any scenes with the object. This is arguably the most important observer condition, and it is the only one that is added by default to the [example NetworkManager prefab](#). This condition is important because it helps avoid serious de-syncs that would happen if the server tried to tell clients about objects from a scene they haven't yet loaded.

```
<div align="left"><figure><figcaption></figcaption></figure></div>
```

Settings

```
<div align="left"><figure><figcaption><p>Default settings</p></figcaption></figure></div>
```

:gear: Add Order

This controls the order in which this observer condition will be evaluated on an object. This can be very useful when having observer conditions that are more computationally complex than others, as it allows you to choose the order in which they will be evaluated. Timed conditions are always evaluated after non-timed conditions.

:gear: Is Constant

Is used to declare whether the condition's settings or data will remain unchanged at runtime. Its purpose is to optimize performance by avoiding unnecessary updates or recalculations for conditions that do not change during execution. It is currently not implemented, but is available for future use and can already be set.

DefaultPrefabObjects

The **Default Prefab Objects** asset is a dynamically populated collection of spawnable network objects. FishNet automatically generates it in the designated directory specified in your Fish-Networking configuration, filling it with networked prefabs from the configured locations.

{% hint style="info" %} You do not need to manually populate or update this collection, FishNet will take care of it automatically. {% endhint %}

<figure><figcaption><p>Example of DefaultPrefabObjects being populated by Networked Prefabs</p></figcaption></figure>

DualPrefabObjects

The Dual Prefab Objects asset can be used to have Fish-Networking spawn a different prefab on the clients as it uses on the server. Whenever you spawn the server selected version, FishNet will spawn the client version on all clients.

{% hint style="success" %} You can select the same prefab in both the Server and Client fields to have the same one spawned on all clients as the server. {% endhint %}

<figure>

alt="" data-bbox="84 252 755 287"/>

<figcaption><p>Example of a Dual Prefab Objects asset being populated</p></figcaption></figure>

SpawnablePrefabs

Fish-Networking uses a collection of prefabs to manage spawning of network objects across the network. Whenever you instantiate and spawn an object on the server, FishNet will use the selected **Spawnable Prefabs** collection to find which object it is and to automatically instantiate and link the objects on all clients.

The **Spawnable Prefabs** collection can be set on your NetworkManager component.

{% hint style="success" %} The NetworkManager will automatically set the **Spawnable Prefabs** to an automatically managed DefaultPrefabObjects asset if a custom one is not selected. {% endhint %}

<figure><figcaption><p>The

Spawnable Prefabs field on the NetworkManager</p></figcaption></figure>

SinglePrefabObjects

The **Single Prefab Objects** is a scriptable object which you can inherit from for custom functionality. It is used by the DefaultPrefabObjects collection and can also be directly used for your own collections. If using it directly, remember to manually add all your spawnable network object prefabs into its collection.

<figure><figcaption><p>A single prefab
objects asset being manually populated</p></figcaption></figure>

Server Hosting

Hosting is a way for players to access your game and play together.

What does server hosting entail?

Server hosting involves renting or utilizing a server to host your game's files and services. A server host is essential for game deployment because it provides the infrastructure and resources necessary to manage server loads, enable online gameplay, and ensure the game is accessible to players around the world. It also facilitates updates, balances traffic, and maintains security, which are crucial for a smooth gaming experience.

FishNet supports any server hosting you choose, and you can also manage it completely yourself, including letting your players host servers themselves or act as a host.

[book-open | Terminology | /docs/v4/server-hosting/terminology] Overview of key server hosting terms, comparing session-based vs persistent architectures, and outlining the roles of relays and dedicated servers. [users | Services | /docs/v4/server-hosting/services] A comparison of third-party hosting and relay services compatible with FishNet, including our recommended providers.

Getting Started with AWS

A community-made guide for using AWS to host a FishNet server.

This tutorial was kindly provided by haseebzahid413#2226.

Verify Permissions for AWS Account

Go to AWS Management Console and launch a Virtual Machine.

Select any free-tier Linux machine and any free-tier instance type.

Edit Security Groups and add two rules:

- **Custom TCP Rule** — port 7770 (or your chosen port), Source set to Anywhere
- **UDP Rule** — port 7770 (or your chosen port), Source set to Anywhere

Press **Review and Launch**.

Create a new Key Pair, give it a name, download the keypair, then press **Launch Instance**.

Setting Up Putty and WinSCP

Download Putty and WinSCP.

Open PuttyGen: go to **Conversions > Import Key**, select the downloaded key, set the correct Key Type, and press **Save Private Key**. This produces a .ppk file used in the steps below.

Open Putty. Copy the IPv4 DNS address for your EC2 instance from the AWS console.

In Putty: go to **Configuration > Session > Host Name** and enter `ec2-user@<your-dns-address>`. Keep port 22 and Connection Type SSH.

Go to **Connection > SSH > Auth** and browse for the .ppk file generated above.

Go back to Session, give it a name, press **Save**, then double-click the session name to open a terminal on the Linux machine.

Open WinSCP. When prompted, choose the single window with two panels option.

In **Login > New Site**: set File Protocol to SCP, paste the same hostname, set Username to `ec2-user`, then under **Advanced > SSH > Authentication** provide the private key file.

Press **Ok**, then **Save** with a name, and double-click the login to connect to the Linux server. This window is used to transfer, replace, or delete builds on the server.

Configuring Fish-Networking

Add the **TugBoat** component on the Network Manager.

Set the IPv4 server bind address to `127.0.0.1` (or leave empty).

Set the port to 7770 (or your chosen port — must match the EC2 security group rule).

Set the client address to the EC2 instance's public IPv4 DNS address.

Save the scene.

Building for Linux

In Build Settings, set the target platform to **Linux**, architecture **x86_64**, and enable **Server Build** (headless).

Ensure the server scene is added and enabled in Build Settings.

Build and save to a folder. Wait for the build to complete.

Deploying

In WinSCP, navigate the left panel to your Unity server build folder.

Right-click and upload the build to the Linux server. This may take a moment.

Once the upload is complete, go to the Putty terminal and navigate into the build folder.

Run the following commands:

```
chmod +x "YourExecutable.x86_64"  
./"YourExecutable.x86_64"
```

The server is now up and running.

Connecting a Client

Open your Unity project.

Press **Client** — it should connect to the server successfully.

Services



We strongly recommend reviewing our [Server Hosting](#) page for terminology as well the Pros/Cons of services mentioned on this page. This information is updated to the best of our knowledge. If you are interested in a service listed below — to better understand their platform — please visit the provided links to their official website.

Our recommended service

PlayFlow Cloud, a company we've had the pleasure of working with firsthand, is recommended as our top-pick for a hosting service. Their outstanding support, reliable service, and superior pricing have consistently impressed our staff and community.

PlayFlow Cloud

?????

- ? Generous free tier with better pricing than competitors.
- ? Lobby & Matchmaking, with an included SDK handling the work.
- ? Worldwide dedicated and instanced servers.
- ?? User-friendly control panel, metrics, support, and more.

Other services

Here is a non-extensive selection of other hosting services that work well with FishNet.

Dedicated servers

These hosts offer session-based servers, as well as persistent servers. Relay services may be available as well.

Hathora

- Pay-per-use pricing.
- Session-based servers available.
- Persistent servers available.
- Relay servers available.
- Match Making / Lobbies.

Edgegap

- Pay-per-use pricing.
- Session-based servers available.

- Persistent servers available.
- Relay servers available.
- Match Making / Lobbies.

AWS (GameLift)

AWS is a bit more difficult to setup and manage. We generally only recommend AWS to experienced developers.

- Pay-per-use pricing — a little more complex.
- Session-based servers available.
- Persistent servers available.
- Relay servers available.
- Match Making / Lobbies.

We have a community guide for AWS: [Community Guide](#)

Relays only

These are relay-only services.

Steam

Steam relays are offered free when you launch your game on Steam; there is a percentage of sales, as well as an initial fee when using Steam.

We have an official FishNet transport for Steam, and there are third-party Unity assets available as well. A popular asset is [Toolkit for Steamworks Foundation by Heathen Engineering](#).

- Free for games launched using Steam.
- Session-based relays.
- Match Making / Lobbies.
- Friends list (Steam).

Epic Online Services (EOS)

EOS provides a number of features for free. The API and documentation is a bit more complex, which may be difficult for beginners. A vetted third-party [EOS transport](#) is available for FishNet, which may help with some of those hurdles.

A cross-platform friends list is possible with EOS but at the cost of complexity; e.g.: you will not be able to directly access your Steam friends via the EOS platform.

- Free to all developers.
- Session-based relays.
- *Match Making / Lobbies.
- *Friends list.

Easy WebGL Hosting with PlayFlow

How PlayFlow makes WebGL hosting easy

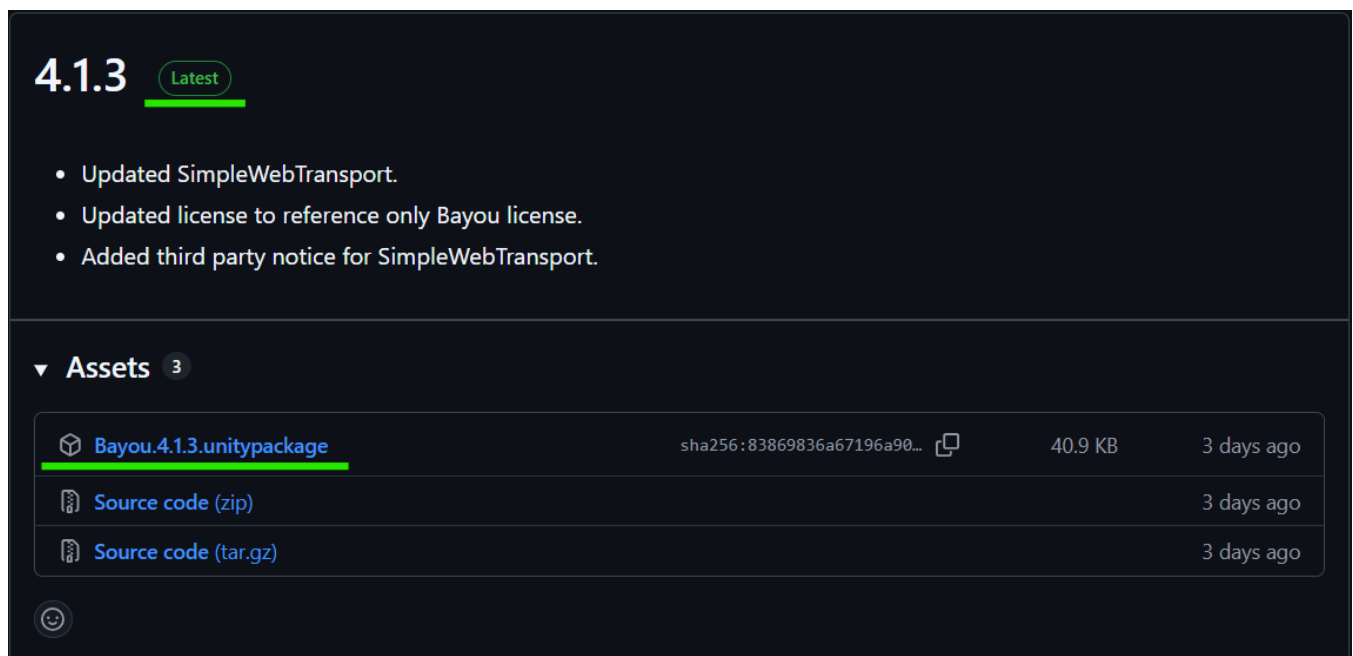
For seamless WebGL gaming experiences in web browsers, secure WebSocket connections are crucial, but configuring them can be complex. Often, developers end up managing certificates and networking issues, which can be time-consuming. **With PlayFlow**, however, you only need to enable a simple option to handle these concerns effortlessly.

Before we begin...

This tutorial assumes you've read the [Getting Started with PlayFlow](#) tutorial already. In the following steps we will convert our project to support WebGL.

1. Install Bayou

For FishNet connections on WebGL we will need to use a web supported transport. This tutorial will be using [Bayou](#). Head over to the [Bayou GitHub repo](#) and download the most recently released .unitypackage file, then import it into your project.

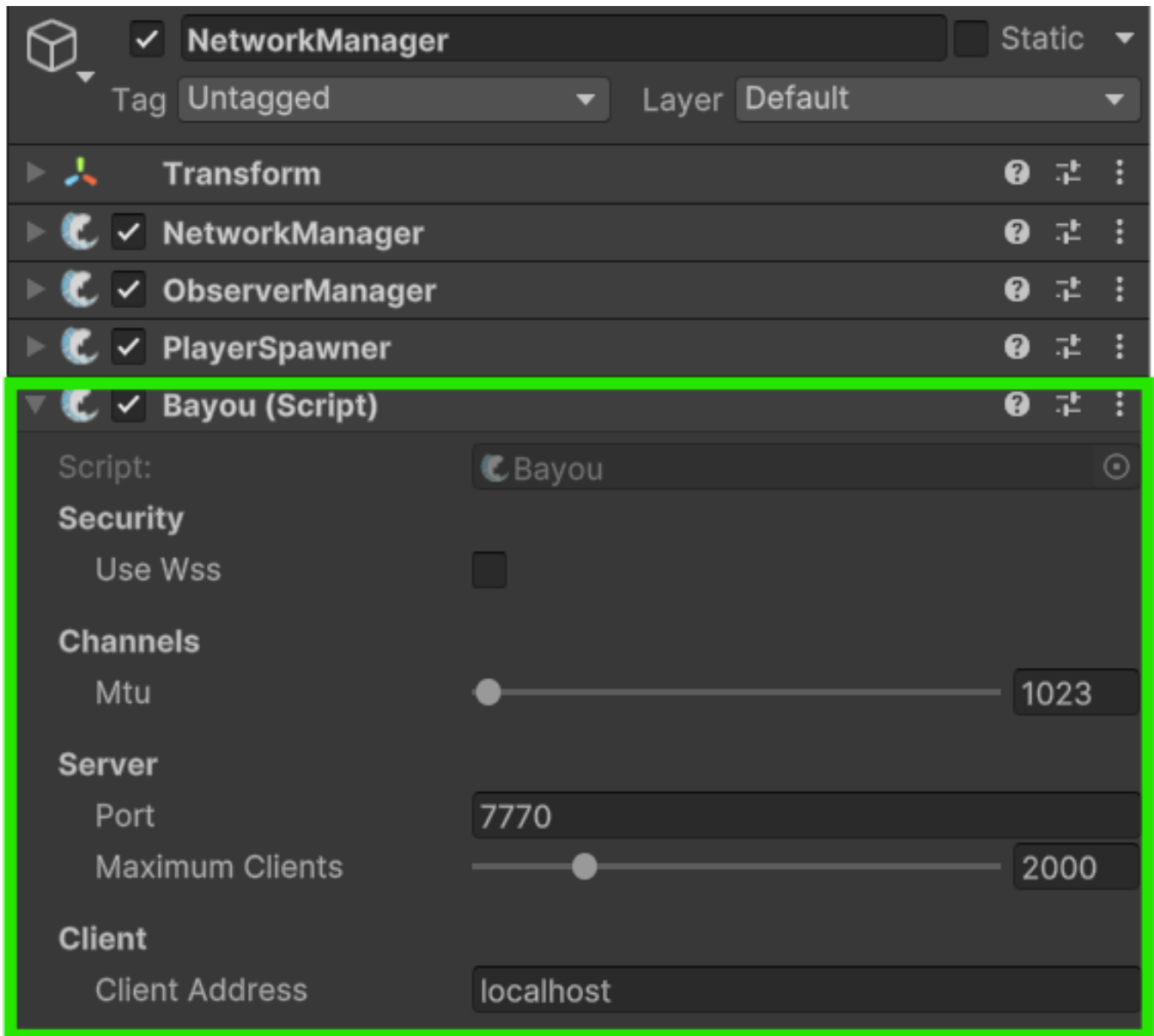


The screenshot shows the Unity Package Manager interface. At the top, version 4.1.3 is highlighted as the 'Latest' version. Below this, a list of updates is shown: 'Updated SimpleWebTransport', 'Updated license to reference only Bayou license', and 'Added third party notice for SimpleWebTransport'. Under the 'Assets' section, three items are listed: 'Bayou.4.1.3.unitypackage' (40.9 KB, 3 days ago), 'Source code (zip)' (3 days ago), and 'Source code (tar.gz)' (3 days ago). The 'Bayou.4.1.3.unitypackage' item is highlighted with a green underline.

Asset Name	SHA256 Hash	Size	Time
Bayou.4.1.3.unitypackage	sha256:83869836a67196a90...	40.9 KB	3 days ago
Source code (zip)			3 days ago
Source code (tar.gz)			3 days ago

2. Add Bayou to your Network Manager

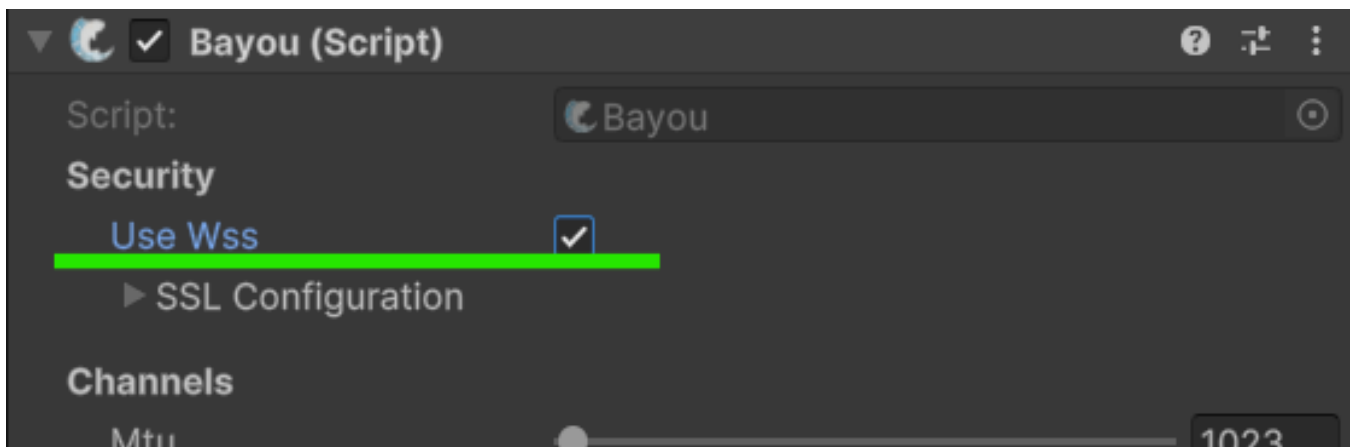
Now that **Bayou** is imported, select your NetworkManager and add the component to it.



[!INFO] If you have more than one transport on your NetworkManager, such as Tugboat, then you will need to add the TransportManager component and drag Bayou into its **Transport** field to set it as the active one.

3. Enable WSS

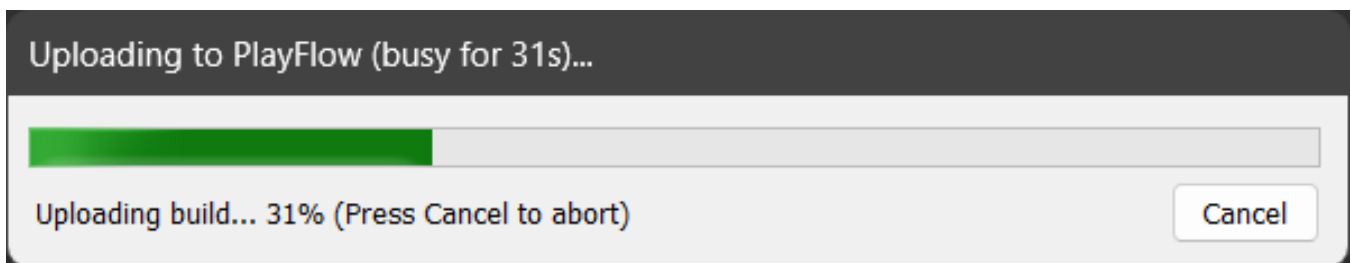
In the Bayou component, enable the **Use WSS** option. This will tell Bayou to use Web Socket Secure. The SSL Configuration options below do not need to be changed for this setup.



4. Build your PlayFlow server

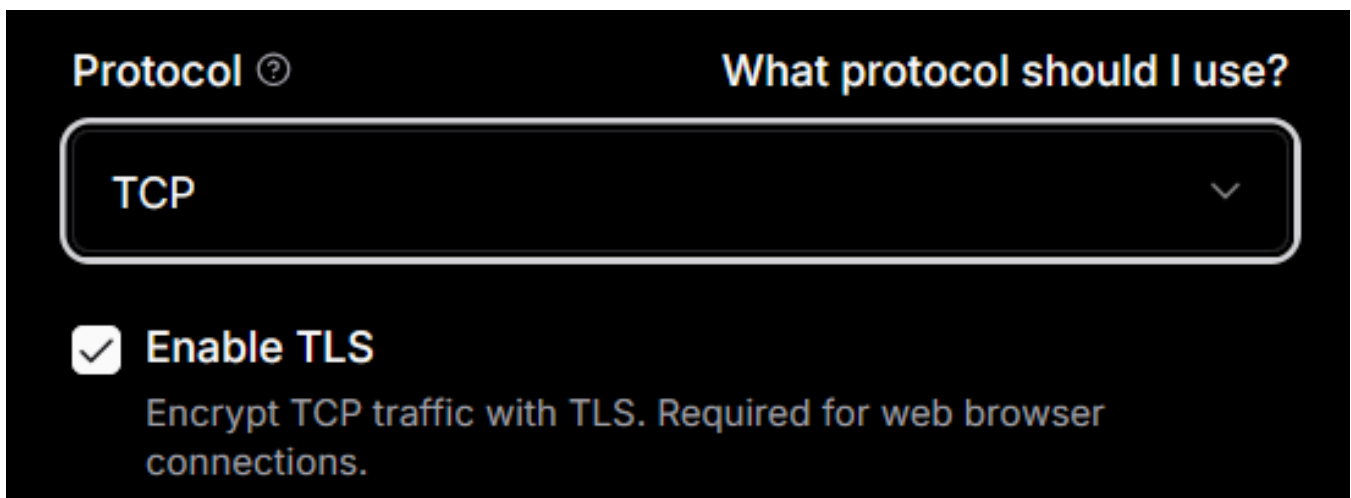
Open the PlayFlow Cloud Window from the Unity toolbar (**PlayFlow ? PlayFlow Cloud**).

Click the **Upload Server** button to build and upload your Unity server.



5. Set the correct PlayFlow port protocol

Open the [PlayFlow dashboard](#) and navigate to the **Configuration** tab. Edit or add your port, choose the **TCP** protocol, and be sure to **Enable TLS**.



6. Start a PlayFlow server

Head over to the **Server** tab in PlayFlow and create your server instance.

Create New Server

×

Configure and launch a new game server instance. Ensure you have uploaded necessary builds.

Name

WebGL Tutorial

Region

North America (US East) ▾

Size

Micro ((1 vCPU, 512MB RAM)) ▾

Free plan is limited to Micro and Small servers.

⚡ Upgrade to Pro for more powerful servers

Build

web-gl ▾

v1 (Latest) ▾

TTL (secs)

Optional: 60-86400 ▾

Auto-stop server after this duration. Leave blank for no TTL.

Auto-restart

☐ Enable auto-restart on failure

If enabled, the server will automatically restart if it crashes.

Startup Args

Optional: -port=7777 -map=MyMap

Cancel

Create Server

7. Prepare your client build

Click on your server instance's **Details** to see the **Host** and **External Port**. Open Unity, select the Bayou

component, and enter these into the **Client Address** and **Port** fields respectively.



WebGL Tutorial
34bb9800-255f-4697-8a73-dc8ede3b4c25

Launching

Details

Metrics

Logs

Configuration

Compute Size

small

Version

web-gl

Region

us-east

Auto-restart

Disabled

Started At

Aug 20, 2025 13:36:20

Time To Live

1h 0m

Network

1 port

webgl-bayou

TCP

TLS Enabled

Host

connect.computeflow.cloud

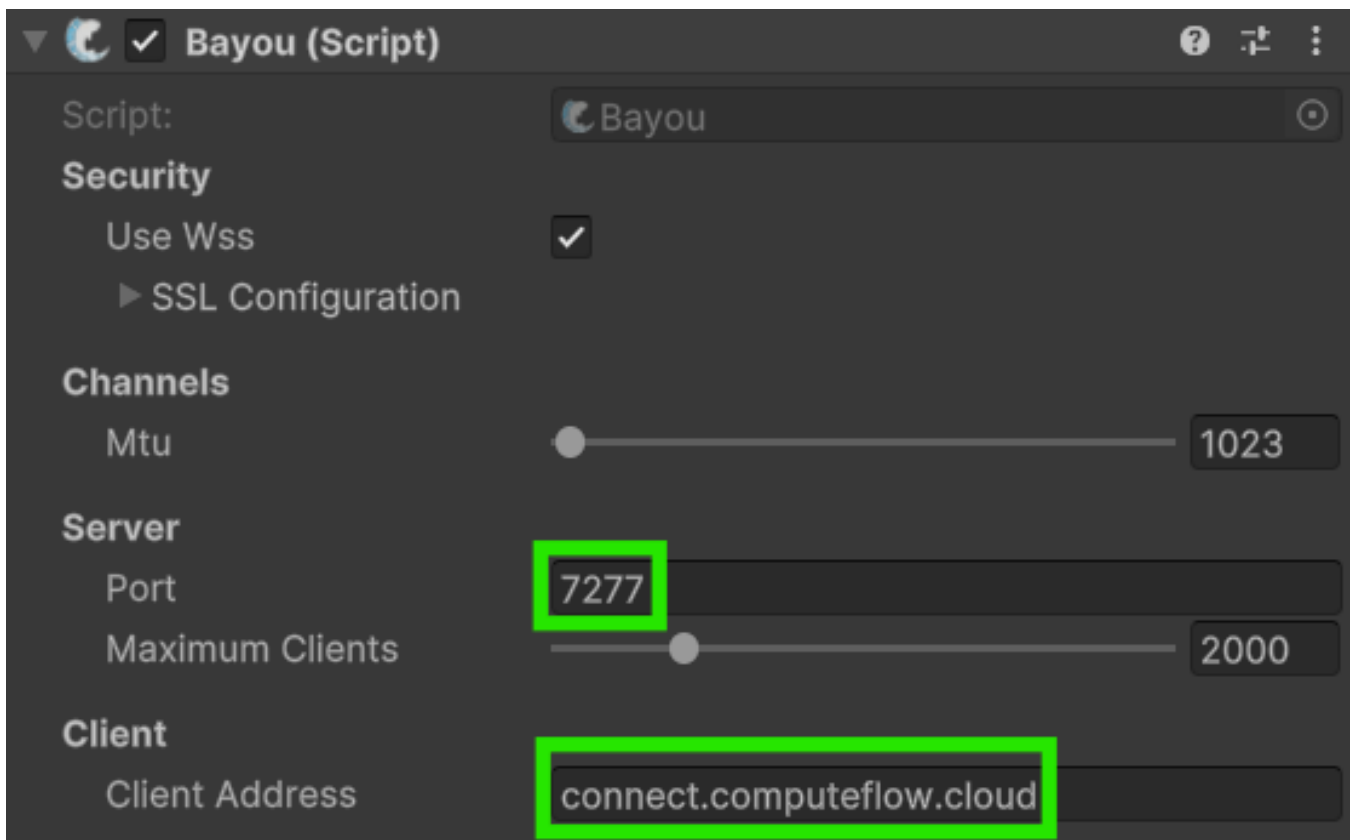
External Port

7277

Connection String

connect.computeflow.cloud:7277

Internal Port: 7770



8. Build your WebGL client

In your Unity build profiles, switch to the **Web Platform** and build your game. Once done, zip the built files and upload to your chosen hosting site.

Name	Date modified	Type	Size
Build	2025-08-20 13:45	File folder	
TemplateData	2025-07-21 13:55	File folder	
index.html	2025-07-21 13:55	Firefox HTML Doc...	6 KB
WebGL.zip	2025-08-20 13:47	Compressed (zipp...	14,213 KB

For this test we created a project on Itch.io and uploaded the client build to it.

Uploads

Upload a ZIP file containing your game. There must be an `index.html` file in the ZIP. Or upload a `.html` file that contains your entire game. [Learn more](#) →

Any additional files you upload will be made available for download. You can apply a minimum price to the project after uploading additional downloadable files.

WebGL.zip · Success

[More...](#)

[Delete file](#)

14mb · [Change display name](#)

Today at 1:50 PM

☒ This file will be played in the browser

[Upload files](#)

or

 [Choose from Dropbox](#)

[Add External file](#) ?

File size limit: 1 GB. [Contact us](#) if you need more space

TIP Use *butler* to upload files: it only uploads what's changed, generates patches for the [itch.io app](#), and you can automate it. [Get started!](#)

Embed options

How should your project be run in your page?

Embed in page ▼

Auto-detect size (Unity HTML only) ▼

Success!

Just like that, we were able to create a server and have our clients in the web browser securely connect to it. If you want to check out more of their features, visit their documentation at <https://documentation.playflowcloud.com/>

You are also encouraged to join their [Discord server](#).

Getting Started with PlayFlow

{% embed url="https://www.youtube.com/watch?v=TyvBHUIYP64" %}

This guide assumes you have some or all of your game already written using FishNet and are now ready to use PlayFlow to implement a dedicated server. You can also use PlayFlow throughout the development process as its one-click deployment makes it easy to setup a temporary server for testing.

Before we begin...

1. Starting project

If you don't have a project already setup but want to follow this tutorial, you can use the Getting Started FishNet tutorial project. The files for it can be downloaded from the [FishNet Getting Started GitHub repository](#).

2. Install the PlayFlow SDK

Now that we're ready to start, let's install [PlayFlow Cloud](#). You can directly import it with the following git URL:

```
https://github.com/PlayFlowCloud/PlayFlow-Multiplayer-Unity-SDK.git
```

Simply add this through the Unity Package Manager and you'll be ready to continue!

[!INFO] If you need more detailed instructions for this step, check out PlayFlow's own [Installation Guide](#).

3. Add the necessary Unity modules

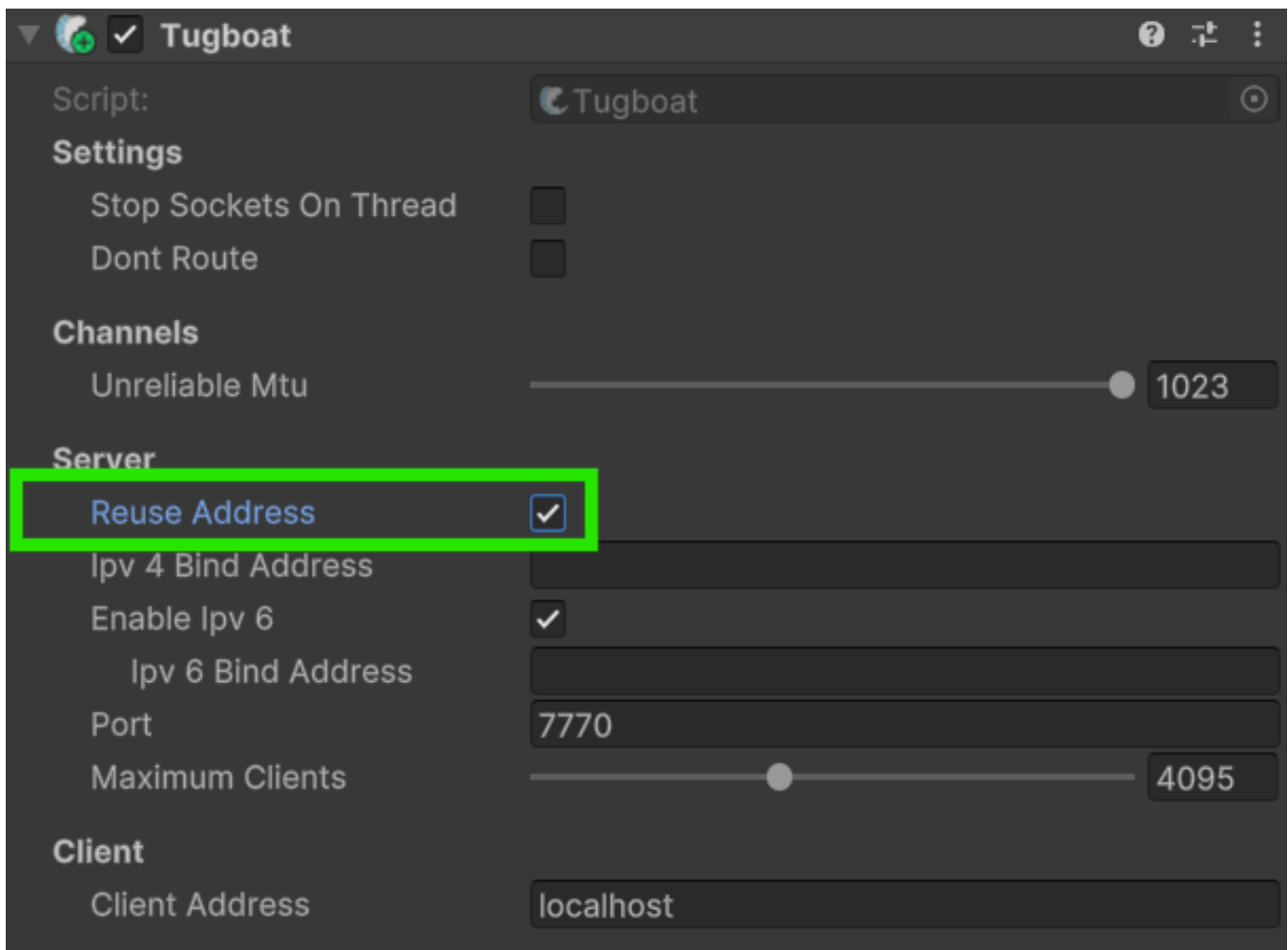
The next thing we need to do is ensure our Unity installation has the correct modules for building a server. Install the **Linux Build Support** modules for your Unity version in the Unity Hub.

[!INFO] Exact details on the required modules depending on your Unity version can be found in PlayFlow's [Unity Modules Setup guide](#).

4. Add Tugboat

Select the NetworkManager in your project and add the Tugboat component if it isn't already there (if you are using a different transport, that is also fine).

Enable the **Reuse Address** checkbox, this will be needed for our server to work correctly.

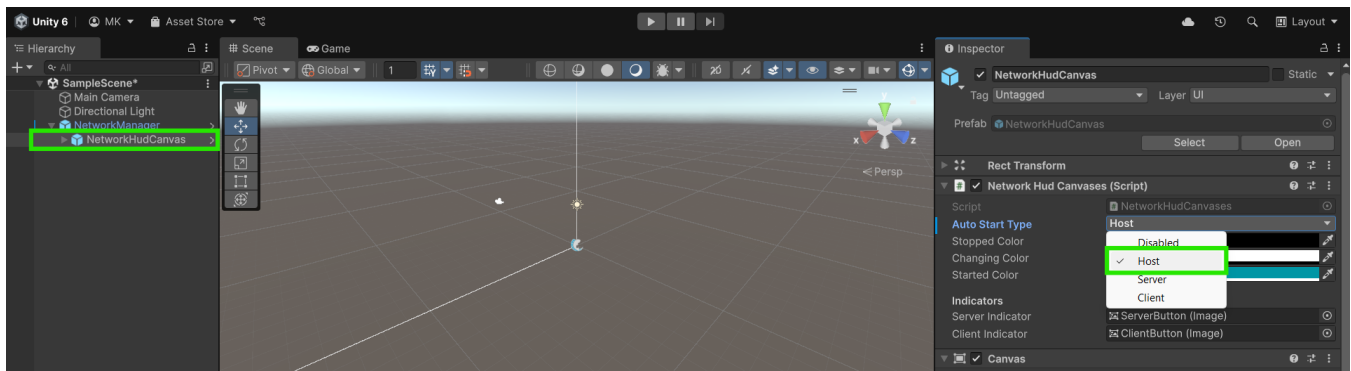


The other two fields we'll be paying attention to are the *Port* and *Client Address* ones. For now you can leave the default settings — we'll remember the port number for later and we'll use the *Client Address* field in our client build to input the URL of our hosted server.

5. Manage Auto Start

We will want FishNet to automatically start as a server in the server build we make. FishNet will actually do this by default with the **Start On Headless** option on the ServerManager. If you disabled this, be sure to manage starting the server itself, or simply re-enable it.

If we have set the NetworkHudCanvas *Auto Start Type* to something other than **Disabled**, then we will want to disable it now, since we want our server to only behave as a server.



Once that's all done we can move on to using PlayFlow Cloud!

Deploying your game with PlayFlow

1. Create your PlayFlow account

Now that PlayFlow is installed, let's go to the website and setup our dashboard.

Go to the [PlayFlow Cloud website](#) and sign-up using your email, or log in with your GitHub or Google account. If asked, confirm your email address to complete the setup.

2. Create a game studio

You'll now be prompted to create a Game Studio. Game Studios make it easy to organize your game projects and work smoothly with your team.

Enter a name for your studio and select either the **Hobby** or **Pro** plan — you can read more about the plan types [here](#). For this tutorial we will be using the **Hobby** plan, but don't worry, you can always change the plan and even the name after it's created.

Once you've done that, click **Create Studio** and you'll be good to continue.

3. Create a new project

Now that you've created your first Studio you'll be prompted to choose it and then we can move forward with creating a new project.

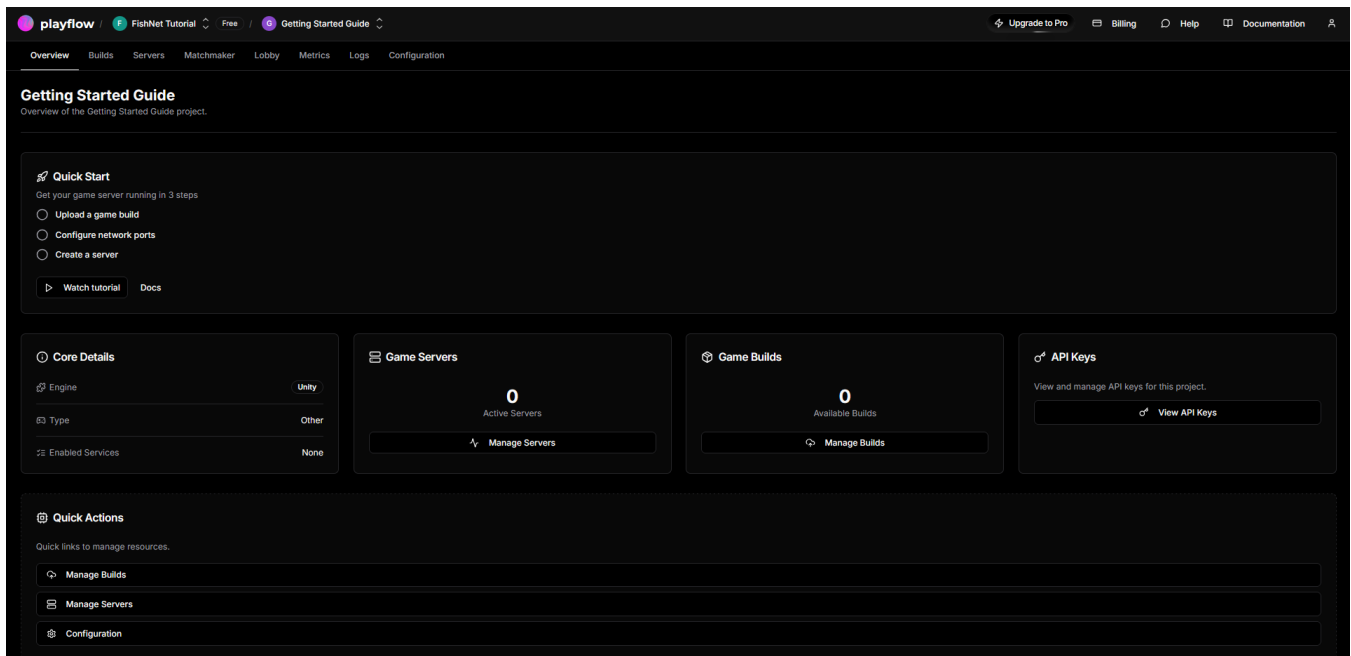
Click the **Create New Project** button and enter the details of your project:

- **Project Name:** Give your project any fitting name you'd like.
- **Game Engine:** Select Unity as the Game Engine.
- **Game Type:** Choose the game type that fits your project best.

Click **Create Project** to proceed.

4. Explore the dashboard

With your new project created and selected, you can check out the PlayFlow dashboard for it. PlayFlow has a lot of powerful features and makes it very easy to view logs, metrics, server details, and more. The PlayFlow dashboard can also be used to directly upload your game's server builds and manage server instances.



5. Link the PlayFlow SDK to your project

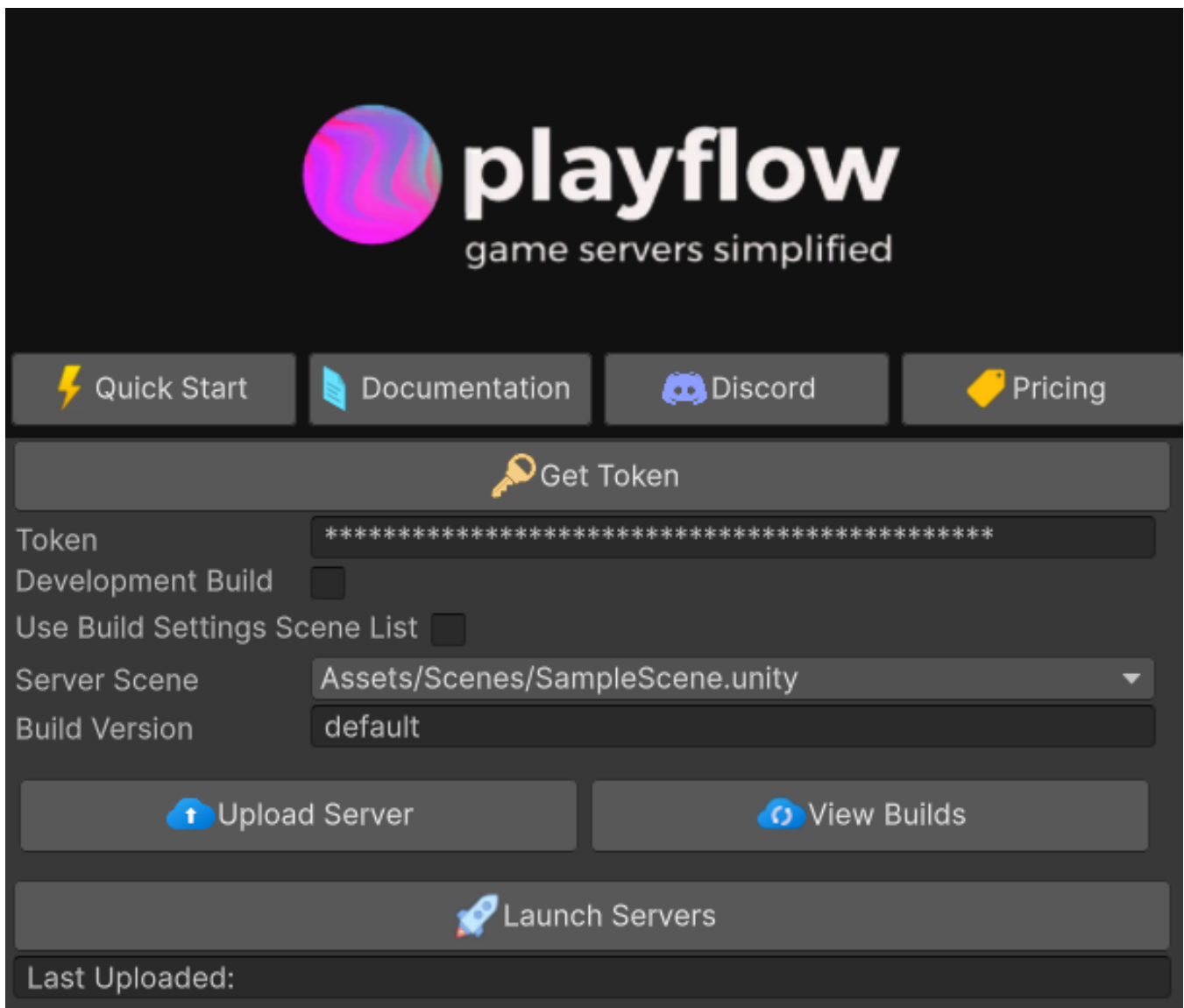
Now let's link our Unity Project to the PlayFlow one we've just created.

Open the **Overview** page in the PlayFlow dashboard.

Find the **API Keys** section and click the **View API Keys** button. Then copy the **PlayFlow API Key** to your clipboard.

Go to the Unity Editor and open the PlayFlow window from the Toolbar at **PlayFlow ? PlayFlow Cloud**.

Inside the *Token* field, paste the **PlayFlow API Key** you copied.



playflow
game servers simplified

Quick Start Documentation Discord Pricing

Get Token

Token *****

Development Build ☐

Use Build Settings Scene List ☐

Server Scene Assets/Scenes/SampleScene.unity ▼

Build Version default

Upload Server View Builds

Launch Servers

Last Uploaded:

6. Build and upload the server

Specify the scene that the server should start in with the **Server Scene** field. You can set it manually via the dropdown or enable **Build Settings' Scene List** to use that instead.

[!INFO] Don't forget to add your scenes to Unity's build settings scene list!

When you're happy with the settings, press the **Upload Server** button to have PlayFlow automatically build and upload a headless Linux server build of your game.

7. Setup the PlayFlow port

Open the PlayFlow dashboard and head to the **Configuration** tab ? **Network Ports** section. Click **Add Your First Port** and fill in the fields:

- **Port Name:** e.g. getting_started_udp
- **Port Number:** The port from your Transport's *Port* field. Tugboat default is 7770.
- **Protocol:** Tugboat uses UDP.

Add New Port

Configure a new port for your game server

Port Name ?

getting_started_udp

Port Number ?

7770

Protocol ?

UDP

Description (Optional) ?

Used for game traffic

Cancel

Add Port

What protocol should I use?

Click **Add Port** to finish.

8. Create the server

Head over to the **Servers** tab and click **Create Your First Server**.

Create New Server



Configure and launch a new game server instance. Ensure you have uploaded necessary builds.

Name FishNet Getting Started

Region North America (US East)

Size Micro ((1 vCPU, 512MB RAM))

Free plan is limited to Micro and Small servers.

 Upgrade to Pro for more powerful servers

Build default

v1 (Latest)

TTL (secs) Optional: 60-86400

Auto-stop server after this duration. Leave blank for no TTL.

Auto-restart



Enable auto-restart on failure

If enabled, the server will automatically restart if it crashes.

Startup Args

Optional: -port=7777 -map=MyMap

Cancel

Create Server

Customize the settings, set a name, and press **Create Server**. You should now see the server in the

Servers tab.

Servers - Getting Started Guide

Manage your game server instances for this project.

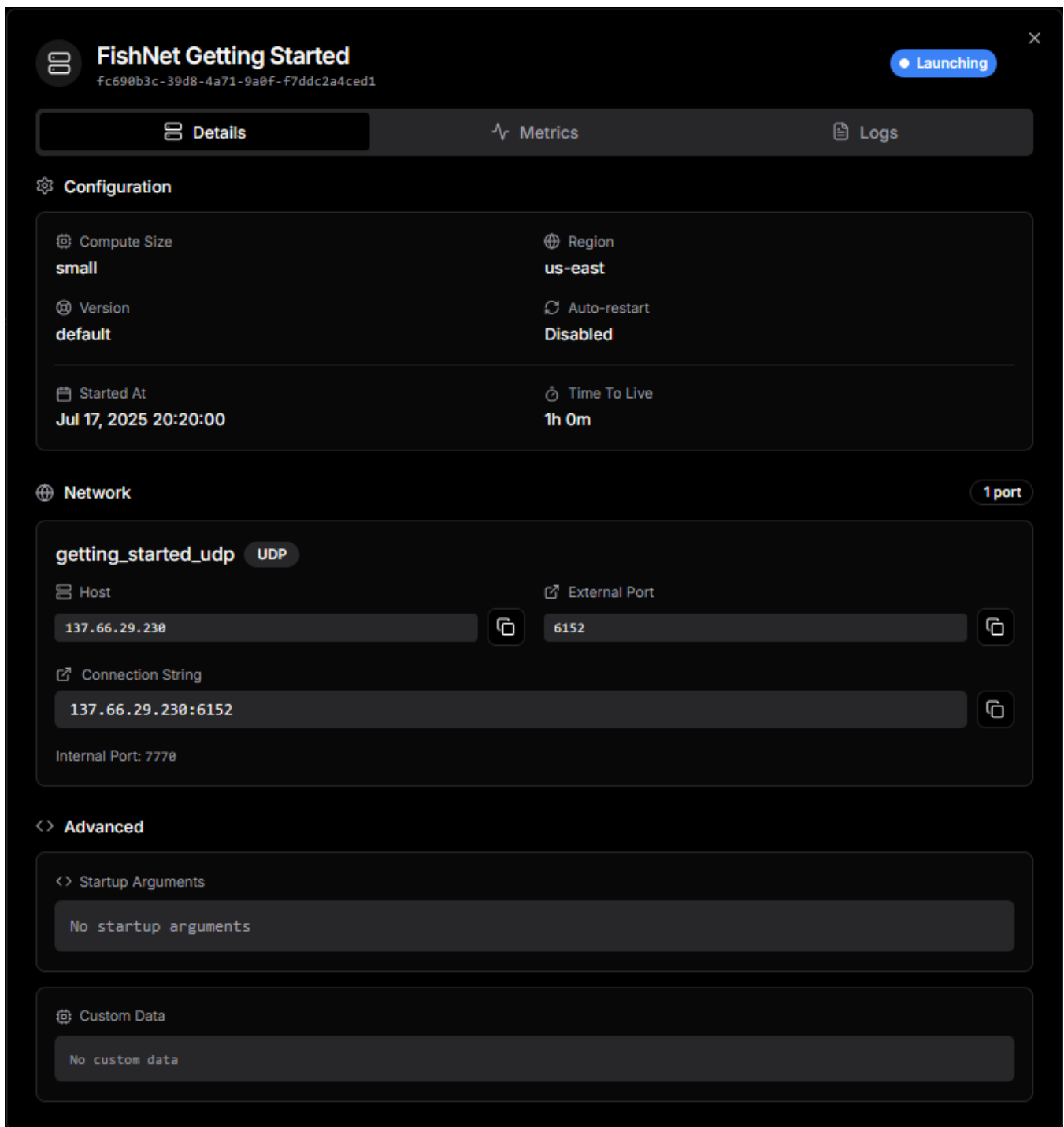
 [Create Server](#)

Showing 1 to 1 of 1 servers

 Name	# Instance ID	 Status	 Region	 Started	 Actions
FishNet Getting Started Size: Small Version: default (v1)	566aaec2-d294-4571-8f8a-a4bbc4dea3b6	Launching	us-east	less than a minute ago TTL: 3600s	Details Stop

9. Connect to the server

Click on **Details** to see the server details. Under **Details ? Network**, copy the *Host* and *External Port* fields.



In the Unity Editor, select your transport component:

- Enter the IP Address from the **Host** field into the Transport's *Client Address* field.
- Enter the **Port** into the Transport's *Port* field.

10. Test the game

Run the game or build and run it and connect as a client. Your game should connect to the PlayFlow server and work correctly!

Take the Next Steps!

Well done! You've now learned how to setup and deploy your server using PlayFlow. If you want to check out more of their features, visit their documentation at <https://documentation.playflowcloud.com/>

You are also encouraged to join their [Discord](#) [server](#).

PlayFlow Cloud (Recommended)

[center]

[Image: untitled]

PlayFlow is the **easiest** way to **build, deploy**, and **scale** multiplayer games globally. Their outstanding support, reliable service, and competitive pricing make PlayFlow our personal recommendation. They have a powerful API with many useful features that make them stand out.

Deploy your first server in under 30 seconds for free. No credit card required.

[Image: untitled]

Instant Scaling with Global Deployment

Effortlessly scale in real time with no surprise costs during traffic surges, and deploy worldwide in seconds using Unity, containers, or your control panel. [/center]

15+

Global Regions

<20ms

Average Latency

1M+

Players Served

[center]

More Key Benefits of PlayFlow

The PlayFlow Unity SDK provides a powerful and convenient way to manage your game servers, lobbies, and matchmaking directly within the Unity Editor and your game's C# scripts.

Find out more about PlayFlow's robust features below! [/center]

[center]

? Better Pricing Than the Competition

No hidden fees. Transparent pricing — paying only for what you use. You can use their cost estimator to get an idea of what running your game might cost you directly from their website. Check out their full pricing page [here](https://www.playflowcloud.com/#pricing). [/center]

[center]

? Matchmaking & Lobby System

Complete game lobby system and matchmaking included so you can match up your players by skill, region, or your own custom rules. [/center]

[center]

? Generous Free Tier & Support That Listens

Build fast, playtest often — all without pulling out your credit card. PlayFlow offers the quickest way to test your game online directly from the Unity Editor, and you can do so for free!

Get help from real people with fast replies, and feedback that actually shapes the platform. [/center]

[center]

? Real-Time Metrics & Powerful Control Panel

Experience seamless monitoring with the advanced dashboard for complete visibility. Instantly track server health, player counts, CPU/memory usage, and network traffic. Features include real-time performance tracking, customizable alerts, log aggregation, and player activity monitoring. [/center]

[center]

? And Still More!

Yet more compelling features, including... [/center]

?? <5min

Getting Started Time

? Auto-Scaling

Intelligent demand-based scaling

? Online Cost Estimator

Instantly Forecast Project Costs

[center] ? [Website](#) ? [Quick Start](#) ? [Documentation](#) ? [PlayFlow SDK](#) ?? [API Reference](#) [/center]

Services

We strongly recommend reviewing our Server Hosting page for terminology as well the Pros/Cons of services mentioned on this page. This information is updated to the best of our knowledge. If you are interested in a service listed below — to better understand their platform — please visit the provided links to their official website.

Our recommended service

PlayFlow Cloud — a company we've had the pleasure of working with firsthand — is recommended as our top-pick for a hosting service. Their outstanding support, reliable service, and superior pricing have consistently impressed our staff and community.

PlayFlow Cloud features: Generous free tier with better pricing than competitors. Lobby & Matchmaking, with an included SDK handling the work. Worldwide dedicated and instanced servers. User-friendly control panel, metrics, support, and more.

Other services

Here is a non-extensive selection of other hosting services that work well with FishNet.

Dedicated servers

These hosts offer session-based servers, as well as persistent servers. Relay services may be available as well.

PlayFlow Cloud

- Pay-per-use pricing
- Session-based servers available
- Persistent servers available
- Relay servers available
- Match Making / Lobbies

GameLift (AWS)

- Pay-per-use pricing — a little more complex
- Session-based servers available
- Persistent servers available
- Relay servers available
- Match Making / Lobbies

AWS is a bit more difficult to setup and manage. We generally only recommend AWS to experienced developers. We have a community guide for AWS available in our documentation.

Relays only

These are relay-only services.

Steam

Steam relays are offered free when you launch your game on Steam; there is a percentage of sales, as well as an initial fee when using Steam.

We have an official FishNet transport for Steam, and there are third-party Unity assets available as well to

help utilize Steam Relays. A popular asset is Toolkit for Steamworks Foundation by Heathen Engineering.

- Free for games launched using Steam
- Session-based relays
- Match Making / Lobbies
- Friends list (Steam)

Epic Online Services (EOS)

EOS provides a number of features for free. The API and documentation is a bit more complex, which may be difficult for beginners. A vetted third-party EOS transport is available for FishNet, which may help with some of those hurdles.

A cross-platform friends list is possible with EOS but at the cost of complexity; e.g.: you will not be able to directly access your Steam friends via the EOS platform. With this in mind, match making with friends may be more difficult.

- Free to all developers
- Session-based relays
- Match Making / Lobbies
- Friends list

Terminology

Overview of key server hosting terms, comparing session-based vs persistent architectures, and outlining the roles of relays and dedicated servers.

Session-based versus persistent

A session-based service, be it relay or dedicated server, will start up and end as needed. Given that these servers are on-demand (session-based) you typically pay-per-use rather than a flat rate, which can save you a substantial amount of money. Session-based services are the most common way online games are hosted and are typically used for matches and lobby based gaming, but can also be used for entire worlds with hundreds of players.

Persistent services are typically always running, no matter the number of players present. Because the servers are always available, this makes it easier to maintain the state of your worlds. A persistent server is the common choice for MMO-like games. While a constant world is very appealing this type of service often comes with significantly higher costs.

Dedicated servers

A dedicated server refers to a server-authoritative game server. There are several advantages to a dedicated server, and very few cons.

Benefits of a dedicated server:

- **Scalability:** Dedicated servers are rarely limited to a set number of players. If you need worlds which support dozens — even hundreds — of players, generally a dedicated server is the recommended route.
- **Security:** A dedicated server is the only way to effectively prevent player cheating, as the server becomes your game's authority and allows you to validate player information.
- **Bandwidth:** Data transfer is rarely limited with dedicated servers. Relays may limit how much total data you can send, or throttle how frequently you can send data.
- **Latency:** For games which require a fast response time a dedicated server is mandatory. These servers can deploy across the world, providing the best latency and most neutrality for players.
- **Persistence:** If you want players to be able to join and leave servers, whether the service is session-based or persistent, a dedicated server will provide a smoother experience. Unlike peer-to-peer hosting, no player-host is required.

Drawbacks of a dedicated server:

- **Cost:** Dedicated servers can be more expensive, especially if a free relay service is available through your platform or other means.

Relays

A relay is a client-authoritative service, where the client acts as the server.

Benefits of a relay:

- **Cost-effective for Casual Games:** Relays are ideal for casual or small-scale games, especially when a free relay service is available. However, using a paid relay service can quickly become impractical, as pay-per-use dedicated servers are often more affordable, while also gaining other advantages.

Drawbacks of a relay:

- **Scalability:** Relays cannot scale. You are often limited to a number of players per session.

- **Security:** The client acting as the host is the authoritative source. Because there is not a neutral authority, cheating is very possible and often easily performed on relay servers. If cheating is a concern, a relay is likely not for you.
- **Bandwidth:** While total bandwidth usage is rarely restricted, most relay services impose limitations on data transfer frequency or size. Games that require rapid data transmission or large data chunks may experience throttling, leading to various performance issues.
- **Latency:** The hosting client's network connection determines latency for the entire session. A weak connection results in higher latency for all players, and players located far from the host will experience additional delay.
- **Persistence:** Relay services do not support state persistence. When the host ends the session, all progress is lost. Additionally, if the host leaves unexpectedly, the game session typically terminates immediately for all connected players.

Showcase

Here you can find games that were made using Fish-Networking. If you'd like to add your game, reach out in the [Discord community](#).

Schedule I

A sandbox drug empire game where you build your operation from the ground up. Start as a small-time dealer and grow your business into a full criminal enterprise.

[View on Steam](#)

Mage Arena

A competitive online arena game where you battle opponents using powerful spells and abilities. Master your magic to climb the ranks and prove yourself the greatest mage.

[View on Steam](#)

Moorhuhn Kart 4

The beloved Moorhuhn chicken returns in a full-featured multiplayer kart racer. Race across creative tracks and battle friends with wacky items.

[View on Steam](#)

RaceWorms

A wild multiplayer racing game where you control a worm through obstacle-filled tracks. Compete online and master the physics to reach the finish first.

[View on Steam](#)

Skygard Arena

A fast-paced multiplayer arena game set in a high-flying fantasy world. Battle enemies with unique abilities as you fight for dominance in the skies.

[View on Steam](#)

Forsaken Frontiers

A multiplayer survival and combat game set in a post-apocalyptic world. Form alliances or go it alone as you explore dangerous frontiers and fight to survive.

[View on Steam](#)

Potion Shop Simulator

Run your very own magical potion shop! Brew concoctions, serve customers, and grow your business in this

charming multiplayer shop simulator.

[View on Steam](#)

Hellcome

A co-op multiplayer shooter where you fight through waves of demonic enemies. Team up with friends to survive increasingly brutal onslaughts from the depths of hell.

[View on Steam](#)

Arcade Sundown

A retro-inspired multiplayer arcade game with tight competitive gameplay. Challenge players online in a variety of classic arcade-style modes.

[View on Steam](#)

FAS

A fast-paced online first-person shooter with fluid movement mechanics and intense gunplay. Jump into quick matches and compete against players worldwide.

[View on Steam](#)

Golfie

A roguelite mini golf game with procedurally generated courses and power-ups. Play solo or challenge friends across an ever-changing set of creative holes.

[View on Steam](#)

Cat Warfare

An online multiplayer battle game where cats go to war. Pick your feline fighter and engage in chaotic team-based combat across a variety of maps.

[View on Steam](#)

Extreme Tag!

A multiplayer game of tag taken to the extreme. Chase or flee from opponents in creative arenas with power-ups and obstacles that keep every round fresh.

[View on Steam](#)

Guns / Blocks / Steel

A multiplayer action game blending block-building with gunplay. Construct defenses and dismantle your opponents in this creative shooter.

[View on Steam](#)

Marble Smashers

A competitive multiplayer marble game where you smash and knock opponents off the arena. Master physics-based gameplay to become the last marble standing.

[View on Steam](#)

Chess For Idiots

An approachable online chess game designed to make the classic strategy game fun and accessible for everyone, including complete beginners.

[View on Steam](#)

Snow Folks

A cozy multiplayer game set in a snowy world. Play with friends in a variety of winter mini-games and activities.



[View on Steam](#)

Meow Meow Wizard Arena

A chaotic multiplayer wizard battle game featuring cats casting powerful spells. Compete against other players in fast and furious magical duels.

[View on Steam](#)

LOCOSOCCER

An over-the-top multiplayer soccer game with wild mechanics and outrageous fun. Score ridiculous goals and outmaneuver opponents in this zany take on the beautiful game.

[View on Steam](#)

Valkyrie Champions

An online action RPG where players take on the role of powerful Valkyrie warriors. Battle through dungeons and PvP arenas with friends in this Norse-inspired adventure.

[View on Steam](#)

Arcadius

A multiplayer arcade game featuring classic gameplay with a modern competitive twist. Compete online in a variety of fast-paced game modes.

[View on Steam](#)

One Up

A mobile multiplayer game available on Android. Challenge friends or compete against players worldwide in this fun and accessible mobile experience.



[View on Google Play](#)

Internet.Game: SAW

A browser-based multiplayer experience inspired by the SAW franchise. Play with friends directly in your browser — no download required.

PARTNERS

LIONSGATE®

twisted pictures

GameStop

Autograph

MZ MAGIC EDEN

1 GAMES EVERYONE CAN PLAY

INTERNET GAME

2 FUN AS F*CK

3 SYNCHRONOUS MULTIPLAYER

[Play Now](#)

Plethora

A browser-based multiplayer game platform with a variety of games to enjoy. Jump in and play with friends directly from your browser.



[Play Now](#)

Upcoming Releases

Games built with Fish-Networking that are currently in development or coming soon. If you'd like to add your game, reach out in the [Discord community](#).

Goblin Cleanup

A cooperative multiplayer game where players work together as goblins cleaning up after a chaotic adventure. Tidy up dungeons, battle messes, and survive the chaos with friends.

[View on Steam](#)

Starlight Re:Volver

A sci-fi multiplayer shooter with stylish combat and fast-paced gunplay. Fight through vibrant arenas as rival agents in an interstellar conflict.

[View on Steam](#)

Alchemy Tunes

A creative multiplayer game blending music and magic. Craft potions, combine melodies, and collaborate with other players in a whimsical world driven by sound.

[View on Steam](#)

Ravenous Horde

A multiplayer strategy survival game where waves of enemies threaten your base. Coordinate with teammates to build defenses and push back the ever-growing horde.

[View on Steam](#)

Purrrifiers: Cleaning Chaos

A chaotic co-op cleaning game starring cats. Work with friends to scrub, sweep, and restore order to increasingly messy environments before time runs out.

[View on Steam](#)

Cryptfall

A multiplayer dungeon crawler with roguelite elements and deep loot systems. Delve into procedurally generated crypts and fight for survival alongside your party.

[View on Steam](#)

Mithrall

An online action RPG set in a rich fantasy world. Battle dangerous creatures, forge powerful gear, and

explore a vast realm with other players.

[View on Steam](#)

Blue Ridge Hunting

A realistic multiplayer hunting simulation set in the scenic Blue Ridge wilderness. Track, stalk, and take down game across expansive open environments with friends.

[View on Steam](#)

Race Project

A competitive online racing game built for speed. Jump into fast-paced multiplayer races across a variety of tracks and prove you have what it takes to win.

[View on Steam](#)

Bayou

Description

Bayou is the transport you are looking for when you want communication on WebGL!\ \ It is an updated and repurposed version of the "SimpleWebTransport" That was originally designed for Mirror Networking.

Compatibility

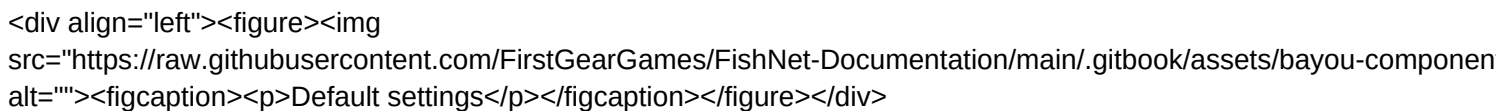
System		Supported?
Windows		Fully Supported
MacOS		Fully Supported
Linux		Fully Supported
Web		Fully Supported
IOS		Fully Supported
Android		Fully Supported
Xbox		Fully Supported
PlayStation		Fully Supported
Nintendo		Not Supported

How to Install

Bayou does not come with FishNet by default, you must download it from GitHub.\ You can find the latest version at the link below!

<https://github.com/FirstGearGames/Bayou/> \ Once Downloaded and Imported into Unity with FishNet. You should add the component to your Network manager, and Assign the "Transport" variable on the Transport Manager to Tugboat.

Settings



Default settings

:gear: Use WSS

When enabled, Bayou will use Secure WebSockets instead of normal WebSockets. When this is enabled the following options will be available: *Enabled: Turns on/off Requiring SSL Certificate.*

Certificate Path: File Path for the certificate *Certificate Password: Password for certificate* **SSL**

Protocol: The protocol you would like to use for encryption.

:gear: MTU

Maximum Transmission Unity for the unreliable channel (in bytes)

:gear: Port

The port you want to bind the server to. Default port is 7770.

:gear: Maximum Clients

The maximum amount of clients allowed to connect to the server. Default is 2000 clients.

:gear: Client Address

The Address the client should use to connect to the server.

FishyEOS (Epic Online Services)

General

EOS transport allows you to utilize Epic's services such as match making and relay. \\ Fishy EOS uses the PlayEverywhere EOS Plugin for Unity SDK, which is a wrapper API/SDK for developers using C# and Unity to take advantage of Epics Online Services API.

Compatibility

System	Supported?	Epic Overlay Support?
Windows	Supported	Yes
MacOS	Supported	No
Linux	Supported	No
Web	No	Not Supported
IOS	Supported	No
Android	Supported	No
Xbox	In Preview	Unknown
PlayStation	In Preview	Unknown
Nintendo	In Preview	Unknown

How to install

Please visit the GitHub locations for official installation and usage instructions.

Main: <https://github.com/ETdoFresh/FishyEOS>

Fork: <https://github.com/FirstGearGames/FishyEOS>.

{% hint style="warning" %} Remember that this transport requires a separate installation of the PlayEverywhere EOS for Unity SDK. Installation for that should also be apart of the GitHub Links above. {% endhint %}

Component settings

- Maximum Clients: Maximum number of players which may be connected at once

EOS

- Socket Name: Socket ID [Must be the same on all clients and server]
- Remote Server Product User ID: Server Product User ID. Must be set for remote clients.

- Auto Authenticate: Automatically Authenticate/Login to EOS Connect when starting server or client.
- Auto Connect Data: All the data necessary to login to EOS and connect to a remote peer.
 - Login Credential Type:
 - ID:
 - Token:
 - Display Name:
 - Automatically Create Device ID:
 - Auth Scope Flags:
 - Timeout:

FishyFacepunch (Steam)

General

FishyFacepunch is a transport to interface with Steam using Facepunch.Steamworks API.\\ You will need FishyFacepunch to be the transport layer, and you will need FacePunch.Steamworks, which is a subjectively simpler wrapper for the Steamworks.Net API.\\ See the How to Install section to setup the Transport and FacePunch.Steamworks.

Compatibility

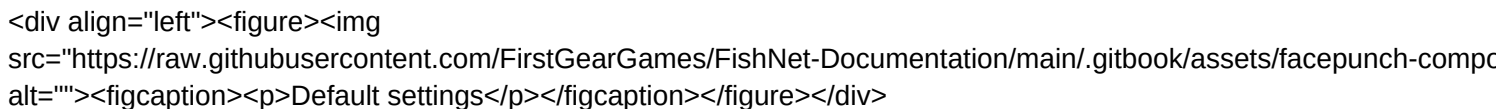
System		Supported?
Windows		Fully Supported
MacOS		Fully Supported
Linux		Fully Supported
Web		Not Supported
IOS		Not Supported
Android		Not Supported
Xbox		Not Supported
PlayStation		Not Supported
Nintendo		Not Supported

How to install

To install visit <https://github.com/FirstGearGames/FishyFacepunch> and follow the Setup instructions.

{% hint style="warning" %} Note that this transport requires Facepunch.Steamworks to be installed separately, as well as .Net 4x or greater. Links to install these should be on the GitHub as well. {% endhint %}

Component settings



- **Steam App ID:** The App ID assigned to your game when registered with Steam. The default is 480 which is Steam's testing game (SpaceWar).

Server

- **Server Bind Address:** Used if you want to manually Bind a Steam ID to the server.
- **Port:** The port you want the server to be binded to. Default is 27015
- **Maximum Clients:** The maximum amount of clients you would like to connect to the server. Default is 16.

Client

- **ClientAddress:** The Steam ID of the server you are connecting to. The Servers Steam ID if its

standalone, or the Hosts Client Steam ID if the server is a host.

- **Timeout (Seconds):** The amount of time allowed to establish a connection, in seconds.

FishyRealtime (Photon)

{% hint style="danger" %} While the FishyRealtime Transport is free under MIT license, Exit Game's Photon Realtime is subject to their own service charges and license. {% endhint %}

General

FishyRealtime is a transport that allows the Photon Realtime service from the company **Exit Games** to work with FishNet!

Compatibility

System	
Windows	Fully Supported
MacOS	Fully Supported
IOS	Fully Supported
Android	Fully Supported
Linux	Fully Supported
Xbox	Fully Supported
PlayStation	Fully Supported
Nintendo	Fully Supported
Web	Not Supported

How to install

Please visit the official GitHub link for Installation and Usage instructions.

Main: <https://github.com/REIO7200/FishyRealtime>

{% hint style="warning" %} To reiterate, This transport does depend on installing the Photon Realtime SDK Separately. {% endhint %}

Component settings

- **Photon APP ID:** The app id of the Photon Realtime product.
- **App Version:** The version of the product(Game), only clients with the same version can connect to each other.

Default room options

- **Max Players:** Default max players.
- **Is Visible:** Whether Room is Visible to Public Room List.
- **Is Open:** Whether the room is public or private.
- **Connection Protocol:** Connection Type.
- **Region:** The Region the Server is hosted on.

FishySteamworks (Steam)

General

FishySteamworks is FizzySteamworks reformed. FishySteamworks has significant performance improvements and has been made to fully support [Toolkit for Steamworks SDK](#) by *Heathen Engineering*.

Compatibility

System	Supported?
Windows	Fully Supported
MacOS	Fully Supported
Linux	Fully Supported
Web	Not Supported
IOS	Not Supported
Android	Not Supported
Xbox	Not Supported
PlayStation	Not Supported
Nintendo	Not Supported

How to install

Github: <https://github.com/FirstGearGames/FishySteamworks/> \ Import the package from Github above into your Unity FishNet project, and add the "FishySteamworks" Component to your [Transport Manager](#).

{% hint style="warning" %} *FishySteamworks is dependent upon [Steamworks.NET](#). An [installation article](#) for Steamworks.NET has kindly been provided by *Heathen Engineering** {endhint %}

Component settings

- **Steam App Id:** is the Id for your steam application. 480 is the default, which is often used as a test App Id.
- **Server Bind Address:** Which IP the server will bind/listen on. The default value is empty.
- **Port:** Indicates which port the server will listen on.
- **Peer To Peer:** Will allow clients to act as host when running your project. IP addresses are **NOT** revealed.
- **Maximum Clients:** is how many players can be connected to the server at once.
- **Client Address:** is the IP of the server, which clients connect to. Empty is the default value for local testing.

FishyUnityTransport

General

A transport to utilize Unity's Official Transport with FishNet. Supports both Unity Transport 1.0 and Unity Transport 2.0 \ Using **FishyUnityTransport** also allows you to utilize some of Unity's Multiplayer Services, like Relay!

Compatibility

System	Supported?
Windows	Fully Supported
MacOS	Fully Supported
Linux	Fully Supported
Web	Fully Supported
IOS	Fully Supported
Android	Fully Supported
Xbox	Fully Supported
PlayStation	Fully Supported
Nintendo	Fully Supported

How to install

For installation instructions and usage please visit the official GitHub site.

Main: <https://github.com/oonush/FishyUnityTransport>

Component settings

{% hint style="warning" %} Settings marked in **Yellow** are only supported on Unity Transport 2.0! {% endhint %}

- **Protocol Type:** Configure whether the transport is done via direct connection or over Unity Relay.
- **Use Web Sockets:** Per default the client/server will communicate over UDP. Set to true to communicate with WebSocket.
- **Use Encryption:** Per default the client/server communication will not be encrypted. Select true to enable DTLS for UDP and TLS for Websocket
- **Max Packet Queue Size:** The maximum amount of packets that can be in the internal send/receive queues. Basically this is how many packets can be sent/received in a single update/frame.
- **Max Payload Size:** The maximum size of an unreliable payload that can be handled by the transport.
- **Heartbeat Timeout (ms):** Timeout in milliseconds after which a heartbeat is sent if there is no activity.
- **Connect Timeout (ms):** Timeout in milliseconds indicating how long we will wait until we send a new connection attempt.
- **Max Connect Attempts:** The maximum amount of connection attempts we will try before disconnecting.
- **Disconnect Timout (ms):** Inactivity timeout after which a connection will be disconnected. The

connection needs to receive data from the connected endpoint within this timeout. Note that with heartbeats enabled, simply not sending any data will not be enough to trigger this timeout (since heartbeats count as connection events)

- **Connection Data:**
 - **Address:** IP address of the server (address to which clients will connect to)
 - **Port:** UDP port of the server.
 - **Server Listen Address:** IP address the server will listen on. If not provided, will use localhost.
- **Maximum Clients:** The Number of Clients allowed to connect to the FishNet server.

FishyWebRTC

This transport is fully functional with the latest Fish-Networking version.
For installation and usage please visit the project's [GitHub here](#).

Transports

The **Transport** is the low-level layer responsible for sending and receiving raw data over the network. It abstracts away the details of TCP/UDP sockets and handles packet delivery, reliability, ordering, etc. FishNet uses events internally to plug into transport messages. Although it be unlikely you would need to access such messages, they are available to you for your development as well.

There are many transports available, some are maintained by the Fish-Networking team and others maintained by the community.

{% hint style="success" %} Tugboat is the default transport and will be added to your project at runtime. If you wish to use a different transport or customize **Tugboat** you must add the component to your NetworkManager object. {% endhint %}

Switching transports at run-time or using multiple transports

FishNet has support for changing transports on your NetworkManager while the network is not running as well as allowing servers to utilize multiple transports at once. For example you may want to use FishyEOS or Unity Transport to support your console users, but also use Tugboat to give a better experience to your Non-Console users.

FishNet uses the Multipass transport to achieve both of these goals.

{% hint style="info" %} Even with **Multipass**, clients can only ever utilize one active **transport**. More info can be found on the Component and Guide pages. {% endhint %}

Multipass

Description

Multipass is a pass-through transport that allows multiple transports to run on the server at once. For example, you may want to run Tugboat for your desktop or mobile users, and Bayou for your web users. With **Multipass** a single server build can run both, joining players from both together in the same game. For usage see the [guide](#).

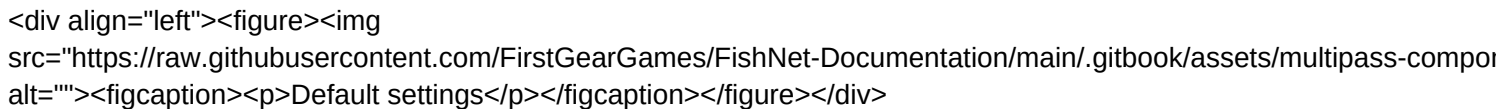
Compatibility

System	Supported?
All Platforms	Fully Supported

How to install

Multipass comes with FishNet by default! No extra downloads needed. \ \ Just add Multipass to your [Transport Manager](#) and follow the usage guide mentioned in General above.

Settings



Default settings

:gear: Global Server Actions

While enabled, this will perform server actions such as start or stop server on all transports.

:gear: Transports

A collection of [transports](#) you wish **Multipass** to support.

Tugboat

Description

Tugboat uses [LiteNetLib](#) to support reliable and unreliable messages. This is the default transport for Fish-Networking and will automatically be added to your [NetworkManager](#) if no other [transport](#) has been added already. It supports a large variety of platforms, and is a good choice for many games.

Compatibility

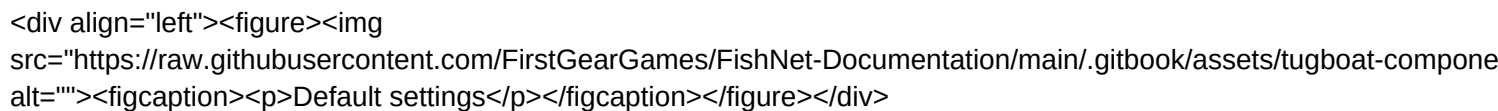
System	Supported?
Windows	Fully Supported
MacOS	Fully Supported
Linux	Fully Supported
Web	Not Supported
IOS	Fully Supported
Android	Fully Supported
Xbox	Fully Supported
PlayStation	Fully Supported
Nintendo	Fully Supported

How to install

Tugboat is the default transport of FishNet! Which means it comes with FishNet by default when Importing FishNet from your favorite source.

The [Transport Manager](#) will automatically make this the default transport at runtime, if you did not add any other Transport to the Network Manager Game Object.

Settings



Default settings

:gear: Stop Sockets On Thread

When enabled, causes the local server and client sockets to be stopped using a new thread. This option can be useful for ensuring that the socket shutdown process does not block the main thread, which may help with smoother shutdowns or prevent the application from freezing during network cleanup.

:gear: Don't Route

Enable **Don't Route** if you want your Tugboat server or client to send data directly to the local interface, avoiding any network routing by the operating system. This is an advanced option primarily for special networking requirements.

:gear: Unreliable MTU

This is the largest size an unreliable packet may be. When a single outbound data exceeds this value it

is sent reliably. Smaller unreliable datas will be automatically split over multiple unreliable sends.

:gear: Reuse Address

Allows the same address and port to be used multiple times by the server. This can be useful if you wish to launch multiple builds or server instances on the same machine using the same configuration.

:gear: IPv4 Bind Address

This is which address to bind the server to. If set and IPv4 is not available this will cause a socket error upon server start.

:gear: Enable IPv6

Enables IPv6 binding when available. In some cases you may want this disabled if you have an IPv6 interface you do not want used.

:gear: IPv6 Bind Address

This is which address to bind the server to. If set and IPv6 is not available this will cause a socket error upon server start.

:gear: Port

Is which port the server will listen on. This is also the port clients will connect to. In some instances you may need to change this at runtime using `TransportManager.SetPort()`.

:gear: Maximum Clients

Maximum Clients is the maximum active clients allowed before the transport begins to deny connections.

:gear: Client Address

This is which address to connect to as a client. This is typically your server address. Localhost is set as default for local testing.

Yak (Pro Feature)

{% hint style="danger" %} Yak is part of Fish-Networking Pro features. {% endhint %}

Description

An offline transport that runs off your multiplayer code. No local or remote connections are made using this transport.

Besides being an offline transport, Yak API behaves exactly as our many other transports. In result, you do not need to change any of your code to take advantage of Yak.

To support offline play and online play you will want to consider using [Multipass](#) with Yak and an online transport of your choice.

Compatibility

System	Supported?
All Platforms	Fully Supported

How to install

Yak is part of FishNet Pro! So you must have purchased pro. See how to purchase and download FishNet Pro [here](#). \ After Installing FishNet Pro, just add Yak to your Transport manager, most likely you will be adding Yak onto Multipass like mentioned in the General Section above.

Settings

No settings needed to be configured with Yak.

Creating Bug Reports

A well-crafted bug report will result in bugs being resolved much quicker.

New bug reports can be created on the [FishNet GitHub Issues page](#). You will need to be logged into GitHub to create a bug report.

Keep an Eye on Your Reports

If your bug report is missing information we may close it out asking you to submit a new report.

Sometimes bug reports perfectly meet our guidelines but require more information. When this happens we will notify you in the bug report of what information we need, and place a "waiting on information" flag on it. If we do request more information and receive no response within 2 weeks the report will likely be closed to keep prioritization and order. You can still comment on closed reports and we will re-open them as needed.

Issue Template

When creating a new issue on our GitHub you will be presented with a template. See below for detailed information on how to complete each section.

Necessary Information

When filling out your report please include the following:

- Unity version
- Fish-Networking version
- Discord link where you troubleshoot the issue

Info: If you are not able to join our Discord to troubleshoot, please specify this in place of the Discord link. We still expect you to troubleshoot the issue locally if you are unable to connect with our helpers.

Leaving your Discord name in our server, or email address (less preferred), will allow us to contact you if you're unresponsive to the bug report while we still need more information.

Description

Provide a brief description of the bug report. This should be a summary of what causes the bug or how it is affecting your project.

Replication

In an ordered fashion, provide step-by-step details on how to reproduce the bug. Be sure to indicate if starting as client or server, if multiple builds must be running, what actions to take, and so on.

Expected Behavior

Describe what you expect the behavior to be. This helps us know if the result is intended, and could give us insight on what may be going wrong.

Media

If there is any media or content that could assist us in resolving your issue, please provide it here. Media could include images, stack traces, videos, links, and more. If you are providing code samples or stack traces please use text.

Sample Project

A sample project is not always required, but at times issues are too complex to reproduce using the issue

template alone. When that is the case we will ask you to create a sample project. When including a sample project, also state how to use it.

Sample Project Guidelines

Sample projects should always follow these guidelines to keep them simple and small.

All samples must be within their own folder so when imported into a Unity project they will not be merged with other files. For example: `MyBugReport\Scripts`, `MyBugReport\Prefabs`, etc.

Provide only the files needed to reproduce the problem. Do not include extra models, prefabs, code, or anything unrelated to the issue. Delete any code in provided scripts that does not directly affect the bug.

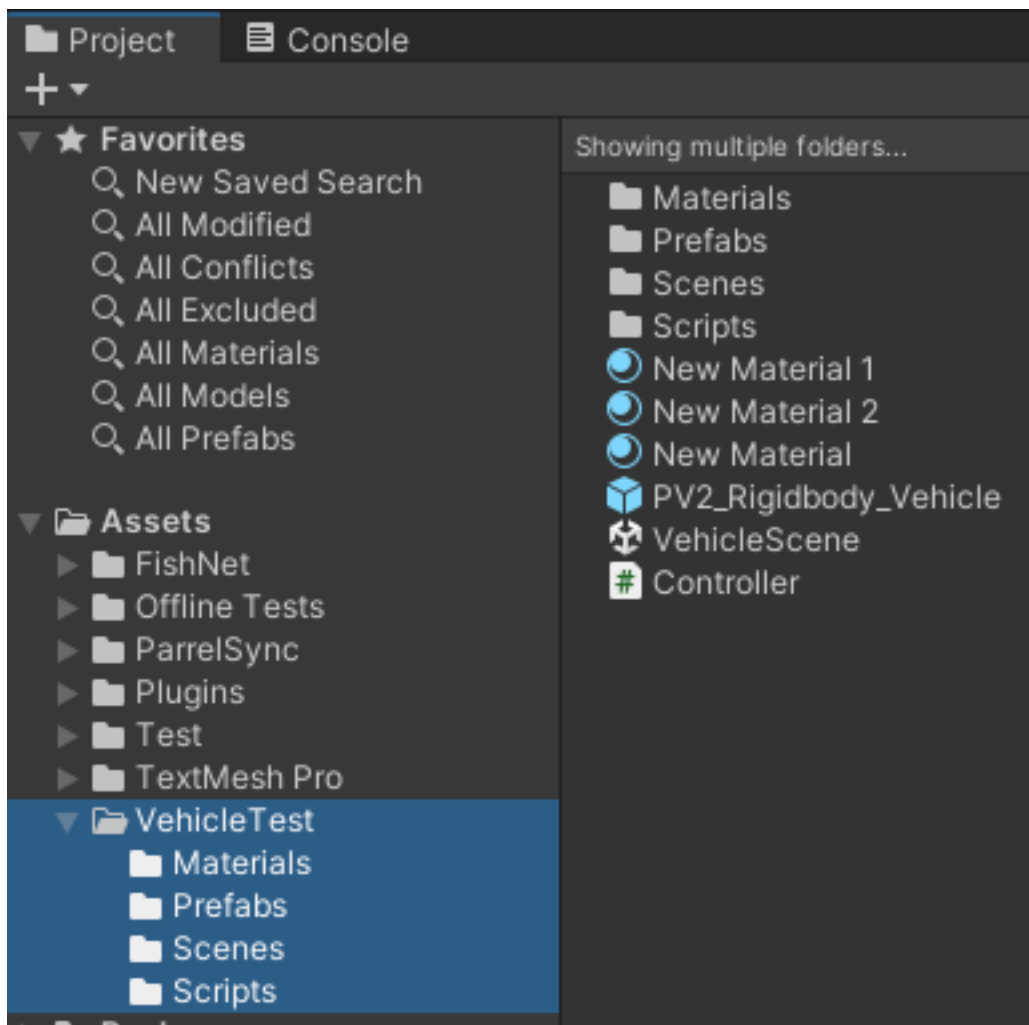
Do not include Fish-Networking within your sample project — only include the files needed to reproduce the problem.

Use the old input system.

Export as a `.unitypackage`: right-click your folder, choose **Export Package**, and uncheck **Include Dependencies** at the bottom.

Info: GitHub does not allow uploading `.unitypackage` files. You may need to archive your exported package before uploading. Please use zip format.

Warning: Model files can be large and pink materials are not fun to look at! If possible, replace models with standard meshes and use the standard render pipeline.



Report Example

This is an example of what a created bug report might look like.

Below is a bug report for a SyncVar which is not sending updates to clients. Headers are shown here for clarity, but you do not need to include them in your actual report.

Info: If troubleshooting was not performed or applicable fields are missing in the bug report there is a fair chance the submitter would be asked to update their issue, and until then the issue would be suspended.

Unity version: Unity 2021.3.10f

FishNet Version: FishNet 4.3.1

Discord Troubleshoot Link / What I've tried: <https://discord.com/channels/123456>

Description: When I set a SyncVar on the server, non-host clients do not get the update. Callbacks are not working either.

Replication:

This can be reproduced in two editors.

Start the 1st editor as host or server.

In a player NetworkBehaviour script, update an int SyncVar in Awake, wrapped in an InstanceFinder.IsServerStarted check.

Start the 2nd editor as client only.

Have the 1st editor spawn the player object for the 2nd editor.

Expectation:

- Both the 1st and 2nd editor get callbacks.
 - The SyncVar value should synchronize to the 2nd editor.
-

At a glance this would suggest everything is working as intended. The server is not active on the 2nd editor (client only), so the InstanceFinder.IsServerStarted check is failing and the value is never applied locally. The 1st editor is not sending the value because SyncTypes set in Awake are assumed to be applied on both server and client, since Awake always fires regardless of network state.

Although this is covered in the SyncType documentation, the behavior could be something of a gotcha. If the developers' troubleshooting steps would not have caught this, they would likely provide the explanation above and ask the developer to close the issue if it were now resolved.

Creating Bug Reports

A well crafted bug report will result in bugs being resolved much quicker.

New bug reports can be created by navigating to our GitHub issues page. You will need to be logged into GitHub to create a bug report.

Keep an eye on your reports

If your bug report is missing information we may close it out asking you to submit a new report.

Sometimes bug reports perfectly meet our guidelines but require more information. When this happens we will notify you in the bug report of what information we need, and place a 'waiting on information' flag on the bug report.

If we do request more information and receive no response within 2 weeks the report will likely be closed to keep prioritization and order. You can still comment on closed reports; we will re-open them as needed.

Issue template

When creating a new issue on our GitHub you will be presented with a template. Provided below is detail information on how to complete the template.

Necessary information

When filling out your report please include the following: Unity version, Fish-Networking version, and a Discord link where you troubleshoot the issue.

If you are not able to join our Discord to troubleshoot please specify this in place of where you would include the Discord link. We still expect you to troubleshoot the issue locally if you are unable to connect with our helpers.

Leaving your Discord name in our server, or email address (less preferred) will allow us to contact you if you're unresponsive to the bug report while we still need more information.

Description

Provide a brief description of the bug report. This should be a summary of what causes the bug or how the bug is affecting your project.

Replication

In an ordered fashion please provide step-by-step details on how to reproduce the bug. Be sure to indicate if starting as client, server, if multiple builds must be running, what actions to take, and so on.

Expected behavior

Describe what you expect the behavior to be. This helps us know if the result is intended, as well could give us insight on what may be going wrong.

Media

If there is any media or content which could assist us in resolving your issue please provide it here. Media could be a variety of things such as images, stack traces, videos, links and more. If you are providing code samples or stack traces please use text.

Sample project

A sample project is not always required; but, at times issues are too complex to reproduce using the issue template. When this is true we will ask you to create a sample project for us to look at. When including a sample project also state how to use the project.

Guidelines

Sample projects should always follow guidelines to keep them simple and small.

All samples must be within their own folder so when imported into a Unity project they will not be merged with other files. For example, `MyBugReport\Scripts`, `MyBugReport\Prefabs`, and so on.

Provide only files needed to reproduce the problem. Do not provide extra models, prefabs, code, etc unrelated to the problem. Delete any code in provided scripts which does not directly affect your issue.

Do not include Fish-Networking within your sample project; only include the files needed to reproduce the problem.

Use the old input system.

Export as a `.unitypackage`. To do so right-click your folder, Export Package, and uncheck **Include Dependencies** at the bottom.

GitHub does not allow uploading unitypackage files. You may have to archive your exported package before uploading. Please use zip format when doing so.

Model files can be large and pink materials are not fun to look at! If possible replace models with standard meshes and use the standard render pipeline.

FAQ

Discover frequently asked questions and their answers.

Miscellaneous

Broadcasts are generally the best choice for sending data without a player object. Many developers trade this data in a custom authenticator class, which allows data to be sent before the client initializes anything at all for the network. See our `PasswordAuthenticator` script for an example of doing this.

Another approach is to use a custom player spawner instead of our `PlayerSpawner`. You may send broadcasts back and forward freely, and only spawn your player when you feel is right. See broadcasts for more information on this feature.

Most often when this occurs you actually are able to control your player, but you simply do not see them because your game could be using the incorrect camera. This is seen when more than one player spawns, and the player prefab has a live camera beneath them.

To resolve this issue simply use one camera in the scene and take over it as needed, or disable the camera on your player prefab and only enable it if you own the object.

You can install Fish-Networking free over Pro, and Pro over free without any issues. Just download free or Pro and import normally. See our Pro section for information on downloading Pro.

If you import a new version of Fish-Networking and there are immediately compile errors, delete your FishNet folder then import the latest version again.

Hosting and connectivity

There are several ways to make rooms within Fish-Networking.

You can use a third party service which creates individual server instances, each acting as their own. There are several services which provide this; an example of one is PlayFlow Cloud.

Another option is to have a single Fish-Networking instance manage rooms in a single build. This reduces the complexity of a third party service but requires you to develop with stacked scenes in mind. Our project Lobby and Worlds accomplishes this, and is available to supporters.

While stacked scenes aren't necessarily difficult, and support for them are built-into Fish-Networking, there are still some Unity limitations around them. An example being, not all physics API are available in stacked scenes.

Mobile apps don't typically close like a normal application. Instead, mobile apps are suspended so that you may "re-open" them quickly.

Clients will likely disconnect if the game remains in the background for longer than the timeout settings on your `Client/ServerManager`.

Often you can utilize the Unity callback `OnApplicationPause` in mobile games to tell the `ClientManager` to disconnect.

A large variety of third party services allow you to host p2p games. Steam and EOS are the two most common ones, but plenty are available. We support both Steam and EOS.

When trying to connect to your IP directly you must allow connections through your firewall. Be sure to adjust your firewall to allow the port used by your game.

You can also join/create LAN games without changing your firewall by using our Fish-Networking Discovery

addon.

Check to make sure you aren't trying to start the FishNet server twice, perhaps due to manually starting it as well as FishNet automatically starting it if enabled in the `ServerManager` component's **Start On Headless** option. You can also try enabling the `ReuseAddress` option if using the Tugboat transport. This will allow the server to bind to the port even if it was recently used, making server restarts and multiple concurrent instances possible without port conflicts.

Missing or hidden objects

By default when the `clientHost` is not an observer of an object the renderers for the object are disabled. This is to simulate as if the object is not spawned for the `clientHost`, though it of course is as the server is still using the object.

You may disable this feature by changing a setting on the `ObserverManager`. It's also possible to disable this per `NetworkObject`.

You can also utilize `NetworkObject` events to manually update renderers.

First and foremost, check the console for any Fish-Networking warnings or errors.

Scene objects become disabled when the client is not an observer of the scene. Our observer system controls what objects clients can see, spawn, and transmit.

The most common reason a client is not an observer of the scene is because the server has not added the client to the scene. This can be done manually if you know the client has the scene loaded, using a Fish-Networking `SceneManager` reference, and calling `AddConnectionToScene`. E.g.:
`sceneManager.AddConnectionToScene(yourClient)`.

You may just as well use our automated system by telling the `SceneManager` to load the scene for the client. See the scene management guide for more information.

If you are starting entering play mode with only one scene you may be manually spawning the player but not adding to starting scene. See our `PlayerSpawner` script within your Fish-Networking installation for an example of how to instantiate player prefabs, and add clients to scenes.

If a client is not an observer of an object then the server does not spawn the object for the client. See our observers guide for more information.

SyncTypes (SyncVar, SyncList, etc.)

Clients may update `SyncTypes` locally, but they are not synchronized over the network; only the server may synchronize `SyncTypes`. Typically, clients will send a Remote Procedure Call to the server indicating it wants to update something, and the server complies. See these guides for more information: Remote Procedure Calls, `SyncTypes`.

`SyncTypes` run on intervals, defaulted to every 100ms if the `SyncType` has changed; the interval may be changed on the `ServerManager`.

However, even if the interval is met, `SyncTypes` always synchronize after remote procedure calls (RPC), even if you set them before calling the RPC. You can change `SyncTypes` to synchronize first on a per `SyncType` basis using `SyncTypeSettings`. Review also the `SyncTypes` guide thoroughly for updating a `SyncType`'s settings.

Remote Procedure Calls (RPCs)

By default only clients which own the objects may send a `ServerRpc`. You may bypass this restriction by setting `RequireOwnership` to `false` in the `ServerRpc` attribute. If you've not bypassed this restriction, the sender will always be owner.

The `ServerRpc` guide shows how to set `RequireOwnership`, as well how to know which spectator might be sending the RPC. You will notice in the guide a `NetworkConnection` is specified at the end of the RPC parameters. You do not pass in a connection when sending the `ServerRpc`; it's set automatically when you receive the RPC call.

Errors and warnings

Typically only the server may spawn networked objects. You will see this warning if you try to network spawn an object on a client, while predicted spawning is not enabled.

Predicted spawning must be turned on in the `ServerManager`. See also the `PredictedSpawn` component.

Most likely you are seeing this error because your `SyncType` does not have an initializer. When declaring `SyncTypes` they must be `readOnly` and initialized.

For example: `private readonly SyncVar<int> _mySv = new();`

You can view more information about `SyncTypes` [here](#).

You will see this warning if the client is not started. It's also possible to see this warning if you are trying to communicate with the server such as using a `ServerRpc` before the `NetworkObject` has finished initializing for the client, even if the client connection has started. (`obj.IsClientInitialized`) See `NetworkBehaviour` API and callback order for more information on this.

It's also possible you have a method decorated with the `[Client]` attribute, such as if you want a method to only run on clients. This will cause the warning, even if your intents are to not have the client connected. If this is true, you may set `LoggingType` to `Off` within the `Client` attribute.

You will see this warning if the server is not started. It's also possible to see this warning if you are trying to communicate with a client such as using a `Target` or `ObserversRpc` before the object is initialized for the server. See `NetworkBehaviour` API and callback order for more information on this.

It's also possible you have a method decorated with the `[Server]` attribute, such as if you want a method to only run on server. This will cause the warning, even if your intents are to not have the server running. If this is true, you may set `LoggingType` to `Off` within the `Server` attribute.

You will see this error if the server thinks your client is in scene, when the client does not have the scene loaded. Using a `SceneCondition` on the `ObserverManager` will typically resolve this problem. You can simply add the `ObserverManager` component to your network manager and give it the scene condition in its `Default Conditions` list.

If you are already using a `SceneCondition` and are certain the correct scenes are being loaded then open the scenes which you are having problems with, and use the **Reserialize NetworkObjects** utility from the (Tools ? Fish-Networking ? Reserialize NetworkObjects) toolbar menu.

You can also troubleshoot this further by adding the `DebugManager` component to your `NetworkManager` and enable **Write Scene Object Details**. The next time you see the error it will also print the scene and object name the spawn or message was intended for.

In very rare cases this is a bug. If you've tried all the troubleshooting steps above without success consider

reaching us on our Discord for help.

On very rare occasion you may encounter this error after making a change to your script. This is a Unity bug that we have no way to resolve internally. The exact cause is unknown, but we know it's related to Unity caching something improperly in a script.

There are a few work-arounds that have worked consistently; most commonly adding a new empty method with a parameter (try without as well) and saving the script will resolve the issue. At some point the Unity cache will fix itself, and the empty method can later be removed.

Clearing the library cache seems to have no benefits of resolving this error. The only success we've seen besides the work around is moving everything to a completely new project. Because creating a new project is so much work it's recommended to use the work-around.

Objects

There unfortunately is not any definitive cause of this error. Usually restarting Unity will resolve the problem.

This message can also be seen when a `NetworkObject` is a prefab which is unable to save changes. If your object is a prefab check to make sure there are no missing scripts, or anything else preventing saving of the prefab.

If issues persist please reach us on our Discord.

You can get a list of all objects owned by a connection by using `conn.Objects`.

If you want to grab the first object spawned for a connection use `conn.FirstObject`. You can set the `FirstObject` at runtime if you have a preference to the 'FirstObject'.

If you want to know what objects you own as a client you can grab your own connection under `clientManager.Connection`.

This is done by writing a custom player spawner. You can use our `PlayerSpawner` as an example, but instead of spawning immediately only spawn after a client has been added to a scene.

A good place to start is using the `ClientPresenceChangeEnd` callback to know when the client has entered the scene and has visibility of objects within it.

In most cases you do not need to pass around a `NetworkObject Id`. The recommended approach, if sending over the network, is to send the `NetworkObject` reference itself. This automatically efficiently sends the `NetworkObject` information, and returns the proper object on the other end.

Should you have reasons to use the `Id` specifically you can look up spawned objects within the `serverManager.Objects.Spawned` collection, or `clientManager.Objects.Spawned` if client only.

Performance

Our framework does not have any CCU limitations. How many players you are able to host on a single server varies greatly depending upon your server hardware, game mechanics, and your coding efficiency.

Several Fish-Networking games have achieved 500+ CCU, and thousands of `NetworkObjects`.

Fish-Networking does not take any special actions to support nor restrict sharding. Some users experienced success with custom implementations of world sharding, but we do not officially support this feature.

With how powerful servers have become world sharding is rarely needed, and generally we discourage against it.

Troubleshooting

This section will go over common limitations and issues you may run into, what FishNet's warning and error logs mean, and how to report problems.

- [Technical Limitations](#)
- [Creating Bug Reports](#)
- [FAQ](#)

Technical Limitations

Limitation is not a word we like to use in Fish-Networking but unfortunately there are a few technical blocks with Unity, or restrictions in place for performance reasons. These limitations are considered reasonably acceptable and may never be resolved.

<details>

<summary>Networked scene objects</summary>

When a scene object is networked they may behave differently.

A networked scene object will be disabled when a scene loads, and will not activate until the client or server is started. For clients specifically networked scene objects will only activate when the server is sure the client has loaded the scene; this is done automatically through the Fish-Networking scene manager.

Our SceneManager allows instantiated networked objects to be moved between scenes, but networked scene objects may not. Unity cannot know a scene objects details without the scene being loaded first, so trying to spawn a scene object without the client having the scene loaded would result in errors.

Due to the movement restrictions just mentioned, networked scene objects may not be marked DontDestroyOnLoad, nor can the NetworkObject.IsGlobal feature be used. Both of these would place the scene object in a new scene, causing errors.

When a networked scene object is despawned it is always disabled, rather than destroyed. This is so you may spawn it at a later time. Manually destroying a scene object on the server is possible and would simply result in it never being spawned on clients.

</details>

<details>

<summary>Nested NetworkObjects and NetworkBehaviours</summary>

NetworkObjects which are nested on a prefab may not be unparented at runtime.

Root NetworkObjects may have their parent updated at runtime.

</details>

<details>

<summary>PredictedSpawn</summary>

Predicted spawns currently cannot be spawned as nested.

Predicted spawns are currently not aligned with prediction interpolation.

</details>

Updating FishNet

Instructions for how to update FishNet to the latest or Pro version.

Before making large changes to your project, it's always advised to back it up by using your version control software or a manual backup. We advise doing this step first before trying to update your FishNet version, even if the likelihood of it breaking anything is quite small.

If you have a project using the free version of FishNet and want to update it to FishNet Pro, you can follow these exact same steps.

Steps

Make a project backup/commit.

Uninstall FishNet. You can do this from the Package Manager or by deleting the FishNet folder in the Unity Editor.

Don't close Unity.

Import the latest or Pro version of FishNet.

You can find instructions for importing the free version of FishNet here: [Installing Fish-Networking And](#)
instructions for purchasing and importing the Pro version here: [Upgrading to FishNet Pro](#)

Your project should now be correctly updated to the latest version. If you run into any issues along the way, please feel free to reach out for help in our Discord server: <https://discord.gg/Ta9HgDh4Hj>

Changes in API

Sometimes logic has to be changed for the better, but it doesn't have to be a rough experience. You may find planned breaks here, and how to resolve them.

4.1.0

NetworkManager CanLog, Log, LogWarning, LogError

Errors may show that these methods no longer exist; they however do but with different usage.

These methods have been moved under the namespace `FishNet.Managing` and are now called using either a `NetworkManager` reference, valid or null, or by using `NetworkManagerExtensions.MethodName`.

GetPooledInstantiated(GameObject, ushort, bool) is missing

Changed to `GetPooledInstantiated(GameObject, bool)`.

GetPooledInstantiated(int, bool) is missing

Changed to `GetPooledInstantiated(int, ushort, bool)`.

GetPooledInstantiated(NetworkObject, ushort, bool) is missing

Changed to `GetPooledInstantiated(NetworkObject, bool)`.

StorePooledInstantiated(NetworkObject, int, bool) is missing

Changed to `StorePooledInstantiated(NetworkObject, bool)`.

Authenticator is missing

Use `ServerManager.GetAuthenticator` or `ServerManager.SetAuthenticator` instead.

GetInstance(bool warn = true) is missing

Use `GetInstance()` or `TryGetInstance(T)`.

TimeManager LastPacketTick

Changed from `uint` to `EstimatedTick`.

You may see errors such as: cannot convert type '`uint`' to '`EstimatedTick`'. Use `LastPacketTick.LastRemoteTick`. Reviewing the `EstimatedTick` API is recommended to leverage its features better.

Misc

PooledReader and PooledWriter

`Dispose` and `DisposeLength` are missing. Use `Store` and `StoreLength`.

ReaderPool and WriterPool

`GetReader`, `GetWriter` have been renamed to be consistent with other API. Changed to `Retrieve`.

`Recycle` has been renamed to be consistent with other API. Changed to `Store`.

Observer Conditions

`DistanceCondition.UpdateFrequently` is missing. Method no longer used.

ObserverCondition.Timed is missing. Changed to GetConditionType().

ObserverCondition.Clone is missing. Method no longer used. Use Awake to initialize custom conditions.

MatchCondition had several fields replaced. The following fields were removed:

- MatchConnections ? Changed to GetMatchConnections(NetworkManager)
- ConnectionMatch ? Changed to GetConnectionMatches(NetworkManager)
- MatchObject ? Changed to GetMatchObjects(NetworkManager)
- ObjectMatch ? Changed to GetObjectMatches(NetworkManager)

SyncTypes

SyncVar and SyncObject attribute are no longer used.

Remove SyncVar/SyncObject attribute. When setting or getting the SyncVar's value be sure to use the .Value of it instead of trying to set or get it directly.

ISyncType, SyncBase.Reset is missing. Changed to ResetState.

SyncBase.Read(PooledReader) is missing. Changed to Read(PooledReader, bool).

SyncVar fields must be declared as SyncVar. Fields previously identified as SyncVars using the SyncVar attributes must now use SyncVar<Type> as the field, and must be initialized like SyncObjects.

```
// Example.
private readonly SyncVar<float> _health = new();

private void MyMethod()
{
    Debug.Log($"Current health is {_health.Value}");
    // Set health to a new value.
    _health.Value = 10f;
}
```

SceneUnloadEventArgs

UnloadedSceneHandles, UnloadedSceneNames, UnloadedScenesV2 is missing. Changed to UnloadedScenes.

PredictedObject

A variety of methods within PredictedObject have been replaced by a simpler variant.

The following methods were removed: GetSmoothTicks, GetInterpolation, SetInterpolation.

The removed methods have been replaced with SetSpectatorSmoothingType.

Misc

Broadcast: cannot convert from 'method group'.

Broadcast handlers now require Channel as the last parameter.

```
// Example:
// Before
private void OnResponseBroadcast(ResponseBroadcast rb)
// After
private void OnResponseBroadcast(ResponseBroadcast rb, Channel channel)
```

`IsClient`, `IsServer`, `IsHost`, `IsServerOnly`, `IsClientOnly` are Obsolete.

These fields return if the socket is started and for many those intentions were not clear. In result all instances have been replaced with Started versions of each; e.g.: `IsClientStarted`.

To check if an object is initialized for the server or client use Initialized variants, such as `IsClientInitialized`.

SceneLoad/UnloadData PreferredScene change.

PreferredScene has been changed to a structure to allow setting separate preferred scenes for server and client.

``NetworkAnimator.ForceSend`` is missing. Method no longer used. `NetworkAnimator.SendAll` may be of use.

``NetworkConnection.Tick`` is missing. Changed to `NetworkConnection.LocalTick`.

ServerObjects ``RebuildObservers`` method is missing. All `IEnumerable` `RebuildObservers` methods have been replaced with `IList` variants.

``ServerManager.Authenticator`` is missing. Changed to `GetAuthenticator` and `SetAuthenticator`.

``NetworkBehaviour.DirtySyncTypes`` is missing. Method no longer used.

``NetworkObject.ResetForObjectPool`` is missing. Changed to `ResetState`.

``NetworkObject.ClientInitialized`` is missing. Changed to `NetworkObject.IsClientInitialized`.

Replicate and Reconcile attribute ``Resends`` is missing. Replaced with `PredictionManager.RedundancyCount`.

``Rollback(PreciseTick, PhysicsType, bool asOwner)`` Changed to `Rollback(PreciseTick, RollbackPhysicsType, bool)`.

``ReadDictionary<TKey, TValue>()`` Changed to `ReadDictionaryAllocated<TKey, TValue>()`.

``ListCache<T>`, `ListCaches` is missing.` Classes replaced with better versions under a different namespace. The following are their replacements: `CollectionCaches`, `ResettableCollectionCaches`, `ObjectCaches`, `ResettableObjectCaches`.

``RetrieveObject(int, bool)`` is missing. Changed to `RetrieveObject(int, ushort, bool)`.

3.3.0

Unscheduled break.

ObserversRpc IncludeOwner is missing

Changed to `ExcludeOwner`. Remember to flip the values.

3.1.1

Prediction Changes

Replicate datas must now implement `IReplicateData`, and Reconcile datas must implement `IRconcileData`.

As of this release prediction methods can now have their Channels set, so you may ensure certain messages never drop. In result your replicate method signatures should now be as shown below.

```
private void MyReplicate(ReplicateData rd, bool asServer, Channel channel = Channel.Unreliable, bool replaying = false)
```

Reconcile methods must also now include a `Channel` parameter as shown.

```
private void MyReconcile(ReconcileData rd, bool asServer, Channel channel = Channel.Unreliable)
```

For more detailed instructions on these changes see the prediction guide.

TimeManager Fields and Events

Many prediction related events and values were moved out of the `TimeManager` and placed into the `PredictionManager`. Use `PredictionManager.XYZ` instead of `TimeManager.XYZ`.

The following events and fields were moved: `LastReconcileTick`, `LastReplicateTick`, `IsReplaying()`, `IsReplaying(Scene)`, `OnPreReconcile`, `OnPostReconcile`, `OnPreReplicateReplay`, `OnPostReplicateReplay`.

In addition two events had their signatures changed. `OnPre/PostReplicateReplay` now also includes the tick being replayed.

Changed signature `OnPre/PostReplicateReplay(PhysicsScene, PhysicsScene2D)`. Replaced with `OnPre/PostReplicateReplay(uint, uint)`.

Misc

`LoggingConfiguration` is now a base class. `LoggingConfiguration` can be used to create custom loggers. `LevelLoggingConfiguration` is now the default logger. If you were previously using a `LoggingConfiguration` with custom settings create new as a `LevelLoggingConfiguration`.

Changed signature `NetworkConnection.LoadedStartScenes`. Replaced with `NetworkConnection.LoadedStartScenes()`.

Removed `RigidbodyPauser.SetTimeManager`. Replaced with `RigidbodyPauser.SetPredictionManager`; now takes `PredictionManager` argument.

Removed `NetworkBehaviour/NetworkObject.Deinitializing`. Replaced with `NetworkBehaviour/NetworkObject.IsDeinitializing`.

Removed `NetworkObject.SceneObject`. Replaced with `NetworkObject.IsSceneObject`.

Removed `ServerOnlyCondition`. Replaced with `OwnerOnlyCondition`.

Removed `TimeManager.TickPercent`. Replaced with `TimeManager.GetPreciseTick` or `TimeManager.GetTickPercent`.

Removed `TimeManager.TicksToTime(bool)`. Replaced with `TimeManager.TicksToTime(TickType)`.

Removed `TimeManager.OnPhysicsSimulation`. Replaced with `TimeManager.OnPrePhysicsSimulation`.

Removed `NetworkManager.Authenticator`. Replaced with `ServerManager.Get/SetAuthenticator`.

Removed `ServerManager.Authenticator`. Replaced with `ServerManager.Get/SetAuthenticator`.

Changed signature `Transport.GetServerBindAddress`. Replaced with `Transport.GetServerBindAddress(IPAddressType)`.

Changed signature `Transport.SetServerBindAddress(string)`. Replaced with `Transport.SetServerBindAddress(string, IPAddressType)`.

Removed `ListCaches.XYZCache`. Replaced with `ListCaches.GetXYZCache`, `ListCaches.SetXYZCache`.

2.0.0

Misc

Removed ``NetworkBehaviour/NetworkObject.OwnerIsValid``. Replaced with `NetworkBehaviour/NetworkObject.Owner.IsValid`.

Removed ``NetworkBehaviour/NetworkObject.OwnerIsActive``. Replaced with `NetworkBehaviour/NetworkObject.Owner.IsActive`.

Changed Name ``LoadOptions.DisallowStacking``. Change references from `DisallowStacking` to `AllowStacking` and inverse value.

Changed Name — **Several enums are being changed from plural to singular.** Those changed will be `SceneScopeTypes`, `ServerUnloadModes`, `LocalConnectionStates`, `RemoteConnectionStates`. Remove plural (s) from each enum reference. E.g.: `SceneScopeTypes` will become `SceneScopeType`.

Removed ``TimeManager.TicksToTimeDouble``. Replaced with `TimeManager.TicksToTime`; now returns a double.

Changed signature ``Transport.Initialize(networkManager)``. Add `int` as a second parameter to your transport's `Initialize` method. Include `Transport.Index` within the following callback structures: `ClientConnectionStateArgs`, `ServerConnectionStateArgs`, `RemoteConnectionStateArgs`, `ClientReceivedDataArgs`, `ServerReceivedDataArgs`.

Removed ``Transport.GetChannelCount()``, ``Transport.GetDefaultReliableChannel()``, ``Transport.GetDefaultUnreliableChannel()``. Delete call or overrides to methods. There will always only be two channel counts, 0 for reliable and 1 for unreliable.

Removed ``Transport.GetDefaultReliableChannel()`` and ``GetDefaultUnreliableChannel()``. Delete calls or overrides to methods. `Channel.Reliable` should always be id 0, and `Channel.Unreliable` id 1.

Upgrading to FishNet Pro





Instructions for how to purchase and then install FishNet Pro in your Unity project.

Purchasing FishNet Pro

Fish-Networking Pro can be obtained by purchasing it from the Unity Asset Store or from sponsoring the project on GitHub or Patreon.

Unity Asset Store — Purchase directly on the Unity Asset Store to get lifetime access to future updates and be able to import it through Unity's Package Manager.

GitHub Sponsors — Sponsor FishNet on GitHub to get access to FishNet Pro on a monthly or lifetime basis. This option also gives the most back to the developers.

Patreon — Get access to FishNet Pro on a monthly or lifetime basis by sponsoring the developer on Patreon.

Unity Asset Store

Purchasing and installing via the Unity Asset Store should be very familiar to a regular Unity user. With this method you will have lifetime access to FishNet and be able to download the latest version available on the Asset Store through the Unity Package Manager.

Open your Unity project.

Go to **Window > Asset Store** or visit the Unity Asset Store online.

Search for **FishNet Pro** or navigate to

<https://assetstore.unity.com/packages/tools/network/fishnet-pro-networking-evolved-287711>.

Click **Buy Now**, proceed with the payment, and then click **Open in Unity**.

In Unity, go to **Package Manager** (Window > Package Manager).

Select **My Assets**, find FishNet Pro, and click **Download** and then **Import**.

You can also gain access to Priority Support with your asset store purchase! Your priority support months begin only after you claim your perks on our Discord. The amount of months received are

directly related to what goes back into the project after Unity's fees. If you purchase Fish-Networking Pro at \$50 you will receive 4 months of Priority Support, and 2 months at \$25.

GitHub Sponsors

Through GitHub Sponsors you can get lifetime or monthly access to FishNet at various prices. When obtaining Fish-Networking Pro through GitHub you will have access to both Pro and the paid Projects.

Head over to the FirstGearGames GitHub Sponsors page.

Select one of the monthly or one-time options and then proceed to the payment.

You can now go to the Fish-Networking website.

Login with the **GitHub Login** option and then download the FishNet .unitypackage file.

In Unity, go to **Assets > Import Package > Custom Package...**

Select the downloaded .unitypackage file.

Click **Import** to bring all the FishNet files into your project.

You must make a one-time payment of at least \$10 for your initial downloads. Afterwards you're welcome to switch to \$2/month for updates. You may leave the updates tier and join us again at any time. When on any \$10 tier or higher you will gain access to Priority Support, this includes your initial purchase.

Patreon

When obtaining Fish-Networking Pro through Patreon you will have access to both Pro and the paid Projects.

Head over to the FirstGearGames Patreon page.

Select a membership tier and then proceed to the payment.

You can now go to the Fish-Networking website.

Login with the **Patreon Login** option and then download the FishNet .unitypackage file.

In Unity, go to **Assets > Import Package > Custom Package...**

Select the downloaded .unitypackage file.

Click **Import** to bring all the FishNet files into your project.

You must make a one-time payment of at least \$10 for your initial downloads. Afterwards you're welcome to switch to \$2/month for updates. You may leave the updates tier and join us again at any time. When on any \$10 tier or higher you will gain access to Priority Support, this includes your initial purchase.